

AI ENGINEERING FROM SCRATCH.

Language: NLP Foundations and the Transformer

VOLUME III / VI · 45 CHAPTERS · PHASES 05 · 07

EDITION 2026.09 · A SNAPSHOT OF A LIVING COURSE · NEWEST BUILD ALWAYS AT THE LINK BELOW

aiengineeringfromscratch.com

github.com/rohitg00/ai-engineering-from-scratch

Contents

About This Volume	1
How to Use This Book	2
Part I — NLP, Foundations to Advanced	3
Text Processing — Tokenization, Stemming, Lemmatization	4
Bag of Words, TF-IDF, and Text Representation	10
Word Embeddings — Word2Vec from Scratch	16
GloVe, FastText, and Subword Embeddings	22
Sentiment Analysis	28
Named Entity Recognition	34
POS Tagging and Syntactic Parsing	41
CNNs and RNNs for Text	47
Sequence-to-Sequence Models	52
Attention Mechanism — The Breakthrough	57
Machine Translation	63
Text Summarization	69
Question Answering Systems	75
Information Retrieval and Search	81
Topic Modeling — LDA and BERTopic	87
Text Generation Before Transformers — N-gram Language Models	92
Chatbots — Rule-Based to Neural to LLM Agents	99
Multilingual NLP	107
Subword Tokenization — BPE, WordPiece, Unigram, SentencePiece	114
Structured Outputs & Constrained Decoding	119
Natural Language Inference — Textual Entailment	125
Embedding Models — The 2026 Deep Dive	130

Chunking Strategies for RAG	136
Coreference Resolution	142
Entity Linking & Disambiguation	147
Relation Extraction & Knowledge Graph Construction	152
LLM Evaluation — RAGAS, DeepEval, G-Eval	158
Long-Context Evaluation — NIAH, RULER, LongBench, MRCR	164
Dialogue State Tracking	169
Part II — Transformers Deep Dive	174
Why Transformers — The Problems with RNNs	175
Self-Attention from Scratch	180
Multi-Head Attention	187
Positional Encoding — Sinusoidal, RoPE, ALiBi	192
The Full Transformer — Encoder + Decoder	198
BERT — Masked Language Modeling	203
GPT — Causal Language Modeling	208
T5, BART — Encoder-Decoder Models	214
Vision Transformers (ViT)	220
Audio Transformers — Whisper Architecture	225
Mixture of Experts (MoE)	231
KV Cache, Flash Attention & Inference Optimization	236
Scaling Laws	242
Build a Transformer from Scratch — The Capstone	247
Attention Variants — Sliding Window, Sparse, Differential	253
Speculative Decoding — Draft, Verify, Repeat	259

About This Volume

This is Volume 3 of *AI Engineering from Scratch*, a six-volume compilation of the open course of the same name. Each volume stands alone; cross-references cite course phase numbers, which map to volumes like this:

Vol	Title	Course phases
1	Foundations — Math, Tooling, and Classical Machine Learning	00, 01, 02
2	Deep Learning — Networks, Vision, and Speech	03, 04, 06
3	Language — NLP Foundations and the Transformer	05, 07
4	Large Language Models — Generation, Reinforcement, Pretraining, and Engineering	08, 09, 10, 11
5	Agents — Multimodality, Protocols, Autonomy, and Swarms	12, 13, 14, 15, 16
6	Production — Infrastructure, Safety, and Capstones	17, 18, 19

The chapters in this volume come from course phases 05, 07. Chapter prerequisites name phases, not volumes; use the table above to translate.

How to Use This Book

This volume is one loop of a larger machine, and it works best when you run the whole loop:

1. **Read the chapter here.** The prose, the derivations, and the code walkthroughs are complete on the page.
2. **Run the code from the repository.** Every chapter has a code/ directory with a working implementation you can run and break: <https://github.com/rohitg00/ai-engineering-from-scratch>
3. **Open the web edition for what paper cannot do.** Animated figures you can watch and drag, and a quiz per chapter that grades itself: <https://aiengineeringfromscratch.com>

The repository is the living edition. Lessons are updated as the field moves; the book is a snapshot with a version number. When they disagree, the repo is right.

Learning with an AI

This course is built to be read by agents as well as people. The machine-readable index of every lesson lives at <https://aiengineeringfromscratch.com/llms.txt>. If you learn with an AI assistant, paste this and go:

I am working through *AI Engineering from Scratch, Volume 3: Language*. Fetch <https://aiengineeringfromscratch.com/llms.txt>, find the lesson I name, and act as my tutor: quiz me on its Key Terms, review my solutions to its Exercises, and walk me through its code from the repository.

Part I — NLP, Foundations to Advanced

Course phase 05. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>

Text Processing — Tokenization, Stemming, Lemmatization

Language is continuous. Models are discrete. Preprocessing is the bridge.

Type: Build **Languages:** Python **Prerequisites:** Phase 2 · 14 (Naive Bayes) **Time:** ~45 minutes

The Problem

A model cannot read “The cats were running.” It reads integers.

Every NLP system opens with the same three questions. Where does a word start. What is the root of the word. How do we treat “run”, “running”, “ran” as the same thing when it helps, and as different things when it doesn’t.

Get tokenization wrong and the model learns from garbage. If your tokenizer treats don't as one token but do n't as two, the training distribution splits. If your stemmer collapses organization and organ to the same stem, topic modeling dies. If your lemmatizer needs part-of-speech context but you don't pass it, verbs get treated as nouns.

This lesson builds the three preprocessing steps from scratch, then shows how NLTK and spaCy do the same work so you can see the tradeoffs.

The Concept

Three operations. Each has a job and a failure mode.

Tokenization splits a string into tokens. “Token” is deliberately vague because the right granularity depends on the task. Word-level for classical NLP. Subword for transformers. Character for languages without whitespace.

Stemming chops suffixes with rules. Fast, aggressive, dumb. running -> run. organization -> organ. That second one is the failure mode.

Lemmatization reduces a word to its dictionary form using grammar knowledge. Slower, accurate, needs a lookup table or morphological analyzer. ran -> run (needs to know “ran” is past tense of “run”). better -> good (needs to know comparative forms).

Rule of thumb. Stem when speed matters and you can tolerate noise (search indexing, rough classification). Lemmatize when meaning matters (question answering, semantic search, anything the user will read).

Interactive figure: edit-distance. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/01-text-processing>

Build It

Step 1: a regex word tokenizer

The simplest useful tokenizer splits on non-alphanumeric characters while keeping punctuation as its own tokens. Not perfect, not final, but it runs in one line.

```
import re

def tokenize(text):
    return re.findall(r"[A-Za-z]+(?:'[A-Za-z]+)?|[0-9]+|[\^\sA-Za-z0-9]", text)
```

Three patterns in order of precedence. Words with optional inner apostrophe (don't, it's). Pure numbers. Any single non-whitespace non-alphanumeric character as a standalone token (punctuation).

```
>>> tokenize("The cats weren't running at 3pm.")
['The', 'cats', "weren't", 'running', 'at', '3', 'pm', '.']
```

Failure modes to notice. 3pm splits to ['3', 'pm'] because we alternated between letter runs and digit runs. Good enough for most tasks. URLs, emails, hashtags all break. For production, add patterns before the general ones.

Step 2: a Porter stemmer (step 1a only)

The full Porter algorithm has five phases of rules. Step 1a alone covers the most frequent English suffixes and teaches the pattern.

```
def stem_step_1a(word):
    if word.endswith("sses"):
        return word[:-2]
    if word.endswith("ies"):
        return word[:-2]
    if word.endswith("ss"):
        return word
    if word.endswith("s") and len(word) > 1:
        return word[:-1]
    return word

>>> [stem_step_1a(w) for w in ["caresses", "ponies", "caress", "cats"]]
['caress', 'poni', 'caress', 'cat']
```

Read the rules top-down. The ies -> i rule is why ponies -> poni, not pony. Real Porter has step 1b that would fix it. Rules compete. Earlier rules win. The order matters more than any single rule.

Step 3: a lookup-based lemmatizer

Lemmatization proper needs morphology. A tractable teaching version uses a small lemma table and a fallback.

```
LEMMA_TABLE = {
    ("running", "VERB"): "run",
    ("ran", "VERB"): "run",
    ("runs", "VERB"): "run",
    ("better", "ADJ"): "good",
    ("best", "ADJ"): "good",
```

```

    ("cats", "NOUN"): "cat",
    ("cat", "NOUN"): "cat",
    ("were", "VERB"): "be",
    ("was", "VERB"): "be",
    ("is", "VERB"): "be",
}

def lemmatize(word, pos):
    key = (word.lower(), pos)
    if key in LEMMA_TABLE:
        return LEMMA_TABLE[key]
    if pos == "VERB" and word.endswith("ing"):
        return word[:-3]
    if pos == "NOUN" and word.endswith("s"):
        return word[:-1]
    return word.lower()

>>> lemmatize("running", "VERB")
'run'
>>> lemmatize("cats", "NOUN")
'cat'
>>> lemmatize("better", "ADJ")
'good'
>>> lemmatize("watched", "VERB")
'watched'

```

The last case is the key teaching moment. `watched` is not in our table and our fallback only handles `ing`. Real lemmatization covers `ed`, irregular verbs, comparative adjectives, plurals with sound changes (`children` -> `child`). This is why production systems use WordNet, spaCy's morphologizer, or a full morphological analyzer.

Step 4: pipe them together

```

def preprocess(text, pos_tagger=None):
    tokens = tokenize(text)
    stems = [stem_step_1a(t.lower()) for t in tokens]
    tags = pos_tagger(tokens) if pos_tagger else [(t, "NOUN") for t in tokens]
    lemmas = [lemmatize(word, pos) for word, pos in tags]
    return {"tokens": tokens, "stems": stems, "lemmas": lemmas}

```

The missing piece is a POS tagger. Phase 5 · 07 (POS Tagging) builds one. For now, default everything to NOUN and acknowledge the limitation.

Use It

NLTK and spaCy ship the production versions. A few lines each.

NLTK

```

import nltk
nltk.download("punkt_tab")
nltk.download("wordnet")
nltk.download("averaged_perceptron_tagger_eng")

```

```

from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag

```

```

text = "The cats were running."
tokens = word_tokenize(text)
stems = [PorterStemmer().stem(t) for t in tokens]
lemmatizer = WordNetLemmatizer()
tagged = pos_tag(tokens)

```

```

def nltk_pos_to_wordnet(tag):
    if tag.startswith("V"):
        return "v"
    if tag.startswith("J"):
        return "a"
    if tag.startswith("R"):
        return "r"
    return "n"

```

```

lemmas = [lemmatizer.lemmatize(t, nltk_pos_to_wordnet(tag)) for t, tag in tagged]

```

word_tokenize handles contractions, Unicode, edge cases your regex misses. PorterStemmer runs all five phases. WordNetLemmatizer needs the POS tag translated from NLTK's Penn Treebank scheme to WordNet's abbreviation set. The translation wiring above is the bit most tutorials skip.

spaCy

```
import spacy
```

```

nlp = spacy.load("en_core_web_sm")
doc = nlp("The cats were running.")

```

```

for token in doc:
    print(token.text, token.lemma_, token.pos_)

```

```

The      the      DET
cats     cat       NOUN
were     be        AUX
running  run       VERB
.        .         PUNCT

```

spaCy hides the whole pipeline behind `nlp(text)`. Tokenization, POS tagging, and lemmatization all run. Faster than NLTK at scale. More accurate out of the box. The tradeoff is that you cannot easily swap individual components.

When to pick which

Situation	Pick
Teaching, research, swapping components	NLTK
Production, multi-language, speed matters	spaCy

Situation	Pick
Transformer pipeline (you'll tokenize with the model's tokenizer anyway)	Use tokenizers / transformers and skip classical preprocessing

The two failure modes nobody warns you about

Most tutorials teach the algorithms and stop. Two things will bite a real preprocessing pipeline, and they are almost never covered.

Reproducibility drift. NLTK and spaCy change tokenization and lemmatizer behavior between versions. What produced `['do', 'n't']` in spaCy 2.x may produce `["don't"]` in 3.x. Your model was trained on one distribution. Inference now runs on a different one. Accuracy quietly degrades and nobody knows why. Pin library versions in `requirements.txt`. Write a preprocessing regression test that freezes expected tokenization of 20 sample sentences. Run it on every upgrade.

Training / inference mismatch. Train with aggressive preprocessing (lowercase, stopword removal, stemming), deploy on raw user input, watch performance crater. This is the single most common production NLP failure. If you preprocess during training, you must run the identical function during inference. Ship preprocessing as a function inside the model package, not as a notebook cell the serving team rewrites.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/01-text-processing>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/01-text-processing/code>

1. **Easy.** Extend `tokenize` to keep URLs as single tokens. Test: `tokenize("Visit https://example.com today.")` should produce one URL token.
2. **Medium.** Implement Porter step 1b. If a word contains a vowel and ends in `ed` or `ing`, remove it. Handle the double-consonant rule (hopping $\rightarrow$ hop, not hopp).
3. **Hard.** Build a lemmatizer that uses WordNet as a lookup table but falls back to your Porter stemmer when WordNet has no entry. Measure accuracy on a tagged corpus against plain WordNet and plain Porter.

Key Terms

Term	What people say	What it actually means
Token	A word	Whatever unit the model consumes. Can be word, subword, character, or byte.
Stem	Root of a word	Result of rule-based suffix stripping. Not always a real word.
Lemma	Dictionary form	The form you'd look up. Requires grammatical context to compute correctly.

Term	What people say	What it actually means
POS tag	Part of speech	Category like NOUN, VERB, ADJ. Needed to lemmatize accurately.
Morphology	Word shape rules	How a word changes form based on tense, number, case. Lemmatization depends on it.

Further Reading

- Porter, M. F. (1980). An algorithm for suffix stripping — the original paper, five pages, still the clearest explanation.
- spaCy 101 — linguistic features — how a real pipeline is wired.
- NLTK book, chapter 3 — tokenization edge cases you haven't thought of yet.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/01-text-processing>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/01-text-processing/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/01-text-processing>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Bag of Words, TF-IDF, and Text Representation

Count first, think later. TF-IDF still beats embeddings on well-defined tasks in 2026.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 01 (Text Processing), Phase 2 · 02 (Linear Regression from Scratch) **Time:** ~75 minutes

The Problem

The model needs numbers. You have strings.

Every NLP pipeline has to answer the same question. How do we turn a variable-length stream of tokens into a fixed-size vector that a classifier can consume. The first answer the field landed on was the dumbest one that works. Count the words. Make a vector.

That vector has carried more production NLP than any embedding model. Spam filters, topic classifiers, log anomaly detection, search ranking (before BM25), the first wave of sentiment analysis, the first decade of academic NLP benchmarks. 2026 practitioners still reach for it first on narrow classification tasks. It is fast, interpretable, and often indistinguishable from a 400M-parameter embedding model on tasks where word presence is what matters.

This lesson builds bag of words, then TF-IDF, from scratch. Then shows scikit-learn doing the same in three lines. Then names the failure mode that makes you reach for embeddings.

The Concept

Bag of Words (BoW) throws away order. For each document, count how many times each vocabulary word appears. Vector length is the vocabulary size. Position i is the count of word i .

TF-IDF reweights BoW. A word that appears in every document is uninformative, so scale it down. A word rare across the corpus but frequent in a single document is signal, so scale it up.

$$\begin{aligned}\text{TF-IDF}(w, d) &= \text{TF}(w, d) * \text{IDF}(w) \\ &= \text{count}(w \text{ in } d) / |d| * \log(N / \text{df}(w))\end{aligned}$$

Where TF is term frequency in the document, df is document frequency (how many docs contain the word), N is total documents. The log keeps the weight bounded for ubiquitous words.

Key property: both produce sparse vectors with interpretable axes. You can look at a trained classifier's weights and read which words push a document toward each class. You cannot do this with a 768-dimensional BERT embedding.

Interactive figure: bow-tfidf. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/02-bag-of-words-tfidf>

Build It

Step 1: build the vocabulary

```
def build_vocab(docs):
    vocab = {}
    for doc in docs:
        for token in doc:
            if token not in vocab:
                vocab[token] = len(vocab)
    return vocab
```

Input: list of tokenized documents (any word-level tokenizer will do; the code/main.py in this lesson uses a simplified lowercase variant). Output: {word: index} dict. Stable insertion order means word index 0 is the first word seen in the first document. Convention varies; scikit-learn sorts alphabetically.

Step 2: bag of words

```
def bag_of_words(docs, vocab):
    matrix = [[0] * len(vocab) for _ in docs]
    for i, doc in enumerate(docs):
        for token in doc:
            if token in vocab:
                matrix[i][vocab[token]] += 1
    return matrix

>>> docs = [["cat", "sat", "on", "mat"], ["cat", "cat", "ran"]]
>>> vocab = build_vocab(docs)
>>> bag_of_words(docs, vocab)
[[1, 1, 1, 1, 0], [2, 0, 0, 0, 1]]
```

Rows are documents. Columns are vocabulary indices. Entry [i][j] is “how many times word j appears in document i.” Doc 1 has cat twice because it did. Doc 0 has ran zero times because it did not.

Step 3: term frequency and document frequency

```
import math

def term_frequency(doc_bow, doc_length):
    return [c / doc_length if doc_length else 0 for c in doc_bow]

def document_frequency(bow_matrix):
    df = [0] * len(bow_matrix[0])
    for row in bow_matrix:
        for j, count in enumerate(row):
            if count > 0:
                df[j] += 1
    return df
```

```
def inverse_document_frequency(df, n_docs):
    return [math.log((n_docs + 1) / (d + 1)) + 1 for d in df]
```

Two smoothing tricks worth naming. The $(n+1)/(d+1)$ avoids $\log(x/0)$. The trailing $+1$ ensures a word in every document still has IDF 1 (not 0), matching scikit-learn's default. Other implementations use raw $\log(N/df)$. Both work; the smoothed version is friendlier.

Step 4: TF-IDF

```
def tfidf(bow_matrix):
    n_docs = len(bow_matrix)
    df = document_frequency(bow_matrix)
    idf = inverse_document_frequency(df, n_docs)
    out = []
    for row in bow_matrix:
        length = sum(row)
        tf = term_frequency(row, length)
        out.append([tf_j * idf_j for tf_j, idf_j in zip(tf, idf)])
    return out
```

```
>>> docs = [
...     ["the", "cat", "sat"],
...     ["the", "dog", "sat"],
...     ["the", "cat", "ran"],
... ]
>>> vocab = build_vocab(docs)
>>> bow = bag_of_words(docs, vocab)
>>> tfidf(bow)
```

Three documents, five vocab words (the, cat, sat, dog, ran). the appears in all three, so its IDF is low. dog appears in one, so its IDF is high. The vectors are sparse (most entries are small) and the discriminative words pop.

Step 5: L2-normalize rows

```
def l2_normalize(matrix):
    out = []
    for row in matrix:
        norm = math.sqrt(sum(x * x for x in row))
        out.append([x / norm if norm else 0 for x in row])
    return out
```

Without normalization, a longer document gets a larger vector and dominates similarity scores. L2 normalization puts every document on the unit hypersphere. Cosine similarity between rows is now just a dot product.

Use It

scikit-learn ships the production version.

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
```

```
docs = ["the cat sat on the mat", "the dog sat on the mat", "the cat ran"]
```

```
bow_vectorizer = CountVectorizer()
```

```
bow = bow_vectorizer.fit_transform(docs)
print(bow_vectorizer.get_feature_names_out())
print(bow.toarray())
```

```
tfidf_vectorizer = TfidfVectorizer()
tfidf = tfidf_vectorizer.fit_transform(docs)
print(tfidf.toarray().round(3))
```

CountVectorizer does tokenization, vocabulary, and BoW in one call. TfidfVectorizer adds IDF weighting and L2 normalization. Both return sparse matrices. For 100k documents, the dense version does not fit in memory; stay sparse until the classifier demands dense.

Knobs that change everything:

Arg	Effect
ngram_range=(1, 2)	Include bigrams. Usually boosts classification.
min_df=2	Drop words in fewer than 2 docs. Trims vocabulary on noisy data.
max_df=0.95	Drop words in more than 95% of docs. Approximates stopword removal without a hardcoded list.
stop_words="english"	scikit-learn's builtin stopword list. Task-dependent — sentiment analysis should <i>not</i> drop negations.
sublinear_tf=True	Use $1 + \log(\text{tf})$ instead of raw tf. Helps when a term repeats many times in one doc.

When TF-IDF still wins (as of 2026)

- Spam detection, topic labeling, log anomaly flagging. Word presence is what matters; semantic nuance does not.
- Low-data regimes (hundreds of labeled examples). TF-IDF plus logistic regression has no pretraining cost.
- Anywhere latency matters. TF-IDF plus a linear model answers in microseconds. Embedding a document through a transformer takes 10-100ms.
- Systems that must explain their predictions. Inspect the classifier's coefficients. Top positive words are the reason.

When TF-IDF fails

The semantic blindness failure. Consider these two documents:

- "The movie was not good at all."
- "The movie was excellent."

One is a negative review. One is positive. Their TF-IDF overlap is exactly {the, movie, was}. A bag-of-words classifier has to memorize that the word not near good flips the label. It can learn this on enough data, but never as gracefully as a model that understands syntax.

The other failure: out-of-vocabulary words at inference. A BoW model trained on IMDb reviews has no idea what to do with Zoomer-approved if that token never appeared in training. Subword embeddings (lesson 04) handle this. TF-IDF cannot.

Hybrid: TF-IDF weighted embeddings

The 2026 pragmatic default for medium-data classification: use TF-IDF weights as attention over word embeddings.

```
def tfidf_weighted_embedding(doc, tfidf_scores, embedding_table, dim):
    vec = [0.0] * dim
    total_weight = 0.0
    for token in doc:
        if token not in embedding_table or token not in tfidf_scores:
            continue
        weight = tfidf_scores[token]
        emb = embedding_table[token]
        for i in range(dim):
            vec[i] += weight * emb[i]
        total_weight += weight
    if total_weight == 0:
        return vec
    return [v / total_weight for v in vec]
```

You get semantic capacity from embeddings, and rare-word emphasis from TF-IDF. Classifier trains on the pooled vector. This outperforms either on its own for sentiment, topic, and intent classification below about 50k labeled examples.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/02-bag-of-words-tfidf>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/02-bag-of-words-tfidf/code>

1. **Easy.** Implement `cosine_similarity(doc_vec_a, doc_vec_b)` on the L2-normalized TF-IDF output. Verify that identical documents score 1.0 and disjoint-vocabulary documents score 0.0.
2. **Medium.** Add n-gram support to `bag_of_words`. Parameter `n` produces counts over n-grams. Test that `n=2` on `["the", "cat", "sat"]` produces bigram counts for `["the cat", "cat sat"]`.
3. **Hard.** Build the TF-IDF-weighted-embedding hybrid above using GloVe 100d vectors (download once, cache). Compare classification accuracy against plain TF-IDF and plain mean-pooled embeddings on the 20 Newsgroups dataset. Report which wins where.

Key Terms

Term	What people say	What it actually means
BoW	Word frequency vector	Counts of vocabulary words in one document. Throws away order.
TF	Term frequency	Count of a word in a document, optionally normalized by document length.
DF	Document frequency	Count of documents containing the word at least once.
IDF	Inverse document frequency	$\log(N / df)$ smoothed. Downweights words that appear everywhere.

Term	What people say	What it actually means
Sparse vector	Mostly zeros	Vocabulary is typically 10k-100k words; most are absent from any given document.
Cosine similarity	Vector angle	Dot product of L2-normalized vectors. 1 is identical, 0 is orthogonal.

Further Reading

- scikit-learn — feature extraction from text — the canonical API reference, plus notes on every knob.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval — the paper that made TF-IDF the default for a decade.
- “Why TF-IDF Still Beats Embeddings” — Ashfaq Thonikkadavan (Medium) — 2026 take on when the old method wins and why.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/02-bag-of-words-tfidf>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/02-bag-of-words-tfidf/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/02-bag-of-words-tfidf>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Word Embeddings — Word2Vec from Scratch

A word is the company it keeps. Train a shallow net on that idea and geometry falls out.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 02 (BoW + TF-IDF), Phase 3 · 03 (Backpropagation from Scratch) **Time:** ~75 minutes

The Problem

TF-IDF knows dog and puppy are different words. It does not know they mean nearly the same thing. A classifier trained on dog cannot generalize to a review about puppy. You can paper over this by listing synonyms, but that fails on rare terms, domain jargon, and every language you did not anticipate.

You want a representation where dog and puppy land close together in space. Where king - man + woman lands near queen. Where a model trained on dog transfers some signal to puppy for free.

Word2Vec gave us that space. Two layer neural network, trillion-token training runs, published in 2013. The architecture is almost embarrassingly simple. The results reshaped NLP for a decade.

The Concept

Distributional hypothesis (Firth, 1957): “You shall know a word by the company it keeps.” If two words appear in similar contexts, they probably mean similar things.

Word2Vec comes in two flavors, both exploiting that idea.

- **Skip-gram.** Given a center word, predict the surrounding words. cat -> (the, sat, on) with window size 2.
- **CBOW (continuous bag of words).** Given surrounding words, predict the center. (the, sat, on) -> cat.

Skip-gram is slower to train but handles rare words better. It became the default.

The network has one hidden layer with no nonlinearity. Input is a one-hot vector over the vocabulary. Output is a softmax over the vocabulary. After training, you throw away the output layer. The hidden layer weights are the embeddings.

```
one-hot(center) — W → hidden (d-dim) — W' → softmax(vocab)
                        ^
                        this is the embedding
```

The trick: softmax over 100k words is prohibitively expensive. Word2Vec uses **negative sampling** to turn it into a binary classification task. Predict “did this context word appear

near this center word, yes or no”. Sample a handful of negative (non-co-occurring) words per training pair instead of computing softmax over the whole vocabulary.

Interactive figure: word-vector-arithmetic. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/03-word-embeddings-word2vec>

Build It

Step 1: training pairs from a corpus

```
def skipgram_pairs(docs, window=2):
    pairs = []
    for doc in docs:
        for i, center in enumerate(doc):
            for j in range(max(0, i - window), min(len(doc), i + window + 1)):
                if i == j:
                    continue
                pairs.append((center, doc[j]))
    return pairs

>>> skipgram_pairs(["the", "cat", "sat", "on", "mat"], window=2)
[('the', 'cat'), ('the', 'sat'),
 ('cat', 'the'), ('cat', 'sat'), ('cat', 'on'),
 ('sat', 'the'), ('sat', 'cat'), ('sat', 'on'), ('sat', 'mat'),
 ...]
```

Every (center, context) pair in a window is a positive training example.

Step 2: embedding tables

Two matrices. W is the center-word embedding table (the one you keep). W' is the context-word table (often discarded, sometimes averaged with W).

```
import numpy as np

def init_embeddings(vocab_size, dim, seed=0):
    rng = np.random.default_rng(seed)
    W = rng.normal(0, 0.1, size=(vocab_size, dim))
    W_prime = rng.normal(0, 0.1, size=(vocab_size, dim))
    return W, W_prime
```

Small random init. Vocab size 10k and dim 100 is realistic; for teaching, 50 vocab x 16 dim is enough to see the geometry.

Step 3: negative sampling objective

For each positive pair (center, context), sample k random words from the vocabulary as negatives. Train the model so the dot product $W[\text{center}] \cdot W'[\text{context}]$ is high for positives and low for negatives.

```
def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -20, 20)))
```

```
def train_pair(W, W_prime, center_idx, context_idx, negative_indices, lr):
    v_c = W[center_idx]
    u_pos = W_prime[context_idx]
    u_negs = W_prime[negative_indices]

    pos_score = sigmoid(v_c @ u_pos)
    neg_scores = sigmoid(u_negs @ v_c)

    grad_center = (pos_score - 1) * u_pos
    for i, u in enumerate(u_negs):
        grad_center += neg_scores[i] * u

    W[context_idx] = W[context_idx]
    W_prime[context_idx] -= lr * (pos_score - 1) * v_c
    for i, neg_idx in enumerate(negative_indices):
        W_prime[neg_idx] -= lr * neg_scores[i] * v_c
    W[center_idx] -= lr * grad_center
```

The magic formula: logistic loss on positive pair (want sigmoid near 1) plus logistic loss on negative pairs (want sigmoid near 0). Gradients flow to both tables. Full derivation is in the original paper; walk through it once with pencil and paper if you want it to stick.

Step 4: train on a toy corpus

```
def train(docs, dim=16, window=2, k_neg=5, epochs=100, lr=0.05, seed=0):
    vocab = build_vocab(docs)
    vocab_size = len(vocab)
    rng = np.random.default_rng(seed)
    W, W_prime = init_embeddings(vocab_size, dim, seed=seed)
    pairs = skipgram_pairs(docs, window=window)

    for epoch in range(epochs):
        rng.shuffle(pairs)
        for center, context in pairs:
            c_idx = vocab[center]
            ctx_idx = vocab[context]
            negs = rng.integers(0, vocab_size, size=k_neg)
            negs = [n for n in negs if n != ctx_idx and n != c_idx]
            train_pair(W, W_prime, c_idx, ctx_idx, negs, lr)
    return vocab, W
```

After enough epochs on a large corpus, words that share contexts have similar center embeddings. On a toy corpus, you see the effect faintly. On billions of tokens, you see it dramatically.

Step 5: the analogy trick

```
def nearest(vocab, W, target_vec, topk=5, exclude=None):
    exclude = exclude or set()
    inv_vocab = {i: w for w, i in vocab.items()}
    norms = np.linalg.norm(W, axis=1, keepdims=True) + 1e-9
    W_norm = W / norms
    target = target_vec / (np.linalg.norm(target_vec) + 1e-9)
    sims = W_norm @ target
```

```

order = np.argsort(-sims)
out = []
for i in order:
    if i in exclude:
        continue
    out.append((inv_vocab[i], float(sims[i])))
    if len(out) == topk:
        break
return out

def analogy(vocab, W, a, b, c, topk=5):
    v = W[vocab[b]] - W[vocab[a]] + W[vocab[c]]
    return nearest(vocab, W, v, topk=topk, exclude={vocab[a], vocab[b], vocab[c]})

```

On pre-trained 300d Google News vectors:

```

>>> analogy(vocab, W, "man", "king", "woman")
[('queen', 0.71), ('monarch', 0.62), ('princess', 0.59), ...]

```

king - man + woman = queen. Not because the model knows what royalty is. Because the vector (king - man) captures something like “royal”, and adding it to woman lands near the royal-female region.

Use It

Writing Word2Vec from scratch is teaching. Production NLP uses `gensim`.

```

from gensim.models import Word2Vec

sentences = [
    ["the", "cat", "sat", "on", "the", "mat"],
    ["the", "dog", "ran", "across", "the", "room"],
]

model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=1,
    sg=1,
    negative=5,
    workers=4,
    epochs=30,
)

print(model.wv["cat"])
print(model.wv.most_similar("cat", topn=3))

```

For real work, you almost never train Word2Vec yourself. You download pre-trained vectors.

- **GloVe** — Stanford’s co-occurrence-matrix factorization approach. 50d, 100d, 200d, 300d checkpoints. Good general coverage. Lesson 04 covers GloVe specifically.
- **fastText** — Facebook’s Word2Vec extension that embeds character n-grams. Handles out-of-vocabulary words by composing subwords. Lesson 04.

- **Pretrained Word2Vec on Google News** — 300d, 3M word vocabulary, published 2013. Still downloaded daily.

When Word2Vec still wins in 2026

- Lightweight domain-specific retrieval. Train on medical abstracts in an hour on a laptop, get specialized vectors no general model captures.
- Analogy-style feature engineering. `gender_vector = mean(man - woman pairs)`. Subtract it from other words to get a gender-neutral axis. Still used in fairness research.
- Interpretability. 100d is small enough to plot via PCA or t-SNE and actually see clusters form.
- Anywhere inference has to run on-device with no GPU. Word2Vec lookup is a single row fetch.

Where Word2Vec fails

The polysemy wall. `bank` has one vector. `river bank` and `financial bank` share it. `table` (spreadsheet vs. furniture) shares it. A classifier downstream cannot distinguish the senses from the vector.

Contextual embeddings (ELMo, BERT, every transformer since) solved this by producing a different vector for each occurrence of the word based on surrounding context. That is the jump from Word2Vec to BERT: from static to contextual. Phase 7 covers the transformer half.

The out-of-vocabulary problem is the other failure. Word2Vec has never seen `Zoomer`-approved if it was not in training data. No fallback. `fastText` fixes this with subword composition (lesson 04).

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/03-word-embeddings-word2vec>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/03-word-embeddings-word2vec/code>

1. **Easy.** Run the training loop on a tiny corpus (20 sentences about cats and dogs). After 200 epochs, verify `nearest(vocab, W, W[vocab["cat"]])` returns `dog` in its top 3. If not, increase epochs or vocabulary.
2. **Medium.** Add subsampling of frequent words. Words with frequency above 10^{-5} are dropped from training pairs with probability proportional to their frequency. Measure the effect on rare-word similarity.
3. **Hard.** Train a model on the 20 Newsgroups corpus. Compute two bias axes: `he - she` and `doctor - nurse`. Project occupation words onto both axes. Report which occupations have the largest bias gap. This is the kind of probe fairness researchers use.

Key Terms

Term	What people say	What it actually means
Word embedding	Word as a vector	A dense, low-dim (typically 100-300) representation learned from context.
Skip-gram	Word2Vec trick	Predict context words from center word. Slower than CBOW, better for rare words.
Negative sampling	Training shortcut	Replace softmax over full vocab with binary classification against k random words.
Static embedding	One vector per word	Same vector regardless of context. Fails on polysemy.
Contextual embedding	Context-sensitive vector	Different vector for each occurrence based on surrounding words. What transformers produce.
OOV	Out of vocabulary	Word not seen in training. Word2Vec cannot produce a vector for these.

Further Reading

- Mikolov et al. (2013). Distributed Representations of Words and Phrases and their Compositionality — the negative-sampling paper. Short and readable.
- Rong, X. (2014). word2vec Parameter Learning Explained — the clearest derivation of the gradients, if the original paper's math feels dense.
- gensim Word2Vec tutorial — production training settings that actually work.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/03-word-embeddings-word2vec>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/03-word-embeddings-word2vec/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/03-word-embeddings-word2vec>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

GloVe, FastText, and Subword Embeddings

Word2Vec trained one embedding per word. GloVe factorized the co-occurrence matrix. FastText embedded the pieces. BPE bridged to transformers.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 03 (Word2Vec from Scratch)
Time: ~45 minutes

The Problem

Word2Vec left two open questions.

First, there was a parallel line of research that factorized the co-occurrence matrix directly (LSA, HAL) rather than doing online skip-gram updates. Was Word2Vec's iterative approach fundamentally better, or was the difference an artifact of how the two methods handled counts? **GloVe** answered that: matrix factorization with a thoughtfully chosen loss matches or beats Word2Vec, and costs less to train.

Second, neither method had a story for words it had never seen. Zoomer-approved, dogecoin, any proper noun coined last week, every inflected form of a rare root. **FastText** fixed this by embedding character n-grams: a word is the sum of its parts, including morphemes, so even out-of-vocabulary words get a sensible vector.

Third, once transformers arrived, the question shifted again. Word-level vocabularies cap out around a million entries; real language is more open than that. **Byte-pair encoding (BPE)** and its relatives solved this by learning a vocabulary of frequent subword units that covers everything. Every modern tokenizer for every modern LLM is a subword tokenizer.

This lesson walks all three, then explains which to reach for when.

The Concept

GloVe (Global Vectors). Build the word-word co-occurrence matrix X where $X[i][j]$ is how often word j appears in the context of word i . Train vectors such that $v_i \cdot v_j + b_i + b_j \approx \log(X[i][j])$. Weight the loss so frequent pairs do not dominate. Done.

FastText. A word is the sum of its character n-grams plus the word itself. `where` becomes `<wh, whe, her, ere, re>, <where>`. The word vector is the sum of those component vectors. Train as Word2Vec. Benefit: unseen words (`whereupon`) compose from known n-grams.

BPE (Byte-Pair Encoding). Start with a vocabulary of individual bytes (or characters). Count every adjacent pair in the corpus. Merge the most frequent pair into a new token. Repeat for k iterations. Result: a vocabulary of $k + 256$ tokens where frequent sequences (in `g, tion, the`) are single tokens and rare words are broken into familiar pieces. Every sentence tokenizes into something.

Interactive figure: n5-subword-merge. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/04-glove-fasttext-subword>

Build It

GloVe: factorize the co-occurrence matrix

```
import numpy as np
from collections import Counter

def build_cooccurrence(docs, window=5):
    pair_counts = Counter()
    vocab = {}
    for doc in docs:
        for token in doc:
            if token not in vocab:
                vocab[token] = len(vocab)
    for doc in docs:
        indexed = [vocab[t] for t in doc]
        for i, center in enumerate(indexed):
            for j in range(max(0, i - window), min(len(indexed), i + window + 1)):
                if i != j:
                    distance = abs(i - j)
                    pair_counts[(center, indexed[j])] += 1.0 / distance
    return vocab, pair_counts

def glove_train(vocab, pair_counts, dim=16, epochs=100, lr=0.05, x_max=100,
    alpha=0.75, seed=0):
    n = len(vocab)
    rng = np.random.default_rng(seed)
    W = rng.normal(0, 0.1, size=(n, dim))
    W_tilde = rng.normal(0, 0.1, size=(n, dim))
    b = np.zeros(n)
    b_tilde = np.zeros(n)

    for epoch in range(epochs):
        for (i, j), x_ij in pair_counts.items():
            weight = (x_ij / x_max) ** alpha if x_ij < x_max else 1.0
            diff = W[i] @ W_tilde[j] + b[i] + b_tilde[j] - np.log(x_ij)
            coef = weight * diff

            grad_W_i = coef * W_tilde[j]
            grad_W_tilde_j = coef * W[i]
            W[i] -= lr * grad_W_i
            W_tilde[j] -= lr * grad_W_tilde_j
            b[i] -= lr * coef
            b_tilde[j] -= lr * coef

    return W + W_tilde
```

Two moving pieces worth naming. The weighting function $f(x) = (x/x_{\max})^{\alpha}$ down-weights very frequent pairs (like (the, and)) so they do not dominate the loss. The final embedding is the sum of W (center) and W_{tilde} (context) tables. Summing both is a published trick that tends to outperform using just one.

FastText: subword-aware embeddings

```
def char_ngrams(word, n_min=3, n_max=6):
    wrapped = f"<{word}>"
    grams = {wrapped}
    for n in range(n_min, n_max + 1):
        for i in range(len(wrapped) - n + 1):
            grams.add(wrapped[i:i + n])
    return grams

>>> char_ngrams("where")
{'<where>', '<wh>', 'whe', 'her', 'ere', 're>', '<whe>', 'wher', 'here', 'ere>',
 → '<wher>', 'where', 'here>'}
```

Each word is represented by its set of n-grams (typically 3 to 6 characters). The word embedding is the sum of its n-gram embeddings. For skip-gram training, plug this in where Word2Vec used a single vector.

```
def fasttext_vector(word, ngram_table):
    grams = char_ngrams(word)
    vecs = [ngram_table[g] for g in grams if g in ngram_table]
    if not vecs:
        return None
    return np.sum(vecs, axis=0)
```

For an unseen word, you still get a vector as long as some of its n-grams are known. whereupon shares <wh, her, ere, and <where with where, so the two land near each other.

BPE: learned subword vocabulary

```
def learn_bpe(corpus, k_merges):
    vocab = Counter()
    for word, freq in corpus.items():
        tokens = tuple(word) + ("</w>")
        vocab[tokens] = freq

    merges = []
    for _ in range(k_merges):
        pair_freq = Counter()
        for tokens, freq in vocab.items():
            for a, b in zip(tokens, tokens[1:]):
                pair_freq[(a, b)] += freq
        if not pair_freq:
            break
        best = pair_freq.most_common(1)[0][0]
        merges.append(best)

    new_vocab = Counter()
    for tokens, freq in vocab.items():
        new_tokens = []
```

```

    i = 0
    while i < len(tokens):
        if i + 1 < len(tokens) and (tokens[i], tokens[i + 1]) == best:
            new_tokens.append(tokens[i] + tokens[i + 1])
            i += 2
        else:
            new_tokens.append(tokens[i])
            i += 1
    new_vocab[tuple(new_tokens)] = freq
    vocab = new_vocab
    return merges

```

```

def apply_bpe(word, merges):
    tokens = list(word) + ["</w>"]
    for a, b in merges:
        new_tokens = []
        i = 0
        while i < len(tokens):
            if i + 1 < len(tokens) and tokens[i] == a and tokens[i + 1] == b:
                new_tokens.append(a + b)
                i += 2
            else:
                new_tokens.append(tokens[i])
                i += 1
        tokens = new_tokens
    return tokens

```

```

>>> corpus = Counter({"low": 5, "lower": 2, "newest": 6, "widest": 3})
>>> merges = learn_bpe(corpus, k_merges=10)
>>> apply_bpe("lowest", merges)
['low', 'est</w>']

```

First iteration merges the most common adjacent pair. After enough iterations, frequent substrings (low, est, tion) become single tokens and rare words break cleanly.

The real GPT / BERT / T5 tokenizers learn 30k-100k merges. Result: any text tokenizes into a bounded-length sequence of known IDs, no OOV ever.

Use It

In practice, you rarely train any of these yourself. You load pre-trained checkpoints.

```

import fasttext.util
fasttext.util.download_model("en", if_exists="ignore")
ft = fasttext.load_model("cc.en.300.bin")
print(ft.get_word_vector("whereupon").shape)
print(ft.get_word_vector("zoomerapproved").shape)

```

For BPE-style subword tokenization in the transformer era:

```

from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
print(tok.tokenize("unbelievably tokenized"))

```

```
['un', 'bel', 'iev', 'ably', 'Ġtoken', 'ized']
```

The Ġ prefix marks word boundaries (a GPT-2 convention). Every modern tokenizer is a BPE variant, WordPiece (BERT), or SentencePiece (T5, LLaMA).

When to pick which

Situation	Pick
Pretrained general-purpose word vectors, no OOV tolerance needed	GloVe 300d
Pretrained general-purpose word vectors, must handle misspellings / neologisms / morphologically rich languages	FastText
Anything going into a transformer (training or inference)	Whatever tokenizer the model shipped with. Never swap.
Training your own language model from scratch	Train a BPE or SentencePiece tokenizer on your corpus first
Production text classification with a linear model	Still TF-IDF. Lesson 02.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/04-glove-fasttext-subword>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/04-glove-fasttext-subword/code>

1. **Easy.** Run `char_ngrams("playing")` and `char_ngrams("played")`. Compute the Jaccard overlap of the two n-gram sets. You should see substantial shared pieces (pla, lay, play), which is why FastText transfers well across morphological variants.
2. **Medium.** Extend `learn_bpe` to track vocabulary growth. Plot tokens-per-corpus-character as a function of number of merges. You should see rapid compression at first, asymptoting near ~2-3 chars per token.
3. **Hard.** Train a 1k-merge BPE on Shakespeare's complete works. Compare tokenization of common words vs. rare proper nouns. Measure average tokens per word before and after. Write up what surprised you.

Key Terms

Term	What people say	What it actually means
Co-occurrence matrix	Word-word frequency table	$X[i][j]$ = how often word j appears in a window around word i .
Subword	Piece of a word	A character n-gram (FastText) or learned token (BPE/WordPiece/SentencePiece).

Term	What people say	What it actually means
BPE	Byte-pair encoding	Iterative merging of most-frequent adjacent pairs until vocabulary hits target size.
OOV	Out of vocabulary	Word the model has never seen. Word2Vec/GloVe fail. FastText and BPE handle it.
Byte-level BPE	BPE on raw bytes	GPT-2's scheme. Vocabulary starts with 256 bytes, so nothing is ever OOV.

Further Reading

- Pennington, Socher, Manning (2014). GloVe: Global Vectors for Word Representation — the GloVe paper, seven pages, still the best derivation of the loss.
- Bojanowski et al. (2017). Enriching Word Vectors with Subword Information — FastText.
- Sennrich, Haddow, Birch (2016). Neural Machine Translation of Rare Words with Subword Units — the paper that introduced BPE to modern NLP.
- Hugging Face tokenizer summary — how BPE, WordPiece, and SentencePiece actually differ in practice.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/04-glove-fasttext-subword>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/04-glove-fasttext-subword/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/04-glove-fasttext-subword>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Sentiment Analysis

The canonical NLP task. Most of what you need to know about classical text classification shows up here.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 02 (BoW + TF-IDF), Phase 2 · 14 (Naive Bayes) **Time:** ~75 minutes

The Problem

“The food was not great.” Positive or negative?

Sentiment sounds simple. A reviewer said they liked or did not like something. Label the sentence. The reason it became the canonical NLP task is that every easy-looking case hides a hard one. Negation flips meaning. Sarcasm inverts it. “Not bad at all” is positive despite two negative-coded words. Emojis carry more signal than surrounding text. Domain vocabulary matters (tight in music review versus tight in fashion review).

Sentiment is a working lab for classical NLP. If you understand why every naive baseline has a specific failure mode, you understand why every richer model was invented. This lesson builds a Naive Bayes baseline from scratch, adds logistic regression, and names the traps that make production sentiment a compliance-grade problem.

The Concept

Classical sentiment is a two-step recipe.

1. **Represent.** Turn the text into a feature vector. BoW, TF-IDF, or n-grams.
2. **Classify.** Fit a linear model (Naive Bayes, logistic regression, SVM) on labeled examples.

Naive Bayes is the dumbest model that works. Assume every feature is independent given the label. Estimate $P(\text{word} \mid \text{positive})$ and $P(\text{word} \mid \text{negative})$ from counts. At inference, multiply the probabilities. The “naive” independence assumption is laughably wrong and yet the results are shockingly strong. The reason: with sparse text features and moderate data, the classifier cares about which side each word leans toward more than how much.

Logistic regression fixes the independence assumption. It learns a weight per feature, including negative weights. not good as a bigram feature gets a negative weight. Naive Bayes cannot do that for bigrams it has never labeled.

Interactive figure: sentiment-logits. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/05-sentiment-analysis>

Build It

Step 1: a real mini-dataset

```

POSITIVE = [
    "absolutely loved this movie",
    "beautiful cinematography and a great story",
    "one of the best films of the year",
    "brilliant acting from the lead",
    "heartwarming and funny",
]

NEGATIVE = [
    "boring and far too long",
    "not worth your time",
    "the plot made no sense",
    "terrible acting, awful script",
    "i want my two hours back",
]

```

Small on purpose. Real work uses tens of thousands of examples (IMDb, SST-2, Yelp polarity). The math is identical.

Step 2: multinomial Naive Bayes from scratch

```

import math
from collections import Counter

def train_nb(docs_by_class, vocab, alpha=1.0):
    class_priors = {}
    class_word_probs = {}
    total_docs = sum(len(d) for d in docs_by_class.values())

    for cls, docs in docs_by_class.items():
        class_priors[cls] = len(docs) / total_docs
        counts = Counter()
        for doc in docs:
            for token in doc:
                counts[token] += 1
        total = sum(counts.values()) + alpha * len(vocab)
        class_word_probs[cls] = {
            w: (counts[w] + alpha) / total for w in vocab
        }
    return class_priors, class_word_probs

def predict_nb(doc, class_priors, class_word_probs):
    scores = {}
    for cls in class_priors:
        s = math.log(class_priors[cls])
        for token in doc:
            if token in class_word_probs[cls]:
                s += math.log(class_word_probs[cls][token])

```

```

    scores[cls] = s
    return max(scores, key=scores.get)

```

Additive smoothing ($\alpha=1.0$) is Laplace smoothing. Without it, a word unseen in a class has probability zero and the log explodes. $\alpha=0.01$ is common in practice. $\alpha=1.0$ is the teaching default.

Step 3: logistic regression from scratch

```
import numpy as np
```

```

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -20, 20)))

```

```

def train_lr(X, y, epochs=500, lr=0.05, l2=0.01):
    n_features = X.shape[1]
    w = np.zeros(n_features)
    b = 0.0
    for _ in range(epochs):
        logits = X @ w + b
        preds = sigmoid(logits)
        err = preds - y
        grad_w = X.T @ err / len(y) + l2 * w
        grad_b = err.mean()
        w -= lr * grad_w
        b -= lr * grad_b
    return w, b

```

```

def predict_lr(X, w, b):
    return (sigmoid(X @ w + b) >= 0.5).astype(int)

```

L2 regularization matters here. Text features are sparse; without L2 the model memorizes training examples. Start at 0.01 and tune.

Step 4: handling negation (the failure mode)

Consider “not good” and “not bad”. A BoW classifier sees {not, good} and {not, bad} and learns from whichever showed up more in training. A bigram classifier sees not_good and not_bad and learns them as distinct features. That is usually enough.

A cruder fix that works when you do not have bigrams: **negation scoping**. Prefix tokens following a negation word with NOT_ up to the next punctuation.

```

NEGATION_WORDS = {"not", "no", "never", "nor", "none", "nothing", "neither"}
NEGATION_TERMINATORS = {".", "!", "?", ",", ";"}

```

```

def apply_negation(tokens):
    out = []
    negate = False
    for token in tokens:
        if token in NEGATION_TERMINATORS:

```

```

        negate = False
        out.append(token)
        continue
    if token in NEGATION_WORDS:
        negate = True
        out.append(token)
        continue
    out.append(f"NOT_{token}" if negate else token)
return out

>>> apply_negation(["not", "good", "at", "all", ".", "but", "funny"])
['not', 'NOT_good', 'NOT_at', 'NOT_all', '.', 'but', 'funny']

```

Now good and NOT_good are different features. The classifier can weight them opposite. Three lines of preprocessing, measurable accuracy jump on sentiment benchmarks.

Step 5: evaluation metrics that matter

Accuracy alone is misleading if classes are imbalanced. Real sentiment corpora are usually 70-80% positive or 70-80% negative; a constant-majority classifier gets 80% accuracy and is worthless. Report every one of the following:

- **Per-class precision and recall.** One pair per class. Macro-average them to get a single number that respects class balance.
- **Macro-F1 (primary metric for imbalanced data).** Mean of per-class F1 scores, equally weighted. Use this instead of accuracy when classes are imbalanced.
- **Weighted-F1 (alternative).** Same as macro but weighted by class frequency. Report alongside macro-F1 when the imbalance itself has business meaning.
- **Confusion matrix.** Raw counts. Always inspect before trusting any scalar metric; it reveals which pair of classes the model confuses.
- **Per-class error samples.** Pull 5 wrong predictions per class. Read them. Nothing replaces reading the actual errors.

For severely imbalanced data (> 95-5 ratio), report **AUROC** and **AUPRC** instead of accuracy. AUPRC is more sensitive to the minority class, which is what you usually care about (spam, fraud, rare sentiment).

Common bug to avoid. Reporting micro-F1 instead of macro-F1 on imbalanced data gives a number that looks high because it is dominated by the majority class. Macro-F1 forces you to see the minority-class performance.

```

def evaluate(y_true, y_pred):
    tp = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 1)
    fp = sum(1 for t, p in zip(y_true, y_pred) if t == 0 and p == 1)
    fn = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 0)
    tn = sum(1 for t, p in zip(y_true, y_pred) if t == 0 and p == 0)
    precision = tp / (tp + fp) if tp + fp else 0
    recall = tp / (tp + fn) if tp + fn else 0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0
    return {"tp": tp, "fp": fp, "tn": tn, "fn": fn, "precision": precision,
            "recall": recall, "f1": f1}

```

Use It

scikit-learn does it in six lines, correctly.

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True,
    ↪ stop_words=None)),
    ("clf", LogisticRegression(C=1.0, max_iter=1000)),
])
pipe.fit(X_train, y_train)
print(pipe.score(X_test, y_test))

```

Three things to notice. `stop_words=None` keeps negations. `ngram_range=(1, 2)` adds bigrams so `not_good` becomes a feature. `sublinear_tf=True` dampens repeated words. These three flags are the difference between a 75%-accurate baseline and an 85%-accurate baseline on SST-2.

When to reach for a transformer

- Sarcasm detection. Classical models fail here. Period.
- Long reviews where sentiment shifts mid-document.
- Aspect-based sentiment. “Camera was great but battery was terrible.” You need to attribute sentiment to aspects. Transformers or structured output models only.
- Non-English, low-resource languages. Multilingual BERT gives you a zero-shot baseline for free.

If you need any of the above, skip ahead to phase 7 (transformers deep dive). Otherwise, Naive Bayes or logistic regression on TF-IDF plus bigrams plus negation handling is your 2026 production baseline.

The reproducibility trap (again)

Retraining sentiment models is routine. Re-evaluating them is not. Accuracy numbers reported in papers use specific splits, specific preprocessing, specific tokenizers. If you compare your new model to a baseline without using the identical pipeline, you will get misleading deltas. Always regenerate the baseline on your pipeline, not the paper’s number.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/05-sentiment-analysis>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/05-sentiment-analysis/code>

1. **Easy.** Add `apply_negation` as a preprocessing step in the scikit-learn pipeline and measure the F1 delta on a small sentiment dataset.
2. **Medium.** Implement class-weighted logistic regression (pass `class_weight="balanced"` to scikit-learn, or derive the gradient yourself). Measure the effect on a synthetic 90-10 class imbalance.

3. **Hard.** Build a sarcasm detector by training a second classifier on the residuals of the sentiment model. Document your experimental setup. Warn the reader when your accuracy is below chance (chance-level on 2-class sarcasm is ~50%, and most first attempts land there).

Key Terms

Term	What people say	What it actually means
Polarity	Positive or negative	Binary label; sometimes extended to neutral or fine-grained (5-star).
Aspect-based sentiment	Per-aspect polarity	Attribute sentiment to specific entities or attributes mentioned in text.
Negation scoping	Reversing nearby tokens	Prefix tokens after “not” with NOT_ until punctuation.
Laplace smoothing	Adding 1 to counts	Prevents zero-probability features in Naive Bayes.
L2 regularization	Shrinking weights	Adds $\lambda * \sum(w^2)$ to loss. Essential for sparse text features.

Further Reading

- Pang and Lee (2008). Opinion Mining and Sentiment Analysis — the foundational survey. Long, but the first four sections cover everything classical.
- Wang and Manning (2012). Baselines and Bigrams: Simple, Good Sentiment and Topic Classification — the paper that showed bigrams + Naive Bayes is hard to beat on short text.
- scikit-learn text feature extraction docs — reference for CountVectorizer, TfidfVectorizer, and every knob you’ll tune.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/05-sentiment-analysis>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/05-sentiment-analysis/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/05-sentiment-analysis>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Named Entity Recognition

Pull the names out. Sounds easy until you deal with ambiguous boundaries, nested entities, and domain jargon.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 02 (BoW + TF-IDF), Phase 5 · 03 (Word Embeddings) **Time:** ~75 minutes

The Problem

“Apple sued Google over its iPhone search deal in the US.” Five entities: Apple (ORG), Google (ORG), iPhone (PRODUCT), search deal (maybe), US (GPE). A good NER system extracts all of them with correct types. A bad one misses iPhone, confuses Apple the fruit with Apple the company, and labels “US” as a PERSON.

NER is the workhorse underneath every structured extraction pipeline. Resume parsing, compliance log scanning, medical record anonymization, search query understanding, grounding for chatbot responses, legal contract extraction. You never quite see it; you always depend on it.

This lesson walks the classical path (rule-based, HMM, CRF) into the modern one (BiLSTM-CRF, then transformers). Each step solves a specific limitation of the one before it. The pattern is the lesson.

The Concept

BIO tagging (or BILOU) turns entity extraction into a sequence-labeling problem. Label each token with B-TYPE (beginning of entity), I-TYPE (inside entity), or O (outside any entity).

Apple	B-ORG
sued	O
Google	B-ORG
over	O
its	O
iPhone	B-PRODUCT
search	O
deal	O
in	O
the	O
US	B-GPE
.	O

Multi-token entities chain: New B-GPE, York I-GPE, City I-GPE. A model that understands BIO can extract arbitrary spans.

The architecture progression:

- **Rule-based.** Regex + gazetteer lookups. High precision on known entities, zero coverage on new ones.

- **HMM.** Hidden Markov Model. Emission probability of token given tag, transition probability of tag-to-tag. Viterbi decode. Trained on labeled data.
- **CRF.** Conditional Random Field. Like HMM but discriminative, so you can mix arbitrary features (word shape, capitalization, neighboring words). Still the classical production workhorse in 2026 for low-resource deployments.
- **BiLSTM-CRF.** Neural features instead of hand-crafted. LSTM reads the sentence both directions, CRF layer on top enforces consistent tag sequences.
- **Transformer-based.** Fine-tune BERT with a token-classification head. Best accuracy. Most compute.

Interactive figure: ner-bio-tagging. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/06-named-entity-recognition>

Build It

Step 1: BIO tagging helpers

```
def spans_to_bio(tokens, spans):
    labels = ["0"] * len(tokens)
    for start, end, label in spans:
        labels[start] = f"B-{label}"
        for i in range(start + 1, end):
            labels[i] = f"I-{label}"
    return labels

def bio_to_spans(tokens, labels):
    spans = []
    current = None
    for i, label in enumerate(labels):
        if label.startswith("B-"):
            if current:
                spans.append(current)
            current = (i, i + 1, label[2:])
        elif label.startswith("I-") and current and current[2] == label[2:]:
            current = (current[0], i + 1, current[2])
        else:
            if current:
                spans.append(current)
            current = None
    if current:
        spans.append(current)
    return spans

>>> tokens = ["Apple", "sued", "Google", "over", "iPhone", "sales", "."]
>>> labels = ["B-ORG", "0", "B-ORG", "0", "B-PRODUCT", "0", "0"]
>>> bio_to_spans(tokens, labels)
[(0, 1, 'ORG'), (2, 3, 'ORG'), (4, 5, 'PRODUCT')]
```

Step 2: hand-crafted features

For classical (non-neural) NER, features are the game. Useful ones:

```
def token_features(token, prev_token, next_token):
    return {
        "lower": token.lower(),
        "is_upper": token.isupper(),
        "is_title": token.istitle(),
        "has_digit": any(c.isdigit() for c in token),
        "suffix_3": token[-3:].lower(),
        "shape": word_shape(token),
        "prev_lower": prev_token.lower() if prev_token else "<BOS>",
        "next_lower": next_token.lower() if next_token else "<EOS>",
    }
```

```
def word_shape(word):
    out = []
    for c in word:
        if c.isupper():
            out.append("X")
        elif c.islower():
            out.append("x")
        elif c.isdigit():
            out.append("d")
        else:
            out.append(c)
    return "".join(out)
```

word_shape("iPhone") returns xXxxxx. word_shape("USA-2024") returns XXX-dddd. Capitalization patterns are high-signal for proper nouns.

Step 3: a simple rule-based + dictionary baseline

```
ORG_GAZETTEER = {"Apple", "Google", "Microsoft", "OpenAI", "Meta", "Amazon",
    ↪ "Netflix"}
GPE_GAZETTEER = {"US", "USA", "UK", "India", "Germany", "France"}
PRODUCT_GAZETTEER = {"iPhone", "Android", "Windows", "ChatGPT", "Claude"}
```

```
def rule_based_ner(tokens):
    labels = []
    for token in tokens:
        if token in ORG_GAZETTEER:
            labels.append("B-ORG")
        elif token in GPE_GAZETTEER:
            labels.append("B-GPE")
        elif token in PRODUCT_GAZETTEER:
            labels.append("B-PRODUCT")
        else:
            labels.append("O")
    return labels
```

Production gazetteers have millions of entries scraped from Wikipedia and DBpedia. Coverage is good. Disambiguation (Apple the company vs the fruit) is terrible. That is why statistical models won.

Step 4: the CRF step (sketch, not full impl)

Full CRF from scratch in 50 lines is not enlightening without the probability-theory foundations. Use `sklearn-crfsuite` instead:

```
import sklearn_crfsuite

def to_features(tokens):
    out = []
    for i, tok in enumerate(tokens):
        prev = tokens[i - 1] if i > 0 else ""
        nxt = tokens[i + 1] if i + 1 < len(tokens) else ""
        out.append({
            "word.lower()": tok.lower(),
            "word.isupper()": tok.isupper(),
            "word.istitle()": tok.istitle(),
            "word.isdigit()": tok.isdigit(),
            "word.suffix3": tok[-3:].lower(),
            "word.shape": word_shape(tok),
            "prev.word.lower()": prev.lower(),
            "next.word.lower()": nxt.lower(),
            "BOS": i == 0,
            "EOS": i == len(tokens) - 1,
        })
    return out
```

```
crf = sklearn_crfsuite.CRF(algorithm="lbfgs", c1=0.1, c2=0.1, max_iterations=100,
    ↪ all_possible_transitions=True)
X_train = [to_features(s) for s in sentences_tokenized]
crf.fit(X_train, bio_labels_train)
```

`c1` and `c2` are L1 and L2 regularization. `all_possible_transitions=True` lets the model learn illegal sequences (e.g., I-ORG after O) are unlikely, which is how a CRF enforces BIO consistency without you writing the constraint.

Step 5: what a BiLSTM-CRF adds

Features become learned. Inputs: token embeddings (GloVe or fastText). LSTM reads left-to-right and right-to-left. Concatenated hidden states go through a CRF output layer. The CRF still enforces tag-sequence consistency; the LSTM replaces hand-crafted features with learned ones.

```
import torch
import torch.nn as nn

class BiLSTM_CRF_Head(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_labels):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, bidirectional=True,
            ↪ batch_first=True)
        self.fc = nn.Linear(hidden_dim * 2, n_labels)

    def forward(self, token_ids):
```

```

e = self.embed(token_ids)
h, _ = self.lstm(e)
emissions = self.fc(h)
return emissions

```

For the CRF layer, use `torchcrf.CRF` (pip install `pytorch-crf`). The gain over hand-crafted CRF is measurable but smaller than you expect unless you have tens of thousands of labeled sentences.

Use It

spaCy ships production-grade NER out of the box.

```
import spacy
```

```

nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple sued Google over its iPhone search deal in the US.")
for ent in doc.ents:
    print(f"{ent.text:20s} {ent.label_}")

```

Apple	ORG
Google	ORG
iPhone	ORG
US	GPE

Notice iPhone labeled ORG rather than PRODUCT — spaCy's small model has weak product-entity coverage. The large model (`en_core_web_lg`) does better. The transformer model (`en_core_web_trf`) does better still.

Hugging Face for BERT-based NER:

```
from transformers import pipeline
```

```

ner = pipeline("ner", model="dslim/bert-base-NER", aggregation_strategy="simple")
print(ner("Apple sued Google over its iPhone in the US."))

[{'entity_group': 'ORG', 'word': 'Apple', ...},
 {'entity_group': 'ORG', 'word': 'Google', ...},
 {'entity_group': 'MISC', 'word': 'iPhone', ...},
 {'entity_group': 'LOC', 'word': 'US', ...}]

```

`aggregation_strategy="simple"` merges contiguous B-X, I-X tokens into a span. Without it, you get token-level labels and have to merge yourself.

LLM-based NER (the 2026 option)

Zero-shot and few-shot LLM NER is now competitive with fine-tuned models on many domains, and dramatically better when labeled data is scarce.

- **Zero-shot prompting.** Give the LLM a list of entity types and an example schema. Ask for JSON output. Works out of the box; accuracy is moderate on novel domains.
- **ZeroTuneBio-style prompting.** Decompose the task into candidate extraction → meaning explanation → judgment → re-check. A multi-stage prompt (not one-shot) lifts accuracy substantially on biomedical NER. The same pattern works for legal, financial, and scientific domains.

- **Dynamic prompting with RAG.** Retrieve the most similar labeled examples from a small annotated seed set for every inference call; build the few-shot prompt on the fly. In 2026 benchmarks, this lifts GPT-4 biomedical NER F1 by 11-12% over static prompting.
- **Per-entity-type decomposition.** For long documents, a single call that extracts all entity types at once loses recall as length grows. Run one extraction pass per entity type. Higher inference cost, substantially higher accuracy. This is the standard pattern for clinical notes and legal contracts.

Production recommendation as of 2026: start with an LLM zero-shot baseline before you collect training data. Often the F1 is good enough that you never need to fine-tune.

Where classical NER still wins

Even with LLMs available, classical NER wins when:

- Latency budget is under 50ms.
- You have thousands of labeled examples and need 98%+ F1.
- The domain has a stable ontology where a pretrained CRF or BiLSTM transfers well.
- Regulatory constraints require an on-prem, non-generative model.

Where it falls apart

- **Domain shift.** CoNLL-trained NER on legal contracts performs worse than a gazetteer. Fine-tune on your domain.
- **Nested entities.** “Bank of America Tower” is simultaneously an ORG and a FACILITY. Standard BIO cannot represent overlapping spans. You need nested NER (multi-pass or span-based models).
- **Long entities.** “United States Federal Deposit Insurance Corporation.” Token-level models sometimes split this. Use `aggregation_strategy` or post-process.
- **Sparse types.** Medical NER labels like DRUG_BRAND, ADVERSE_EVENT, DOSE. General-purpose models have no idea. Scispacy and BioBERT are the starting points there.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/06-named-entity-recognition>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/06-named-entity-recognition/code>

1. **Easy.** Implement `bio_to_spans` (the inverse of `spans_to_bio`) and verify round-trip consistency on 10 sentences.
2. **Medium.** Train the `sklearn-crfsuite` CRF above on the CoNLL-2003 English NER dataset. Report per-entity F1 using `segeval`. Typical result: ~84 F1.
3. **Hard.** Fine-tune `distilbert-base-cased` on a domain-specific NER dataset (medical, legal, or financial). Compare against the `spaCy` small model. Document data leakage checks and write up what surprised you.

Key Terms

Term	What people say	What it actually means
NER	Extract names	Label token spans with types (PERSON, ORG, GPE, DATE, ...).
BIO	Tagging scheme	B-X begins, I-X continues, O outside.
BILOU	Better BIO	Adds L-X (last), U-X (unit) for cleaner boundaries.
CRF	Structured classifier	Models transitions between labels, not just emissions. Enforces valid sequences.
Nested NER	Overlapping entities	One span is a different entity than a sub-span of it. BIO cannot express this.
Entity-level F1	Proper NER metric	Predicted span must match true span exactly. Token-level F1 overstates accuracy.

Further Reading

- Lample et al. (2016). Neural Architectures for Named Entity Recognition — the BiLSTM-CRF paper. Canonical.
- Devlin et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers — introduces the token-classification pattern that became standard.
- spaCy linguistic features — named entities — practical reference for every attribute on `Doc.ents` and `Span`.
- `segeval` — the correct metric library. Use it always.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/06-named-entity-recognition>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/06-named-entity-recognition/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/06-named-entity-recognition>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

POS Tagging and Syntactic Parsing

Grammar was unfashionable for a while. Then every LLM pipeline needed to validate structured extraction, and it came back.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 01 (Text Processing), Phase 2 · 14 (Naive Bayes) **Time:** ~45 minutes

The Problem

Lesson 01 promised that lemmatization needs a part-of-speech tag. Without knowing running is a verb, a lemmatizer cannot reduce it to run. Without knowing better is an adjective, it cannot reduce to good.

That promise hid a whole subfield. Part-of-speech tagging assigns grammatical categories. Syntactic parsing recovers the sentence's tree structure: which word modifies which, which verb governs which arguments. Classical NLP spent twenty years refining both. Then deep learning collapsed them into a token-classification task on top of a pretrained transformer, and the research community moved on.

Not the applied community. Every structured-extraction pipeline still uses POS and dependency trees under the hood. LLM-generated JSON gets validated against grammatical constraints. Question-answering systems decompose queries using dependency parses. Machine translation quality evaluators check alignment of parse trees.

Worth knowing. This lesson introduces the tagsets, the baselines, and the point where you stop implementing from scratch and call spaCy.

The Concept

POS tagging labels each token with a grammatical category. The **Penn Treebank (PTB)** tagset is the English default. 36 tags with distinctions the casual reader finds fussy: NN singular noun, NNS plural noun, NNP proper noun singular, VBD verb past tense, VBZ verb 3rd person singular present, and so on. The **Universal Dependencies (UD)** tagset is coarser (17 tags) and language-agnostic; it became the default for cross-lingual work.

The/DET cats/NOUN were/AUX running/VERB at/ADP 3pm/NOUN ./PUNCT

Syntactic parsing produces a tree. Two major styles:

- **Constituency parsing.** Noun phrases, verb phrases, prepositional phrases nest inside each other. Output is a tree of non-terminal categories (NP, VP, PP) with words as leaves.
- **Dependency parsing.** Each word has a single head word it depends on, labeled with a grammatical relation. Output is a tree where every edge is a (head, dependent, relation) triple.

Dependency parsing won in the 2010s because it generalizes cleanly across languages, especially free-word-order ones.

```

running is ROOT
cats is nsubj of running
were is aux of running
at is prep of running
3pm is pobj of at

```

Interactive figure: pos-tagger. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/07-pos-tagging-parsing>

Interactive figure: dependency-arcs. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/07-pos-tagging-parsing>

Build It

Step 1: most-frequent-tag baseline

The dumbest POS tagger that works. For each word, predict the tag it had most often in training.

```

from collections import Counter, defaultdict

def train_mft(train_examples):
    word_tag_counts = defaultdict(Counter)
    all_tags = Counter()
    for tokens, tags in train_examples:
        for token, tag in zip(tokens, tags):
            word_tag_counts[token.lower()][tag] += 1
            all_tags[tag] += 1
    word_best = {w: c.most_common(1)[0][0] for w, c in word_tag_counts.items()}
    default_tag = all_tags.most_common(1)[0][0]
    return word_best, default_tag

def predict_mft(tokens, word_best, default_tag):
    return [word_best.get(t.lower(), default_tag) for t in tokens]

```

On the Brown corpus, this baseline hits ~85% accuracy. Not good, but the floor below which no serious model should fall.

Step 2: bigram HMM tagger

Model the joint probability of the sequence:

$$P(\text{tags}, \text{words}) = \prod P(\text{tag}_i \mid \text{tag}_{\{i-1\}}) * P(\text{word}_i \mid \text{tag}_i)$$

Two tables: transition probabilities (tag given previous tag), emission probabilities (word given tag). Estimate both from counts with Laplace smoothing. Decode with Viterbi (dynamic programming over the tag lattice).

```

import math

def train_hmm(train_examples, alpha=0.01):

```

```

transitions = defaultdict(Counter)
emissions = defaultdict(Counter)
tags = set()
vocab = set()

for tokens, ts in train_examples:
    prev = "<BOS>"
    for token, tag in zip(tokens, ts):
        transitions[prev][tag] += 1
        emissions[tag][token.lower()] += 1
        tags.add(tag)
        vocab.add(token.lower())
        prev = tag
    transitions[prev]["<EOS>"] += 1

return transitions, emissions, tags, vocab

def log_prob(table, given, key, smooth_denom, alpha):
    return math.log((table[given].get(key, 0) + alpha) / smooth_denom)

def viterbi(tokens, transitions, emissions, tags, vocab, alpha=0.01):
    tags_list = list(tags)
    n = len(tokens)
    V = [[0.0] * len(tags_list) for _ in range(n)]
    back = [[0] * len(tags_list) for _ in range(n)]

    for j, tag in enumerate(tags_list):
        em_denom = sum(emissions[tag].values()) + alpha * (len(vocab) + 1)
        tr_denom = sum(transitions["<BOS>"].values()) + alpha * (len(tags_list) + 1)
        tr = log_prob(transitions, "<BOS>", tag, tr_denom, alpha)
        em = log_prob(emissions, tag, tokens[0].lower(), em_denom, alpha)
        V[0][j] = tr + em
        back[0][j] = 0

    for i in range(1, n):
        for j, tag in enumerate(tags_list):
            em_denom = sum(emissions[tag].values()) + alpha * (len(vocab) + 1)
            em = log_prob(emissions, tag, tokens[i].lower(), em_denom, alpha)
            best_prev = 0
            best_score = -1e30
            for k, prev_tag in enumerate(tags_list):
                tr_denom = sum(transitions[prev_tag].values()) + alpha *
→ (len(tags_list) + 1)
                tr = log_prob(transitions, prev_tag, tag, tr_denom, alpha)
                score = V[i - 1][k] + tr + em
                if score > best_score:
                    best_score = score
                    best_prev = k
            V[i][j] = best_score
            back[i][j] = best_prev

    last_best = max(range(len(tags_list)), key=lambda j: V[n - 1][j])

```

```

path = [last_best]
for i in range(n - 1, 0, -1):
    path.append(back[i][path[-1]])
return [tags_list[j] for j in reversed(path)]

```

Bigram HMM on Brown hits ~93% accuracy. The jump from 85% to 93% is mostly transition probabilities — the model learns DET NOUN is common and NOUN DET is rare.

Step 3: why modern taggers beat this

Transition + emission probabilities are local. They cannot capture that saw is a noun in “I bought a saw” but a verb in “I saw the movie.” A CRF with arbitrary features (suffix, word shape, word before and after, word itself) hits ~97%. A BiLSTM-CRF or transformer hits ~98%+.

The ceiling on this task is set by annotator disagreement. Human annotators agree about 97% of the time on Penn Treebank. Models past 98% are probably overfitting the test set.

Step 4: dependency parsing sketch

Full dependency parsing from scratch is out of scope; the canonical textbook treatment is in Jurafsky and Martin. Two classical families to know:

- **Transition-based** parsers (arc-eager, arc-standard) act like a shift-reduce parser: they read tokens, shift them onto a stack, and apply reduce actions that create arcs. Greedy decoding is fast. Classic implementation is MaltParser. Modern neural version: Chen and Manning’s transition-based parser.
- **Graph-based** parsers (Eisner’s algorithm, Dozat-Manning biaffine) score every possible head-dependent edge and pick the maximum spanning tree. Slower but more accurate.

For most applied work, call spaCy:

```

import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("The cats were running at 3pm.")
for token in doc:
    print(f"{token.text:10s} tag={token.tag_:5s} pos={token.pos_:6s}
          ↪ dep={token.dep_:10s} head={token.head.text}")

```

The	tag=DT	pos=DET	dep=det	head=cats
cats	tag=NNS	pos=NOUN	dep=nsubj	head=running
were	tag=VBD	pos=AUX	dep=aux	head=running
running	tag=VBG	pos=VERB	dep=R00T	head=running
at	tag=IN	pos=ADP	dep=prep	head=running
3pm	tag=NN	pos=NOUN	dep=pobj	head=at
.	tag=.	pos=PUNCT	dep=punct	head=running

Read the dep column bottom to top and the sentence’s grammatical structure falls out.

Use It

Every production NLP library ships POS and dependency parsers as part of a standard pipeline.

- **spaCy** (en_core_web_sm / md / lg / trf). Fast, accurate, integrated with tokenization + NER + lemmatization. token.tag_ (Penn), token.pos_ (UD), token.dep_ (dependency relation).
- **Stanford NLP (stanza)**. Stanford's successor to CoreNLP. State-of-the-art on 60+ languages.
- **trankit**. Transformer-based, good UD accuracy.
- **NLTK**. pos_tag. Usable, slow, older. Fine for teaching.

Where this still matters in 2026

- **Lemmatization**. Lesson 01 needs POS to lemmatize correctly. Always.
- **Structured extraction from LLM outputs**. Validate that a generated sentence respects grammatical constraints (e.g., subject-verb agreement, required modifiers).
- **Aspect-based sentiment**. Dependency parses tell you which adjective modifies which noun.
- **Query understanding**. “movies directed by Wes Anderson starring Bill Murray” decomposes into structured constraints via the parse.
- **Cross-lingual transfer**. UD tags and dependency relations are language-agnostic, enabling zero-shot structured analysis of new languages.
- **Low-compute pipelines**. If you cannot ship a transformer, POS + dependency parse + gazetteer gets you surprisingly far.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/07-pos-tagging-parsing>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/07-pos-tagging-parsing/code>

1. **Easy**. Using the most-frequent-tag baseline on a small tagged corpus (e.g., NLTK's Brown subset), measure accuracy on held-out sentences. Verify the ~85% result.
2. **Medium**. Train the bigram HMM above and report per-tag precision/recall. Which tags does the HMM confuse most?
3. **Hard**. Use spaCy's dependency parse to extract subject-verb-object triples from a 1000-sentence sample. Evaluate on 50 manually labeled triples. Document where extraction fails (often passives, coordinations, and elided subjects).

Key Terms

Term	What people say	What it actually means
POS tag	Word's type	Grammatical category. PTB has 36; UD has 17.
Penn Treebank	Standard tagset	English-specific. Fine-grained verb tenses and noun number.
Universal Dependencies	Multilingual tagset	Coarser than PTB; language-neutral; defaults for cross-lingual work.

Term	What people say	What it actually means
Dependency parse	Sentence tree	Each word has one head, each edge has a grammatical relation.
Viterbi	Dynamic programming	Finds the highest-probability tag sequence given emissions and transitions.

Further Reading

- Jurafsky and Martin — Speech and Language Processing, chapters 8 and 18 — the canonical textbook treatment of POS and parsing.
- Universal Dependencies project — the cross-lingual tagset and treebank collection used by every multilingual parser.
- spaCy linguistic features guide — practical reference for every attribute exposed on Token.
- Chen and Manning (2014). A Fast and Accurate Dependency Parser using Neural Networks — the paper that brought neural parsers into the mainstream.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/07-pos-tagging-parsing>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/07-pos-tagging-parsing/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/07-pos-tagging-parsing>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

CNNs and RNNs for Text

Convolutions learn n-grams. Recurrences remember. Both are superseded by attention. Both still matter on constrained hardware.

Type: Build **Languages:** Python **Prerequisites:** Phase 3 · 11 (PyTorch Intro), Phase 5 · 03 (Word Embeddings), Phase 4 · 02 (Convolutions from Scratch) **Time:** ~75 minutes

The Problem

TF-IDF and Word2Vec produced flat vectors that ignored word order. A classifier built on them could not tell dog bites man from man bites dog. Word order sometimes carries the signal.

Two families of architectures filled that gap before transformers arrived.

Convolutional nets for text (TextCNN). Apply 1D convolutions over sequences of word embeddings. A filter of width 3 is a learnable trigram detector: it spans three words and outputs a score. Stack different widths (2, 3, 4, 5) to detect multi-scale patterns. Max-pool to a fixed-size representation. Flat, parallel, fast.

Recurrent nets (RNN, LSTM, GRU). Process tokens one at a time, maintaining a hidden state that carries information forward. Sequential, memory-bearing, flexible input lengths. Dominated sequence modeling from 2014 to 2017, then attention happened.

This lesson builds both, then names the failure that motivated attention.

The Concept

TextCNN (Kim, 2014). Tokens get embedded. A width-k 1D convolution slides a filter over consecutive k-grams of embeddings, producing a feature map. Global max-pooling over that map picks the strongest activation. Concatenate max-pooled outputs from several filter widths. Feed to a classifier head.

Why it works. A filter is a learnable n-gram. Max-pooling is position-invariant, so “not good” fires the same feature at the start or middle of a review. Three filter widths with 100 filters each gives you 300 learned n-gram detectors. Training is parallel; no sequential dependency.

RNN. At each time step t , the hidden state $h_t = f(W * x_t + U * h_{t-1} + b)$. Share W , U , b across time. The hidden state at time T is a summary of the entire prefix. For classification, pool across $h_1 \dots h_T$ (max, mean, or last).

Plain RNNs suffer vanishing gradients. The **LSTM** adds gates that decide what to forget, what to store, and what to output, stabilizing gradients through long sequences. The **GRU** simplifies LSTM to two gates; performs similarly with fewer parameters.

Bidirectional RNNs run one RNN forward and another backward, concatenating hidden states. Every token’s representation sees both left and right context. Essential for tagging tasks.

Interactive figure: rnn-unroll. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/08-cnns-rnns-for-text>

Build It

Step 1: TextCNN in PyTorch

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class TextCNN(nn.Module):
    def __init__(self, vocab_size, embed_dim, n_classes, filter_widths=(2, 3, 4),
        ↪ n_filters=64, dropout=0.3):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.convs = nn.ModuleList([
            nn.Conv1d(embed_dim, n_filters, kernel_size=k)
            for k in filter_widths
        ])
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(n_filters * len(filter_widths), n_classes)

    def forward(self, token_ids):
        x = self.embed(token_ids).transpose(1, 2)
        pooled = []
        for conv in self.convs:
            c = F.relu(conv(x))
            p = F.max_pool1d(c, c.size(2)).squeeze(2)
            pooled.append(p)
        h = torch.cat(pooled, dim=1)
        return self.fc(self.dropout(h))
```

The `transpose(1, 2)` reshapes `[batch, seq_len, embed_dim]` to `[batch, embed_dim, seq_len]` because `nn.Conv1d` treats the middle axis as channels. The pooled output is fixed-size regardless of input length.

Step 2: LSTM classifier

```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, n_classes,
        ↪ bidirectional=True, dropout=0.3):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
            ↪ bidirectional=bidirectional)
        factor = 2 if bidirectional else 1
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * factor, n_classes)

    def forward(self, token_ids):
```

```

x = self.embed(token_ids)
out, _ = self.lstm(x)
pooled = out.max(dim=1).values
return self.fc(self.dropout(pooled))

```

Max-pool over the sequence, not last-state pool. For classification, max-pooling usually beats taking the last hidden state because information at the end of a long sequence tends to dominate the last state.

Step 3: the vanishing gradient demo (intuition)

A plain RNN without gating cannot learn long-range dependencies. Consider a toy task: predict whether token A appeared anywhere in a sequence. If A is at position 1 and the sequence is 100 tokens long, the gradient from the loss has to flow back through 99 multiplications of the recurrent weight. If the weight is less than 1, the gradient vanishes. If more than 1, it explodes.

```

def vanishing_gradient_sim(seq_len, recurrent_weight=0.9):
    import math
    return math.pow(recurrent_weight, seq_len)

```

```

# At weight=0.9 over 100 steps:
# 0.9 ^ 100 ≈ 2.7e-5
# The gradient from step 100 to step 1 is effectively zero.

```

LSTMs fix this with a **cell state** that runs through the network with only additive interactions (the forget gate scales it multiplicatively, but gradients still flow along the “highway”). GRUs do something similar with fewer parameters. Both give you stable training through 100+ step sequences.

Step 4: why this still was not enough

Three problems persisted even with LSTMs.

1. **Sequential bottleneck.** Training an RNN on a sequence of length 1000 requires 1000 serial forward/backward steps. Cannot parallelize across time.
2. **Fixed-size context vector in encoder-decoder setups.** The decoder sees only the final hidden state of the encoder, compressed over the entire input. Long inputs lose detail. Lesson 09 covers this directly.
3. **Distant-dependency accuracy ceiling.** LSTMs outperform plain RNNs but still struggle to propagate specific information across 200+ steps.

Attention solved all three. Transformers dropped recurrence entirely. Lesson 10 is the pivot.

Use It

PyTorch’s `nn.LSTM`, `nn.GRU`, and `nn.Conv1d` are production-ready. Training code is standard.

Hugging Face ships pretrained embeddings you plug in as the input layer:

```

from transformers import AutoModel

encoder = AutoModel.from_pretrained("bert-base-uncased")
for param in encoder.parameters():
    param.requires_grad = False

```

```

class BertCNN(nn.Module):
    def __init__(self, n_classes, filter_widths=(2, 3, 4), n_filters=64):
        super().__init__()
        self.encoder = encoder
        self.convs = nn.ModuleList([nn.Conv1d(768, n_filters, kernel_size=k) for k
        ↪ in filter_widths])
        self.fc = nn.Linear(n_filters * len(filter_widths), n_classes)

    def forward(self, input_ids, attention_mask):
        with torch.no_grad():
            out = self.encoder(input_ids=input_ids,
            ↪ attention_mask=attention_mask).last_hidden_state
            x = out.transpose(1, 2)
            pooled = [F.max_pool1d(F.relu(conv(x)),
            ↪ kernel_size=conv(x).size(2)).squeeze(2) for conv in self.convs]
            return self.fc(torch.cat(pooled, dim=1))

```

Use-when-it-fits-the-constraint checklist.

- **Edge / on-device inference.** TextCNN with GloVe embeddings is 10-100x smaller than a transformer. If your deploy target is a phone, this is the stack.
- **Streaming / online classification.** RNN processes one token at a time; transformers need the full sequence. For real-time incoming text, LSTMs still win.
- **Tiny models for baselines.** Fast iteration on a new task. Train a TextCNN in 5 minutes on a CPU.
- **Sequence labeling with limited data.** BiLSTM-CRF (lesson 06) is still a production-grade NER architecture for 1k-10k labeled sentences.

Everything else goes to a transformer.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/08-cnns-rnns-for-text>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/08-cnns-rnns-for-text/code>

1. **Easy.** Train a TextCNN on a 3-class toy dataset (you invent the data). Verify that filter widths (2, 3, 4) outperform a single width (3) on average F1.
2. **Medium.** Implement max-pool, mean-pool, and last-state pooling for the LSTM classifier. Compare on a small dataset; document which pooling wins and hypothesize why.
3. **Hard.** Build a BiLSTM-CRF NER tagger (combine lesson 06 and this one). Train on CoNLL-2003. Compare to the CRF-alone baseline from lesson 06 and to a BERT fine-tune. Report training time, memory, and F1.

Key Terms

Term	What people say	What it actually means
TextCNN	CNN for text	Stack of 1D convolutions over word embeddings with global max-pool. Kim (2014).
RNN	Recurrent net	Hidden state updated at each time step: $h_t = f(W x_t + U h_{t-1})$.
LSTM	Gated RNN	Adds input / forget / output gates + a cell state. Trains stably through long sequences.
GRU	Simpler LSTM	Two gates instead of three. Similar accuracy, fewer parameters.
Bidirectional	Both directions	Forward + backward RNN concatenated. Every token sees both sides of its context.
Vanishing gradient	Training signal dies	Repeated multiplication by <1 weights in plain RNNs makes early-step gradients effectively zero.

Further Reading

- Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification — the TextCNN paper. Eight pages. Readable.
- Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory — the LSTM paper. Unexpectedly lucid.
- Olah, C. (2015). Understanding LSTM Networks — the diagrams that made LSTMs accessible to everyone.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/08-cnns-rnns-for-text>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/08-cnns-rnns-for-text/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/08-cnns-rnns-for-text>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Sequence-to-Sequence Models

Two RNNs pretending to be a translator. The bottleneck they hit is the reason attention exists.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 08 (CNNs + RNNs for Text), Phase 3 · 11 (PyTorch Intro) **Time:** ~75 minutes

The Problem

Classification maps a variable-length sequence to a single label. Translation maps a variable-length sequence to another variable-length sequence. The input and output live in different vocabularies, possibly different languages, with no guarantee of length parity.

The seq2seq architecture (Sutskever, Vinyals, Le, 2014) cracked this with a deliberately simple recipe. Two RNNs. One reads the source sentence and produces a fixed-size context vector. The other reads that vector and generates the target sentence token by token. Same code you wrote for lesson 08, glued together differently.

This is worth studying for two reasons. First, the context-vector bottleneck is the most pedagogically useful failure in NLP. It motivates everything attention and transformers are good at. Second, the training recipe (teacher forcing, scheduled sampling, beam search at inference) still applies to every modern generation system including LLMs.

The Concept

Encoder. An RNN that reads the source sentence. Its final hidden state is the **context vector** — a fixed-size summary of the entire input. Lose nothing but the source, supposedly.

Decoder. Another RNN initialized from the context vector. At each step it takes the previously generated token as input and produces a distribution over the target vocabulary. Sample or argmax to pick the next token. Feed it back in. Repeat until an <EOS> token is produced or max length is hit.

Training: Cross-entropy loss at each decoder step, summed over the sequence. Standard backprop through time through both networks.

Teacher forcing. During training, the decoder's input at step t is the *ground-truth* token at position $t-1$, not the decoder's own previous prediction. This stabilizes training; without it, early mistakes cascade and the model never learns. At inference, you have to use the model's own predictions, so there is always a train/inference distribution gap. That gap is called **exposure bias**.

The bottleneck. Everything the encoder learned about the source must be squeezed into that one context vector. Long sentences lose detail. Rare words get blurred. Reordering (chat noir vs. black cat) has to be memorized, not computed.

Attention (lesson 10) fixes this by letting the decoder look at *every* encoder hidden state, not just the last one. That is the whole pitch.

Interactive figure: lstm-gates. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/09-sequence-to-sequence>

Build It

Step 1: an encoder

```
import torch
import torch.nn as nn

class Encoder(nn.Module):
    def __init__(self, src_vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embed = nn.Embedding(src_vocab_size, embed_dim, padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim, batch_first=True)

    def forward(self, src):
        e = self.embed(src)
        outputs, hidden = self.gru(e)
        return outputs, hidden
```

outputs has shape [batch, seq_len, hidden_dim] — one hidden state per input position. hidden has shape [1, batch, hidden_dim] — the final step. Lesson 08 said “pool over outputs for classification.” Here we keep the last hidden state as the context vector, and ignore the per-step outputs.

Step 2: a decoder

```
class Decoder(nn.Module):
    def __init__(self, tgt_vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embed = nn.Embedding(tgt_vocab_size, embed_dim, padding_idx=0)
        self.gru = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, tgt_vocab_size)

    def forward(self, token, hidden):
        e = self.embed(token)
        out, hidden = self.gru(e, hidden)
        logits = self.fc(out)
        return logits, hidden
```

Decoder is called one step at a time. Input: a batch of single tokens and the current hidden state. Output: vocabulary logits for the next token and the updated hidden state.

Step 3: training loop with teacher forcing

```
def train_batch(encoder, decoder, src, tgt, bos_id, optimizer,
    ↪ teacher_forcing_ratio=0.9):
    optimizer.zero_grad()
    _, hidden = encoder(src)
    batch_size, tgt_len = tgt.shape
```

```

input_token = torch.full((batch_size, 1), bos_id, dtype=torch.long)
loss = 0.0
loss_fn = nn.CrossEntropyLoss(ignore_index=0)

for t in range(tgt_len):
    logits, hidden = decoder(input_token, hidden)
    step_loss = loss_fn(logits.squeeze(1), tgt[:, t])
    loss += step_loss
    use_teacher = torch.rand(1).item() < teacher_forcing_ratio
    if use_teacher:
        input_token = tgt[:, t].unsqueeze(1)
    else:
        input_token = logits.argmax(dim=-1)

loss.backward()
optimizer.step()
return loss.item() / tgt_len

```

Two knobs worth naming. `ignore_index=0` skips loss on padding tokens. `teacher_forcing_ratio` is the probability of using the true token vs. the model's prediction at each step. Start at 1.0 (full teacher forcing) and anneal down to ~0.5 over training to close the exposure-bias gap.

Step 4: inference loop (greedy)

```

@torch.no_grad()
def greedy_decode(encoder, decoder, src, bos_id, eos_id, max_len=50):
    _, hidden = encoder(src)
    batch_size = src.shape[0]
    input_token = torch.full((batch_size, 1), bos_id, dtype=torch.long)
    output_ids = []
    for _ in range(max_len):
        logits, hidden = decoder(input_token, hidden)
        next_token = logits.argmax(dim=-1)
        output_ids.append(next_token)
        input_token = next_token
        if (next_token == eos_id).all():
            break
    return torch.cat(output_ids, dim=1)

```

Greedy decoding picks the highest-probability token at every step. It can wander off: once you commit to a token, you cannot unsay it. **Beam search** keeps the top-k partial sequences alive and picks the highest-scoring complete one at the end. Beam width 3-5 is standard.

Step 5: the bottleneck, demonstrated

Train the model on a toy copy task: source [a, b, c, d, e], target [a, b, c, d, e]. Increase sequence length. Observe accuracy.

```

seq_len=5    copy accuracy: 98%
seq_len=10   copy accuracy: 91%
seq_len=20   copy accuracy: 62%
seq_len=40   copy accuracy: 23%

```

A single GRU hidden state cannot losslessly memorize a 40-token input. The information is

there at every encoder step, but the decoder only sees the last state. Attention fixes this directly.

Use It

PyTorch has `nn.Transformer` and `nn.LSTM`-based seq2seq templates. Hugging Face's `transformers` library ships full encoder-decoder models (BART, T5, mBART, NLLB) trained on billions of tokens.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tok = AutoTokenizer.from_pretrained("facebook/bart-base")
model = AutoModelForSeq2SeqLM.from_pretrained("facebook/bart-base")

src = tok("Translate this to French: Hello, how are you?", return_tensors="pt")
out = model.generate(**src, max_new_tokens=50, num_beams=4)
print(tok.decode(out[0], skip_special_tokens=True))
```

Modern encoder-decoders dropped RNNs for transformers. The high-level shape (encoder, decoder, generate-token-by-token) is identical to the 2014 seq2seq paper. The mechanism inside each block is different.

When to still reach for RNN-based seq2seq

Almost never, for new projects. Specific exceptions:

- Streaming translation where you consume input one token at a time with bounded memory.
- On-device text generation where transformer memory cost is prohibitive.
- Pedagogy. Understanding the encoder-decoder bottleneck is the fastest path to understanding why transformers won.

Exposure bias and its mitigations

- **Scheduled sampling.** Anneal teacher forcing ratio during training so the model learns to recover from its own mistakes.
- **Minimum risk training.** Train on sentence-level BLEU score instead of token-level cross-entropy. Closer to what you actually want.
- **Reinforcement learning fine-tuning.** Reward the sequence generator with a metric. Used in modern LLM RLHF.

All three still apply to transformer-based generation.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/09-sequence-to-sequence>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/09-sequence-to-sequence/code>

1. **Easy.** Implement the toy copy task. Train a GRU seq2seq on input-output pairs where the target equals the source. Measure accuracy at lengths 5, 10, 20. Reproduce the bottleneck.
2. **Medium.** Add beam search decoding with beam width 3. Measure BLEU on a small parallel corpus against greedy. Document where beam search wins (usually last tokens) and where it makes no difference.
3. **Hard.** Fine-tune facebook/bart-base on a 10k-pair paraphrase dataset. Compare the fine-tuned model's beam-4 output to the base model's on held-out inputs. Report BLEU and pick 10 qualitative examples.

Key Terms

Term	What people say	What it actually means
Encoder	Input RNN	Reads source. Produces per-step hidden states and a final context vector.
Decoder	Output RNN	Initialized from context vector. Generates target tokens one at a time.
Context vector	The summary	Final encoder hidden state. Fixed size. The bottleneck attention solves.
Teacher forcing	Use true tokens	Feed the ground-truth previous token at training time. Stabilizes learning.
Exposure bias	Train/test gap	Model trained on true tokens never practiced recovering from its own mistakes.
Beam search	Better decoding	Keep top-k partial sequences alive at each step instead of committing greedily.

Further Reading

- Sutskever, Vinyals, Le (2014). Sequence to Sequence Learning with Neural Networks — the original seq2seq paper. Four pages.
- Cho et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation — introduced the GRU and the encoder-decoder framing.
- Bahdanau, Cho, Bengio (2014). Neural Machine Translation by Jointly Learning to Align and Translate — the attention paper. Read immediately after this lesson.
- PyTorch NLP from Scratch tutorial — buildable seq2seq + attention code.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/09-sequence-to-sequence>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/09-sequence-to-sequence/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/09-sequence-to-sequence>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Attention Mechanism — The Breakthrough

The decoder stops squinting at a compressed summary and starts looking at the whole source. Everything after this is attention plus engineering.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 09 (Sequence-to-Sequence Models) **Time:** ~45 minutes

The Problem

Lesson 09 ended on a measured failure. A GRU encoder-decoder trained on a toy copy task goes from 89% accuracy at length 5 to near-chance at length 80. The reason is structural, not a training bug: every bit of information the encoder gleaned has to fit in one fixed-size hidden state, and the decoder never sees anything else.

Bahdanau, Cho, and Bengio published a three-line fix in 2014. Instead of giving the decoder only the final encoder state, keep every encoder state. At each decoder step, compute a weighted average of encoder states where the weights say “how much does the decoder need to look at encoder position i right now?” That weighted average is the context, and it changes every decoder step.

That is the whole idea. Transformers extended it. Self-attention applied it to a single sequence. Multi-head attention ran it in parallel. But the 2014 version already broke the bottleneck, and once you have it, the pivot to transformers is engineering, not conceptual.

The Concept

At each decoder step t :

1. Use the previous decoder hidden state s_{t-1} as a **query**.
2. Score it against every encoder hidden state $h_1, \dots, h_T$. One scalar per encoder position.
3. Softmax the scores to get attention weights $\alpha_{t,1}, \dots, \alpha_{t,T}$ that sum to 1.
4. Context vector $c_t = \sum \alpha_{t,i} * h_i$. Weighted average of encoder states.
5. Decoder takes c_t plus the previous output token, produces the next token.

The weighted average is the point. When the decoder needs to translate “Je” to “I”, it weights the encoder state over “Je” high and the others low. When it needs “not”, it weights “pas” high. The context vector reshapes each step.

Shapes (the thing that bites everyone)

This is where every attention implementation goes wrong the first time. Read slowly.

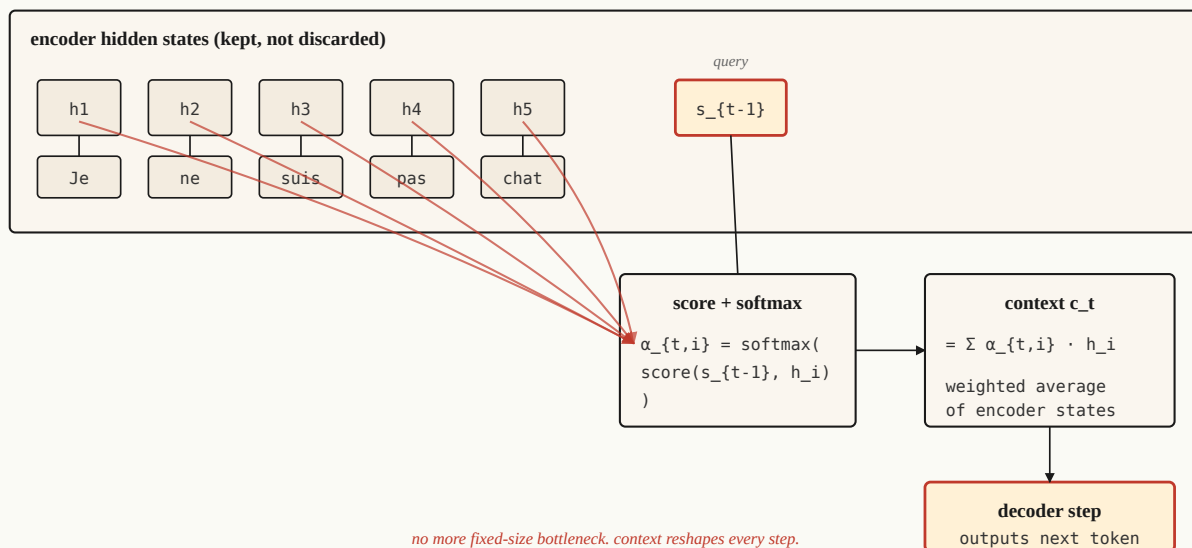


Figure 1: Bahdanau attention: decoder queries all encoder states

Thing	Shape	Notes
Encoder hidden states H	(T_{enc}, d_h)	If BiLSTM, $d_h = 2 * d_{\text{hidden}}$
Decoder hidden state s_{t-1}	$(d_s,)$	One vector
Attention score $e_{t,i}$	scalar	One per encoder position
Attention weight $\alpha_{t,i}$	scalar	After softmax over all i
Context vector c_t	$(d_h,)$	Same shape as an encoder state

Bahdanau (additive) score. $e_{t,i} = v_\alpha^T * \tanh(W_a * s_{t-1} + U_a * h_i)$.

- s_{t-1} has shape $(d_s,)$, h_i has shape $(d_h,)$.
- W_a has shape (d_{attn}, d_s) . U_a has shape (d_{attn}, d_h) .
- Their sum inside the $\tanh$ has shape $(d_{\text{attn}},)$.
- v_α has shape $(d_{\text{attn}},)$. The inner product with v_α collapses to a scalar. **This is what v_α does.** It is not magic. It is the projection that turns an attention-dim vector into a scalar score.

Luong (multiplicative) score. Three variants:

- dot: $e_{t,i} = s_t^T * h_i$. Requires $d_s == d_h$. Hard constraint. Skip if your encoder is bidirectional.
- general: $e_{t,i} = s_t^T * W * h_i$ with W shape (d_s, d_h) . Removes the equal-dim constraint.
- concat: essentially the Bahdanau form. Rarely used since the first two are cheaper.

One Bahdanau / Luong gotcha worth naming. Bahdanau uses s_{t-1} (the decoder state *before* generating the current word). Luong uses s_t (the state *after*). Mixing them up produces subtly wrong gradients that are extremely hard to debug. Pick one paper and stick to its convention.

Interactive figure: attention-heatmap. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/10-attention-mechanism>

Build It

Step 1: additive (Bahdanau) attention

```
import numpy as np

def additive_attention(decoder_state, encoder_states, W_a, U_a, v_a):
    projected_dec = W_a @ decoder_state
    projected_enc = encoder_states @ U_a.T
    combined = np.tanh(projected_enc + projected_dec)
    scores = combined @ v_a
    weights = softmax(scores)
    context = weights @ encoder_states
    return context, weights

def softmax(x):
    x = x - np.max(x)
    e = np.exp(x)
    return e / e.sum()
```

Check your shapes against the table above. `encoder_states` has shape (T_{enc}, d_h) . `projected_dec` has shape $(d_{attn},)$ and broadcasts. `combined` has shape (T_{enc}, d_{attn}) . `scores` has shape $(T_{enc},)$. `weights` has shape $(T_{enc},)$. `context` has shape $(d_h,)$. Ship it.

Step 2: Luong dot and general

```
def dot_attention(decoder_state, encoder_states):
    scores = encoder_states @ decoder_state
    weights = softmax(scores)
    return weights @ encoder_states, weights

def general_attention(decoder_state, encoder_states, W):
    projected = W.T @ decoder_state
    scores = encoder_states @ projected
    weights = softmax(scores)
    return weights @ encoder_states, weights
```

Three lines each. This is why Luong’s paper landed. Same accuracy on most tasks, a lot less code.

Step 3: a worked numerical example

Given three encoder states (roughly “cat”, “sat”, “mat”) and a decoder state that aligns most with the first, the attention distribution concentrates on position 0. If the decoder state shifts to align with the last, attention moves to position 2. The context vector tracks.

```
H = np.array([
    [1.0, 0.0, 0.2],
    [0.5, 0.5, 0.1],
    [0.1, 0.9, 0.3],
])

s_close_to_cat = np.array([0.9, 0.1, 0.2])
ctx, w = dot_attention(s_close_to_cat, H)
print("weights:", w.round(3))
weights: [0.464 0.305 0.231]
```

First row wins. Then move the decoder state closer to the third encoder state and watch the weights shift. That is it. Attention is explicit alignment.

Step 4: why this is the bridge to transformers

Translate the language above into Q/K/V:

- **Query** = decoder state $s_{\{t-1\}}$
- **Key** = encoder states (what we score against)
- **Value** = encoder states (what we weight and sum)

In classical attention, keys and values are the same thing. Self-attention separates them: you can query a sequence against itself, with different learned projections for K and V. Multi-head attention runs it in parallel with different learned projections. Transformers stack the whole stage many times and drop RNNs.

The math is the same. The shapes are the same. The pedagogical jump from Bahdanau attention to scaled dot-product attention is mostly notation.

Use It

PyTorch and TensorFlow ship attention directly.

```
import torch
import torch.nn as nn

mha = nn.MultiheadAttention(embed_dim=128, num_heads=8, batch_first=True)
query = torch.randn(2, 5, 128)
key = torch.randn(2, 10, 128)
value = torch.randn(2, 10, 128)

output, weights = mha(query, key, value)
print(output.shape, weights.shape)

torch.Size([2, 5, 128]) torch.Size([2, 5, 10])
```

That is a transformer attention layer. Query batch of 5 positions, key/value batch of 10 positions, 128-dim each, 8 heads. output is the new context-augmented queries. weights is the 5x10 alignment matrix you can visualize.

When classical attention still matters

- Pedagogy. The single-head, single-layer, RNN-based version makes every concept visible.

- On-device sequence tasks where transformers do not fit.
- Any paper from 2014-2017. You will misread it without knowing Bahdanau’s convention.
- Fine-grained alignment analysis in MT. Raw attention weights are an interpretability tool even on transformer models, and reading them requires knowing what they are.

The attention-weight-as-explanation trap

Attention weights look interpretable. They are weights that sum to one across positions; you can plot them; high means “looked at this.” Reviewers love them.

They are not as interpretable as they look. Jain and Wallace (2019) showed that attention distributions can be permuted and replaced by arbitrary alternatives without changing model predictions for some tasks. Never report attention weights as evidence of reasoning without an ablation or counterfactual check.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/10-attention-mechanism>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/10-attention-mechanism/code>

1. **Easy.** Implement softmax masking so padding tokens in the encoder get attention weight zero. Test on a batch with variable-length sequences.
2. **Medium.** Add multi-head attention to the Luong general form. Split d_h into n_{heads} groups, run attention per head, concatenate. Verify the single-head case matches your earlier implementation.
3. **Hard.** Train a GRU encoder-decoder with Bahdanau attention on the toy copy task from lesson 09. Plot accuracy vs sequence length. Compare against the no-attention baseline. You should see the gap widen as length grows, confirming attention lifts the bottleneck.

Key Terms

Term	What people say	What it actually means
Attention	Looking at things	Weighted average of a value sequence, weights computed from a query-key similarity.
Query, Key, Value	QKV	Three projections: Q asks, K is what to match, V is what to return.
Additive attention	Bahdanau	Feed-forward score: $v^T \tanh(W q + U k)$.
Multiplicative attention	Luong dot / general	Score is $q^T k$ or $q^T W k$. Cheaper, same accuracy on most tasks.
Alignment matrix	The pretty picture	Attention weights as a (T_{dec}, T_{enc}) grid. Read it to see what the model attended to.

Further Reading

- Bahdanau, Cho, Bengio (2014). Neural Machine Translation by Jointly Learning to Align and Translate — the paper.
- Luong, Pham, Manning (2015). Effective Approaches to Attention-based Neural Machine Translation — the three score variants and their comparison.
- Jain and Wallace (2019). Attention is not Explanation — the interpretability caveat.
- Dive into Deep Learning — Bahdanau Attention — runnable walkthrough with PyTorch.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/10-attention-mechanism>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/10-attention-mechanism/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/10-attention-mechanism>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Machine Translation

Translation is the task that paid for NLP research for thirty years and keeps paying now.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 10 (Attention Mechanism), Phase 5 · 04 (GloVe, FastText, Subword) **Time:** ~75 minutes

The Problem

A model reads a sentence in one language and produces a sentence in another. Length varies. Word order varies. Some source words map to multiple target words and vice versa. Idioms refuse one-to-one mapping. “I miss you” in French is “tu me manques” — literally “you are lacking to me.” No word-level alignment survives that.

Machine translation is the task that forced NLP to invent encoder-decoders, attention, transformers, and eventually the whole LLM paradigm. Every step forward arrived because translation quality was measurable and the gap between human and machine was stubborn.

This lesson skips the history lesson and teaches the working pipeline of 2026: pretrained multilingual encoder-decoder (NLLB-200 or mBART), subword tokenization, beam search, BLEU and chrF evaluation, and the handful of failure modes that still ship to production uncaught.

The Concept

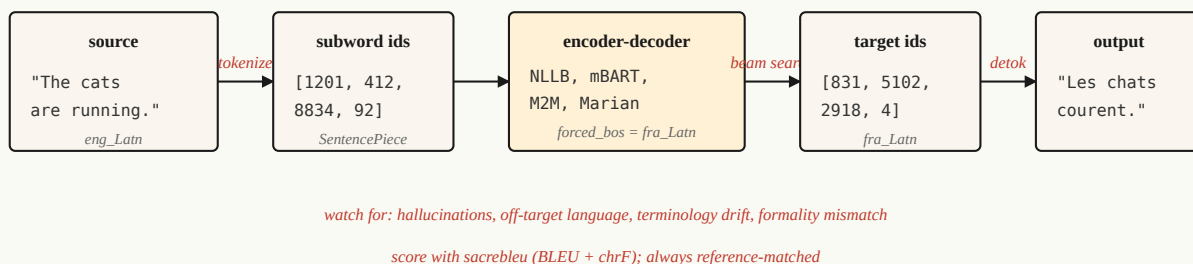


Figure 2: MT pipeline: tokenize → encode → decode with attention → detokenize

Modern MT is a transformer encoder-decoder trained on parallel text. The encoder reads the source in its language’s tokenization. The decoder generates the target, one subword at a time, using the encoder’s output via cross-attention (lesson 10). Decoding uses beam search to avoid the greedy-decoding trap. The output is detokenized, detruccased, and scored against a reference.

Three operational choices drive real-world MT quality.

- **Tokenizer.** SentencePiece BPE trained on a mixed-language corpus. Shared vocabulary across languages is what enables zero-shot pairs in NLLB.
- **Model size.** NLLB-200 distilled 600M fits on a laptop. NLLB-200 3.3B is the published production default. 54.5B is the research ceiling.
- **Decoding.** Beam width 4-5 for general content. Length penalty to avoid too-short output. Constrained decoding when you need terminology consistency.

Interactive figure: seq2seq-alignment. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/11-machine-translation>

Build It

Step 1: a pretrained MT call

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_id = "facebook/nllb-200-distilled-600M"
tok = AutoTokenizer.from_pretrained(model_id, src_lang="eng_Latn")
model = AutoModelForSeq2SeqLM.from_pretrained(model_id)

src = "The cats are running."
inputs = tok(src, return_tensors="pt")

out = model.generate(
    **inputs,
    forced_bos_token_id=tok.convert_tokens_to_ids("fra_Latn"),
    num_beams=5,
    length_penalty=1.0,
    max_new_tokens=64,
)
print(tok.batch_decode(out, skip_special_tokens=True)[0])

Les chats courent.
```

Three things matter here. `src_lang` tells the tokenizer which script and segmentation to apply. `forced_bos_token_id` tells the decoder which language to generate. Both are NLLB-specific tricks; mBART and M2M-100 use their own conventions and they are not interchangeable.

Step 2: BLEU and chrF

BLEU measures n-gram overlap between output and reference. Four reference n-gram sizes (1-4), geometric mean of precisions, brevity penalty for too-short output. The score is in [0, 100]. Commonly used. Frustrating to interpret: 30 BLEU is “usable”; 40 is “good”; 50 is “exceptional”; differences under 1 BLEU are noise.

chrF measures character-level F-score. More sensitive to morphologically rich languages where BLEU undercounts matches. Often reported alongside BLEU.

```
import sacrebleu

hypotheses = ["Les chats courent."]
references = [["Les chats courent."]]
```

```

bleu = sacrebleu.corpus_bleu(hypotheses, references)
chrf = sacrebleu.corpus_chrf(hypotheses, references)
print(f"BLEU: {bleu.score:.1f}  chrF: {chrf.score:.1f}")

```

Always use sacrebleu. It normalizes tokenization so scores are comparable across papers. Rolling your own BLEU computation is how misleading benchmarks happen.

The three-tier evaluation hierarchy (2026)

Modern MT evaluation uses three complementary metric families. Ship with at least two.

- **Heuristic** (BLEU, chrF). Fast, reference-based, interpretable, insensitive to paraphrase. Use for legacy comparison and regression detection.
- **Learned** (COMET, BLEURT, BERTScore). Neural models trained on human judgment; compare semantic similarity of translation to source and reference. COMET has the highest association with MT research since 2023 and is the 2026 production default where quality matters.
- **LLM-as-judge** (reference-free). Prompt a large model to score translations on fluency, adequacy, tone, cultural appropriateness. GPT-4-as-judge matches human agreement ~80% of the time when the rubric is well designed. Use for open-ended content where no reference exists.

Practical 2026 stack: sacrebleu for BLEU and chrF, unbabel-comet for COMET, and a prompted LLM for the final human-facing signal. Calibrate every metric against 50-100 human-labeled examples before trusting it on production data.

Reference-free metrics (COMET-QE, BLEURT-QE, LLM-as-judge) let you evaluate translations without a reference, which matters for long-tail language pairs where reference translations do not exist.

Step 3: what breaks in production

The working pipeline above will translate fluently 80% of the time and silently fail the remaining 20%. Named failure modes:

- **Hallucination.** Model invents content that was not in the source. Common in unfamiliar domain vocabulary. Symptom: output is fluent but claims facts the source did not state. Mitigation: constrained decoding on domain terms, human review on regulated content, monitoring for output much longer than input.
- **Off-target generation.** Model translates into the wrong language. NLLB is surprisingly prone to this on rare language pairs. Mitigation: verify `forced_bos_token_id` and always decode with a language-ID model check on output.
- **Terminology drift.** “Sign up” becomes “s’inscrire” in doc 1 and “créer un compte” in doc 2. For UI text and user-facing strings, consistency matters more than raw quality. Mitigation: glossary-constrained decoding or post-edit dictionary.
- **Formality mismatch.** French “tu” vs “vous”, Japanese politeness levels. The model picks whichever form was more common in training. For customer-facing content this is usually wrong. Mitigation: prompt prefix with a formality token if the model supports it, or fine-tune a small model on formal-only corpora.
- **Length explosion on short input.** Very short input sentences often produce overlong translations because the length penalty falls off a cliff below ~5 source tokens. Mitigation: hard max-length cap proportional to source length.

Step 4: fine-tuning for a domain

Pretrained models are generalists. Legal, medical, or game-dialog translation benefits measurably from fine-tuning on domain parallel data. The recipe is not exotic:

```
from transformers import Trainer, TrainingArguments
from datasets import Dataset

pairs = [
    {"src": "The defendant pleaded guilty.", "tgt": "L'accusé a plaidé coupable."},
]

ds = Dataset.from_list(pairs)

def preprocess(ex):
    return tok(
        ex["src"],
        text_target=ex["tgt"],
        truncation=True,
        max_length=128,
        padding="max_length",
    )

ds = ds.map(preprocess, remove_columns=["src", "tgt"])

args = TrainingArguments(output_dir="out", per_device_train_batch_size=4,
    ↪ num_train_epochs=3, learning_rate=3e-5)
Trainer(model=model, args=args, train_dataset=ds).train()
```

A few thousand high-quality parallel examples beats a few hundred thousand noisy web-scraped ones. Quality of training data is the single largest production lever.

Use It

The 2026 production stack for MT:

Use case	Recommended starting point
Any-to-any, 200 languages	facebook/nllb-200-distilled-600M (laptop) or nllb-200-3.3B (production)
English-centric, high quality, 50 languages	facebook/mbart-large-50-many-to-many-mmt
Short runs, cheap inference, English-French/German/Spanish	Helsinki-NLP / Marian models
Latency-critical browser-side	ONNX-quantized Marian (~50 MB)
Maximum quality, willing to pay	GPT-4 / Claude / Gemini with translation prompts

LLMs now outperform specialized MT models on several language pairs as of 2026, particularly on idiomatic content and long context. The tradeoff is per-token cost and latency. Pick an

LLM when context length, stylistic consistency, or domain adaptation via prompting matters more than throughput.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/11-machine-translation>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/11-machine-translation/code>

1. **Easy.** Translate a 5-sentence English paragraph to French and back to English using `nllb-200-distilled-600M`. Measure how close the round-trip is to the original. You should see semantic preservation with word-choice drift.
2. **Medium.** Implement a language-ID check on translation outputs using `fasttext lid.176` or `langdetect`. Integrate into the MT call so off-target generations are caught before returning.
3. **Hard.** Fine-tune `nllb-200-distilled-600M` on a 5,000-pair domain corpus of your choice. Measure BLEU on a held-out set before and after fine-tuning. Report which kinds of sentences improved and which regressed.

Key Terms

Term	What people say	What it actually means
BLEU	Translation score	N-gram precision with brevity penalty. [0, 100].
chrF	Character F-score	Character-level F-score. More sensitive for morphologically rich languages.
NMT	Neural MT	Transformer encoder-decoder trained on parallel text. The 2017+ default.
NLLB	No Language Left Behind	Meta’s 200-language MT model family.
Constrained decoding	Controlled output	Force specific tokens or n-grams to appear / not appear in the output.
Hallucination	Invented content	Model output that is not supported by the source.

Further Reading

- Costa-jussà et al. (2022). No Language Left Behind: Scaling Human-Centered Machine Translation — the NLLB paper.
- Post (2018). A Call for Clarity in Reporting BLEU Scores — why `sacrebleu` is the only correct way to report BLEU.
- Popović (2015). chrF: character n-gram F-score for automatic MT evaluation — the chrF paper.
- Hugging Face MT guide — practical fine-tuning walkthrough.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/11-machine-translation>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/11-machine-translation/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/11-machine-translation>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Text Summarization

Extractive systems tell you what the document said. Abstractive systems tell you what the author meant. Different tasks, different pitfalls.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 02 (BoW + TF-IDF), Phase 5 · 11 (Machine Translation) **Time:** ~75 minutes

The Problem

A 2,000-word news article lands in your feed. You need 120 words that capture it. You can either pick the three most important sentences from the article (extractive) or rewrite the content in your own words (abstractive). Both are called summarization. They are completely different problems.

Extractive summarization is a ranking problem. Score every sentence, return the top-k. The output is always grammatical because it is lifted verbatim. The risk is missing content that is distributed across the article.

Abstractive summarization is a generation problem. A transformer produces new text conditioned on the input. The output is fluent and compressive but may hallucinate facts that were not in the source. The risk is confident fabrication.

This lesson builds both, with the failure mode each one owns.

The Concept

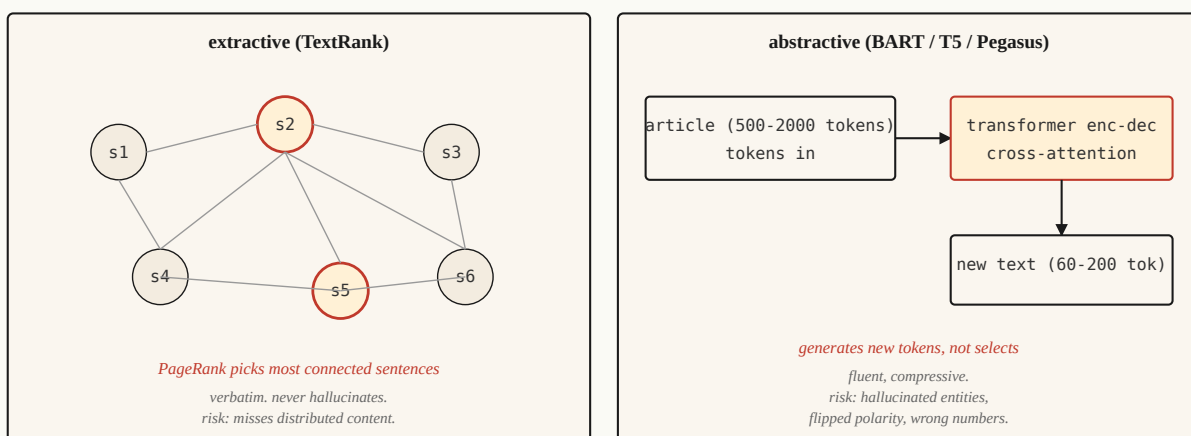


Figure 3: Extractive TextRank vs abstractive transformer

Extractive. Treat the article as a graph where nodes are sentences and edges are similarities. Run PageRank (or something like it) over the graph to score sentences by how connected they

are to everything else. Highest-scoring sentences are the summary. The canonical implementation is **TextRank** (Mihalcea and Tarau, 2004).

Abstractive. Fine-tune a transformer encoder-decoder (BART, T5, Pegasus) on document-summary pairs. At inference, the model reads the document and generates the summary token-by-token via cross-attention. Pegasus in particular uses a gap-sentence pretraining objective that makes it excellent at summarization without much fine-tuning.

Evaluation with **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation). ROUGE-1 and ROUGE-2 score unigram and bigram overlap. ROUGE-L scores longest common subsequence. Higher is better but 40 ROUGE-L is “good” and 50 is “exceptional.” Every paper reports all three. Use the rouge-score package.

Interactive figure: summarize-collapse. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/12-text-summarization>

Build It

Step 1: TextRank (extractive)

```
import math
import re
from collections import Counter

def sentence_split(text):
    return re.split(r"(?<=[.!?])\s+", text.strip())

def similarity(s1, s2):
    w1 = Counter(s1.lower().split())
    w2 = Counter(s2.lower().split())
    intersection = sum((w1 & w2).values())
    denom = math.log(len(w1) + 1) + math.log(len(w2) + 1)
    if denom == 0:
        return 0.0
    return intersection / denom

def textrank(text, top_k=3, damping=0.85, iterations=50, epsilon=1e-4):
    sentences = sentence_split(text)
    n = len(sentences)
    if n <= top_k:
        return sentences

    sim = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i != j:
                sim[i][j] = similarity(sentences[i], sentences[j])

    scores = [1.0] * n
    for _ in range(iterations):
```

```

new_scores = [1 - damping] * n
for i in range(n):
    total_out = sum(sim[i]) or 1e-9
    for j in range(n):
        if sim[i][j] > 0:
            new_scores[j] += damping * sim[i][j] / total_out * scores[i]
if max(abs(s - ns) for s, ns in zip(scores, new_scores)) < epsilon:
    scores = new_scores
    break
scores = new_scores

ranked = sorted(range(n), key=lambda k: scores[k], reverse=True)[:top_k]
ranked.sort()
return [sentences[i] for i in ranked]

```

Two things worth naming. The similarity function uses log-normalized word overlap, which is the original TextRank variant. Cosine of TF-IDF vectors works too. The damping factor 0.85 and iteration count are the PageRank defaults.

Step 2: abstractive with BART

```

from transformers import pipeline

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

article = """(long news article text)"""

summary = summarizer(article, max_length=120, min_length=60, do_sample=False)
print(summary[0]["summary_text"])

```

BART-large-CNN is fine-tuned on the CNN/DailyMail corpus. It produces news-style summaries out of the box. For other domains (scientific papers, dialog, legal), use the corresponding Pegasus checkpoint or fine-tune on your target data.

Step 3: ROUGE evaluation

```

from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"],
    ↪ use_stemmer=True)
scores = scorer.score(reference_summary, generated_summary)
print({k: round(v.fmeasure, 3) for k, v in scores.items()})

```

Always use stemming. Without it, “running” and “run” count as different words and ROUGE undercounts.

Beyond ROUGE (2026 summarization eval)

ROUGE has been the dominant summarization metric for twenty years and it is insufficient on its own in 2026. A large-scale meta-analysis of NLG papers showed:

- **BERTScore** (contextual embedding similarity) gained ground through 2023 and is now reported alongside ROUGE in most summarization papers.
- **BARTScore** treats evaluation as generation: score the summary by how likely a pre-trained BART assigns it given the source.

- **MoverScore** (Earth Mover's Distance over contextual embeddings) reached the top spot in 2025 summarization benchmarks because it captures semantic overlap better than ROUGE.
- **FactCC** and **QA-based faithfulness** were common 2021-2023, now often replaced by **G-Eval** (a GPT-4 prompt chain that scores coherence, consistency, fluency, relevance with chain-of-thought reasoning).
- **G-Eval** and similar LLM-judge approaches match human judgment ~80% of the time when rubrics are well-designed.

Production recommendation: report ROUGE-L for legacy comparison, BERTScore for semantic overlap, G-Eval for coherence and factuality. Calibrate against 50-100 human-labeled summaries.

Step 4: the factuality problem

Abstractive summaries are prone to hallucination. Extractive summaries carry a much lower hallucination risk because the output is lifted verbatim from the source, though they can still mislead if source sentences are decontextualized, outdated, or quoted out of order. This is the single biggest reason production systems still prefer extractive methods for compliance-adjacent content.

Hallucination types to name:

- **Entity swap.** Source says "John Smith." Summary says "John Brown."
- **Number drift.** Source says "25,000." Summary says "25 million."
- **Polarity flip.** Source says "rejected the offer." Summary says "accepted the offer."
- **Fact invention.** Source does not mention the CEO. Summary says the CEO approved.

Evaluation approaches that work:

- **FactCC.** A binary classifier trained on entailment between source sentence and summary sentence. Predicts factual/not-factual.
- **QA-based factuality.** Ask a QA model questions whose answers are in the source. If the summary supports different answers, flag.
- **Entity-level F1.** Compare named entities in source vs summary. Entities present only in the summary are suspect.

For anything user-facing where factuality matters (news, medical, legal, financial), extractive is the safer default. Abstractive needs a factuality check in the loop.

Use It

The 2026 stack:

Use case	Recommended
News, 3-5 sentence summary, English	facebook/bart-large-cnn
Scientific papers	google/pegasus-pubmed or a tuned T5
Multi-document, long-form	Any LLM with 32k+ context, prompted
Dialog summarization	philschmid/bart-large-cnn-samsum
Extractive, low hallucination risk by construction	TextRank or sumy's LSA / LexRank

LLMs with long context often beat specialized models in 2026 when compute is not a constraint. The tradeoff is cost and reproducibility; specialized models give more consistent outputs.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/12-text-summarization>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/12-text-summarization/code>

1. **Easy.** Run TextRank on 5 news articles. Compare the top-3 sentences to a reference summary. Measure ROUGE-L. You should see 30-45 ROUGE-L on CNN/DailyMail-style articles.
2. **Medium.** Implement entity-level factuality: extract named entities from source and summary (spaCy), compute recall of source entities in summary and precision of summary entities against source. High precision and low recall mean safe but terse; low precision means hallucinated entities.
3. **Hard.** Compare BART-large-CNN against an LLM (Claude or GPT-4) on 50 CNN/DailyMail articles. Report ROUGE-L, factuality (by entity F1), and cost per summary. Document where each wins.

Key Terms

Term	What people say	What it actually means
Extractive	Pick sentences	Return sentences verbatim from the source. Never hallucinates.
Abstractive	Rewrite	Generate new text conditioned on source. Can hallucinate.
ROUGE	Summary metric	N-gram / LCS overlap between system output and reference.
TextRank	Graph-based extractive	PageRank over sentence similarity graph.
Factuality	Is it right	Whether summary claims are supported by the source.
Hallucination	Made-up content	Content in the summary that the source does not support.

Further Reading

- Mihalcea and Tarau (2004). TextRank: Bringing Order into Texts — the extractive canonical paper.
- Lewis et al. (2019). BART: Denoising Sequence-to-Sequence Pre-training — the BART paper.
- Zhang et al. (2019). PEGASUS: Pre-training with Extracted Gap-sentences — Pegasus and the gap-sentence objective.
- Lin (2004). ROUGE: A Package for Automatic Evaluation of Summaries — ROUGE paper.

- Maynez et al. (2020). On Faithfulness and Factuality in Abstractive Summarization — the factuality landscape paper.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/12-text-summarization>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/12-text-summarization/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/12-text-summarization>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Question Answering Systems

Three systems shaped modern QA. Extractive found spans. Retrieval-augmented grounded them in documents. Generative produced answers. Every modern AI assistant is a mix of the three.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 11 (Machine Translation), Phase 5 · 10 (Attention Mechanism) **Time:** ~75 minutes

The Problem

A user types “When did the first iPhone launch?” and expects “June 29, 2007.” Not “Apple’s history is long and varied.” Not “2007” sitting in isolation with no sentence. A direct, grounded, correct answer.

Three architectures have dominated QA over the last decade.

- **Extractive QA.** Given a question and a passage that is known to contain the answer, find the start and end indices of the answer span in the passage. SQuAD is the canonical benchmark.
- **Open-domain QA.** The passage is not given. Retrieve the relevant passage first, then extract or generate an answer. This is the bedrock of every RAG pipeline today.
- **Generative / Closed-book QA.** A large language model answers from its parametric memory. No retrieval. Fastest at inference, least reliable on facts.

The trend in 2026 is hybrid: retrieve the best few passages, then prompt a generative model to answer grounded in those passages. That is RAG, and lesson 14 covers the retrieval half in depth. This lesson builds the QA half.

The Concept

Extractive. Encode question and passage together with a transformer (BERT family). Train two heads that predict start and end token indices of the answer. Loss is cross-entropy over valid positions. Output is a span from the passage. Never hallucinates (by construction), never handles questions the passage cannot answer (by construction).

Retrieval-augmented (RAG). Two stages. First, a retriever finds the top-k passages from a corpus. Second, a reader (extractive or generative) produces the answer using those passages. The retriever-reader split lets each be trained and evaluated independently. Modern RAG often adds a reranker between them.

Generative. A decoder-only LLM (GPT, Claude, Llama) answers from learned weights. No retrieval step. Excellent on common knowledge, catastrophic on rare or recent facts. The hallucination rate is inversely correlated with fact frequency in the pretraining data.

Interactive figure: qa-span. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/13-question-answering>

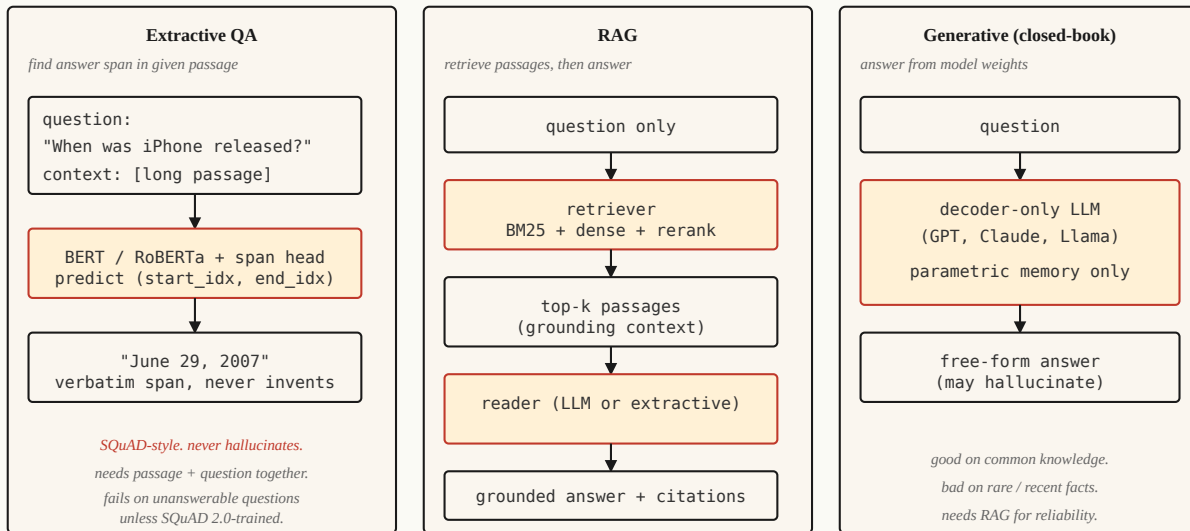


Figure 4: QA architectures: extractive, retrieval-augmented, generative

Build It

Step 1: extractive QA with a pretrained model

from transformers import pipeline

```
qa = pipeline("question-answering", model="deepset/roberta-base-squad2")
```

```
passage = (
    "Apple Inc. released the first iPhone on June 29, 2007. "
    "The device was announced by Steve Jobs at Macworld in January 2007."
)
question = "When was the first iPhone released?"
```

```
answer = qa(question=question, context=passage)
print(answer)
```

```
{'score': 0.98, 'start': 57, 'end': 70, 'answer': 'June 29, 2007'}
```

deepset/roberta-base-squad2 is trained on SQuAD 2.0, which includes unanswerable questions. By default, the question-answering pipeline returns the highest-scoring span even when the model's null score wins — it does *not* automatically return an empty answer. To get explicit “no answer” behavior, pass `handle_impossible_answer=True` to the pipeline call: the pipeline then returns an empty answer only when the null score exceeds every span score. Always check the score field either way.

Step 2: a retrieval-augmented pipeline (sketch)

```
from sentence_transformers import SentenceTransformer
import numpy as np
```

```
encoder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
```

```
corpus = [
    "Apple Inc. released the first iPhone on June 29, 2007.",
    "Macworld 2007 featured the iPhone announcement by Steve Jobs.",
    "Android launched in 2008 as Google's mobile operating system.",
    "The first iPod was released in 2001.",
]
corpus_embeddings = encoder.encode(corpus, normalize_embeddings=True)
```

```
def retrieve(question, top_k=2):
    q_emb = encoder.encode([question], normalize_embeddings=True)
    sims = (corpus_embeddings @ q_emb.T).squeeze()
    order = np.argsort(-sims)[:top_k]
    return [corpus[i] for i in order]
```

```
def answer(question):
    passages = retrieve(question, top_k=2)
    combined = " ".join(passages)
    return qa(question=question, context=combined)
```

```
print(answer("When was the first iPhone released?"))
```

Two-stage pipeline. Dense retriever (Sentence-BERT) finds relevant passages by semantic similarity. Extractive reader (RoBERTa-SQuAD) pulls the answer span from the combined top passages. Works on small corpora. For a million-document corpus, use FAISS or a vector database.

Step 3: generative with RAG

```
def rag_generate(question, llm):
    passages = retrieve(question, top_k=3)
    prompt = f"""Context:
{chr(10).join('- ' + p for p in passages)}
```

```
Question: {question}
```

```
Answer using only the context above. If the context does not contain the answer,
↪ say "I don't know."
"""
    return llm(prompt)
```

The prompt pattern matters. Explicitly telling the model to ground in the context and return “I don’t know” when the context is insufficient cuts hallucination rates by 40-60% compared to naive prompting. More elaborate patterns add citations, confidence scores, and structured extraction.

Step 4: evaluation that reflects the real world

SQuAD uses **Exact Match (EM)** and **token-level F1**. EM is a strict match after normalization (lowercase, strip punctuation, remove articles) — either the prediction matches exactly or it scores 0. F1 is computed over token overlap between prediction and reference and gives

partial credit. Both under-credit paraphrases: “June 29, 2007” vs “June 29th, 2007” typically gets 0 EM (the ordinal breaks normalization) but still earns substantial F1 from overlapping tokens.

For production QA:

- **Answer accuracy** (LLM-judged or human-judged, since metrics do not capture semantic equivalence).
- **Citation accuracy.** Does the cited passage actually support the answer? Trivial to check automatically with string match between generated citations and retrieved passages.
- **Refusal calibration.** When the answer is not in the retrieved passages, does the system correctly say “I don’t know”? Measure false confidence rate.
- **Retrieval recall.** Before evaluating the reader, measure whether the retriever gets the right passage into the top-k. A reader cannot fix a missing passage.

RAGAS: the 2026 production eval framework

RAGAS is purpose-built for RAG systems and is the shipping default in 2026. It scores four dimensions without requiring gold references:

- **Faithfulness.** Does each claim in the answer come from the retrieved context? Measured by NLI-based entailment. Your primary hallucination metric.
- **Answer relevance.** Does the answer address the question? Measured by generating hypothetical questions from the answer and comparing to the real question.
- **Context precision.** Of the retrieved chunks, what fraction were actually relevant? Low precision = noise in prompt.
- **Context recall.** Did the retrieved set contain all needed information? Low recall = reader cannot succeed.

Reference-free scoring lets you evaluate on live production traffic without curated gold answers. Layer LLM-as-judge on top for open-ended questions where exact-match metrics are useless.

`pip install ragas`. Plug your retriever + reader. Get four scalars per query. Alert on regressions.

Use It

The 2026 stack.

Use case	Recommended
Given passage, find answer span	deepset/roberta-base-squad2
Over a fixed corpus, closed-book not acceptable	RAG: dense retriever + LLM reader
Real-time over a document store	RAG with hybrid (BM25 + dense) retriever + reranker (lesson 14)
Conversational QA (follow-up questions)	LLM with conversation history + RAG on each turn
Highly factual, regulated domains	Extractive over an authoritative corpus; never generative alone

Extractive QA is unfashionable in 2026 because RAG with LLMs handles more cases. It still ships in contexts where literal quotation is required: legal research, regulatory compliance, audit tools.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/13-question-answering>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/13-question-answering/code>

1. **Easy.** Set up the SQuAD extractive pipeline above on 10 Wikipedia passages. Hand-craft 10 questions. Measure how often the answer is correct. You should see 7-9 correct if passages and questions are clean.
2. **Medium.** Add a refusal classifier. When the top retrieval score is below a threshold (say 0.3 cosine), return “I don’t know” instead of calling the reader. Tune the threshold on a held-out set.
3. **Hard.** Build a RAG pipeline over a 10,000-document corpus of your choice. Implement hybrid retrieval (BM25 + dense) with RRF fusion (see lesson 14). Measure answer accuracy with and without the hybrid step. Document which question types benefit most.

Key Terms

Term	What people say	What it actually means
Extractive QA	Find the answer span	Predict start and end indices of the answer within a given passage.
Open-domain QA	QA over a corpus	No given passage; must retrieve then answer.
RAG	Retrieve then generate	Retrieval-augmented generation. Retriever + reader pipeline.
SQuAD	Canonical benchmark	Stanford Question Answering Dataset. EM + F1 metrics.
Hallucination	Made-up answer	Reader output not supported by retrieved context.
Refusal calibration	Know when to shut up	System correctly says “I don’t know” when unable to answer.

Further Reading

- Rajpurkar et al. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text — the benchmark paper.
- Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain QA — DPR, the canonical dense retriever for QA.
- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — the paper that named RAG.
- Gao et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey — comprehensive RAG survey.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/13-question-answering>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/13-question-answering/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/13-question-answering>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Information Retrieval and Search

BM25 is precise but brittle. Dense casts a wide net but misses keywords. Hybrid is the 2026 default. Everything else is tuning.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 02 (BoW + TF-IDF), Phase 5 · 04 (GloVe, FastText, Subword) **Time:** ~75 minutes

The Problem

The user types “what happens if someone lies to get money” and expects to find the statute that actually covers that: “Section 420 IPC.” A keyword search misses it entirely (no shared vocabulary). A semantic search misses it if the embeddings were not trained on legal text. Real search has to handle both.

IR is the pipeline under every RAG system, every search bar, every docs site’s fuzzy lookup. The 2026 architecture that works in production is not a single method. It is a chain of complementary methods, each catching the failures of the one before.

This lesson builds each piece and names which failures each catches.

The Concept

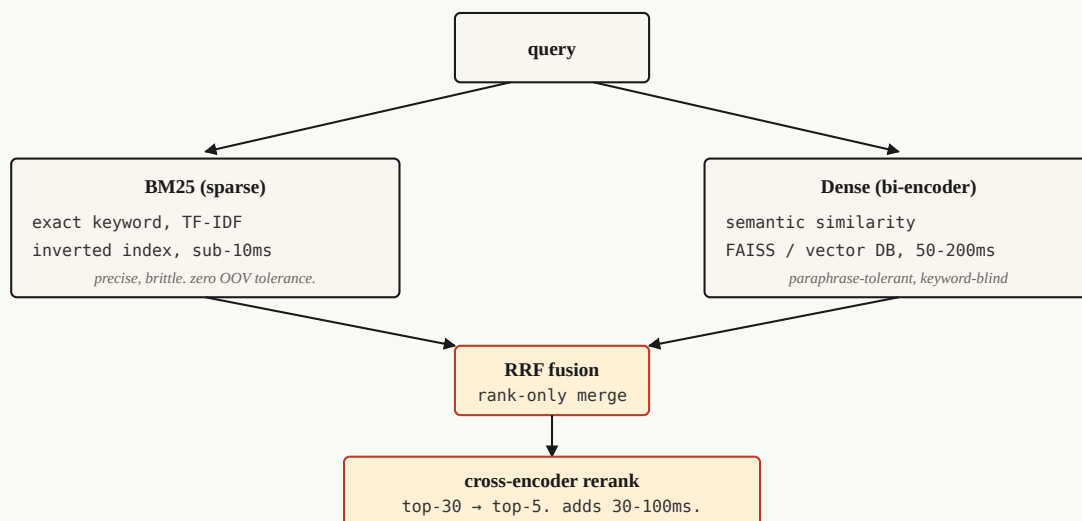


Figure 5: Hybrid retrieval: BM25 + dense + RRF + cross-encoder rerank

Four layers. Pick the ones you need.

1. **Sparse retrieval (BM25).** Fast, precise on exact matches, terrible on semantics. Run over an inverted index. Sub-10ms per query on millions of documents. Gets you statute

- references, product codes, error messages, named entities right.
2. **Dense retrieval.** Encode query and documents into vectors. Nearest neighbor search. Captures paraphrases and semantic similarity. Misses exact keyword matches that differ by one character. 50-200ms per query with FAISS or a vector DB.
 3. **Fusion.** Merge the ranked lists from sparse and dense. Reciprocal Rank Fusion (RRF) is the easy default because it ignores raw scores (which live in different scales) and only uses rank positions. Weighted fusion is an option when you know one signal dominates for your domain.
 4. **Cross-encoder rerank.** Take the top-30 from fusion. Run a cross-encoder (query + document together, scoring each pair). Keep the top-5. Cross-encoders are slower per pair than bi-encoders but far more accurate. You amortize by only running them on the top-30.

Three-way retrieval (BM25 + dense + learned-sparse like SPLADE) outperforms two-way in 2026 benchmarks but needs infrastructure for learned-sparse indexes. For most teams, two-way plus cross-encoder rerank is the sweet spot.

Interactive figure: gx-hybrid-retrieval. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/14-information-retrieval-search>

Build It

Step 1: BM25 from scratch

```
import math
import re
from collections import Counter

TOKEN_RE = re.compile(r"[a-z0-9]+")

def tokenize(text):
    return TOKEN_RE.findall(text.lower())

class BM25:
    def __init__(self, corpus, k1=1.5, b=0.75):
        if not corpus:
            raise ValueError("corpus must not be empty")
        self.corpus = [tokenize(d) for d in corpus]
        self.k1 = k1
        self.b = b
        self.n_docs = len(self.corpus)
        self.avg_dl = sum(len(d) for d in self.corpus) / self.n_docs
        self.df = Counter()
        for doc in self.corpus:
            for term in set(doc):
                self.df[term] += 1

    def idf(self, term):
        n = self.df.get(term, 0)
        return math.log(1 + (self.n_docs - n + 0.5) / (n + 0.5))
```

```

def score(self, query, doc_idx):
    q_tokens = tokenize(query)
    doc = self.corpus[doc_idx]
    dl = len(doc)
    freq = Counter(doc)
    score = 0.0
    for term in q_tokens:
        f = freq.get(term, 0)
        if f == 0:
            continue
        numerator = f * (self.k1 + 1)
        denominator = f + self.k1 * (1 - self.b + self.b * dl / self.avg_dl)
        score += self.idf(term) * numerator / denominator
    return score

def rank(self, query, top_k=10):
    scored = [(self.score(query, i), i) for i in range(self.n_docs)]
    scored.sort(reverse=True)
    return scored[:top_k]

```

Two parameters worth knowing. $k_1=1.5$ controls term-frequency saturation; higher means more weight on term repetition. $b=0.75$ controls length normalization; 0 ignores document length, 1 fully normalizes. The defaults are Robertson's recommendations from the original paper and rarely need tuning.

Step 2: dense retrieval with a bi-encoder

```

from sentence_transformers import SentenceTransformer
import numpy as np

def build_dense_index(corpus, model_id="sentence-transformers/all-MiniLM-L6-v2"):
    encoder = SentenceTransformer(model_id)
    embeddings = encoder.encode(corpus, normalize_embeddings=True)
    return encoder, embeddings

def dense_search(encoder, embeddings, query, top_k=10):
    q_emb = encoder.encode([query], normalize_embeddings=True)
    sims = (embeddings @ q_emb.T).flatten()
    order = np.argsort(-sims)[:top_k]
    return [(float(sims[i]), int(i)) for i in order]

```

L2-normalize embeddings so dot product equals cosine. all-MiniLM-L6-v2 is 384-dim, fast, and strong enough for most English retrieval. For multilingual work, use paraphrase-multilingual-MiniLM-L12-v2. For top accuracy, bge-large-en-v1.5 or e5-large-v2.

Step 3: Reciprocal Rank Fusion

```

def reciprocal_rank_fusion(rankings, k=60):
    scores = {}
    for ranking in rankings:
        for rank, (_, doc_idx) in enumerate(ranking):
            scores[doc_idx] = scores.get(doc_idx, 0.0) + 1.0 / (k + rank + 1)

```

```
fused = sorted(scores.items(), key=lambda x: x[1], reverse=True)
return [(score, doc_idx) for doc_idx, score in fused]
```

The $k=60$ constant comes from the original RRF paper. Higher k flattens the contribution of rank differences; lower k makes top ranks dominate. 60 is the published default and rarely needs tuning.

Step 4: hybrid search + rerank

```
from sentence_transformers import CrossEncoder
```

```
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
```

```
def hybrid_search(query, bm25, encoder, dense_embeddings, corpus, top_k=5,
    pool_size=30, reranker=reranker):
    sparse_ranking = bm25.rank(query, top_k=pool_size)
    dense_ranking = dense_search(encoder, dense_embeddings, query, top_k=pool_size)
    fused = reciprocal_rank_fusion([sparse_ranking, dense_ranking])[pool_size:]

    pairs = [(query, corpus[doc_idx]) for _, doc_idx in fused]
    scores = reranker.predict(pairs)
    reranked = sorted(zip(scores, [doc_idx for _, doc_idx in fused]), reverse=True)
    return reranked[:top_k]
```

Three stages composed. BM25 finds lexical matches. Dense finds semantic matches. RRF merges the two rankings without needing score calibration. Cross-encoder rescores the top-30 using query-document pairs together, which captures fine-grained relevance the bi-encoder missed. Keep top-5.

Step 5: evaluation

Metric	Meaning
Recall@k	Of queries where the correct document exists, how often is it in the top-k?
MRR (Mean Reciprocal Rank)	Average of $1/\text{rank}$ of first relevant document.
nDCG@k	Accounts for relevance gradations, not just binary relevant/not.

For RAG specifically, **Recall@k** of the retriever is the most important number. Your reader cannot answer if the right passage is not in the retrieved set.

Debugging tip: for failing queries, diff the sparse and dense rankings. If one finds the right document and the other does not, you have a vocabulary mismatch (fix: add the missing half) or a semantic ambiguity (fix: better embeddings or a reranker).

Use It

The 2026 stack:

Scale	Stack
1k-100k docs	In-memory BM25 + all-MiniLM-L6-v2 embeddings + RRF. No separate DB.
100k-10M docs	FAISS or pgvector for dense + Elasticsearch / OpenSearch for BM25. Run in parallel.
10M+ docs	Qdrant / Weaviate / Vespa / Milvus with hybrid support. Cross-encoder rerank on top-30.
Best-quality frontier	Three-way (BM25 + dense + SPLADE) + ColBERT late-interaction reranking

Whatever you pick, budget for evaluation. Benchmark retrieval recall before benchmarking end-to-end RAG accuracy. A reader cannot fix what the retriever missed.

The hard-won lessons from 2026 production RAG

- **80% of RAG failures trace to ingestion and chunking, not the model.** Teams spend weeks swapping LLMs and tuning prompts while the retrieval quietly returns the wrong context every third query. Fix chunking first.
- **Chunking strategy matters more than chunk size.** Fixed-size splits break tables, code, and nested headers. Sentence-aware is the default; semantic or LLM-based chunking pays off for technical docs and product manuals.
- **Parent-doc pattern.** Retrieve small “child” chunks for precision. When multiple children from the same parent section appear, swap in the parent block to preserve context. This consistently lifts answer quality without retraining.
- **k_{rerank}=3 is usually optimal.** Every extra chunk past that adds token cost and generation latency without lifting answer quality. If k=8 is still better than k=3 for you, the reranker is underperforming.
- **HyDE / query expansion.** Generate a hypothetical answer from the query, embed that, retrieve. Bridges the phrasing gap between short questions and long documents. Free precision lift with no training.
- **Context budget under 8K tokens.** Consistent hits at that limit mean the reranker threshold is too loose.
- **Version everything.** Prompts, chunking rules, embedding model, reranker. Any drift silently breaks answer quality. CI gates on faithfulness, context precision, and unanswered-question rate block regressions before users see them.
- **Three-way retrieval (BM25 + dense + learned-sparse like SPLADE) outperforms two-way** on 2026 benchmarks, especially for queries mixing proper nouns with semantics. Ship it when infrastructure supports SPLADE indexes.

Proper retrieval design reduces hallucinations by 70-90% according to 2026 industry measurements. Most RAG performance gains come from better retrieval, not model fine-tuning.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/14-information-retrieval-search>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/14-information-retrieval-search>

retrieval-search/code

1. **Easy.** Implement `hybrid_search` above on a 500-document corpus. Test 20 queries. Compare recall at 5 between BM25-only, dense-only, and hybrid.
2. **Medium.** Add MRR calculation. For each test query with a known correct document, find the rank of the correct doc in BM25, dense, and hybrid rankings. Report the MRR for each.
3. **Hard.** Fine-tune a dense encoder on your domain using `MultipleNegativesRankingLoss` (Sentence Transformers). Build a training set from 500 query-document pairs. Compare pre- and post-fine-tune recall.

Key Terms

Term	What people say	What it actually means
BM25	Keyword search	Okapi BM25. Scores documents by term frequency, IDF, and length.
Dense retrieval	Vector search	Encode query + doc into vectors, find nearest neighbors.
Bi-encoder	Embedding model	Encodes query and doc independently. Fast at query time.
Cross-encoder	Reranker model	Encodes query + doc together. Slow but accurate.
RRF	Rank fusion	Combine two rankings by summing $1/(k + \text{rank})$.
Recall@k	Retrieval metric	Fraction of queries where a relevant doc is in the top-k.

Further Reading

- Robertson and Zaragoza (2009). The Probabilistic Relevance Framework: BM25 and Beyond — the definitive BM25 treatment.
- Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain QA — DPR, the canonical bi-encoder.
- Formal et al. (2021). SPLADE: Sparse Lexical and Expansion Model — the learned-sparse retriever that closes the gap with dense.
- Cormack, Clarke, Büttcher (2009). Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods — RRF paper.
- Khattab and Zaharia (2020). ColBERT: Efficient and Effective Passage Search — late-interaction retrieval.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/14-information-retrieval-search>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/14-information-retrieval-search/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/14-information-retrieval-search>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Topic Modeling — LDA and BERTopic

LDA: documents are mixtures of topics, topics are distributions over words.
BERTopic: documents cluster in embedding space, clusters are topics. Same goal, different decompositions.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 02 (BoW + TF-IDF), Phase 5 · 03 (Word2Vec) **Time:** ~45 minutes

The Problem

You have 10,000 customer support tickets, 50,000 news articles, or 200,000 tweets. You need to know what the collection is about without reading it. You do not have labeled categories. You do not even know how many categories exist.

Topic modeling answers that without supervision. Give it a corpus, get back a small set of coherent topics and, for each document, a distribution over those topics.

Two algorithmic families dominate. LDA (2003) treats each document as a mixture of latent topics and each topic as a distribution over words. Inference is Bayesian. It still ships in production where you need mixed-membership topic assignments and explainable word-level probability distributions.

BERTopic (2020) encodes documents with BERT, reduces dimensionality with UMAP, clusters with HDBSCAN, and extracts topic words via class-based TF-IDF. It wins on short text, social media, and anything where semantic similarity matters more than word overlap. One document gets one topic, which is a limitation for long-form content.

This lesson builds intuition for both and names which one to pick for a given corpus.

The Concept

LDA generative story. Each topic is a distribution over words. Each document is a mixture of topics. To generate a word in a document, sample a topic from the document's mixture, then sample a word from that topic's distribution. Inference reverses this: given observed words, infer the topic distribution per document and the word distribution per topic. Collapsed Gibbs sampling or variational Bayes does the math.

Key LDA output:

- `doc_topic`: matrix (`n_docs`, `n_topics`), each row sums to 1 (document's topic mixture).
- `topic_word`: matrix (`n_topics`, `vocab_size`), each row sums to 1 (topic's word distribution).

BERTopic pipeline.

1. Encode each document with a sentence transformer (e.g., `all-MiniLM-L6-v2`). 384-dim vectors.
2. Reduce dimensionality with UMAP to ~5 dimensions. BERT embeddings are too high-dim for clustering.

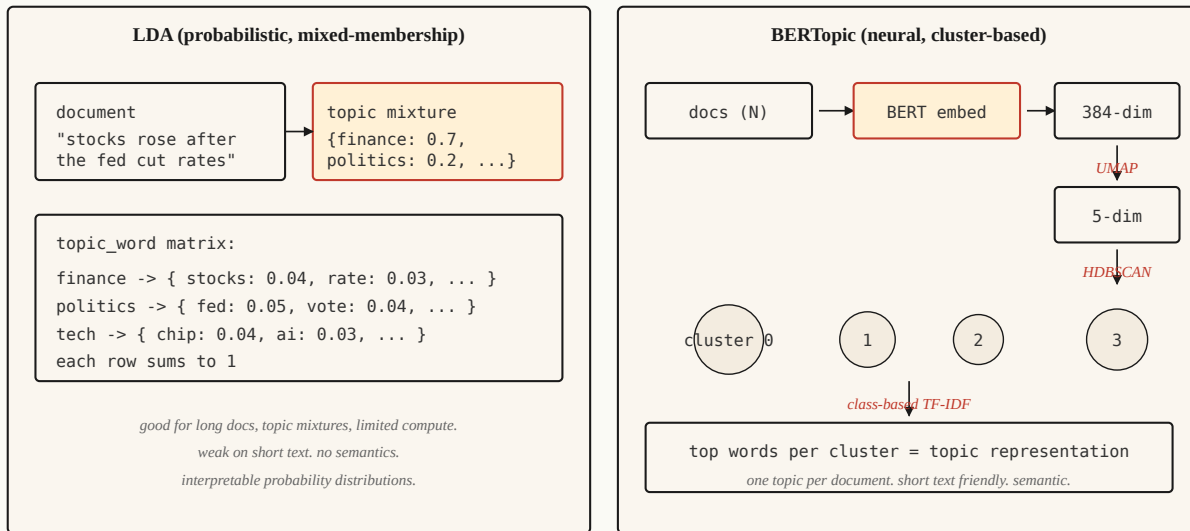


Figure 6: LDA mixture model vs BERTopic clustering

3. Cluster with HDBSCAN. Density-based, produces variable-size clusters and an “outlier” label.
4. For each cluster, compute class-based TF-IDF over the cluster’s documents to extract top words.

Output is one topic per document (plus a -1 outlier label). Optionally, a soft membership via HDBSCAN’s probability vector.

Interactive figure: topic-drift. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/15-topic-modeling>

Build It

Step 1: LDA via scikit-learn

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
import numpy as np
```

```
def fit_lda(documents, n_topics=5, max_features=1000):
    cv = CountVectorizer(
        max_features=max_features,
        stop_words="english",
        min_df=2,
        max_df=0.9,
    )
    X = cv.fit_transform(documents)
    lda = LatentDirichletAllocation(
        n_components=n_topics,
        random_state=42,
        max_iter=50,
```

```

        learning_method="online",
    )
    doc_topic = lda.fit_transform(X)
    feature_names = cv.get_feature_names_out()
    return lda, cv, doc_topic, feature_names

def print_top_words(lda, feature_names, n_top=10):
    for idx, topic in enumerate(lda.components_):
        top_idx = np.argsort(-topic)[:n_top]
        words = [feature_names[i] for i in top_idx]
        print(f"topic {idx}: {' '.join(words)}")

```

Notice: stopwords removed, min_df and max_df filter rare and ubiquitous terms, CountVec- torizer (not TfidfVectorizer) because LDA expects raw counts.

Step 2: BERTopic (production)

```

from bertopic import BERTopic

topic_model = BERTopic(
    embedding_model="sentence-transformers/all-MiniLM-L6-v2",
    min_topic_size=15,
    verbose=True,
)

topics, probs = topic_model.fit_transform(documents)
info = topic_model.get_topic_info()
print(info.head(20))
valid_topics = info[info["Topic"] != -1]["Topic"].tolist()
for topic_id in valid_topics[:5]:
    print(f"topic {topic_id}: {topic_model.get_topic(topic_id)[:10]}")

```

The filter on Topic != -1 drops BERTopic's outlier bucket (documents HDBSCAN could not cluster). min_topic_size controls HDBSCAN's minimum cluster size; BERTopic's library default is 10. This example sets it to 15 explicitly for the lesson's scale. For corpora over 10,000 documents, increase to 50 or 100.

Step 3: evaluation

Both methods output topic words. The question is whether those words cohere.

- **Topic coherence (c_v).** Combines NPMI (normalized pointwise mutual information) of top-word pairs over sliding-window contexts, aggregates the scores into topic vectors, and compares those vectors via cosine similarity. Higher is better. Use `gensim.models.CoherenceModel` with `coherence="c_v"`.
- **Topic diversity.** Fraction of unique words across all topics' top words. Higher is better (topics do not overlap).
- **Qualitative inspection.** Read the top words of each topic. Do they name a real thing? Human judgment is still the last line of defense.

When to pick which

Situation	Pick
Short text (tweets, reviews, headlines)	BERTopic
Long documents with topic mixtures	LDA
No GPU / limited compute	LDA or NMF
Need document-level multi-topic distributions	LDA
LLM integration for topic labeling	BERTopic (direct support)
Resource-constrained edge deployment	LDA
Max semantic coherence	BERTopic

The biggest practical consideration is document length. BERT embeddings truncate; LDA counts work on whatever length. For documents longer than the embedding model’s context, either chunk + aggregate or use LDA.

Use It

The 2026 stack:

- **BERTopic**. Default for short text and anything where semantics matter.
- **gensim.models.LdaModel**. Classic LDA for production, mature, battle-tested.
- **sklearn.decomposition.LatentDirichletAllocation**. Easy LDA for experiments.
- **NMF**. Non-negative matrix factorization. Fast alternative to LDA, comparable quality on short text.
- **Top2Vec**. Similar design to BERTopic. Smaller community but good on some benchmarks.
- **FASTopic**. Newer, faster than BERTopic on very large corpora.
- **LLM-based labeling**. Run any clustering, then prompt a model to name each cluster.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/15-topic-modeling>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/15-topic-modeling/code>

1. **Easy**. Fit LDA with 5 topics on the 20 Newsgroups dataset. Print top 10 words per topic. Label each topic by hand. Did the algorithm find the real categories?
2. **Medium**. Fit BERTopic on the same 20 Newsgroups subset. Compare the number of topics found, top words, and qualitative coherence against LDA. Which surfaces the real categories more cleanly?
3. **Hard**. Compute `c_v` coherence for both LDA and BERTopic on your corpus. Run each with 5, 10, 20, 50 topics. Plot coherence vs topic count. Report which method is more stable across topic counts.

Key Terms

Term	What people say	What it actually means
Topic	A thing the corpus is about	A probability distribution over words (LDA) or a cluster of similar documents (BERTopic).
Mixed membership	Doc is multiple topics	LDA assigns each document a distribution over all topics.
UMAP	Dimensionality reduction	Manifold learning that preserves local structure; used in BERTopic.
HDBSCAN	Density clustering	Finds variable-size clusters; produces “noise” label (-1) for outliers.
c_v coherence	Topic quality metric	Average pointwise mutual information of top topic words within sliding windows.

Further Reading

- Blei, Ng, Jordan (2003). Latent Dirichlet Allocation — the LDA paper.
- Grootendorst (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure — the BERTopic paper.
- Röder, Both, Hinneburg (2015). Exploring the Space of Topic Coherence Measures — the paper that introduced c_v and friends.
- BERTopic documentation — the production reference. Excellent examples.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/15-topic-modeling>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/15-topic-modeling/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/15-topic-modeling>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Text Generation Before Transformers — N-gram Language Models

If a word is surprising, the model is bad. Perplexity makes surprise a number. Smoothing keeps it finite.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 01 (Text Processing), Phase 2 · 14 (Naive Bayes) **Time:** ~45 minutes

The Problem

Before transformers, before RNNs, before word embeddings, a language model predicted the next word by counting how often it followed the previous $n-1$ words. Count “the cat” → “sat” 47 times, “the cat” → “jumped” 12 times, “the cat” → “refrigerator” 0 times. Normalize to get a probability distribution.

That is an n -gram language model. It ran every speech recognizer, every spell checker, and every phrase-based machine translation system from 1980 through 2015. It still runs when you need cheap on-device language modeling.

The interesting problem is what to do about unseen n -grams. A raw count-based model assigns zero probability to anything it has not seen, which is catastrophic because sentences are long and almost every long sentence contains at least one unseen sequence. Fifty years of smoothing research fixed that. Kneser-Ney smoothing is the result, and modern deep learning inherited its empirical tradition.

The Concept

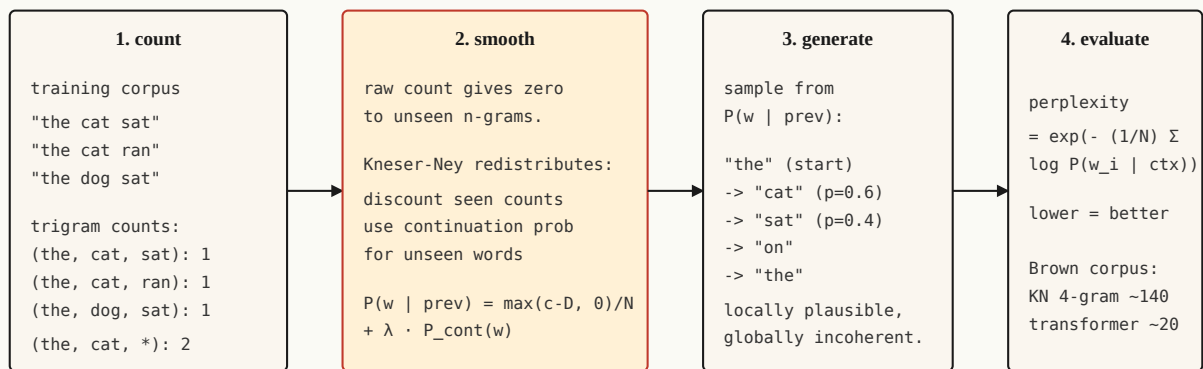
The prediction game

Before any of this machinery existed, one experiment defined what a language model is. Cover the next letter of an English sentence. Ask someone to guess it, one guess at a time, until they get it right. Write down the guess count. Repeat for a few hundred letters.

The guess counts are not trivia. They are a lossless re-encoding of the text: hand the count sequence to a second, identical guesser and they can reconstruct every letter, because at each position they know exactly which guesses come first. A message you can re-encode in fewer symbols carries less information per symbol, so the guess-count statistics put a ceiling on the entropy of English.

Shannon ran this in 1951 and got a number that still governs the field. A 27-symbol alphabet (26 letters plus space) could carry $\log_2(27) \approx 4.75$ bits per letter. Human guessers with 100 letters of context landed between 0.6 and 1.3 bits per letter. English is roughly three-quarters forced moves. The structure a model must learn was measured before any model could learn it.

Every language model since is a mechanical player of this game, and every evaluation number in this lesson is the game scored:



*Kneser-Ney stayed the best n-gram smoother for 20 years.
Transformers closed the perplexity gap by ~10x, not by inventing a better smoother.*

Figure 7: N-gram model: count, smooth, generate

- **Cross-entropy loss** is the average number of bits the model needs per symbol. Training an LM is literally minimizing its score at the guessing game.
- **Perplexity** is 2^{bits} (or e^{nats}): the branching factor still facing the model after its guesswork. Uniform guessing over 27 symbols is perplexity 27; a 1-bit-per-letter player has perplexity 2.
- **Context length is the player's memory.** A trigram model plays with two tokens of memory. A transformer plays the same game with 100K tokens. The rules never changed; the player got better.

One unit switch to track: the game scores per letter in bits ($\log 2$), while the n-gram formulas below score per word token in nats (natural log) — and since perplexity e^H in nats equals 2^H in bits, the two views are the same measurement in different units.

Interactive figure: prediction-game. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/16-text-generation-pre-transformer>

N-gram probability: $P(w_i \mid w_{\{i-n+1\}}, \dots, w_{\{i-1\}})$. Fix n (typically 3 for trigrams, 4 for 4-grams). Compute from counts:

$$P(w \mid \text{context}) = \text{count}(\text{context}, w) / \text{count}(\text{context})$$

The zero-count problem. Any n-gram not seen in training gets probability zero. A 2007 study on the Brown corpus found that even a 4-gram model had 30% of held-out 4-grams unseen in training. You cannot evaluate on any real text without smoothing.

Smoothing approaches, in order of sophistication:

1. **Laplace (add-one).** Add 1 to every count. Simple, terrible on rare events.
2. **Good-Turing.** Reallocate probability mass from higher-frequency events to unseen ones based on frequency-of-frequencies.
3. **Interpolation.** Combine n-gram, (n-1)-gram, etc., estimates with tunable weights.
4. **Backoff.** If n-gram has count zero, fall back to (n-1)-gram. Katz backoff normalizes this.
5. **Absolute discounting.** Subtract a fixed discount D from all counts, redistribute to unseen.
6. **Kneser-Ney.** Absolute discounting plus a clever choice for the lower-order model: use *continuation probability* (how many contexts a word appears in) instead of raw frequency.

The Kneser-Ney insight is deep. “San Francisco” is a common bigram. Unigram “Francisco” appears mostly after “San.” Naive absolute discounting gives “Francisco” high unigram probability (because the count is high). Kneser-Ney notices that “Francisco” appears in only one context and lowers its continuation probability accordingly. Result: a novel bigram ending in “Francisco” gets the appropriate low probability.

Evaluation: perplexity. The exponent of the average negative log-likelihood per word on a held-out test set. Lower is better. A perplexity of 100 means the model is as confused as it would be choosing uniformly among 100 words.

$\text{perplexity} = \exp(- (1/N) * \sum \log P(w_i | \text{context}_i))$

Interactive figure: ngram-backoff. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/16-text-generation-pre-transformer>

Build It

Step 1: trigram counts

```
from collections import Counter, defaultdict

def train_ngram(corpus_tokens, n=3):
    ngrams = Counter()
    contexts = Counter()
    for sentence in corpus_tokens:
        padded = ["<s>"] * (n - 1) + sentence + ["</s>"]
        for i in range(len(padded) - n + 1):
            ctx = tuple(padded[i:i + n - 1])
            word = padded[i + n - 1]
            ngrams[ctx + (word,)] += 1
            contexts[ctx] += 1
    return ngrams, contexts

def raw_probability(ngrams, contexts, context, word):
    ctx = tuple(context)
    if contexts.get(ctx, 0) == 0:
        return 0.0
    return ngrams.get(ctx + (word,), 0) / contexts[ctx]
```

Input is a list of tokenized sentences. Output is n-gram counts and context counts. <s> and </s> are sentence boundaries.

Step 2: Laplace smoothing

```
def laplace_probability(ngrams, contexts, vocab_size, context, word):
    ctx = tuple(context)
    numerator = ngrams.get(ctx + (word,), 0) + 1
    denominator = contexts.get(ctx, 0) + vocab_size
    return numerator / denominator
```

Add 1 to every count. Smooths but over-allocates mass to unseen events, hurting rare-known events too.

Step 3: Kneser-Ney (bigram, interpolated)

```
def kneser_ney_bigram_model(corpus_tokens, discount=0.75):
    unigrams = Counter()
    bigrams = Counter()
    unigram_contexts = defaultdict(set)

    for sentence in corpus_tokens:
        padded = ["<s>"] + sentence + ["</s>"]
        for i, w in enumerate(padded):
            unigrams[w] += 1
            if i > 0:
                prev = padded[i - 1]
                bigrams[(prev, w)] += 1
                unigram_contexts[w].add(prev)

    total_unique_bigrams = sum(len(ctx_set) for ctx_set in
    ↪ unigram_contexts.values())
    continuation_prob = {
        w: len(ctx_set) / total_unique_bigrams for w, ctx_set in
    ↪ unigram_contexts.items()
    }

    context_totals = Counter()
    for (prev, w), count in bigrams.items():
        context_totals[prev] += count

    unique_follow = defaultdict(set)
    for (prev, w) in bigrams:
        unique_follow[prev].add(w)

    def prob(prev, w):
        count = bigrams.get((prev, w), 0)
        denom = context_totals.get(prev, 0)
        if denom == 0:
            return continuation_prob.get(w, 1e-9)
        first_term = max(count - discount, 0) / denom
        lambda_prev = discount * len(unique_follow[prev]) / denom
        return first_term + lambda_prev * continuation_prob.get(w, 1e-9)

    return prob
```

Three moving parts. `continuation_prob` captures “how many different contexts does this word appear in?” (the Kneser-Ney innovation). `lambda_prev` is the mass freed by the discount, used to weight the backoff. The final probability is the discounted main term plus the weighted continuation term.

Step 4: generating text with sampling

```
import random

def generate(prob_fn, vocab, prefix, max_len=30, seed=0):
    rng = random.Random(seed)
```

```

tokens = list(prefix)
for _ in range(max_len):
    candidates = [(w, prob_fn(tokens[-1], w)) for w in vocab]
    total = sum(p for _, p in candidates)
    r = rng.random() * total
    acc = 0.0
    for w, p in candidates:
        acc += p
        if r <= acc:
            tokens.append(w)
            break
    if tokens[-1] == "</s>":
        break
return tokens

```

Sampling proportional to probability. Always gives different output per seed. For beam-search-like output, pick the argmax at each step (greedy) and add a small randomness knob (temperature).

Step 5: perplexity

```
import math
```

```

def perplexity(prob_fn, sentences):
    total_log_prob = 0.0
    total_tokens = 0
    for sentence in sentences:
        padded = ["<s>"] + sentence + ["</s>"]
        for i in range(1, len(padded)):
            p = prob_fn(padded[i - 1], padded[i])
            total_log_prob += math.log(max(p, 1e-12))
            total_tokens += 1
    return math.exp(-total_log_prob / total_tokens)

```

Lower is better. For Brown corpus, a well-tuned 4-gram KN model hits perplexity around 140. A transformer LM hits 15-30 on the same test set. The gap is about 10x. That gap is why the field moved on.

Use It

- **Classical NLP teaching.** The clearest exposure to smoothing, MLE, and perplexity you can get.
- **KenLM.** Production n-gram library. Used as a rescorer in speech and MT systems where low latency matters.
- **On-device autocomplete.** Trigram models in keyboards. Still.
- **Baselines.** Always compute an n-gram LM perplexity before declaring your neural LM good. If your transformer does not beat KN by a wide margin, something is wrong.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/16-text-generation-pre-transformer>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/16-text-generation-pre-transformer/code>

1. **Easy.** Train a trigram LM on a 1,000-sentence Shakespeare corpus. Generate 20 sentences. They will be locally plausible but globally incoherent. This is the canonical demo.
2. **Medium.** Implement perplexity for your KN model on a held-out Shakespeare split. Compare against Laplace. You should see KN lower perplexity by 30-50%.
3. **Hard.** Build a trigram spell corrector: given a misspelled word and its context, generate corrections and rank by context probability under the LM. Evaluate on the Birkbeck spelling corpus (public).

Key Terms

Term	What people say	What it actually means
N-gram	Word sequence	Sequence of n consecutive tokens.
Smoothing	Avoiding zeros	Reallocating probability mass so unseen events get non-zero probability.
Perplexity	LM quality metric	$\exp(-\text{average log-prob})$ on held-out data. Lower is better.
Backoff	Fallback to shorter context	If trigram count is zero, use bigram. Katz backoff formalizes this.
Kneser-Ney	Best smoothing for n-grams	Absolute discounting + continuation probability for the lower-order model.
Continuation probability	KN-specific	$P(w)$ weighted by number of contexts w appears in, not by raw count.
Entropy of text	Information per symbol	Average bits needed to encode the next symbol given the context. Shannon’s 1951 estimate for printed English with up to 100 letters of context: 0.6-1.3 bits/letter, measured before any model existed.

Further Reading

- Shannon (1951). Prediction and Entropy of Printed English — the guessing-game experiment that defined the target every language model still optimizes.
- Jurafsky and Martin — Speech and Language Processing, Chapter 3 (2026 draft) — the canonical treatment of n-gram LMs and smoothing.
- Chen and Goodman (1998). An Empirical Study of Smoothing Techniques for Language Modeling — the paper that settled Kneser-Ney as the best n-gram smoother.
- Kneser and Ney (1995). Improved Backing-off for M-gram Language Modeling — the original KN paper.
- KenLM — fast production n-gram LM, still used in 2026 for latency-sensitive applications.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/16-text-generation-pre-transformer>

- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/16-text-generation-pre-transformer/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/16-text-generation-pre-transformer>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Chatbots — Rule-Based to Neural to LLM Agents

ELIZA replied with pattern matches. DialogFlow mapped intents. GPT answered from weights. Claude runs tools and verifies. Each era solved the previous one's worst failure.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 13 (Question Answering), Phase 5 · 14 (Information Retrieval) **Time:** ~75 minutes

The Problem

A user says “I want to change my flight.” The system has to figure out what they want, what information is missing, how to get it, and how to complete the action. Then the user says “wait, what if I cancel instead?” and the system has to remember the context, switch tasks, and preserve state.

Conversation is hard for an ML system. The input is open-ended. The output has to be coherent over many turns. The system may need to act on the world (change a flight, charge a card). Every wrong step is visible to the user.

Chatbot architectures have cycled through four paradigms, each introduced because the previous one failed too visibly. This lesson walks them in order. The 2026 production landscape is a hybrid of the last two.

The Concept

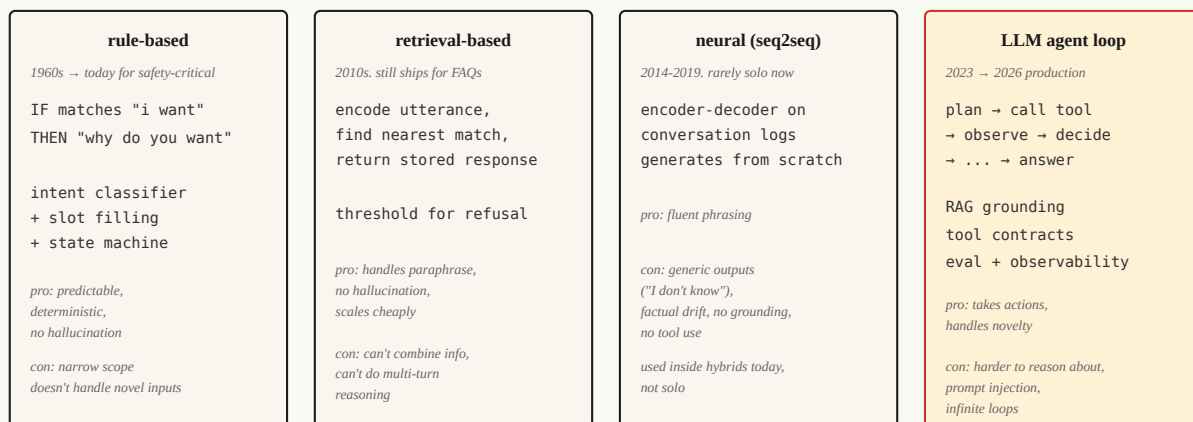


Figure 8: Chatbot evolution: rule-based → retrieval → neural → agent

The scripted half-century, 1950-2001

The first paradigm did not last five years. It lasted fifty. Knowing its arc matters because every system in it is the same machine — match input, emit a canned response, update a little state — and fifty years of adding rules to that machine never produced the general case. That ceiling is why paradigms two through four exist.

1950. Turing sidesteps “can machines think?” by proposing an operational replacement: if an interrogator cannot tell the machine from a person over teletype, the philosophical question is moot. Conversation becomes the field’s benchmark before the field has a name.

1956. The name arrives — a summer workshop at Dartmouth coins “artificial intelligence” on the conjecture that every feature of intelligence “can in principle be so precisely described that a machine can be made to simulate it.” The proposal budgets two months for substantial progress.

1966. ELIZA ships the reflection trick you build in Step 1: decomposition rules pull fragments from the input, reassembly rules echo them back as questions. Around 200 patterns total, zero state, zero understanding — and users confided in it anyway. Weizenbaum spent the rest of his career alarmed by how little machinery it took.

1972. PARRY, built at Stanford to model paranoia, adds the piece ELIZA lacked: internal state. Numeric variables for fear, anger, and mistrust update on every turn and gate which script fires next, so identical inputs produce different responses depending on the conversation so far. In a blinded transcript test, psychiatrists distinguished PARRY from human patients at chance. It is the direct ancestor of persona conditioning — a system prompt implemented as three floats. The same year, the two bots were pointed at each other over ARPANET: a therapist script interviewing a paranoia state machine, the first bot-to-bot conversation on a network.

1995. ALICE scales the ELIZA recipe with AIML, an XML dialect for pattern-template pairs. Roughly 40,000 hand-written categories, three Loebner Prize wins. It proved the scaling law of rule-based systems: more rules buy coverage, never generality. Every rule is a liability someone must maintain.

2001. SmarterChild puts the recipe in front of 30 million instant-messenger users and adds backend lookups — weather, stocks, movie times — spliced into templates. Squint and it is tool calling wearing a 2001 costume: parse intent, call a service, render the result into the reply.

Fifty years, one mechanism, rising rule counts. The paradigm ended not because anyone disproved it but because the maintenance cost of hand-written state machines grows linearly with coverage while user expectations grow with whatever they saw last week.

Interactive figure: chatbot-lineage. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/17-chatbots-rule-to-neural>

Rule-based (ELIZA, AIML, DialogFlow). Hand-authored patterns match user input and produce responses. Intent classifiers route to predefined flows. Slot-filling state machines collect required info. Works brilliantly inside the narrow scope it was designed for. Fails immediately outside it. Still ships in safety-critical domains (banking authentication, airline booking) where hallucination is not tolerated.

Retrieval-based. A FAQ-style system. Encode every pair of (utterance, response). At run-time, encode the user’s message and retrieve the nearest stored response. Think Zendesk’s classic “similar articles” feature. Handles paraphrases better than rules. No generation, so no hallucination.

Neural (seq2seq). Encoder-decoder trained on conversation logs. Generates responses from scratch. Fluent but prone to generic outputs (“I don’t know”) and factual drift. Never reliably on topic. The reason Google, Facebook, and Microsoft all had disappointing chatbots in 2016-2019.

LLM agents. A language model wrapped in a loop that plans, calls tools, and verifies outcomes. Not a chatbot with a long prompt. An agent loop: plan → call tool → observe result → decide next step. Retrieval-first grounding (RAG) keeps it from hallucinating. Tool calls let it actually do things. This is the 2026 architecture.

The four paradigms are not sequential replacements. A 2026 production chatbot routes through all four: rule-based for authentication and destructive actions, retrieval for FAQ, neural generation for natural phrasing, LLM agent for ambiguous open-ended queries.

Build It

Step 1: rule-based pattern matching

```
import re

class RulePattern:
    def __init__(self, pattern, response_template):
        self.regex = re.compile(pattern, re.IGNORECASE)
        self.template = response_template

PATTERNS = [
    RulePattern(r"my name is (\w+)", "Nice to meet you, {0}.",),
    RulePattern(r"i (need|want) (.+)", "Why do you {0} {1}?",),
    RulePattern(r"i feel (.+)", "Why do you feel {0}?",),
    RulePattern(r"(.*)", "Tell me more about that.",),
]

def rule_based_respond(user_input):
    for pattern in PATTERNS:
        m = pattern.regex.match(user_input.strip())
        if m:
            return pattern.template.format(*m.groups())
    return "I don't understand."
```

ELIZA in 20 lines. The reflection trick (“I feel sad” → “Why do you feel sad”) is the canonical psychotherapist demo from Weizenbaum 1966. Still instructive.

Step 2: retrieval-based (FAQ)

This illustrative snippet requires `pip install sentence-transformers` (which pulls in torch). The runnable code/main.py for this lesson uses a stdlib Jaccard similarity instead, so the lesson runs without external dependencies.

```
from sentence_transformers import SentenceTransformer
import numpy as np
```

```
FAQ = [
    ("how do i reset my password", "Go to Settings > Security > Reset Password."),
    ("how do i cancel my order", "Go to Orders, find the order, click Cancel."),
    ("what is your return policy", "30-day returns on unused items, original
    ↪ packaging."),
]
```

```
encoder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
faq_questions = [q for q, _ in FAQ]
faq_embeddings = encoder.encode(faq_questions, normalize_embeddings=True)
```

```
def faq_respond(user_input, threshold=0.5):
    q_emb = encoder.encode([user_input], normalize_embeddings=True)[0]
    sims = faq_embeddings @ q_emb
    best = int(np.argmax(sims))
    if sims[best] < threshold:
        return None
    return FAQ[best][1]
```

Threshold-based refusal is the key design choice. If the best match is not close enough, return None and let the system escalate.

Step 3: neural generation (baseline)

Use a small instruction-tuned encoder-decoder (FLAN-T5) or a fine-tuned conversational model. Production-unusable on its own in 2026 (contradiction, off-topic drift, factual non-sense), but ships inside hybrid systems for natural phrasing. DialoGPT-style decoder-only models need explicit turn separators and EOS handling to produce coherent replies; a FLAN-T5 text2text pipeline works out of the box for a teaching example.

```
from transformers import pipeline

chatbot = pipeline("text2text-generation", model="google/flan-t5-small")

response = chatbot("Respond politely to: Hi there!", max_new_tokens=40)
print(response[0]["generated_text"])
```

Step 4: LLM agent loop

The 2026 production shape:

```
def agent_loop(user_message, tools, llm, max_steps=5):
    history = [{"role": "user", "content": user_message}]
    for _ in range(max_steps):
        response = llm(history, tools=tools)
        tool_call = response.get("tool_call")
        if tool_call:
            tool_name = tool_call.get("name")
            args = tool_call.get("arguments")
            if not isinstance(tool_name, str) or tool_name not in tools:
                history.append({"role": "assistant", "tool_call": tool_call})
                history.append({"role": "tool", "name": str(tool_name), "content":
                ↪ f"error: unknown tool {tool_name!r}"})
```

```

        continue
    if not isinstance(args, dict):
        history.append({"role": "assistant", "tool_call": tool_call})
        history.append({"role": "tool", "name": tool_name, "content":
↪ f"error: arguments must be a dict, got {type(args).__name__}"})
        continue
    fn = tools[tool_name]
    result = fn(**args)
    history.append({"role": "assistant", "tool_call": tool_call})
    history.append({"role": "tool", "name": tool_name, "content": result})
else:
    return response["content"]
return "I could not complete the task in the step budget."

```

Three things to name. Tools are callable functions the LLM can invoke. The loop terminates when the LLM returns a final answer instead of a tool call. The step budget prevents infinite loops on ambiguous tasks.

Real production adds: retrieval-first grounding (inject relevant docs before each LLM call), guardrails (refuse destructive actions without confirmation), observability (log every step), and evaluations (automated checks that agent behavior stays on-spec).

Step 5: hybrid routing

```

def hybrid_chat(user_input):
    if is_destructive_action(user_input):
        return structured_flow(user_input)

    faq_answer = faq_respond(user_input, threshold=0.6)
    if faq_answer:
        return faq_answer

    return agent_loop(user_input, tools, llm)

def is_destructive_action(text):
    danger_words = ["delete", "cancel", "charge", "refund", "transfer"]
    return any(w in text.lower() for w in danger_words)

```

The pattern: deterministic rules for anything destructive, retrieval for canned FAQs, LLM agents for everything else. This is what ships in 2026 customer-support systems.

Use It

The 2026 stack:

Use case	Architecture
Booking, payment, authentication	Rule-based state machines + slot filling
Customer support FAQs	Retrieval over curated answers
Open-ended help chat	LLM agent with RAG + tool calls
Internal tools / IDE assistants	LLM agent with tool calls (search, read, write)
Companion / character chatbots	Tuned LLM with persona system prompt, retrieval on knowledge

Always use hybrid routing in production. No single architecture handles every request well. The routing layer itself is typically a small intent classifier.

Failure modes that still ship

- **Confident fabrication.** LLM agent claims it completed an action it did not. Mitigation: verify outcomes, log tool calls, never let the LLM claim to have done something without a successful tool return.
- **Prompt injection.** User inserts text that overrides the system prompt. Ranked LLM01 in the OWASP Top 10 for LLM Applications 2025. Two flavors: direct injection (pasted into the chat) and indirect injection (hidden in documents, emails, or tool outputs the agent reads).

Attack rates vary by scenario. Measured success rates range ~0.5-8.5% across frontier models in general tool-use and coding benchmarks. Specific high-risk setups (adaptive attacks against AI coding agents, vulnerable orchestration) have reached ~84%. Production CVEs include EchoLeak (CVE-2025-32711, CVSS 9.3) — a zero-click data-exfiltration flaw in Microsoft 365 Copilot triggered by an attacker-controlled email.

Mitigations: treat user input as untrusted throughout the loop; sanitize before tool calls; isolate tool outputs from the main prompt; use the Plan-Verify-Execute (PVE) pattern where the agent plans first, then verifies each action against that plan before executing (this stops tool results from injecting new unplanned actions); require user confirmation for destructive actions; apply least-privilege to tool scopes.

No amount of prompt engineering fully eliminates this risk. External runtime defense layers (LLM Guard, allowlist validation, semantic anomaly detection) are required.

- **Scope creep.** Agent goes off-task because a tool call returned tangentially related info. Mitigation: narrow tool contracts; keep the system prompt focused; add evaluations for off-task rate.
- **Infinite loops.** Agent keeps calling the same tool. Mitigation: step budget, tool-call deduplication, LLM judge on “are we making progress.”
- **Context window exhaustion.** Long conversations push the earliest turns out of context. Mitigation: summarize older turns, retrieve relevant past turns by similarity, or use a long-context model.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/17-chatbots-rule-to-neural>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/17-chatbots-rule-to-neural/code>

1. **Easy.** Implement the rule-based respond above with 10 patterns for a coffee-shop ordering bot. Test edge cases: double orders, modifications, cancellation, unclear intent.
2. **Medium.** Build a hybrid FAQ + LLM fallback. 50 canned FAQ entries for a SaaS product, LLM fallback with retrieval over the docs site. Measure refusal rate and accuracy on 100 real support questions.

3. **Hard.** Implement the agent loop above with three tools (search, read-user-data, send-email). Run an evaluation with 50 test scenarios including prompt injection attempts. Report off-task rate, failed task rate, and any injection success.

Key Terms

Term	What people say	What it actually means
Intent	What the user wants	Categorical label (book_flight, reset_password). Routed to a handler.
Slot	A piece of info	Parameter the bot needs (date, destination). Slot filling is the sequence of asks.
RAG	Retrieval plus generation	Retrieve relevant docs, then ground the LLM's response.
Tool call	Function invocation	LLM emits a structured call with name + args. Runtime executes, returns result.
Agent loop	Plan, act, verify	Controller that runs LLM calls interleaved with tool calls until task complete.
Prompt injection	User attacks prompt	Malicious input that tries to override the system prompt.

Further Reading

- Turing (1950). Computing Machinery and Intelligence — the paper that made conversation the field's benchmark.
- Weizenbaum (1966). ELIZA — A Computer Program For the Study of Natural Language Communication — the original rule-based chatbot paper.
- Colby, Weber, Hilf (1971). Artificial Paranoia — PARRY's affect-variable architecture, the first stateful chatbot.
- Thoppilan et al. (2022). LaMDA: Language Models for Dialog Applications — Google's late neural-chatbot paper, just before LLM agents took over.
- Yao et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models — the paper that named the agent loop pattern.
- Anthropic's guide on building effective agents — 2024 production guidance that still holds in 2026.
- Greshake et al. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection — the prompt-injection paper.
- OWASP Top 10 for LLM Applications 2025 — LLM01 Prompt Injection — the ranking that made prompt injection the top security concern.
- AWS — Securing Amazon Bedrock Agents against Indirect Prompt Injections — practical orchestration-layer defenses including Plan-Verify-Execute and user-confirmation flows.
- EchoLeak (CVE-2025-32711) — the canonical zero-click data-exfiltration CVE from indirect prompt injection. Reference case for why write-access agents need runtime defenses.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/17-chatbots-rule-to-neural>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/17-chatbots-rule-to-neural/code>

- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/17-chatbots-rule-to-neural>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Multilingual NLP

One model, 100+ languages, zero training data for most of them. Cross-lingual transfer is the practical miracle of the 2020s.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 04 (GloVe, FastText, Subword), Phase 5 · 11 (Machine Translation) **Time:** ~45 minutes

The Problem

English has billions of labeled examples. Urdu has thousands. Maithili has almost none. Any practical NLP system that serves a global audience has to work on the long tail of languages where task-specific training data does not exist.

Multilingual models solve this by training one model on many languages simultaneously. The shared representation lets the model transfer skills learned in high-resource languages to low-resource ones. Fine-tune the model on English sentiment analysis, and it produces surprisingly good sentiment predictions on Urdu out of the box. That is zero-shot cross-lingual transfer, and it has reshaped how NLP ships to the world.

This lesson names the tradeoffs, the canonical models, and the one decision that trips up teams new to multilingual work: picking a source language for transfer.

The Concept

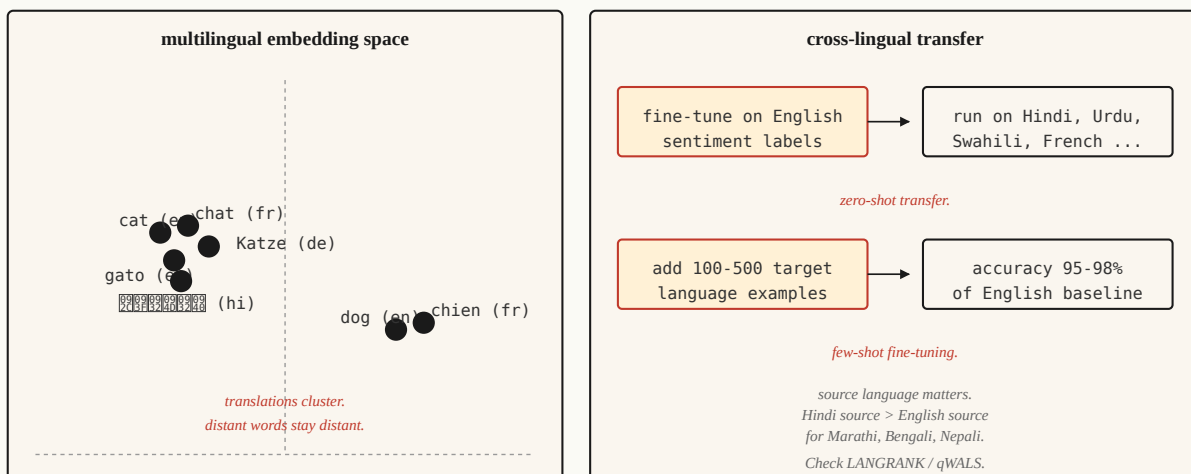


Figure 9: Cross-lingual transfer via shared multilingual embedding space

Shared vocabulary. Multilingual models use a SentencePiece or WordPiece tokenizer trained on text from all target languages. The vocabulary is shared: the same subword unit

represents the same morpheme across related languages. `anti-` in English and Italian gets the same token.

Shared representation. A transformer pretrained on masked language modeling across many languages learns that semantically similar sentences in different languages produce similar hidden states. mBERT, XLM-R, and NLLB all exhibit this. Embeddings for “cat” in English cluster near “chat” in French and “gato” in Spanish, and so do full-sentence embeddings.

Zero-shot transfer. Fine-tune the model on labeled data in one language (usually English). At inference, run it on any other language the model supports. No target-language labels needed. Results are strong for typologically related languages and weaker for distant ones.

Few-shot fine-tuning. Add 100-500 labeled examples in the target language. Accuracy jumps to 95-98% of the English baseline on classification tasks. This is the single most cost-effective lever in multilingual NLP.

The models

Model	Year	Coverage	Notes
mBERT	2018	104 languages	Trained on Wikipedia. First practical multilingual LM. Weak on low-resource.
XLM-R	2019	100 languages	Trained on CommonCrawl (much larger than Wikipedia). Sets the cross-lingual baseline. Base 270M, Large 550M.
XLM-V	2023	100 languages	XLM-R with 1M-token vocabulary (vs 250k). Better on low-resource.
mT5	2020	101 languages	T5 architecture for multilingual generation.
NLLB-200	2022	200 languages	Meta’s translation model; includes 55 low-resource languages.
BLOOM	2022	46 languages + 13 programming	Open 176B LLM trained multilingually.
Aya-23	2024	23 languages	Cohere’s multilingual LLM. Strong on Arabic, Hindi, Swahili.

Pick by use case. Classification works well with XLM-R-base as the sane default. Generation tasks call for mT5 or NLLB depending on translation vs open generation. LLM-style work pairs with Aya-23 or Claude using explicit multilingual prompting.

The source-language decision (2026 research)

Most teams default to English as the fine-tuning source. Recent research (2026) shows this is often wrong.

Language similarity predicts transfer quality better than raw corpus size. For Slavic targets, German or Russian often beat English. For Indic targets, Hindi often beats English. The **qWALS** similarity metric (2026, based on World Atlas of Language Structures features) quantifies this. **LANGRANK** (Lin et al., ACL 2019) is a separate, earlier method that ranks candidate source languages from a combination of linguistic similarity, corpus size, and genetic relatedness.

Practical rule: if your target language has a typologically close high-resource relative, try fine-tuning on that one first, then compare to English fine-tune.

Interactive figure: n5-crosslingual-bridge. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/18-multilingual-nlp>

Build It

Step 1: zero-shot cross-lingual classification

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

tok = AutoTokenizer.from_pretrained("joeddav/xlm-roberta-large-xnli")
model = AutoModelForSequenceClassification.from_pretrained("joeddav/xlm-roberta-large-xnli")

def classify(text, candidate_labels, hypothesis_template="This text is about {}."):
    scores = {}
    for label in candidate_labels:
        hypothesis = hypothesis_template.format(label)
        inputs = tok(text, hypothesis, return_tensors="pt", truncation=True)
        with torch.no_grad():
            logits = model(**inputs).logits[0]
            entail_score = torch.softmax(logits, dim=-1)[2].item()
            scores[label] = entail_score
    return dict(sorted(scores.items(), key=lambda x: -x[1]))

print(classify("I love this product!", ["positive", "negative", "neutral"]))
print(classify("□□□□ □□ □□□□□□ □□□□ □□!", ["positive", "negative", "neutral"]))
print(classify("J'adore ce produit !", ["positive", "negative", "neutral"]))
```

One model, three languages, same API. XLM-R trained on NLI data transfers well to classification via the entailment trick.

Step 2: multilingual embedding space

```

from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-
    ↪ L12-v2")

pairs = [
    ("The cat is sleeping.", "Le chat dort."),
    ("The cat is sleeping.", "El gato está durmiendo."),
    ("The cat is sleeping.", "Die Katze schläft."),
    ("The cat is sleeping.", "The dog is barking."),
]

for eng, other in pairs:
    emb_eng = model.encode([eng], normalize_embeddings=True)[0]
    emb_other = model.encode([other], normalize_embeddings=True)[0]
    sim = float(np.dot(emb_eng, emb_other))
    print(f" {eng!r} <-> {other!r}: cos={sim:.3f}")

```

Translations land close in embedding space. A different English sentence lands further. This is what makes cross-lingual retrieval, clustering, and similarity work.

Step 3: few-shot fine-tuning strategy

```

from transformers import TrainingArguments, Trainer
from datasets import Dataset

def few_shot_finetune(base_model, base_tokenizer, examples):
    ds = Dataset.from_list(examples)

    def tokenize_fn(ex):
        out = base_tokenizer(ex["text"], truncation=True, max_length=128)
        out["labels"] = ex["label"]
        return out

    ds = ds.map(tokenize_fn)
    args = TrainingArguments(
        output_dir="out",
        per_device_train_batch_size=8,
        num_train_epochs=5,
        learning_rate=2e-5,
        save_strategy="no",
    )
    trainer = Trainer(model=base_model, args=args, train_dataset=ds)
    trainer.train()
    return base_model

```

For 100-500 target-language examples, `num_train_epochs=5` and `learning_rate=2e-5` are the safe defaults. Higher learning rates cause the multilingual alignment to collapse and you get an English-only model.

Evaluation that actually works

- **Per-language accuracy on held-out sets.** Not aggregated. The aggregate hides the long tail.
- **Benchmark against monolingual baseline.** For languages with enough data, a monolingual model trained from scratch sometimes beats the multilingual one. Test.
- **Entity-level tests.** Named entities in the target language. Multilingual models often have weak tokenization for scripts far from Latin.
- **Cross-lingual consistency.** Same meaning in two languages should produce the same prediction. Measure the gap.

Use It

The 2026 stack:

Task	Recommended
Classification, 100 languages	XLNet-R-base (~270M) fine-tuned
Zero-shot text classification	joeddav/xlm-roberta-large-xnli
Multilingual sentence embeddings	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Translation, 200 languages	facebook/nllb-200-distilled-600M (see lesson 11)
Generative multilingual	Claude, GPT-4, Aya-23, mT5-XXL
Low-resource language NLP	XLNet-V or a domain-specific fine-tune on related high-resource language

Always budget for fine-tuning in the target language if performance matters. Zero-shot is a starting point, not a final answer.

The tokenization tax (what goes wrong for low-resource languages)

Multilingual models share one tokenizer across all their languages. That vocabulary is trained on a corpus dominated by English, French, Spanish, Chinese, German. For any language outside the dominant set, three taxes compound silently:

- **Fertility tax.** Low-resource language text tokenizes into far more tokens per word than English. A Hindi sentence can need 3-5x the tokens of an equivalent English sentence. That 3-5x eats your context window, training efficiency, and latency.
- **Variant recovery tax.** Every typo, diacritic variant, Unicode normalization mismatch, or case variation becomes a cold-start unrelated sequence in embedding space. The model cannot learn orthographic correspondences that a native speaker takes as obvious.
- **Capacity spillover tax.** Taxes 1 and 2 consume context positions, layer depth, and embedding dimensions. What remains for actual reasoning is systematically smaller than what a high-resource language gets from the same model.

The practical symptom: your model trains normally on Hindi, the loss curve looks right, eval perplexity looks reasonable, and production outputs are subtly wrong. Morphology collapses mid-sentence. Rare inflections stay unrecoverable. **You cannot data-scale your way out of a broken tokenizer.**

Mitigations: pick a tokenizer with good coverage for your target language (XLM-V’s 1M-token vocabulary is a direct fix); verify tokenization fertility on held-out target text before training; use byte-level fallback (SentencePiece `byte_fallback=True`, GPT-2-style byte-level BPE) for truly long-tail scripts so nothing is ever OOV.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/18-multilingual-nlp>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/18-multilingual-nlp/code>

1. **Easy.** Run the zero-shot classification pipeline on 10 sentences per language across English, French, Hindi, and Arabic. Report accuracy on each. You should see strong French, decent Hindi, variable Arabic.
2. **Medium.** Use `paraphrase-multilingual-MiniLM-L12-v2` to build a cross-lingual retriever over a small mixed-language corpus. Query in English, retrieve documents in any language. Measure `recall@5`.
3. **Hard.** Compare English-source and Hindi-source fine-tuning for a Hindi classification task. Use 500 target-language examples for few-shot fine-tuning under both regimes. Report which source produces better Hindi accuracy and by how much. This is the LANGRANK thesis in miniature.

Key Terms

Term	What people say	What it actually means
Multilingual model	One model, many languages	Shared vocabulary and parameters across languages.
Cross-lingual transfer	Train on one language, run on another	Fine-tune on source, evaluate on target without target-language labels.
Zero-shot	No target-language labels	Transfer without fine-tuning on the target language.
Few-shot	Small target labels	100-500 target-language examples used for fine-tuning.
mBERT	First multilingual LM	104-language BERT pretrained on Wikipedia.
XLM-R	Standard cross-lingual baseline	100-language RoBERTa pretrained on CommonCrawl.
NLLB	Meta’s 200-language MT	No Language Left Behind. Includes 55 low-resource languages.

Further Reading

- Conneau et al. (2019). Unsupervised Cross-lingual Representation Learning at Scale — the XLM-R paper.

- Pires, Schlinger, Garrette (2019). How Multilingual is Multilingual BERT? — the analysis paper that started the cross-lingual transfer research line.
- Costa-jussà et al. (2022). No Language Left Behind — NLLB-200 paper.
- Üstün et al. (2024). Aya Model: An Instruction Finetuned Open-Access Multilingual Language Model — Aya, Cohere’s multilingual LLM.
- Language Similarity Predicts Cross-Lingual Transfer Learning Performance (2026) — the qWALS / LANGRANK source-language paper.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/18-multilingual-nlp>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/18-multilingual-nlp/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/18-multilingual-nlp>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Subword Tokenization — BPE, WordPiece, Unigram, SentencePiece

Word tokenizers choke on unseen words. Character tokenizers blow up sequence length. Subword tokenizers split the difference. Every modern LLM ships on one.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 01 (Text Processing), Phase 5 · 04 (GloVe / FastText / Subword) **Time:** ~60 minutes

The Problem

Your vocabulary has 50,000 words. A user types “untokenizable”. Your tokenizer returns [UNK]. The model now has no signal about the word. Worse: the 90th-percentile document in your corpus has 40 rare words, which means 40 bits of dropped information per document.

Subword tokenization solves this. Common words stay single tokens. Rare words decompose into meaningful pieces: untokenizable → un, token, izable. Training data covers everything because any string is ultimately a sequence of bytes.

Every frontier LLM in 2026 ships on one of three algorithms (BPE, Unigram, WordPiece), wrapped in one of three libraries (tiktoken, SentencePiece, HF Tokenizers). You cannot ship a language model without picking one.

The Concept

BPE (Byte-Pair Encoding). Start with a character-level vocabulary. Count every adjacent pair. Merge the most frequent pair into a new token. Repeat until you hit the target vocabulary size. Dominant algorithm: GPT-2/3/4, Llama, Gemma, Qwen2, Mistral.

Byte-level BPE. Same algorithm but over raw bytes (256 base tokens) instead of Unicode characters. Guarantees zero [UNK] tokens — any byte sequence encodes. GPT-2 uses 50,257 tokens (256 bytes + 50,000 merges + 1 special).

Unigram. Start with a huge vocabulary. Assign each token a unigram probability. Iteratively prune tokens whose removal least increases the corpus log-likelihood. Probabilistic at inference: can sample tokenizations (useful for data augmentation via subword regularization). Used by T5, mBART, ALBERT, XLNet, Gemma.

WordPiece. Merge pairs that maximize likelihood of the training corpus rather than raw frequency. Used by BERT, DistilBERT, ELECTRA.

SentencePiece vs tiktoken. SentencePiece is the library that *trains* vocabularies (BPE or Unigram) directly on raw Unicode text, encoding whitespace as `_`. tiktoken is OpenAI’s fast *encoder* against pre-built vocabularies; it does not train.

Rule of thumb:

- **Training a new vocabulary:** SentencePiece (multilingual, no pre-tokenization) or HF Tokenizers.

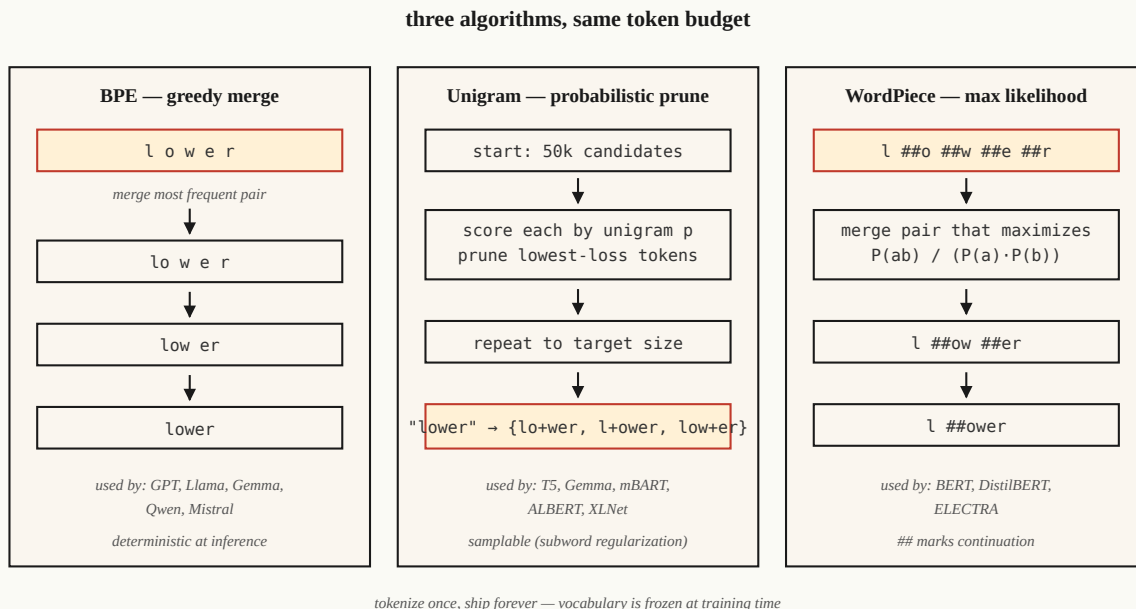


Figure 10: BPE vs Unigram vs WordPiece, character-by-character

- **Fast inference against GPT vocab:** tiktoken (cl100k_base, o200k_base).
- **Both:** HF Tokenizers — one library, training + serving.

Interactive figure: bpe-merge. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/19-subword-tokenization>

Build It

Step 1: BPE from scratch

See code/main.py. The loop:

```
def train_bpe(corpus, num_merges):
    vocab = {(tuple(word) + ("</w>"),): count for word, count in corpus.items()}
    merges = []
    for _ in range(num_merges):
        pairs = Counter()
        for symbols, freq in vocab.items():
            for a, b in zip(symbols, symbols[1:]):
                pairs[(a, b)] += freq
        if not pairs:
            break
        best = pairs.most_common(1)[0][0]
        merges.append(best)
        vocab = apply_merge(vocab, best)
    return merges
```

Three facts the algorithm encodes. `</w>` marks word end so “low” (suffix) and “lower” (prefix) stay distinct. Frequency weighting makes high-frequency pairs win early. The merge list is ordered — inference applies merges in training order.

Step 2: encode with the learned merges

```
def encode_bpe(word, merges):
    symbols = list(word) + ["</w>"]
    for a, b in merges:
        i = 0
        while i < len(symbols) - 1:
            if symbols[i] == a and symbols[i + 1] == b:
                symbols = symbols[:i] + [a + b] + symbols[i + 2:]
            else:
                i += 1
    return symbols
```

Naive $O(n \cdot |\text{merges}|)$. Production implementations (tiktoken, HF Tokenizers) use merge-rank lookup with priority queues and run in near-linear time.

Step 3: SentencePiece in practice

```
import sentencepiece as spm

spm.SentencePieceTrainer.train(
    input="corpus.txt",
    model_prefix="my_tokenizer",
    vocab_size=8000,
    model_type="bpe",           # or "unigram"
    character_coverage=0.9995, # lower for CJK (e.g. 0.9995 for English, 0.995 for
    ↪ Japanese)
    normalization_rule_name="nmt_nfkc",
)

sp = spm.SentencePieceProcessor(model_file="my_tokenizer.model")
print(sp.encode("untokenizable", out_type=str))
# ['_un', 'token', 'izable']
```

Notice: no pre-tokenization required, space encoded as `_`, `character_coverage` controls how aggressively rare characters are preserved vs mapped to `<unk>`.

Step 4: tiktoken for OpenAI-compatible vocabs

```
import tiktoken
enc = tiktoken.get_encoding("o200k_base")
print(enc.encode("untokenizable"))      # [127340, 101028]
print(len(enc.encode("Hello, world!"))) # 4
```

Encoding-only. Fast (Rust backend). Exact match with GPT-4/5 tokenization for byte-counting, cost estimation, context-window budgeting.

Pitfalls that still ship in 2026

- **Tokenizer drift.** Training on vocab A, deploying against vocab B. Token IDs differ; model outputs garbage. Check `tokenizer.json` hash in CI.
- **Whitespace ambiguity.** BPE “hello” vs “ hello” produce different tokens. Always specify `add_special_tokens` and `add_prefix_space` explicitly.

- **Multilingual undertraining.** English-heavy corpora produce vocabularies that split non-Latin scripts into 5-10x more tokens. Same prompt costs 5-10x more in Japanese/Arabic on GPT-3.5. o200k_base partially fixed this.
- **Emoji splits.** A single emoji can take 5 tokens. Checkpoint emoji handling when budgeting context.

Use It

The 2026 stack:

Situation	Pick
Training a monolingual model from scratch	HF Tokenizers (BPE)
Training a multilingual model	SentencePiece (Unigram, character_coverage=0.9995)
Serving an OpenAI-compatible API	tiktoken (o200k_base for GPT-4+)
Domain-specific vocab (code, math, protein)	Train custom BPE on domain corpus, merge with base vocab
Edge inference, small model	Unigram (smaller vocabularies work better)

Vocabulary size is a scaling decision, not a constant. Rough heuristic: 32k for <1B params, 50-100k for 1-10B, 200k+ for multilingual/frontier.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/19-subword-tokenization>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/19-subword-tokenization/code>

1. **Easy.** Train a 500-merge BPE on code/main.py’s tiny corpus. Encode three held-out words. How many produced exactly 1 token vs >1 token?
2. **Medium.** Compare token counts on 100 English Wikipedia sentences between cl100k_base, o200k_base, and a SentencePiece BPE you train with vocab=32k. Report the compression ratio of each.
3. **Hard.** Train the same corpus with BPE, Unigram, and WordPiece. Measure downstream accuracy when using each on a small sentiment classifier. Does the choice move the needle by more than 1 point F1?

Key Terms

Term	What people say	What it actually means
BPE	Byte-Pair Encoding	Greedy merge of most-frequent character pairs until target vocab size hit.
Byte-level BPE	No unknown tokens ever	BPE over raw 256 bytes; GPT-2 / Llama use this.
Unigram	Probabilistic tokenizer	Prunes from a large candidate set using log-likelihood; used by T5, Gemma.
SentencePiece	The whitespace one	Library that trains BPE/Unigram on raw text; space encoded as <code>_</code> .
tiktoken	The fast one	OpenAI's Rust-backed BPE encoder for pre-built vocabs. No training.
Merge list	The magic numbers	Ordered list of (a, b) → ab merges; inference applies in order.
Character coverage	How rare is too rare?	Fraction of characters in training corpus the tokenizer must cover; ~0.9995 typical.

Further Reading

- Sennrich, Haddow, Birch (2015). Neural Machine Translation of Rare Words with Subword Units — the BPE paper.
- Kudo (2018). Subword Regularization with Unigram Language Model — the Unigram paper.
- Kudo, Richardson (2018). SentencePiece: A simple and language independent subword tokenizer — the library.
- Hugging Face — Summary of the tokenizers — concise reference.
- OpenAI tiktoken repo — cookbook + encoding list.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/19-subword-tokenization>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/19-subword-tokenization/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/19-subword-tokenization>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Structured Outputs & Constrained Decoding

Ask an LLM for JSON. Get JSON most of the time. In production, “most” is the problem. Constrained decoding turns “most” into “always” by editing the logits before sampling.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 17 (Chatbots), Phase 5 · 19 (Subword Tokenization) **Time:** ~60 minutes

The Problem

A classifier prompts an LLM: “Return one of {positive, negative, neutral}.” The model returns “The sentiment is positive — this review is overwhelmingly favorable because the customer explicitly states that they ...”. Your parser crashes. Your classifier’s F1 is 0.0.

Free-form generation is not a contract. It is a suggestion. A production system needs a contract.

Three layers exist in 2026.

1. **Prompting.** Ask nicely. “Return only the JSON object.” Works ~80% on frontier models, less on smaller ones.
2. **Native structured output APIs.** OpenAI `response_format`, Anthropic tool use, Gemini JSON mode. Reliable on supported schemas. Vendor-locked.
3. **Constrained decoding.** Modify the logits at every generation step so the model *cannot* emit invalid tokens. 100% valid by construction. Works on any local model.

This lesson builds intuition for all three and names when to reach for which.

The Concept

How constrained decoding works. At each generation step, the LLM produces a logit vector over the full vocabulary (~100k tokens). A *logit processor* sits between the model and the sampler. It computes which tokens are valid given the current position in the target grammar — JSON Schema, regex, context-free grammar — and sets the logits of all invalid tokens to negative infinity. The softmax over the remaining logits puts probability mass only on valid continuations.

Implementations in 2026:

- **Outlines.** Compiles JSON Schema or regex into a finite-state machine. Every token gets an O(1) valid-next-token lookup. FSM-based, so recursive schemas need flattening.
- **XGrammar / Ilguidance.** Context-free grammar engines. Handle recursive JSON Schema. Near-zero decoding overhead. OpenAI credited Ilguidance in their 2025 structured output implementation.
- **vLLM guided decoding.** Built-in `guided_json`, `guided_regex`, `guided_choice`, `guided_grammar` via Outlines, XGrammar, or `lm-format-enforcer` backends.

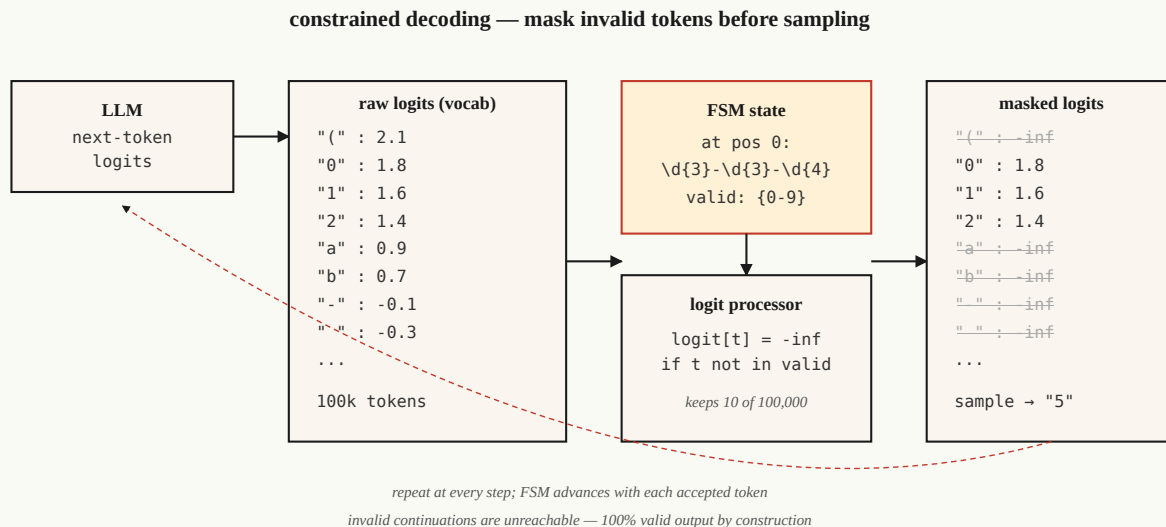


Figure 11: Constrained decoding masking invalid tokens at each step

- **Instructor.** Pydantic-based wrapper over any LLM. Retries on validation failure. Cross-provider, but does not modify logits — it relies on retries + structured-output-aware prompts.

The counterintuitive result

Constrained decoding is often *faster* than unconstrained generation. Two reasons. First, it shrinks the next-token search space. Second, clever implementations skip token generation entirely for forced tokens (scaffolding like `{"name": " — every byte is determined}`).

The pitfall that costs you

Field order matters. Put answer before reasoning, and the model commits to an answer before it thinks. JSON is valid. Answer is wrong. No validation catches it.

```
// BAD
{"answer": "yes", "reasoning": "because ..."}

```

```
// GOOD
{"reasoning": "... therefore ...", "answer": "yes"}

```

Schema field order is logic, not formatting.

Interactive figure: constrained-decoder. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/20-structured-outputs-constrained-decoding>

Build It

Step 1: regex-constrained generation from scratch

See `code/main.py` for a standalone FSM implementation. The core idea in 30 lines:

```
def mask_logits(logits, valid_token_ids):
    mask = [float("-inf")] * len(logits)
    for tid in valid_token_ids:
        mask[tid] = logits[tid]
    return mask

def generate_constrained(model, tokenizer, prompt, fsm):
    ids = tokenizer.encode(prompt)
    state = fsm.initial_state
    while not fsm.is_accept(state):
        logits = model.next_token_logits(ids)
        valid = fsm.valid_tokens(state, tokenizer)
        logits = mask_logits(logits, valid)
        tok = sample(logits)
        ids.append(tok)
        state = fsm.transition(state, tok)
    return tokenizer.decode(ids)
```

The FSM tracks what parts of the grammar we have satisfied so far. `valid_tokens(state, tokenizer)` computes which vocabulary tokens can advance the FSM without leaving an accepting path.

Step 2: Outlines for JSON Schema

```
from pydantic import BaseModel
from typing import Literal
import outlines

class Review(BaseModel):
    sentiment: Literal["positive", "negative", "neutral"]
    confidence: float
    evidence_span: str

model = outlines.models.transformers("meta-llama/Llama-3.2-3B-Instruct")
generator = outlines.generate.json(model, Review)

result = generator("Classify: 'The wait staff was attentive and the food arrived  
↪ hot.'")
print(result)
# Review(sentiment='positive', confidence=0.93, evidence_span='attentive ... hot')
```

Zero validation errors. Ever. The FSM makes invalid output unreachable.

Step 3: Instructor for provider-agnostic Pydantic

```
import instructor
from anthropic import Anthropic
from pydantic import BaseModel, Field

class Invoice(BaseModel):
```

```

vendor: str
total_usd: float = Field(ge=0)
line_items: list[str]

client = instructor.from_anthropic(Anthropic())
invoice = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=1024,
    response_model=Invoice,
    messages=[{"role": "user", "content": "Extract from: 'Acme Corp $420. Widget,
↪ Gizmo.'"}],
)

```

Different mechanism. Instructor does not touch logits. It formats the schema into the prompt, parses the output, and retries on validation failure (default 3 times). Works with any provider. Retries add latency and cost. Cross-provider portability is the selling point.

Step 4: native vendor APIs

```

from openai import OpenAI

client = OpenAI()
response = client.responses.create(
    model="gpt-5",
    input=[{"role": "user", "content": "Classify: 'The food was cold.'"}],
    text={"format": {"type": "json_schema", "name": "sentiment",
        "schema": {"type": "object", "required": ["sentiment"],
            "properties": {"sentiment": {"type": "string",
                "enum": ["positive", "negative",
↪ "neutral"]}}}}},
)
print(response.output_parsed)

```

Server-side constrained decoding. Reliability parity with Outlines for supported schemas. No local model management. Locks you to the vendor.

Pitfalls

- **Recursive schemas.** Outlines flattens recursion to a fixed depth. Tree-structured outputs (nested comments, AST) need XGrammar or llguidance (CFG-based).
- **Huge enums.** 10,000-option enum compiles slowly or times out. Switch to a retriever: predict top-k candidates first, constrain to those.
- **Grammar too strict.** Force date: "YYYY-MM-DD" regex and the model cannot output "unknown" for missing dates. Model compensates by inventing a date. Allow null or a sentinel.
- **Premature commitment.** See field-order pitfall above. Always put reasoning first.
- **Vendor JSON mode without schema.** Pure JSON mode only guarantees valid JSON, not valid *for your use case*. Always provide a full schema.

Use It

The 2026 stack:

Situation	Pick
OpenAI/Anthropic/Google model, simple schema	Native vendor structured output
Any provider, Pydantic workflow, can tolerate retries	Instructor
Local model, need 100% validity, flat schema	Outlines (FSM)
Local model, recursive schema	XGrammar or llguidance
Self-hosted inference server	vLLM guided decoding
Batch processing with retries acceptable	Instructor + cheapest model

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/20-structured-outputs-constrained-decoding>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/20-structured-outputs-constrained-decoding/code>

1. **Easy.** Prompt a small open-weights model (e.g., Llama-3.2-3B) without constrained decoding for Review(sentiment, confidence, evidence_span). Measure the fraction that parse as valid JSON on 100 reviews.
2. **Medium.** Same corpus with Outlines JSON mode. Compare compliance rate, latency, and semantic accuracy.
3. **Hard.** Implement a regex-constrained decoder from scratch for phone numbers (`\d{3}-\d{3}-\d{4}`). Verify 0 invalid outputs on 1000 samples.

Key Terms

Term	What people say	What it actually means
Constrained decoding	Force valid output	Mask invalid-token logits at every generation step.
Logit processor	The thing that constrains	Function: (logits, state) -> masked_logits.
FSM	Finite-state machine	Compiled grammar representation; O(1) valid-next-token lookup.
CFG	Context-free grammar	Grammar that handles recursion; slower but more expressive than FSM.
Schema field order	Does it matter?	Yes — first field commits; always put reasoning before answer.
Guided decoding	vLLM's name for it	Same concept, integrated into the inference server.
JSON mode	OpenAI's early version	Guarantees JSON syntax; does NOT guarantee schema match.

Further Reading

- Willard, Louf (2023). Efficient Guided Generation for LLMs — the Outlines paper.
- XGrammar paper (2024) — fast CFG-based constrained decoding.
- vLLM — Structured Outputs — inference server integration.
- OpenAI — Structured Outputs guide — API reference + gotchas.
- Instructor library — Pydantic + retries across providers.
- JSONSchemaBench (2025) — benchmarking 6 constrained decoding frameworks.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/20-structured-outputs-constrained-decoding>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/20-structured-outputs-constrained-decoding/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/20-structured-outputs-constrained-decoding>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Natural Language Inference — Textual Entailment

“t entails h” means a human reading t would conclude h is true. NLI is the task of predicting entailment / contradiction / neutral. Boring on the surface, load-bearing in production.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 05 (Sentiment Analysis), Phase 5 · 13 (Question Answering) **Time:** ~60 minutes

The Problem

You built a summarizer. It produced a summary. How do you know the summary does not contain a hallucination?

You built a chatbot. It answered “yes.” How do you know the answer is supported by the retrieved passage?

You need to classify 10,000 news articles by topic. You have no training labels. Can you reuse a model?

All three problems reduce to Natural Language Inference. NLI asks: given a premise t and a hypothesis h, is h entailed by t, contradicted, or neutral (unrelated)?

- **Hallucination check:** t = source document, h = summary claim. Not entailment = hallucination.
- **Grounded QA:** t = retrieved passage, h = generated answer. Not entailment = fabrication.
- **Zero-shot classification:** t = document, h = verbalized label (“This is about sports”). Entailment = predicted label.

One task, three production uses. This is why every RAG evaluation framework ships an NLI model under the hood.

The Concept

The three labels.

- **Entailment.** $t \rightarrow h$. “The cat is on the mat” entails “There is a cat.”
- **Contradiction.** $t \rightarrow \neg h$. “The cat is on the mat” contradicts “There is no cat.”
- **Neutral.** No inference either way. “The cat is on the mat” is neutral to “The cat is hungry.”

Not logical entailment. NLI is *natural* language inference — what a typical human reader would infer, not strict logic. “John walked his dog” entails “John has a dog” in NLI, but strict first-order logic would only admit it if you axiomatize possession.

Datasets.

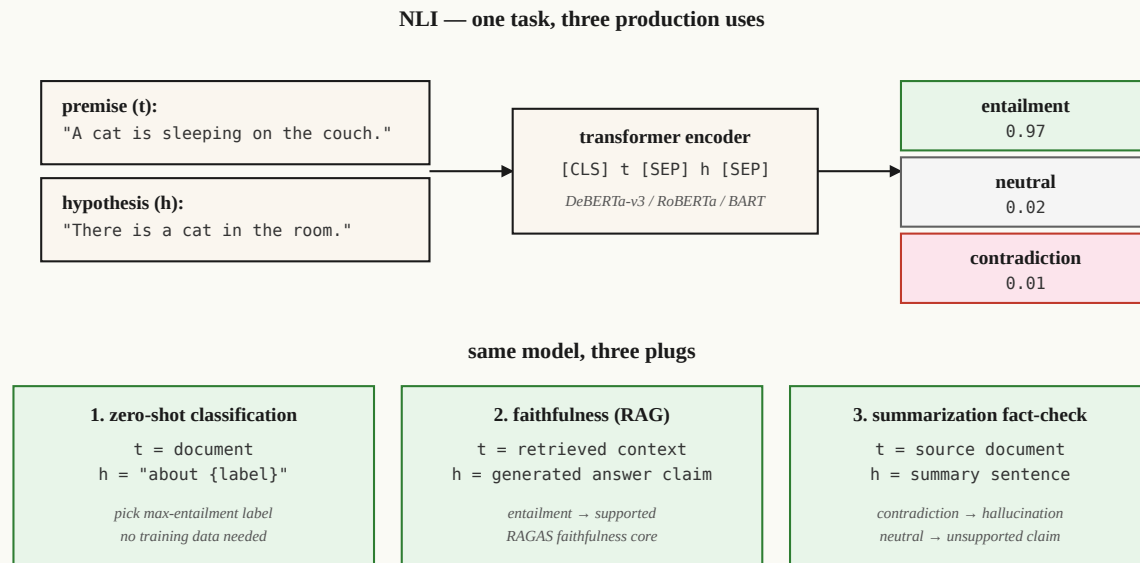


Figure 12: NLI: three-way classification, premise vs hypothesis

- **SNLI** (2015). 570k human-annotated pairs, image captions as premises. Narrow domain.
- **MultiNLI** (2017). 433k pairs across 10 genres. The standard training corpus in 2026.
- **ANLI** (2019). Adversarial NLI. Humans wrote examples specifically designed to break existing models. Harder.
- **DocNLI, ConTRoL** (2020-21). Document-length premises. Tests multi-hop and long-range inference.

The architecture. A transformer encoder (BERT, RoBERTa, DeBERTa) reads [CLS] premise [SEP] hypothesis [SEP]. The [CLS] representation feeds a 3-way softmax. Train on MNLI, evaluate on held-out benchmarks, get 90%+ accuracy on in-distribution pairs.

Zero-shot via NLI. Given a document and candidate labels, turn each label into a hypothesis ("This text is about sports"). Compute entailment probability for each. Pick the max. This is the mechanism behind Hugging Face's zero-shot-classification pipeline.

Interactive figure: nli-router. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/21-nli-textual-entailment>

Build It

Step 1: run a pretrained NLI model

```
from transformers import pipeline

nli = pipeline("text-classification",
               model="facebook/bart-large-mnli",
               top_k=None) # return all labels; replaces deprecated
↪ return_all_scores=True

premise = "The cat is sleeping on the couch."
```

```

hypothesis = "There is a cat in the room."

result = nli({"text": premise, "text_pair": hypothesis})[0]
print(result)
# [{'label': 'entailment', 'score': 0.97},
#  {'label': 'neutral', 'score': 0.02},
#  {'label': 'contradiction', 'score': 0.01}]

```

For production NLI, facebook/bart-large-mnli and microsoft/deberta-v3-large-mnli are the open defaults. DeBERTa-v3 tops leaderboards.

Step 2: zero-shot classification

```

zs = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")

text = "The stock market rallied after the central bank cut interest rates."
labels = ["finance", "sports", "politics", "technology"]

result = zs(text, candidate_labels=labels)
print(result)
# {'labels': ['finance', 'politics', 'technology', 'sports'],
#  'scores': [0.92, 0.05, 0.02, 0.01]}

```

The template is “This example is about {label}.” by default. Customize with hypothesis_template. No training data required. No fine-tuning. Works out of the box.

Step 3: faithfulness check for RAG

```

def is_faithful(answer, context, threshold=0.5):
    result = nli({"text": context, "text_pair": answer})[0]
    entail = next(s for s in result if s["label"] == "entailment")
    return entail["score"] > threshold

```

This is the core of RAGAS faithfulness. Split the generated answer into atomic claims. Check each claim against the retrieved context. Report the fraction that entail.

Step 4: hand-rolled NLI classifier (conceptual)

See code/main.py for a stdlib-only toy: premise and hypothesis are compared via lexical overlap + negation detection. Not competitive with transformer models — but it shows the shape of the task: two texts in, 3-way label out, loss = cross-entropy over {entail, contradict, neutral}.

Pitfalls

- **Hypothesis-only shortcuts.** Models can predict the label from the hypothesis alone at ~60% on SNLI because “not”, “nobody”, “never” correlate with contradiction. Strong baseline for detecting label leakage.
- **Lexical overlap heuristic.** The subsequence heuristic (“every subsequence is entailed”) passes SNLI but fails HANS/ANLI. Use adversarial benchmarks.
- **Document-length degradation.** Single-sentence NLI models drop 20+ F1 on document-length premises. Use DocNLI-trained models for long context.
- **Zero-shot template sensitivity.** “This example is about {label}” vs “{label}” vs “The topic is {label}” can swing accuracy by 10+ points. Tune the template.

- **Domain mismatch.** MNLI trains on general English. Legal, medical, and scientific text need domain-specific NLI models (e.g., SciNLI, MedNLI).

Use It

The 2026 stack:

Use case	Model
General-purpose NLI	microsoft/deberta-v3-large-mnli
Fast / edge	cross-encoder/nli-deberta-v3-base
Zero-shot classification (lightweight)	facebook/bart-large-mnli
Document-level NLI	MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli
Multilingual	MoritzLaurer/multilingual-MiniLMv2-L6-mnli-xnli
Hallucination detection in RAG	NLI layer inside RAGAS / DeepEval

The 2026 meta-pattern: NLI is the duct tape of text understanding. Whenever you need “does A support B?” or “does A contradict B?” — reach for NLI before you reach for another LLM call.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/21-nli-textual-entailment>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/21-nli-textual-entailment/code>

1. **Easy.** Run facebook/bart-large-mnli on 20 hand-crafted (premise, hypothesis, label) triples covering all three classes. Measure accuracy. Add adversarial “subsequence heuristic” traps (“I did not eat the cake” vs “I ate the cake”) and see if it breaks.
2. **Medium.** Compare the zero-shot template “This text is about {label}” against “The topic is {label}” and “{label}” on 100 AG News headlines. Report accuracy swing.
3. **Hard.** Build a RAG faithfulness checker: atomic-claim decomposition + NLI per claim. Evaluate on 50 RAG-generated answers with gold context. Measure false-positive and false-negative rates vs hand labels.

Key Terms

Term	What people say	What it actually means
NLI	Natural Language Inference	3-way classification of premise-hypothesis relationship.
RTE Entailment	Recognizing Textual Entailment “t implies h”	Older name for NLI; same task. A typical reader would conclude h is true given t.

Term	What people say	What it actually means
Contradiction	“t rules out h”	A typical reader would conclude h is false given t.
Neutral	“undecided”	No inference from t to h either way.
Zero-shot classification	NLI as classifier	Verbalize labels as hypotheses, pick max entailment.
Faithfulness	Is the answer supported?	NLI over (retrieved context, generated answer).

Further Reading

- Bowman et al. (2015). A large annotated corpus for learning natural language inference — SNLI.
- Williams, Nangia, Bowman (2017). A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference — MultiNLI.
- Nie et al. (2019). Adversarial NLI — the ANLI benchmark.
- Yin, Hay, Roth (2019). Benchmarking Zero-shot Text Classification — NLI-as-classifier.
- He et al. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention — the 2026 NLI workhorse.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/21-nli-textual-entailment>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/21-nli-textual-entailment/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/21-nli-textual-entailment>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Embedding Models — The 2026 Deep Dive

Word2Vec gave you a vector per word. Modern embedding models give you a vector per passage, cross-lingual, with sparse, dense, and multi-vector views, sized to fit your index. Pick wrong and your RAG retrieves the wrong thing.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 03 (Word2Vec), Phase 5 · 14 (Information Retrieval) **Time:** ~60 minutes

The Problem

Your RAG system retrieves the wrong passage 40% of the time. The culprit is rarely the vector database or the prompt. It is the embedding model.

Choosing an embedding in 2026 means picking across five axes:

1. **Dense vs sparse vs multi-vector.** One vector per passage, or one per token, or a sparse weighted bag of words.
2. **Language coverage.** Monolingual English models still win on English-only tasks. Multilingual models win when corpora are mixed.
3. **Context length.** 512 tokens vs 8,192 vs 32,768 — and real effective capacity is often 60-70% of the advertised max.
4. **Dimension budget.** 3,072 floats at full precision = 12 KB per vector. At 100M vectors, storage is \$1,300/month. Matryoshka truncation cuts this 4×.
5. **Open vs hosted.** Open-weight means you control the stack and data. Hosted means you trade control for always-latest.

This lesson names the tradeoffs so you can pick on evidence, not on whatever was popular last quarter.

The Concept

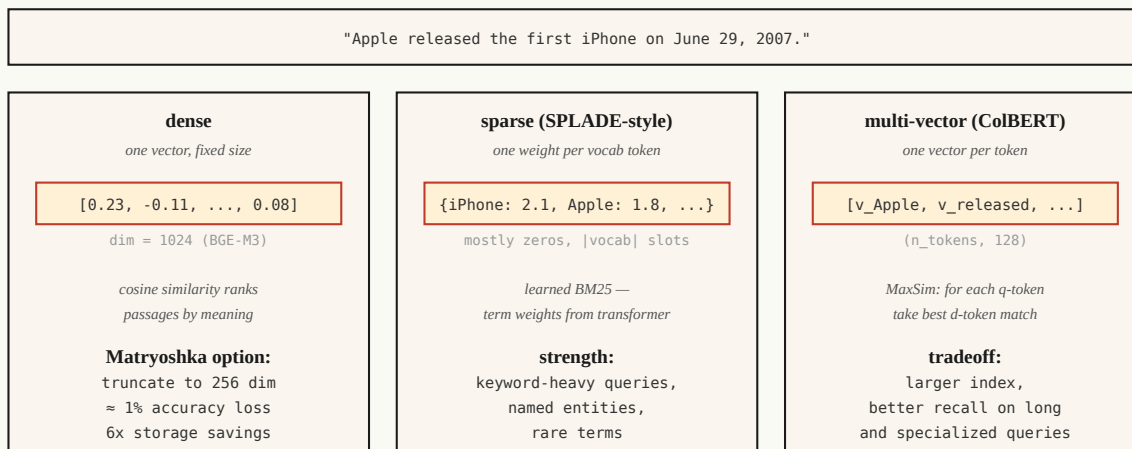
Dense embeddings. One vector per passage (usually 384-3,072 dimensions). Cosine similarity ranks passages by semantic proximity. OpenAI text-embedding-3-large, BGE-M3 dense mode, Voyage-3. Default choice.

Sparse embeddings. SPLADE-style. A transformer predicts a weight for every vocab token, then zeros out most of them. Result is a sparse vector of size |vocab|. Captures lexical matching (like BM25) but with learned term weights. Strong on keyword-heavy queries.

Multi-vector (late interaction). ColBERTv2, Jina-ColBERT. One vector per token. Scoring with MaxSim: for each query token, find the most similar document token, sum the scores. More expensive to store and score, but wins on long queries and domain-specific corpora.

BGE-M3: all three at once. Single model outputs dense, sparse, and multi-vector representations simultaneously. Each can be queried independently; scores fuse via weighted sum. The 2026 default when you want flexibility from one checkpoint.

three embedding modes — same passage, three vectors



BGE-M3 emits all three from one forward pass — fuse with weighted sum or RRF

Figure 13: Dense, sparse, and multi-vector embeddings

Matryoshka Representation Learning. Trained so the first N dimensions of the vector form a useful standalone embedding. Truncate a 1,536-dim vector to 256 dim and pay $\sim 1\%$ accuracy for $6\times$ storage savings. Supported by OpenAI text-3, Cohere v4, Voyage-4, Jina v5, Gemini Embedding 2, Nomic v1.5+.

The MTEB leaderboard tells a partial story

Massive Text Embedding Benchmark — 56 tasks across 8 task types at launch (2022), expanded to 100+ tasks in MTEB v2. In early 2026, Gemini Embedding 2 tops retrieval (67.71 MTEB-R). Cohere embed-v4 leads general (65.2 MTEB). BGE-M3 leads open-weight multilingual (63.0). The leaderboard is necessary but not sufficient — always benchmark on your domain.

The three-tier pattern

Use case	Pattern
Fast first-pass	Dense bi-encoder (BGE-M3, text-3-small)
Recall boost	Sparse (SPLADE, BGE-M3 sparse) + RRF fuse
Precision on top-50	Multi-vector (ColBERTv2) or cross-encoder reranker

Most production stacks use all three.

Interactive figure: gx-matryoshka. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/22-embedding-models-deep-dive>

Build It

Step 1: baseline — dense embeddings with Sentence-BERT

```
from sentence_transformers import SentenceTransformer
import numpy as np

encoder = SentenceTransformer("BAAI/bge-small-en-v1.5")
corpus = [
    "The first iPhone launched in 2007.",
    "Apple released the iPod in 2001.",
    "Android is an operating system from Google.",
]
emb = encoder.encode(corpus, normalize_embeddings=True)

query = "When was the iPhone released?"
q_emb = encoder.encode([query], normalize_embeddings=True)[0]
scores = emb @ q_emb
print(sorted(enumerate(scores), key=lambda x: -x[1]))
```

`normalize_embeddings=True` makes the dot product equal cosine similarity. Always set it.

Step 2: Matryoshka truncation

```
def truncate(vectors, dim):
    out = vectors[:, :dim]
    return out / np.linalg.norm(out, axis=1, keepdims=True)

emb_256 = truncate(emb, 256)
emb_128 = truncate(emb, 128)
```

Re-normalize after truncation. Nomic v1.5, OpenAI text-3, and Voyage-4 are trained so this is lossless for the first few levels. Non-Matryoshka models (original Sentence-BERT) degrade sharply when truncated.

Step 3: BGE-M3 multi-functionality

```
from FlagEmbedding import BGEM3FlagModel

model = BGEM3FlagModel("BAAI/bge-m3", use_fp16=True)

output = model.encode(
    corpus,
    return_dense=True,
    return_sparse=True,
    return_colbert_vecs=True,
)
# output["dense_vecs"]: (n_docs, 1024)
# output["lexical_weights"]: list of dict {token_id: weight}
# output["colbert_vecs"]: list of (n_tokens, 1024) arrays
```

Three indexes, one inference call. Score fusion:

```
dense_score = ... # cosine over dense_vecs
sparse_score = model.compute_lexical_matching_score(q_lex, d_lex)
```

```
colbert_score = model.colbert_score(q_col, d_col)
final = 0.4 * dense_score + 0.2 * sparse_score + 0.4 * colbert_score
```

Tune the weights on your domain.

Step 4: MTEB eval on a custom task

```
from mteb import MTEB
```

```
tasks = ["ArguAna", "SciFact", "NFCorpus"]
evaluation = MTEB(tasks=tasks)
results = evaluation.run(encoder, output_folder="./mteb-results")
```

Run your candidate models on a *representative* subset. Do not trust leaderboard rank alone — your domain matters.

Step 5: hand-rolled cosine from scratch

See code/main.py. Averaged Hashing Trick embeddings (stdlib-only). Not competitive with transformer embeddings, but shows the shape: tokenize → vector → normalize → dot product.

Pitfalls

- **Same model for query and doc.** Some models (Voyage, Jina-ColBERT) use asymmetric encoding — query and document pass through different paths. Always check the model card.
- **Missing prefix.** bge-* models need "Represent this sentence for searching relevant passages: " prepended to queries. 3-5 point recall gap if you forget.
- **Over-trimming Matryoshka.** 1,536 → 256 is usually safe. 1,536 → 64 is not. Validate on your eval set.
- **Context truncation.** Most models silently truncate inputs over their max length. Long docs need chunking (see lesson 23).
- **Ignoring latency tail.** MTEB scores hide p99 latency. A 600M model might beat a 335M model by 2 points but cost 3× more per query.

Use It

The 2026 stack:

Situation	Pick
English-only, fast, API	text-embedding-3-large or voyage-3-large
Open-weight, English	BAAI/bge-large-en-v1.5
Open-weight, multilingual	BAAI/bge-m3 or Qwen3-Embedding-8B
Long context (32k+)	Voyage-3-large, Cohere embed-v4, Qwen3-Embedding-8B
CPU-only deployment	Nomic Embed v2 (137M params, MoE)
Storage-constrained	Matryoshka-truncated + int8 quantization

Situation	Pick
Keyword-heavy queries	Add SPLADE sparse, RRF-fuse with dense

2026 pattern: start with BGE-M3 or text-3-large, evaluate on your domain with MTEB, swap if a domain-specific model wins by more than 3 points.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/22-embedding-models-deep-dive>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/22-embedding-models-deep-dive/code>

1. **Easy.** Encode 100 sentences with bge-small-en-v1.5 at full dim (384), then at Matryoshka 128. Measure MRR drop on 10 queries.
2. **Medium.** Compare BGE-M3 dense, sparse, and colbert on 500 passages from your domain. Which wins on recall@10? Does RRF fusion beat the best single mode?
3. **Hard.** Run MTEB on three candidate models across your top-2 domain tasks. Report MTEB score, p99 latency on a 100-query batch, and \$/1M queries. Pick the Pareto-optimal one.

Key Terms

Term	What people say	What it actually means
Dense embedding	The vector	One fixed-size vector per text. Cosine similarity for ranking.
Sparse embedding	Learned BM25	One weight per vocab token; mostly zeros; trained end-to-end.
Multi-vector	ColBERT-style	One vector per token; MaxSim scoring; bigger index, better recall.
Matryoshka	Russian doll trick	First N dims are a valid smaller embedding on their own.
MTEB	The benchmark	Massive Text Embedding Benchmark — 56 tasks at launch, 100+ in v2.
BEIR	The retrieval benchmark	18 zero-shot retrieval tasks; often cited for cross-domain robustness.
Asymmetric encoding	Query $\neq$ doc path	Model uses different projections for queries and documents.

Further Reading

- Reimers, Gurevych (2019). Sentence-BERT — the bi-encoder paper.

- Muennighoff et al. (2022). MTEB: Massive Text Embedding Benchmark — the leaderboard paper.
- Chen et al. (2024). BGE-M3: Multi-lingual, Multi-functionality, Multi-granularity — the unified three-mode model.
- Kusunoki et al. (2022). Matryoshka Representation Learning — the dimension-ladder training objective.
- Santhanam et al. (2022). ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction — late interaction in production.
- MTEB leaderboard on Hugging Face — live rankings.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/22-embedding-models-deep-dive>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/22-embedding-models-deep-dive/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/22-embedding-models-deep-dive>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Chunking Strategies for RAG

Chunking configuration influences retrieval quality as much as the choice of embedding model (Vectara NAACL 2025). Get chunking wrong and no amount of reranking saves you.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 14 (Information Retrieval), Phase 5 · 22 (Embedding Models) **Time:** ~60 minutes

The Problem

You put a 50-page contract into a RAG system. User asks: “What is the termination clause?” The retriever returns the cover page. Why? Because the model was trained on 512-token chunks and the termination clause sits 20 pages in, split across a page break, with no local keywords tying it to the query.

The fix is not “buy a better embedding model.” The fix is chunking. How big? Overlap? Where to split? With surrounding context?

Feb 2026 benchmarks show surprising results:

- Vectara’s 2026 study: recursive 512-token chunking beat semantic chunking 69% → 54% accuracy.
- SPLADE + Mistral-8B on Natural Questions: overlap provided zero measurable benefit.
- Context cliff: response quality drops sharply around 2,500 tokens of context.

The “obvious” answer (semantic chunking, 20% overlap, 1000 tokens) is often wrong. This lesson builds intuition for six strategies and tells you when to reach for which.

The Concept

Fixed chunking. Split every N characters or tokens. Simplest baseline. Breaks mid-sentence. Good compression, bad coherence.

Recursive. LangChain’s RecursiveCharacterTextSplitter. Try splitting on `\n\n` first, then `\n`, then `.`, then space. Falls back cleanly. The 2026 default.

Semantic. Embed each sentence. Compute cosine similarity between adjacent sentences. Split where similarity drops below a threshold. Preserves topic coherence. Slower; sometimes produces tiny 40-token fragments that hurt retrieval.

Sentence. Split on sentence boundaries. One sentence per chunk or a window of N sentences. Matches semantic chunking up to ~5k tokens at a fraction of the cost.

Parent-document. Store small child chunks for retrieval *and* the larger parent chunk for context. Retrieve by child; return parent. Degrades gracefully: bad child chunks still return reasonable parents.

Late chunking (2024). Embed the whole document at the token level first, then pool token embeddings into chunk embeddings. Preserves cross-chunk context. Works with long-context embedders (BGE-M3, Jina v3). Higher compute.

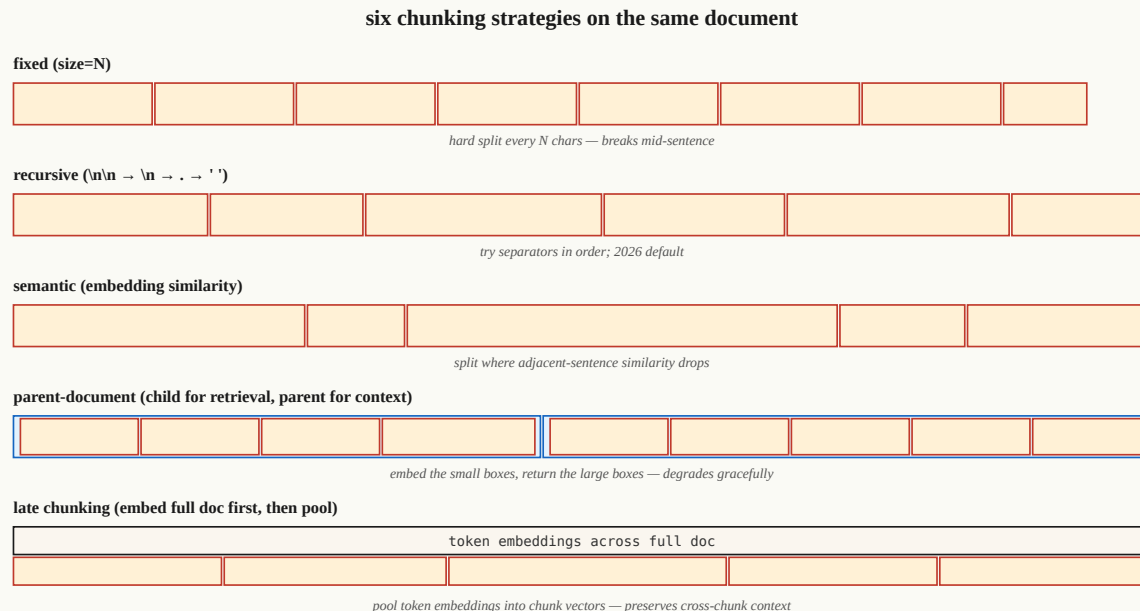


Figure 14: Six chunking strategies visualized on one passage

Contextual retrieval (Anthropic, 2024). Prepend each chunk with an LLM-generated summary of its position in the document (“This chunk is section 3.2 of the termination clauses...”). 35-50% retrieval improvement in Anthropic’s own benchmark. Expensive to index.

The rule that beats every default

Match the chunk size to the query type:

Query type	Chunk size
Factoid (“what is the CEO’s name?”)	256-512 tokens
Analytical / multi-hop	512-1024 tokens
Whole-section comprehension	1024-2048 tokens

NVIDIA’s 2026 benchmark. The chunk should be big enough to contain the answer plus local context, small enough that the retriever’s top-K returns focus on the answer rather than context noise.

Interactive figure: n5-chunk-cuts. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/23-chunking-strategies-rag>

Build It

Step 1: fixed and recursive chunking

```
def chunk_fixed(text, size=512, overlap=0):
    step = size - overlap
    return [text[i:i + size] for i in range(0, len(text), step)]
```

```

def chunk_recursive(text, size=512, seps=("\\n\\n", "\\n", ". ", " ")):
    if len(text) <= size:
        return [text]
    for sep in seps:
        if sep not in text:
            continue
        parts = text.split(sep)
        chunks = []
        buf = ""
        for p in parts:
            if len(p) > size:
                if buf:
                    chunks.append(buf)
                    buf = ""
                chunks.extend(chunk_recursive(p, size=size, seps=seps[1:] or ("
↪ ",)))
                continue
            candidate = buf + sep + p if buf else p
            if len(candidate) <= size:
                buf = candidate
            else:
                if buf:
                    chunks.append(buf)
                    buf = p
        if buf:
            chunks.append(buf)
        return [c for c in chunks if c.strip()]
    return chunk_fixed(text, size)

```

Step 2: semantic chunking

```

def chunk_semantic(text, encoder, threshold=0.6, min_chars=200, max_chars=2048):
    sentences = split_sentences(text)
    if not sentences:
        return []
    embs = encoder.encode(sentences, normalize_embeddings=True)
    chunks = [[sentences[0]]]
    for i in range(1, len(sentences)):
        sim = float(embs[i] @ embs[i - 1])
        current_len = sum(len(s) for s in chunks[-1])
        if sim < threshold and current_len >= min_chars:
            chunks.append([sentences[i]])
        else:
            chunks[-1].append(sentences[i])

    result = []
    for group in chunks:
        text_group = " ".join(group)
        if len(text_group) > max_chars:
            result.extend(chunk_recursive(text_group, size=max_chars))
        else:
            result.append(text_group)

```

```
return result
```

Tune threshold on your domain. Too high → fragments. Too low → one giant chunk.

Step 3: parent-document

```
def chunk_parent_child(text, parent_size=2048, child_size=256):
    parents = chunk_recursive(text, size=parent_size)
    mapping = []
    for p_idx, parent in enumerate(parents):
        children = chunk_recursive(parent, size=child_size)
        for child in children:
            mapping.append({"child": child, "parent_idx": p_idx, "parent": parent})
    return mapping

def retrieve_parent(child_query, mapping, encoder, top_k=3):
    child_embs = encoder.encode([m["child"] for m in mapping],
    ↪ normalize_embeddings=True)
    q_emb = encoder.encode([child_query], normalize_embeddings=True)[0]
    scores = child_embs @ q_emb
    top = np.argsort(-scores)[:top_k]
    seen, parents = set(), []
    for i in top:
        if mapping[i]["parent_idx"] not in seen:
            parents.append(mapping[i]["parent"])
            seen.add(mapping[i]["parent_idx"])
    return parents
```

Key insight: dedupe parents. Multiple children can map to the same parent; returning all would waste context.

Step 4: contextual retrieval (Anthropic pattern)

```
def contextualize_chunks(document, chunks, llm):
    context_prompts = [
        f"""<document>{document}</document>
Here is the chunk to situate: <chunk>{c}</chunk>
Write 50-100 words placing this chunk in the document's context."""
        for c in chunks
    ]
    contexts = llm.batch(context_prompts)
    return [f"{ctx}\n\n{c}" for ctx, c in zip(contexts, chunks)]
```

Index the contextualized chunks. At query time, retrieval benefits from the extra surrounding signal.

Step 5: evaluate

```
def recall_at_k(queries, corpus_chunks, encoder, k=5):
    chunk_embs = encoder.encode(corpus_chunks, normalize_embeddings=True)
    hits = 0
    for q_text, gold_idx in queries:
        q_emb = encoder.encode([q_text], normalize_embeddings=True)[0]
```

```

top = np.argsort(-(chunk_embs @ q_emb))[:k]
if any(i in gold_idx for i in top):
    hits += 1
return hits / len(queries)

```

Always benchmark. The “best” strategy for your corpus may not match any blog post.

Pitfalls

- **Chunking evaluated only on factoid queries.** Multi-hop queries reveal very different winners. Use a query-type-stratified eval set.
- **Semantic chunking without a minimum size.** Produces 40-token fragments that hurt retrieval. Always enforce `min_tokens`.
- **Overlap as cargo cult.** 2026 studies find overlap often provides zero benefit and doubles index cost. Measure, do not assume.
- **No min/max enforcement.** Chunks of 5 tokens or 5000 tokens both break retrieval. Clamp.
- **Cross-doc chunking.** Never let a chunk span two documents. Always chunk per-doc, then merge.

Use It

The 2026 stack:

Situation	Strategy
First build, unknown corpus	Recursive, 512 tokens, no overlap
Factoid QA	Recursive, 256-512 tokens
Analytical / multi-hop	Recursive, 512-1024 tokens + parent-document
Heavy cross-reference (contracts, papers)	Late chunking or contextual retrieval
Conversational / dialog corpus	Turn-level chunks + speaker metadata
Short utterances (tweets, reviews)	One document = one chunk

Start with recursive 512. Measure `recall@5` on a 50-query eval set. Tune from there.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/23-chunking-strategies-rag>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/23-chunking-strategies-rag/code>

1. **Easy.** Chunk one 20-page document with `fixed(512, 0)`, `recursive(512, 0)`, and `recursive(512, 100)`. Compare chunk counts and boundary quality.
2. **Medium.** Build a 30-query eval set over 5 documents. Measure `recall@5` for recursive, semantic, and parent-document. Which wins? Does it match the blog posts?

3. **Hard.** Implement contextual retrieval. Measure MRR improvement over baseline recursive. Report index cost (LLM calls) vs accuracy gain.

Key Terms

Term	What people say	What it actually means
Chunk	A piece of a doc	Sub-document unit that gets embedded, indexed, and retrieved.
Overlap	Safety margin	N tokens shared between adjacent chunks; often useless in 2026 benchmarks.
Semantic chunking	Smart chunking	Split where adjacent-sentence embedding similarity drops.
Parent-document	Two-level retrieval	Retrieve small children, return larger parents.
Late chunking	Chunk after embedding	Embed full doc at token level, pool into chunk vectors.
Contextual retrieval	Anthropic's trick	LLM-generated summary prepended to each chunk before indexing.
Context cliff	2500-token wall	Quality drop observed around 2.5k context tokens in RAG (Jan 2026).

Further Reading

- Yepes et al. / LangChain — Recursive Character Splitting docs — the default in production.
- Vectara (2024, NAACL 2025). Chunking configurations analysis — chunking matters as much as embedding choice.
- Jina AI — Late Chunking in Long-Context Embedding Models (2024) — the late chunking paper.
- Anthropic — Contextual Retrieval — 35-50% retrieval improvement with LLM-generated context prefixes.
- NVIDIA 2026 chunk-size benchmark — Premai summary — chunk size by query type.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/23-chunking-strategies-rag>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/23-chunking-strategies-rag/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/23-chunking-strategies-rag>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Coreference Resolution

“She called him. He did not answer. The doctor was at lunch.” Three references to two people and nobody is named. Coreference resolution figures out who is who.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 06 (NER), Phase 5 · 07 (POS & Parsing) **Time:** ~60 minutes

The Problem

Extract every mention of Apple Inc. from a 300-word article. Easy when the article says “Apple.” Hard when it says “the company,” “they,” “Cupertino’s technology giant,” or “Jobs’s firm.” Without resolving these mentions to the same entity, your NER pipeline misses 60-80% of the mentions.

Coreference resolution links every expression that refers to the same real-world entity into one cluster. It is the glue between surface-level NLP (NER, parsing) and downstream semantics (IE, QA, summarization, KG).

Why it matters in 2026:

- Summarization: “The CEO announced...” vs “Tim Cook announced...” — the summary should name the CEO.
- Question answering: “Who did she call?” requires resolving “she.”
- Information extraction: a knowledge graph with “PER1 founded Apple” and “Jobs founded Apple” as separate entries is wrong.
- Multi-document IE: merging mentions across articles about the same event is cross-document coreference.

The Concept

The task. Input: a document. Output: a clustering of mentions (spans) where each cluster refers to one entity.

Mention types.

- **Named entity.** “Tim Cook”
- **Nominal.** “the CEO”, “the company”
- **Pronominal.** “he”, “she”, “they”, “it”
- **Appositive.** “Tim Cook, Apple’s CEO,”

Architectures.

1. **Rule-based (Hobbs, 1978).** Syntactic-tree-based pronoun resolution using grammar rules. Good baseline. Surprisingly hard to beat on pronouns.
2. **Mention-pair classifier.** For every pair of mentions (m_i , m_j), predict whether they corefer. Cluster by transitive closure. Standard pre-2016.
3. **Mention-ranking.** For each mention, rank candidate antecedents (including “no antecedent”). Pick the top.

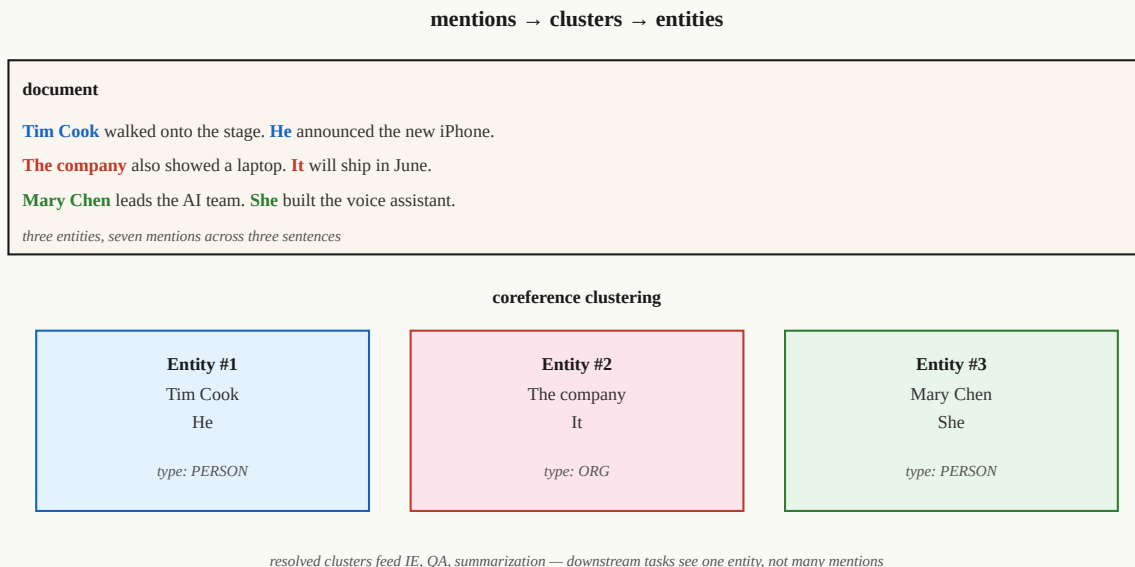


Figure 15: Coreference clustering: mentions → entities

4. **Span-based end-to-end (Lee et al., 2017).** Transformer encoder. Enumerate all candidate spans up to a length cap. Predict mention scores. Predict antecedent-probability for each span. Cluster greedily. The modern default.
5. **Generative (2024+).** Prompt an LLM: “List every pronoun in this text and its antecedent.” Works well on easy cases, struggles on long documents and rare referents.

The evaluation metrics. Five standard metrics (MUC, B³, CEAF, BLANC, LEA) because no single metric captures clustering quality. Report the average of the first three as CoNLL F1. State-of-the-art in 2026 on CoNLL-2012: ~83 F1.

Known hard cases.

- Definite descriptions referring to entities introduced pages earlier.
- Bridging anaphora (“the wheels” → a previously mentioned car).
- Zero anaphora in languages like Chinese and Japanese.
- Cataphora (pronoun before referent): “When **she** walked in, Mary smiled.”

Interactive figure: coref-links. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/24-coreference-resolution>

Build It

Step 1: pretrained neural coreference (AllenNLP / spaCy-experimental)

```
import spacy
nlp = spacy.load("en_coreference_web_trf") # experimental model
doc = nlp("Apple announced new products. The company said they would ship soon.")
for cluster in doc._.coref_clusters:
    print(cluster, "->", [m.text for m in cluster])
```

On a longer document, you get something like: - Cluster 1: [Apple, The company, they] - Cluster 2: [new products]

Step 2: rule-based pronoun resolver (teaching)

See `code/main.py` for a `stdlib`-only implementation:

1. Extract mentions: named entities (capitalized spans), pronouns (dict lookup), definite descriptions (“the X”).
2. For each pronoun, look at the previous K mentions and score them by:
 - gender/number agreement (heuristic)
 - recency (closer wins)
 - syntactic role (subjects preferred)
3. Link the highest-scoring antecedent.

Not competitive with neural models. But it shows the search space and the decisions an end-to-end model must make.

Step 3: using LLMs for coreference

```
prompt = f"""Text: {text}
```

```
List every pronoun and noun phrase that refers to a person or company.
Cluster them by what they refer to. Output JSON:
[{"entity": "Apple", "mentions": ["Apple", "the company", "it"]}], ...]
"""
```

Two failure modes to watch. First, LLMs over-merge (“him” and “her” referring to two distinct people). Second, LLMs silently drop mentions in long documents. Always verify with span-offset checks.

Step 4: evaluation

The standard `conll-2012` script computes MUC, B³, CEAF-φ4 and reports the average. For an in-house eval, start with span-level precision and recall on your annotated test set, then add mention-linking F1.

Pitfalls

- **Singleton explosion.** Some systems report every mention as its own cluster. B³ is lenient. MUC punishes this. Always check all three metrics.
- **Pronouns in long context.** Performance drops ~15 F1 on documents over 2,000 tokens. Chunk carefully.
- **Gender assumptions.** Hard-coded gender rules break on non-binary referents, organizations, animals. Use learned models or neutral scoring.
- **LLM drift on long docs.** A single API call cannot reliably cluster mentions across 50+ paragraphs. Use sliding-window + merge.

Use It

The 2026 stack:

Situation	Pick
English, single document	en_coreference_web_trf (spaCy-experimental) or AllenNLP neural coref
Multilingual	SpanBERT / XLM-R trained on OntoNotes or Multilingual CoNLL
Cross-document event coref	Specialized end-to-end models (2025-26 SOTA)
Quick LLM baseline	GPT-4o / Claude with structured-output coref prompt
Production dialog systems	Rule-based fallback + neural primary + manual review for critical slots

The integration pattern that ships in 2026: run NER first, run coref, merge coref clusters into NER entities. Downstream tasks see one entity per cluster, not one entity per mention.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/24-coreference-resolution>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/24-coreference-resolution/code>

1. **Easy.** Run the rule-based resolver in `code/main.py` on 5 hand-crafted paragraphs. Measure mention-link accuracy against ground truth.
2. **Medium.** Use a pretrained neural coref model on a news article. Compare clusters against your own manual annotation. Where did it fail?
3. **Hard.** Build a coref-enhanced NER pipeline: NER first, then merge via coref clusters. Measure entity-coverage improvement vs NER-only on 100 articles.

Key Terms

Term	What people say	What it actually means
Mention	A reference	A span of text that refers to an entity (name, pronoun, noun phrase).
Antecedent	What "it" refers to	The earlier mention a later one corefers with.
Cluster	The entity's mentions	Set of mentions that all refer to the same real-world entity.
Anaphora	Backward reference	Later mention refers to earlier ("he" → "John").
Cataphora	Forward reference	Earlier mention refers to later ("When he arrived, John...").

Term	What people say	What it actually means
Bridging	Implicit reference	“I bought a car. The wheels were bad.” (wheels of THAT car.)
CoNLL F1	The number on leaderboards	Average of MUC, B ³ , CEAF- ϕ 4 F1 scores.

Further Reading

- Jurafsky & Martin, SLP3 Ch. 26 — Coreference Resolution and Entity Linking — canonical textbook chapter.
- Lee et al. (2017). End-to-end Neural Coreference Resolution — span-based end-to-end.
- Joshi et al. (2020). SpanBERT — pretraining that improves coref.
- Pradhan et al. (2012). CoNLL-2012 Shared Task — the benchmark.
- Hobbs (1978). Resolving Pronoun References — the rule-based classic.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/24-coreference-resolution>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/24-coreference-resolution/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/24-coreference-resolution>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Entity Linking & Disambiguation

NER found “Paris.” Entity linking decides: Paris, France? Paris Hilton? Paris, Texas? Paris (the Trojan prince)? Without linking, your knowledge graph stays ambiguous.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 06 (NER), Phase 5 · 24 (Coreference Resolution) **Time:** ~60 minutes

The Problem

A sentence reads: “Jordan beat the press.” Your NER tags “Jordan” as PERSON. Good. But *which* Jordan?

- Michael Jordan (basketball)?
- Michael B. Jordan (actor)?
- Michael I. Jordan (Berkeley ML professor — yes, this confusion is real in ML papers)?
- Jordan (the country)?
- Jordan (Hebrew first name)?

Entity linking (EL) resolves each mention to a unique entry in a knowledge base: Wikidata, Wikipedia, DBpedia, or your domain KB. Two subtasks:

1. **Candidate generation.** Given “Jordan,” which KB entries are plausible?
2. **Disambiguation.** Given the context, which candidate is the right one?

Both steps are learnable. Both are benchmarked. The combined pipeline has been stable for a decade — what changes is the quality of the disambiguator.

The Concept

Candidate generation. Given the mention surface form (“Jordan”), look up candidates in an alias index. Wikipedia alias dictionaries cover most named entities: “JFK” → John F. Kennedy, Jacqueline Kennedy, JFK airport, JFK (movie). Typical index returns 10-30 candidates per mention.

Disambiguation: three approaches.

1. **Prior + context (Milne & Witten, 2008).** $P(\text{entity} \mid \text{mention}) \times \text{context-similarity}(\text{entity}, \text{text})$. Works well, fast, no training.
2. **Embedding-based (ESS / REL / Blink).** Encode mention + context. Encode each candidate’s description. Pick max cosine. The 2020-2024 default.
3. **Generative (GENRE, 2021; LLM-based, 2023+).** Decode the entity’s canonical name token-by-token. Constrained to a trie of valid entity names so output is guaranteed to be a valid KB id.

End-to-end vs pipeline. Modern models (ELQ, BLINK, ExtEnD, GENRE) run NER + candidate generation + disambiguation in one pass. Pipeline systems still dominate in production because you can swap components.

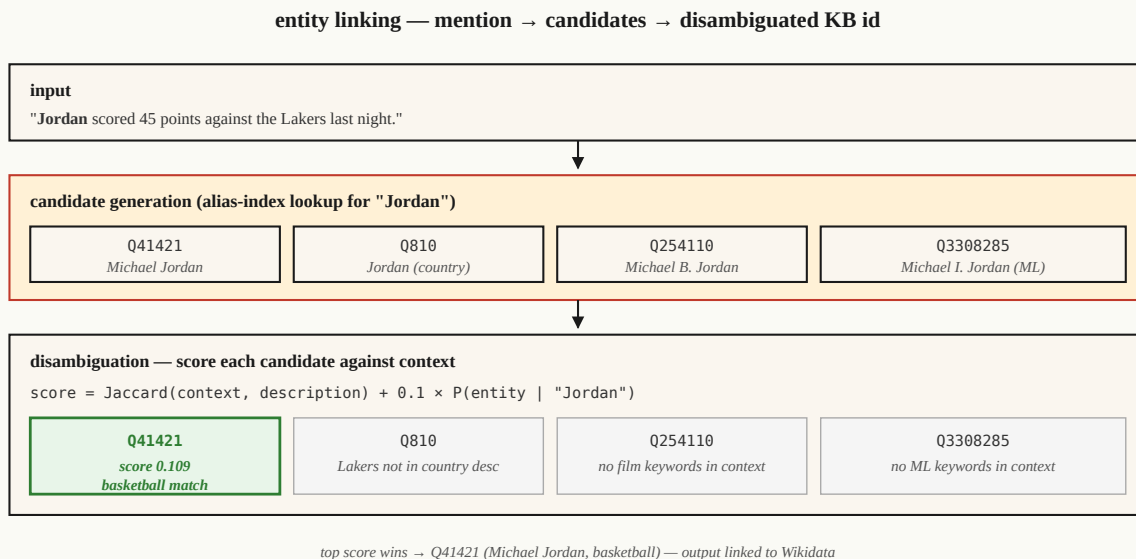


Figure 16: Entity linking pipeline: mention → candidates → disambiguated entity

The two measurements

- **Mention recall (candidate gen).** Fraction of gold mentions where the correct KB entry appears in the candidate list. Floor for the whole pipeline.
- **Disambiguation accuracy / F1.** Given correct candidates, how often the top-1 is right.

Always report both. A system with 99% disambiguation on 80% candidate recall is an 80% pipeline.

Interactive figure: gx-entity-linking. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/25-entity-linking>

Build It

Step 1: build an alias index from Wikipedia redirects

```
alias_to_entities = {
    "jordan": ["Q41421 (Michael Jordan)", "Q810 (Jordan, country)", "Q254110 (Michael B. Jordan)"],
    "paris": ["Q90 (Paris, France)", "Q663094 (Paris, Texas)", "Q55411 (Paris Hilton)"],
    "apple": ["Q312 (Apple Inc.)", "Q89 (apple, fruit)"],
}
```

Wikipedia alias data: ~18M (alias, entity) pairs. Download from Wikidata dumps. Store as inverted index.

Step 2: context-based disambiguation

```
def disambiguate(mention, context, alias_index, entity_desc):
    candidates = alias_index.get(mention.lower(), [])
```

```

if not candidates:
    return None, 0.0
context_words = set(tokenize(context))
best, best_score = None, -1
for entity_id in candidates:
    desc_words = set(tokenize(entity_desc[entity_id]))
    union = len(context_words | desc_words)
    score = len(context_words & desc_words) / union if union else 0.0
    if score > best_score:
        best, best_score = entity_id, score
return best, best_score

```

The Jaccard overlap is a toy. Replace with cosine similarity on embeddings (see code/main.py step-2 for the transformer version).

Step 3: embedding-based (BLINK-style)

```

from sentence_transformers import SentenceTransformer
encoder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

def embed_mention(text, mention_span):
    start, end = mention_span
    marked = f"{text[:start]} [MENTION] {text[start:end]} [/MENTION] {text[end:]}"
    return encoder.encode([marked], normalize_embeddings=True)[0]

def embed_entity(entity_id, description):
    return encoder.encode([f"{entity_id}: {description}"],
        ↪ normalize_embeddings=True)[0]

```

At index time, embed every KB entity once. At query time, embed the mention + context once, dot-product against the candidate pool, pick max.

Step 4: generative entity linking (concept)

GENRE decodes the entity's Wikipedia title character-by-character. Constrained decoding (see lesson 20) ensures only valid titles can be output. Tight integration with a KB-backed trie. The modern descendant is REL-GEN and LLM-prompted EL with structured output.

```

prompt = f"""Text: {text}
Mention: {mention}
List the best Wikipedia title for this mention.
Respond with JSON: {{"title": "..."}}"""

```

Combined with a whitelist (Outlines choice), this is the simplest EL pipeline to ship in 2026.

Step 5: evaluate on AIDA-CoNLL

AIDA-CoNLL is the standard EL benchmark: 1,393 Reuters articles, 34k mentions, Wikipedia entities. Report in-KB accuracy (P@1) and out-of-KB NIL-detection rate.

Pitfalls

- **NIL handling.** Some mentions are not in the KB (emerging entities, obscure people). Systems must predict NIL instead of guessing the wrong entity. Measured separately.

- **Mention boundary errors.** Upstream NER misses partial spans (“Bank of America” tagged as just “Bank”). EL recall drops.
- **Popularity bias.** Trained systems over-predict frequent entities. A mention of “Michael I. Jordan” on an ML paper often links to basketball Jordan.
- **Cross-lingual EL.** Mapping mentions in Chinese text to English Wikipedia entities. Requires a multilingual encoder or a translation step.
- **KB staleness.** New companies, events, people are not in last year’s Wikipedia dump. Production pipelines need a refresh loop.

Use It

The 2026 stack:

Situation	Pick
General-purpose English + Wikipedia Cross-lingual, KB = Wikipedia LLM-friendly, few mentions/day	BLINK or REL mGENRE Prompt Claude/GPT-4 with candidate list + constrained JSON
Domain-specific KB (medical, legal)	Custom BERT with KB-aware retrieval + fine-tune on domain AIDA-style set
Extremely low-latency	Exact-match prior only (Milne-Witten baseline)
Research SOTA	GENRE / ExtEnD / generative LLM-EL

Production pattern that ships in 2026: NER → coref → EL on each mention → collapse clusters to one canonical entity per cluster. Output: one KB id per entity in the document, not one per mention.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/25-entity-linking>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/25-entity-linking/code>

1. **Easy.** Implement the prior+context disambiguator in `code/main.py` on 10 ambiguous mentions (Paris, Jordan, Apple). Hand-label the correct entity. Measure accuracy.
2. **Medium.** Encode 50 ambiguous mentions with a sentence transformer. Embed each candidate’s description. Compare embedding-based disambiguation to Jaccard context overlap.
3. **Hard.** Build a 1k-entity domain KB (e.g. employees + products in your company). Implement NER + EL end-to-end. Measure precision and recall on 100 held-out sentences.

Key Terms

Term	What people say	What it actually means
Entity linking (EL)	Link to Wikipedia	Map a mention to a unique KB entry.
Candidate generation	Who could it be?	Return a shortlist of plausible KB entries for a mention.
Disambiguation	Pick the right one	Score candidates using context, pick the winner.
Alias index	The lookup table	Map from surface form → candidate entities.
NIL	Not in KB	Explicit prediction that no KB entry matches.
KB	Knowledge base	Wikidata, Wikipedia, DBpedia, or your domain KB.
AIDA-CoNLL	The benchmark	1,393 Reuters articles with gold entity links.

Further Reading

- Milne, Witten (2008). Learning to Link with Wikipedia — the foundational prior+context approach.
- Wu et al. (2020). Zero-shot Entity Linking with Dense Entity Retrieval (BLINK) — the embedding-based workhorse.
- De Cao et al. (2021). Autoregressive Entity Retrieval (GENRE) — generative EL with constrained decoding.
- Hoffart et al. (2011). Robust Disambiguation of Named Entities in Text (AIDA) — the benchmark paper.
- REL: An Entity Linker Standing on the Shoulders of Giants (2020) — the open production stack.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/25-entity-linking>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/25-entity-linking/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/25-entity-linking>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Relation Extraction & Knowledge Graph Construction

NER found the entities. Entity linking anchored them. Relation extraction finds the edges between them. A knowledge graph is the sum of nodes, edges, and their provenance.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 06 (NER), Phase 5 · 25 (Entity Linking) **Time:** ~60 minutes

The Problem

An analyst reads: “Tim Cook became CEO of Apple in 2011.” Four facts:

- (Tim Cook, role, CEO)
- (Tim Cook, employer, Apple)
- (Tim Cook, start_date, 2011)
- (Apple, type, Organization)

Relation Extraction (RE) turns free text into structured triples (subject, relation, object). Aggregate across a corpus and you have a knowledge graph. Aggregate and query and you have a reasoning substrate for RAG, analytics, or compliance audits.

The 2026 problem: LLMs extract relations enthusiastically. Too enthusiastically. They hallucinate triples that the source text does not support. Without provenance, you cannot tell real triples from plausible fiction. The 2026 answer is AEVS-style anchor-and-verify pipelines.

The Concept

Triple form. (subject_entity, relation_type, object_entity). Relations come from a closed ontology (Wikidata properties, FIBO, UMLS) or an open set (OpenIE-style, anything goes).

Three extraction approaches.

1. **Rule / pattern-based.** Hearst patterns: “X such as Y” $\rightarrow$ (Y, isA, X). Plus hand-crafted regex. Brittle, precise, explainable.
2. **Supervised classifier.** Given two entity mentions in a sentence, predict the relation from a fixed set. Trained on TACRED, ACE, KBP. Standard 2015–2022.
3. **Generative LLM.** Prompt the model to emit triples. Works out of the box. Needs provenance, or hallucinates plausible-looking junk.

AEVS (Anchor-Extraction-Verification-Supplement, 2026). The current hallucination-mitigation framework:

- **Anchor.** Identify every entity span and relation-phrase span with exact positions.
- **Extract.** Generate triples linked to anchor spans.
- **Verify.** Match each triple element back to the source text; reject anything unsupported.
- **Supplement.** A coverage pass ensures no anchored span is dropped.

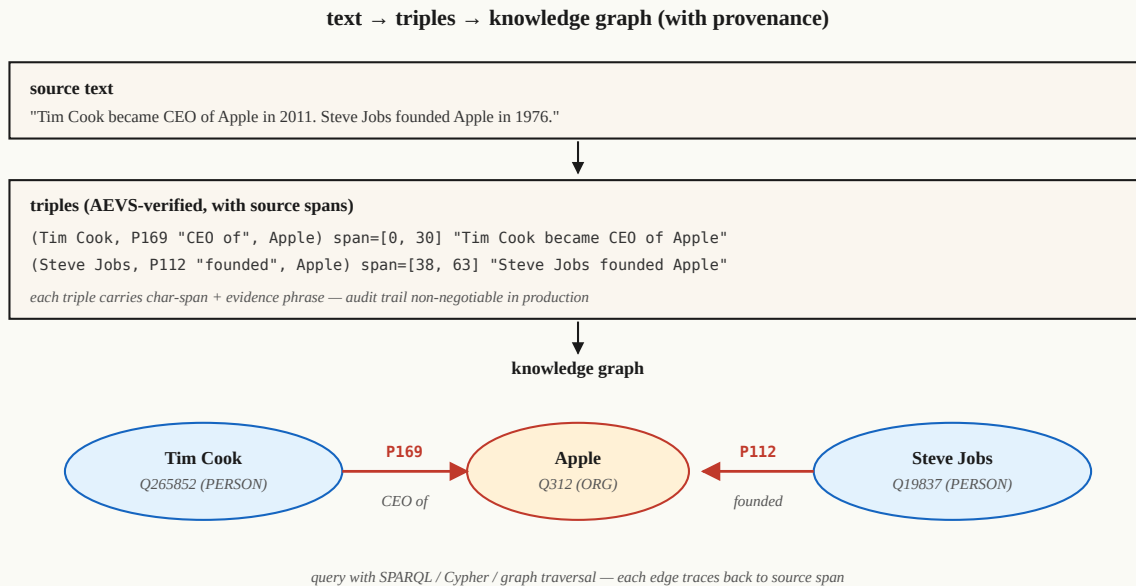


Figure 17: Text → triples → knowledge graph

Hallucinations drop sharply. Requires more compute but is auditable.

The open-vs-closed tradeoff.

- **Closed ontology.** Fixed property list (e.g., Wikidata's 11,000+ properties). Predictable. Queryable. Hard to invent.
- **Open IE.** Any verbal phrase becomes a relation. High recall. Low precision. Messy to query.

Production KGs usually mix: open IE for discovery, then canonicalize relations onto a closed ontology before merging into the main graph.

Interactive figure: relation-triples. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/26-relation-extraction-kg>

Build It

Step 1: pattern-based extraction

```

PATTERNS = [
    (r"(?P<s>[A-Z]\w+) (?[:is|was]) (?[:a|an|the]) (?P<o>[A-Z]?[w+]", "isA"),
    (r"(?P<s>[A-Z]\w+) (?[:is|was]) born in (?P<o>[w+]", "bornIn"),
    (r"(?P<s>[A-Z]\w+) works? (?[:at|for]) (?P<o>[A-Z]\w+)", "worksAt"),
    (r"(?P<s>[A-Z]\w+) founded (?P<o>[A-Z]\w+)", "founded"),
]
  
```

See code/main.py for the full toy extractor. Hearst patterns still ship in domain-specific pipelines because they are debuggable.

Step 2: supervised relation classification

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

tok = AutoTokenizer.from_pretrained("Babelscape/rebel-large")
model =
    ↪ AutoModelForSequenceClassification.from_pretrained("Babelscape/rebel-large")

text = "Tim Cook was born in Alabama. He later became CEO of Apple."
encoded = tok(text, return_tensors="pt", truncation=True)
output = model.generate(**encoded, max_length=200)
triples = tok.batch_decode(output, skip_special_tokens=False)
```

REBEL is a seq2seq relation extractor: text in, triples out, already in Wikidata property ids. Fine-tuned on distant-supervision data. Standard open-weights baseline.

Step 3: LLM-prompted extraction with anchoring

```
prompt = f"""Extract (subject, relation, object) triples from the text.
For each triple, include the exact character span in the source text.
```

```
Text: {text}
```

```
Output JSON:
```

```
[{"subject": {"text": "...", "span": [start, end]}},
  "relation": "...",
  "object": {"text": "...", "span": [start, end]}}, ...]
```

```
Only include triples fully supported by the text. No inference beyond what is
    ↪ stated.
"""
```

Verify every returned span against the source. Reject anything where `text[start:end] != triple_entity`. This is the AEVS “verify” step in its minimal form.

Step 4: canonicalize onto a closed ontology

```
RELATION_MAP = {
    "is the CEO of": "P169",          # "chief executive officer"
    "was born in":   "P19",           # "place of birth"
    "founded":       "P112",          # "founded by" (inverted subject/object)
    "works at":      "P108",          # "employer"
}
```

```
def canonicalize(relation):
    rel_low = relation.lower().strip()
    if rel_low in RELATION_MAP:
        return RELATION_MAP[rel_low]
    return None  # drop unmapped open relations or route to manual review
```

Canonicalization is often 60-80% of the engineering work. Budget for it.

Step 5: build a small graph and query

```
triples = extract(text)
graph = {}
for s, r, o in triples:
    graph.setdefault(s, []).append((r, o))

def neighbors(node, relation=None):
    return [(r, o) for r, o in graph.get(node, []) if relation is None or r ==
            ↪ relation]

print(neighbors("Tim Cook", relation="P108"))    # -> [(P108, Apple)]
```

This is the atom of every RAG-over-KG system. Scale it with RDF triple stores (Blazegraph, Virtuoso), property graphs (Neo4j), or vector-augmented graph stores.

Pitfalls

- **Coreference before RE.** “He founded Apple” — RE needs to know who “he” is. Run coref first (lesson 24).
- **Entity canonicalization.** “Apple Inc” and “Apple” must resolve to the same node. Entity linking first (lesson 25).
- **Hallucinated triples.** LLMs emit triples the text does not support. Enforce span verification.
- **Relation canonicalization drift.** Open IE relations are inconsistent (“was born in,” “came from,” “is a native of”). Collapse to canonical ids or the graph is unqueryable.
- **Temporal errors.** “Tim Cook is CEO of Apple” — true now, false in 2005. Many relations are time-bounded. Use qualifiers (P580 start time, P582 end time in Wikidata).
- **Domain mismatch.** REBEL trained on Wikipedia. Legal, medical, and scientific text often need domain-fine-tuned RE models.

Use It

The 2026 stack:

Situation	Pick
Fast production, general domain	REBEL or LlamaPred with Wikidata canonicalization
Domain-specific (biomed, legal)	SciREX-style domain fine-tune + custom ontology
LLM-prompted, audited output	AEVS pipeline: anchor → extract → verify → supplement
High-volume news IE	Pattern-based + supervised hybrid
Building a KG from scratch	Open IE + manual canonicalization pass
Temporal KG	Extract with qualifiers (start/end time, point in time)

The integration pattern: NER → coref → entity linking → relation extraction → ontology mapping → graph load. Every stage is a potential quality gate.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/26-relation-extraction-kg>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/26-relation-extraction-kg/code>

1. **Easy.** Run the pattern extractor in `code/main.py` on 5 news-article sentences. Hand-check precision.
2. **Medium.** Use REBEL (or a small LLM) on the same sentences. Compare triples. Which extractor has higher precision? Higher recall?
3. **Hard.** Build the AEVS pipeline: extract with LLM + verify spans against source. Measure hallucination rate before vs after the verify step on 50 Wikipedia-style sentences.

Key Terms

Term	What people say	What it actually means
Triple	Subject-relation-object	(s, r, o) tuple that is the atomic unit of a KG.
Open IE	Extract anything	Open-vocabulary relation phrases; high recall, low precision.
Closed ontology	Fixed schema	Bounded set of relation types (Wikidata, UMLS, FIBO).
Canonicalization	Normalize everything	Map surface names / relations to canonical ids.
AEVS	Grounded extraction	Anchor-Extraction-Verification-Supplement pipeline (2026).
Provenance	Source-of-truth link	Every triple carries a doc id + char-span to its source.
Distant supervision	Cheap labels	Align text with an existing KG to create training data.

Further Reading

- Mintz et al. (2009). Distant supervision for relation extraction without labeled data — the distant-supervision paper.
- Huguet Cabot, Navigli (2021). REBEL: Relation Extraction By End-to-end Language generation — seq2seq RE workhorse.
- Wadden et al. (2019). Entity, Relation, and Event Extraction with Contextualized Span Representations (DyGIE++) — joint IE.
- AEVS — Anchor-Extraction-Verification-Supplement framework — 2026 hallucination-mitigation design.
- Wikidata SPARQL tutorial — canonical graph queries.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/26-relation-extraction-kg>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/26-relation-extraction-kg/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/26-relation-extraction-kg>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

LLM Evaluation — RAGAS, DeepEval, G-Eval

Exact-match and F1 miss semantic equivalence. Human review does not scale. LLM-as-judge is the production answer — with enough calibration to trust the number.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 13 (Question Answering), Phase 5 · 14 (Information Retrieval) **Time:** ~75 minutes

The Problem

Your RAG system answers: “June 29th, 2007.” The gold reference is: “June 29, 2007.” Exact Match scores 0. F1 scores ~75%. A human would score 100%.

Now multiply by 10,000 test cases. Multiply again by every change to the retriever, chunking, prompt, or model. You need an evaluator that understands meaning, runs cheaply at scale, does not lie about regressions, and surfaces the right failure modes.

2026 has three frameworks that own this problem.

- **RAGAS.** Retrieval-Augmented Generation ASsessment. Four RAG metrics (faithfulness, answer-relevance, context-precision, context-recall) with NLI + LLM-judge backends. Research-backed, lightweight.
- **DeepEval.** Pytest for LLMs. G-Eval, task-completion, hallucination, bias metrics. CI/CD-native.
- **G-Eval.** A method (and a DeepEval metric): LLM-as-judge with chain-of-thought, custom criteria, 0-1 score.

All three lean on LLM-as-judge. This lesson builds intuition for the method and the trust layer around it.

The Concept

LLM-as-judge. Replace a static metric with an LLM that scores outputs given a rubric. Given (query, context, answer), prompt a judge LLM: “Score 0-1 on faithfulness.” Return the score.

Why it works: LLMs approximate human judgment at a tiny fraction of the cost. GPT-4o-mini at ~\$0.003 per scored case enables 1000-sample regression eval runs for under \$5.

Why it fails silently:

1. **Judge bias.** Judges prefer longer answers, answers from their own model family, answers that match the prompt style.
2. **JSON parsing failures.** Bad JSON → NaN score → silently excluded from the aggregate. RAGAS users know this pain. Gate with try/except + explicit failure mode.
3. **Drift over model versions.** Upgrading the judge changes every metric. Freeze judge model + version.

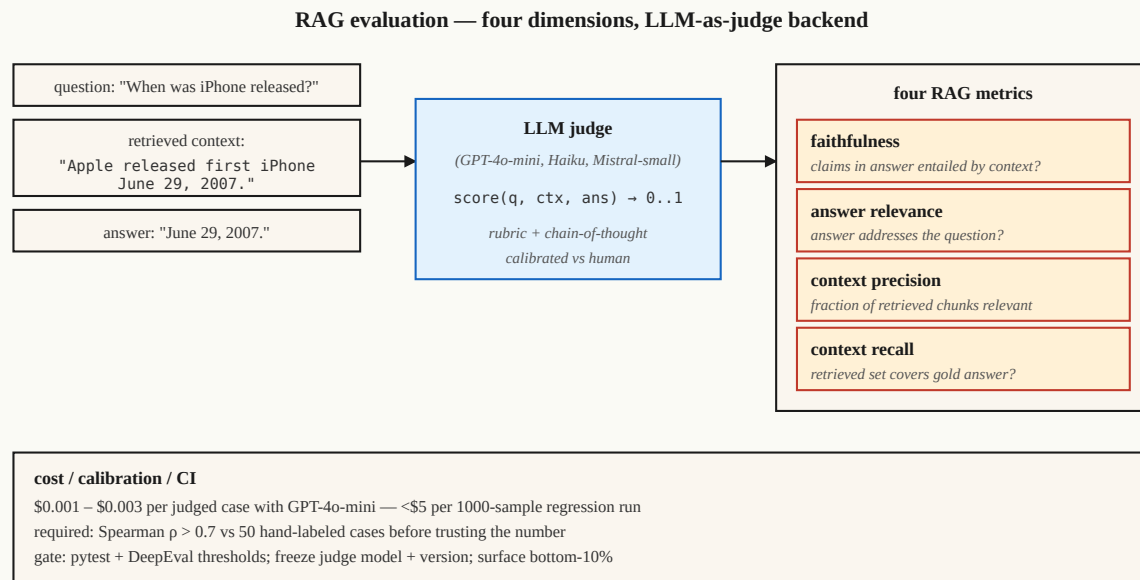


Figure 18: Four evaluation dimensions, LLM-as-judge architecture

The RAG four.

Metric	Question	Backend
Faithfulness	Does each claim in the answer come from the retrieved context?	NLI-based entailment
Answer relevance	Does the answer address the question?	Generate hypothetical questions from answer; compare to real question
Context precision	Of retrieved chunks, what fraction were relevant?	LLM-judge
Context recall	Did retrieval return everything needed?	LLM-judge against gold answer

G-Eval. Define a custom criterion: “Did the answer cite the correct source?” The framework auto-expands into chain-of-thought evaluation steps, then scores 0-1. Good for domain-specific quality dimensions RAGAS does not cover.

Calibration. Never trust the raw judge score until you have a correlation against human labels. Run 100 hand-labeled examples. Plot judge vs human. Compute Spearman rho. If $\rho < 0.7$, your judge rubric needs work.

Interactive figure: n5-judge-gauge. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/27-llm-evaluation-frameworks>

Build It

Step 1: faithfulness with NLI (RAGAS-style)

```

from typing import Callable
from transformers import pipeline

nli = pipeline("text-classification",
               model="MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli",
               top_k=None)

# `llm` is any callable: prompt str -> generated str.
# Example: llm = lambda p: client.messages.create(model="claude-haiku-4-5",
#          ↪ ...).content[0].text
LLM = Callable[[str], str]

def atomic_claims(answer: str, llm: LLM) -> list[str]:
    prompt = f"""Break this answer into simple factual claims (one per line):
{answer}
"""
    return llm(prompt).splitlines()

def faithfulness(answer: str, context: str, llm: LLM) -> float:
    claims = atomic_claims(answer, llm)
    if not claims:
        return 0.0
    supported = 0
    for claim in claims:
        result = nli({"text": context, "text_pair": claim})[0]
        entail = next((s for s in result if s["label"] == "entailment"), None)
        if entail and entail["score"] > 0.5:
            supported += 1
    return supported / len(claims)

```

Decompose the answer into atomic claims. NLI-check each claim against the retrieved context. Faithfulness = fraction supported.

Step 2: answer relevance

```

import numpy as np
from sentence_transformers import SentenceTransformer

# encoder: any model implementing .encode(texts, normalize_embeddings=True) ->
#          ndarray
# e.g., encoder = SentenceTransformer("BAAI/bge-small-en-v1.5")

def answer_relevance(question: str, answer: str, encoder, llm: LLM, n: int = 3) ->
# float:
    prompt = f"Write {n} questions this answer could be the answer to:\n{answer}"
    generated = [line for line in llm(prompt).splitlines() if line.strip()][:n]
    if not generated:
        return 0.0

```

```

q_emb = np.asarray(encoder.encode([question], normalize_embeddings=True)[0])
g_embs = np.asarray(encoder.encode(generated, normalize_embeddings=True))
sims = [float(q_emb @ g_emb) for g_emb in g_embs]
return sum(sims) / len(sims)

```

If the answer implies different questions than the one asked, relevance drops.

Step 3: G-Eval custom metric

```

from deepeval.metrics import GEval
from deepeval.test_case import LLMTestCaseParams, LLMTestCase

metric = GEval(
    name="Correctness",
    criteria="The answer should be factually accurate and match the expected
    ↪ output.",
    evaluation_steps=[
        "Read the expected output.",
        "Read the actual output.",
        "List factual claims in the actual output.",
        "For each claim, mark supported or unsupported by the expected output.",
        "Return score = fraction supported.",
    ],
    evaluation_params=[LLMTestCaseParams.INPUT, LLMTestCaseParams.ACTUAL_OUTPUT,
    ↪ LLMTestCaseParams.EXPECTED_OUTPUT],
)

test = LLMTestCase(input="When was the first iPhone released?",
    actual_output="June 29th, 2007.",
    expected_output="June 29, 2007.")
metric.measure(test)
print(metric.score, metric.reason)

```

The evaluation steps are the rubric. Explicit steps are more stable than implicit “score 0-1” prompts.

Step 4: CI gate

```

import deepeval
from deepeval.metrics import FaithfulnessMetric, ContextualRelevancyMetric

def test_rag_system():
    cases = load_regression_cases()
    faith = FaithfulnessMetric(threshold=0.85)
    rel = ContextualRelevancyMetric(threshold=0.7)
    for case in cases:
        faith.measure(case)
        assert faith.score >= 0.85, f"faithfulness regression on {case.id}"
        rel.measure(case)
        assert rel.score >= 0.7, f"relevancy regression on {case.id}"

```

Ship as a pytest file. Run on every PR. Block merges on regressions.

Step 5: toy eval from scratch

See `code/main.py`. Stdlib-only approximations of faithfulness (overlap of answer claims with context) and relevance (overlap of answer tokens with question tokens). Not production. Shows the shape.

Pitfalls

- **No calibration.** A judge with 0.3 correlation to human labels is noise. Require a calibration run before shipping.
- **Self-evaluation.** Using the same LLM to generate and judge inflates scores by 10-20%. Use a different model family for the judge.
- **Positional bias in pairwise judging.** Judges prefer the first option presented. Always randomize order and run both.
- **Raw aggregate hides failures.** Mean score 0.85 often hides 5% catastrophic failures. Always inspect the bottom quantile.
- **Golden dataset rot.** Unversioned eval sets that drift over time break longitudinal comparison. Tag the dataset with every change.
- **LLM cost.** At scale, judge calls dominate cost. Use the cheapest model that meets calibration threshold. GPT-4o-mini, Claude Haiku, Mistral-small.

Use It

The 2026 stack:

Use case	Framework
RAG quality monitoring	RAGAS (4 metrics)
CI/CD regression gates	DeepEval + pytest
Custom domain criteria	G-Eval within DeepEval
Online live-traffic monitoring	RAGAS with reference-free mode
Human-in-the-loop spot checks	LangSmith or Phoenix with annotation UI
Red-teaming / safety eval	Promptfoo + DeepEval

Typical stack: RAGAS for monitoring, DeepEval for CI, G-Eval for novel dimensions. Run all three; they disagree usefully.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/27-llm-evaluation-frameworks>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/27-llm-evaluation-frameworks/code>

1. **Easy.** Use RAGAS on 10 RAG examples with known hallucinations. Verify the faithfulness metric catches each one.
2. **Medium.** Hand-label 50 QA answers 0-1 for correctness. Score with G-Eval. Measure Spearman rho between judge and human.

3. **Hard.** Build a pytest CI gate with DeepEval. Intentionally regress the retriever. Verify the gate fails. Add bottom-quantile alerting via threshold check on the lowest 10%.

Key Terms

Term	What people say	What it actually means
LLM-as-judge	Scoring with an LLM	Prompt a judge model to score outputs 0-1 given a rubric.
RAGAS	The RAG metric library	Open-source eval framework with 4 reference-free RAG metrics.
Faithfulness	Is the answer grounded?	Fraction of answer claims entailed by retrieved context.
Context precision	Were retrieved chunks relevant?	Fraction of top-K chunks that actually mattered.
Context recall	Did retrieval find everything?	Fraction of gold-answer claims supported by retrieved chunks.
G-Eval	Custom LLM judge	Rubric + chain-of-thought eval steps + 0-1 score.
Calibration	Trust but verify	Spearman correlation between judge score and human score.

Further Reading

- Es et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation — the RAGAS paper.
- Liu et al. (2023). G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment — the G-Eval paper.
- DeepEval docs — open production stack.
- Zheng et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena — biases, calibration, limits.
- MLflow GenAI Scorer — unifying framework that integrates RAGAS, DeepEval, Phoenix.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/27-llm-evaluation-frameworks>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/27-llm-evaluation-frameworks/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/27-llm-evaluation-frameworks>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Long-Context Evaluation — NIAH, RULER, LongBench, MRCR

Gemini 3 Pro advertises 10M tokens of context. At 1M tokens, 8-needle MRCR drops to 26.3%. Advertised $\neq$ usable. Long-context evaluation tells you the actual capacity of the model you are shipping on.

Type: Learn **Languages:** Python **Prerequisites:** Phase 5 · 13 (Question Answering), Phase 5 · 23 (Chunking Strategies) **Time:** ~60 minutes

The Problem

You have a 200-page contract. The model claims a 1M-token context. You paste the contract in and ask: “What is the termination clause?” The model answers — but answers from the cover page because the termination clause sits at 120k tokens deep, past where the model actually attends.

This is the 2026 context-capacity gap. Spec sheets say 1M or 10M. Reality says 60-70% of that is usable, and “usable” depends on the task.

- **Retrieval (single needle in haystack):** near-perfect up to the advertised max on frontier models.
- **Multi-hop / aggregation:** degrades sharply past ~128k on most models.
- **Reasoning over dispersed facts:** the first task to fail.

Long-context evaluation measures these axes. This lesson names the benchmarks, what each actually measures, and how to build a custom needle test for your domain.

The Concept

Needle-in-a-Haystack (NIAH, 2023). Place a fact (“the magic word is pineapple”) at a controlled depth in a long context. Ask the model to retrieve it. Sweep depth $\times$ length. The original long-context benchmark. Frontier models now saturate this; it is a necessary but not sufficient baseline.

RULER (Nvidia, 2024). 13 task types across 4 categories: retrieval (single / multi-key / multi-value), multi-hop tracing (variable tracking), aggregation (common word frequency), QA. Configurable context length (4k to 128k+). Reveals models that saturate NIAH but fail on multi-hop. In the 2024 release, only half of 17 models claiming 32k+ context maintained quality at 32k.

LongBench v2 (2024). 503 multiple-choice questions, 8k-2M word contexts, six task categories: single-doc QA, multi-doc QA, long in-context learning, long dialogue, code repo, long structured data. The production benchmark for real-world long-context behavior.

MRCR (Multi-Round Coreference Resolution). Multi-turn coreference at scale. 8-needle, 24-needle, 100-needle variants. Exposes how many facts a model can juggle before attention degrades.

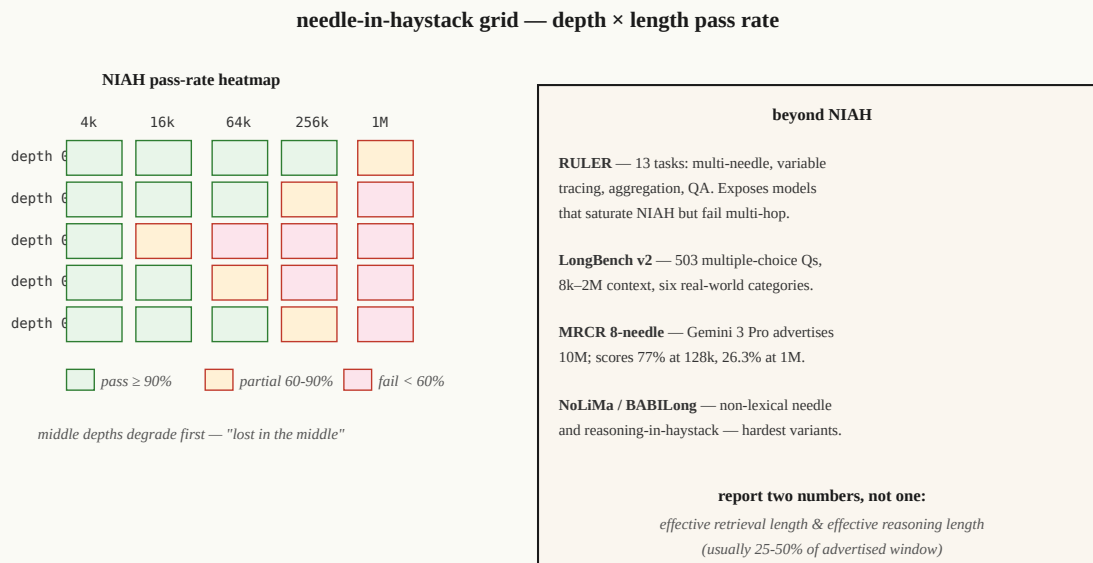


Figure 19: NIAH baseline, RULER multi-task, LongBench holistic

NoLiMa. “Non-lexical needle.” The needle and the query share no literal overlap; retrieval requires one step of semantic reasoning. Harder than NIAH.

HELMET. Concatenates many documents, asks a question from any one. Tests selective attention.

BABILong. Embeds bAbI reasoning chains inside irrelevant haystacks. Tests reasoning-in-a-haystack, not just retrieval.

What to actually report

- **Advertised context window.** The spec-sheet number.
- **Effective retrieval length.** NIAH pass at some threshold (e.g., 90%).
- **Effective reasoning length.** Multi-hop or aggregation pass at that threshold.
- **Degradation curve.** Accuracy vs context length, plotted per task type.

Two numbers for your spec sheet: retrieval-effective and reasoning-effective. Usually the reasoning-effective is 25-50% of the advertised window.

Interactive figure: gx-niah-decay. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/28-long-context-evaluation>

Build It

Step 1: a custom NIAH for your domain

See code/main.py. The skeleton:

```
def build_haystack(filler_text, needle, depth_ratio, total_tokens):
    if not (0.0 <= depth_ratio <= 1.0):
        raise ValueError(f"depth_ratio must be in [0, 1], got {depth_ratio}")
    if total_tokens <= 0:
```

```

    raise ValueError(f"total_tokens must be positive, got {total_tokens}")

    filler_tokens = tokenize(filler_text)
    needle_tokens = tokenize(needle)
    if not filler_tokens:
        raise ValueError("filler_text produced no tokens")

    # Repeat filler until long enough to fill the haystack body.
    body_len = max(total_tokens - len(needle_tokens), 0)
    while len(filler_tokens) < body_len:
        filler_tokens = filler_tokens + filler_tokens
    filler_tokens = filler_tokens[:body_len]

    insert_at = min(int(body_len * depth_ratio), body_len)
    haystack = filler_tokens[:insert_at] + needle_tokens +
    ↪ filler_tokens[insert_at:]
    return " ".join(haystack)

```

```

def score_niah(model, haystack, question, expected):
    answer = model.complete(f"Context: {haystack}\nQ: {question}\nA:",
    ↪ max_tokens=50)
    return 1 if expected.lower() in answer.lower() else 0

```

Sweep $\text{depth_ratio} \in \{0, 0.25, 0.5, 0.75, 1.0\} \times \text{total_tokens} \in \{1\text{k}, 4\text{k}, 16\text{k}, 64\text{k}\}$. Plot the heatmap. That is the NIAH card for your target model.

Step 2: a multi-needle variant

```

def build_multi_needle(filler, needles, total_tokens):
    depths = [0.1, 0.4, 0.7]
    chunks = [filler[:int(total_tokens * 0.1)]]
    for depth, needle in zip(depths, needles):
        chunks.append(needle)
        next_chunk = filler[int(total_tokens * depth): int(total_tokens * (depth +
    ↪ 0.3))]
        chunks.append(next_chunk)
    return " ".join(chunks)

```

Questions like “What are the three magic words?” require retrieving all three. Single-needle success does not predict multi-needle success.

Step 3: multi-hop variable tracing (RULER-style)

```

haystack = """X1 = 42. ... (filler) ... X2 = X1 + 10. ... (filler) ... X3 = X2 *
    ↪ 2."""
question = "What is X3?"

```

The answer requires chaining three assignments. Frontier models at 128k often drop to 50-70% accuracy here.

Step 4: LongBench v2 on your stack

```

from datasets import load_dataset
longbench = load_dataset("THUDM/LongBench-v2")

```

```
def eval_model_on_longbench(model, subset="single-doc-qa"):
    tasks = [x for x in longbench["test"] if x["task"] == subset]
    correct = 0
    for x in tasks:
        answer = model.complete(x["context"] + "\n\nQ: " + x["question"],
    ↪ max_tokens=20)
        if normalize(answer) == normalize(x["answer"]):
            correct += 1
    return correct / len(tasks)
```

Report per-category accuracy. Aggregate scores hide big task-level differences.

Pitfalls

- **NIAH-only evaluation.** Passing NIAH at 1M tokens says nothing about multi-hop. Always run RULER or a custom multi-hop test.
- **Uniform depth sampling.** Many implementations only test depth=0.5. Test depth=0, 0.25, 0.5, 0.75, 1.0 — the “lost in the middle” effect is real.
- **Lexical overlap with filler.** If the needle shares keywords with the filler, retrieval becomes trivial. Use NoLiMa-style non-overlapping needles.
- **Ignoring latency.** 1M-token prompts take 30-120 seconds to prefill. Measure time-to-first-token alongside accuracy.
- **Vendor-self-reported numbers.** OpenAI, Google, Anthropic all publish their own scores. Always re-run independently on your use case.

Use It

The 2026 stack:

Situation	Benchmark
Quick sanity check	Custom NIAH at 3 depths × 3 lengths
Model selection for production	RULER (13 tasks) at your target length
Real-world QA quality	LongBench v2 single-doc-QA subset
Multi-hop reasoning	BABILong or custom variable-tracing
Conversational / dialogue	MRCR 8-needle at your target length
Model upgrade regression	Fixed in-house NIAH + RULER harness, run on every new model

Rule of thumb for production: never trust a context window until you have NIAH + 1 reasoning task at your intended length.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/28-long-context-evaluation>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/28-long-context-evaluation>

context-evaluation/code

1. **Easy.** Build a NIAH with 3 depths (0.25, 0.5, 0.75) $\times$ 3 lengths (1k, 4k, 16k). Run on any model. Plot pass rate as a 3 $\times$ 3 heatmap.
2. **Medium.** Add a 3-needle variant. Measure retrieval of all 3 at each length. Compare to single-needle pass rate at the same length.
3. **Hard.** Construct a variable-tracing task ($X1 \rightarrow X2 \rightarrow X3$, with 3 hops) embedded in 64k of filler. Measure accuracy across 3 frontier models. Report effective reasoning length per model.

Key Terms

Term	What people say	What it actually means
NIAH	Needle in haystack	Plant a fact in filler, ask the model to retrieve it.
RULER	NIAH on steroids	13 task types across retrieval / multi-hop / aggregation / QA.
Effective context	The real capacity	Length at which accuracy still holds above threshold.
Lost in the middle	Depth bias	Models under-attend to content in the middle of long inputs.
Multi-needle	Many facts at once	Multiple plants; tests attention juggling, not retrieval alone.
MRCR	Multi-round coref	8, 24, or 100-needle coreference; exposes attention saturation.
NoLiMa	Non-lexical needle	Needle and query share no literal tokens; requires reasoning.

Further Reading

- Kamradt (2023). Needle in a Haystack analysis — the original NIAH repo.
- Hsieh et al. (2024). RULER: What’s the Real Context Size of Your Long-Context LMs? — the multi-task benchmark.
- Bai et al. (2024). LongBench v2 — real-world long-context eval.
- Modarressi et al. (2024). NoLiMa: Non-lexical needles — harder needles.
- Kuratov et al. (2024). BABILong — reasoning-in-haystack.
- Liu et al. (2024). Lost in the Middle: How Language Models Use Long Contexts — the depth-bias paper.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/28-long-context-evaluation>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/28-long-context-evaluation/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/28-long-context-evaluation>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Dialogue State Tracking

“I want a cheap restaurant in the north... actually make it moderate... and add Italian.” Three turns, three state updates. DST keeps the slot-value dict in sync so the booking works.

Type: Build **Languages:** Python **Prerequisites:** Phase 5 · 17 (Chatbots), Phase 5 · 20 (Structured Outputs) **Time:** ~75 minutes

The Problem

In a task-oriented dialogue system, the user’s goal is encoded as a set of slot-value pairs: {cuisine: italian, area: north, price: moderate}. Every user turn can add, change, or remove a slot. The system must read the whole conversation and output the current state correctly.

Get a single slot wrong and the system books the wrong restaurant, schedules the wrong flight, or charges the wrong card. DST is the hinge between what the user said and what the backend executes.

Why it still matters in 2026 despite LLMs:

- Compliance-sensitive domains (banking, healthcare, airline booking) require deterministic slot values, not free-form generation.
- Tool-use agents still need slot resolution before calling APIs.
- Multi-turn correction is harder than it looks: “actually no, make it Thursday.”

The modern pipeline: classical DST concepts + LLM extractors + structured-output guardrails.

The Concept

Task structure. A schema defines domains (restaurant, hotel, taxi) and their slots (cuisine, area, price, people). Each slot can be empty, filled with a value from a closed set (price: {cheap, moderate, expensive}), or a free-form value (name: “The Copper Kettle”).

Two DST formulations.

- **Classification.** For each (slot, candidate_value) pair, predict yes/no. Works for closed-vocab slots. Standard pre-2020.
- **Generation.** Given the dialogue, generate slot values as free text. Works for open-vocab slots. The modern default.

Metric. Joint Goal Accuracy (JGA) — the fraction of turns where *every* slot is correct. All-or-nothing. MultiWOZ 2.4 leaderboard tops around 83% in 2026.

Architectures.

1. **Rule-based (slot regex + keyword).** Strong baseline for narrow domains. Debuggable.
2. **TripPy / BERT-DST.** Copy-based generation with BERT encoding. Pre-LLM standard.

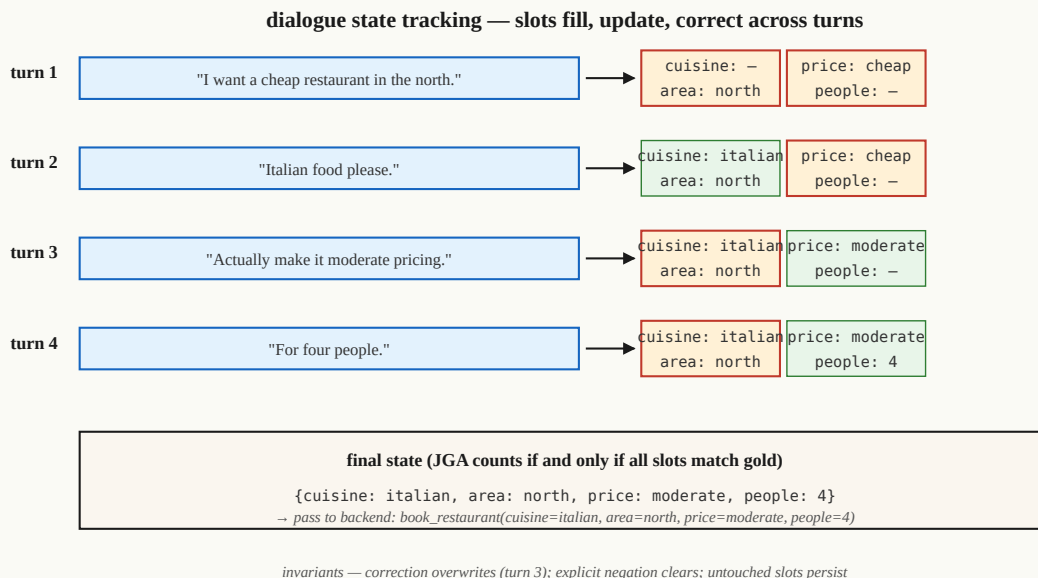


Figure 20: DST: dialog history → slot-value state

3. **LDST (LLaMA + LoRA)**. Instruction-tuned LLM with domain-slot prompting. Reaches ChatGPT-level quality on MultiWOZ 2.4.
4. **Ontology-free (2024-26)**. Skip the schema; generate slot names and values directly. Handles open domains.
5. **Prompt + structured output (2024-26)**. LLM with Pydantic schema + constrained decoding. 5 lines of code, production-ready.

The classic failure modes

- **Co-reference across turns**. "Let's stay with the first option." Needs to resolve which option.
- **Over-write vs append**. User says "add Italian." Do you replace cuisine or append?
- **Implicit confirmations**. "OK cool" — did that accept the offered booking?
- **Correction**. "Actually make it 7 pm." Must update time without clearing other slots.
- **Coreference to previous system utterance**. "Yes, that one." Which "that"?

Interactive figure: n5-slot-tracker. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/29-dialogue-state-tracking>

Build It

Step 1: rule-based slot extractor

See code/main.py. Regex + synonym dictionaries cover 70% of canonical utterances in narrow domains:

```
CUISINE_SYNONYMS = {
    "italian": ["italian", "pasta", "pizza", "italy"],
    "chinese": ["chinese", "chow mein", "noodles"],
```

```
}
```

```
def extract_cuisine(utterance):
    for canonical, synonyms in CUISINE_SYNONYMS.items():
        if any(syn in utterance.lower() for syn in synonyms):
            return canonical
    return None
```

Brittle outside the canonical vocabulary. Works for deterministic slot confirmations.

Step 2: state update loop

```
def update_state(state, utterance):
    new_state = dict(state)
    for slot, extractor in SLOT_EXTRACTORS.items():
        value = extractor(utterance)
        if value is not None:
            new_state[slot] = value
    for slot in NEGATION_CLEARS:
        if is_negated(utterance, slot):
            new_state[slot] = None
    return new_state
```

Three invariants:

- Never reset a slot the user did not touch.
- Explicit negation (“never mind the cuisine”) must clear.
- User correction (“actually...”) must overwrite, not append.

Step 3: LLM-driven DST with structured output

```
from pydantic import BaseModel
from typing import Literal, Optional
import instructor

class RestaurantState(BaseModel):
    cuisine: Optional[Literal["italian", "chinese", "indian", "thai", "any"]] =
    ↪ None
    area: Optional[Literal["north", "south", "east", "west", "center"]] = None
    price: Optional[Literal["cheap", "moderate", "expensive"]] = None
    people: Optional[int] = None
    day: Optional[str] = None

def llm_dst(history, llm):
    prompt = f"""You track the slot values of a restaurant booking across turns.
Dialogue so far:
{render(history)}

Update the state based on the latest user turn. Output only the JSON state."""
    return llm(prompt, response_model=RestaurantState)
```

Instructor + Pydantic guarantees a valid state object. No regex, no schema mismatches, no hallucinated slots.

Step 4: JGA evaluation

```
def joint_goal_accuracy(predicted_states, gold_states):
    correct = sum(1 for p, g in zip(predicted_states, gold_states) if p == g)
    return correct / len(predicted_states)
```

Calibrate: what fraction of turns does the system get ALL slots right? For MultiWOZ 2.4, top 2026 systems: 80-83%. Your in-domain system should exceed that on your narrow vocabulary or the LLM baseline beats you.

Step 5: handling correction

```
CORRECTION_CUES = {"actually", "no wait", "on second thought", "change that to"}
```

```
def is_correction(utterance):
    return any(cue in utterance.lower() for cue in CORRECTION_CUES)
```

On a detected correction, overwrite the last-updated slot rather than appending. Hard to get right without LLM help. The modern pattern: always let the LLM regenerate the whole state from history rather than incrementally updating — this naturally handles corrections.

Pitfalls

- **Full-history regeneration cost.** Letting the LLM regenerate state each turn costs $O(n^2)$ total tokens. Cap history or summarize older turns.
- **Schema drift.** Adding new slots post-hoc breaks old training data. Version your schema.
- **Case sensitivity.** “Italian” vs “italian” vs “ITALIAN” — normalize everywhere.
- **Implicit inheritance.** If the user has previously specified “for 4 people,” a new request for a different time should not clear people. Always pass the full history.
- **Free-form vs closed-set.** Names, times, and addresses need free-form slots; cuisines and areas are closed. Mix both in the schema.

Use It

The 2026 stack:

Situation	Approach
Narrow domain (one or two intents)	Rule-based + regex
Broad domain, labeled data available	LDST (LLaMA + LoRA on MultiWOZ-style data)
Broad domain, no labels, prod-ready	LLM + Instructor + Pydantic schema
Spoken / voice	ASR + normalizer + LLM-DST
Multi-domain booking flow	Schema-guided LLM with per-domain Pydantic models
Compliance-sensitive	Rule-based primary, LLM fallback with confirmation flow

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/29-dialogue-state-tracking>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/29-dialogue-state-tracking/code>

1. **Easy.** Build the rule-based state tracker in `code/main.py` for 3 slots (cuisine, area, price). Test on 10 hand-crafted dialogues. Measure JGA.
2. **Medium.** Same dataset with Instructor + Pydantic + a small LLM. Compare JGA. Inspect the hardest turns.
3. **Hard.** Implement both and route: rule-based primary, LLM fallback when rule-based emits <2 slots with confidence. Measure the combined JGA and inference cost per turn.

Key Terms

Term	What people say	What it actually means
DST	Dialogue state tracking	Maintain the slot-value dict across dialogue turns.
Slot	Unit of user intent	Named parameter the backend needs (cuisine, date).
Domain	The task area	Restaurant, hotel, taxi — sets of slots.
JGA	Joint Goal Accuracy	Fraction of turns where every slot is correct. All-or-nothing.
MultiWOZ	The benchmark	Multi-domain WOZ dataset; standard DST evaluation.
Ontology-free DST	No schema	Generate slot names and values directly, no fixed list.
Correction	"Actually..."	Turn that overwrites a previously-filled slot.

Further Reading

- Budzianowski et al. (2018). MultiWOZ — A Large-Scale Multi-Domain Wizard-of-Oz — the canonical benchmark.
- Feng et al. (2023). Towards LLM-driven Dialogue State Tracking (LDST) — LLaMA + LoRA instruction tuning for DST.
- Heck et al. (2020). TripPy — A Triple Copy Strategy for Value Independent Neural Dialog State Tracking — the copy-based DST workhorse.
- King, Flanigan (2024). Unsupervised End-to-End Task-Oriented Dialogue with LLMs — EM-based unsupervised TOD.
- MultiWOZ leaderboard — canonical DST results.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/29-dialogue-state-tracking>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/05-nlp-foundations-to-advanced/29-dialogue-state-tracking/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/05-nlp-foundations-to-advanced/29-dialogue-state-tracking>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Part II — Transformers Deep Dive

Course phase 07. Live edition with animated figures and quizzes: <https://aiengineeringfromscratch.com/catalog.html>

Why Transformers — The Problems with RNNs

RNNs process tokens one at a time. Transformers process all tokens at once. That single architectural bet changed every scaling curve in deep learning after 2017.

Type: Learn **Languages:** Python **Prerequisites:** Phase 3 (Deep Learning Core), Phase 5 · 09 (Sequence-to-Sequence), Phase 5 · 10 (Attention Mechanism) **Time:** ~45 minutes

The Problem

Before 2017, every state-of-the-art sequence model on the planet — language, translation, speech — was a recurrent neural network. LSTMs and GRUs won ImageNet-equivalent translation benchmarks for half a decade. They were the only tool anyone had.

They had three fatal weaknesses. Sequential computation meant you could not parallelize along the time axis: token $t+1$ needs the hidden state from token t . A 1,024-token sequence meant 1,024 serial steps on a GPU that can do 1,000,000 floating-point ops per cycle. Training wall-clock time scaled linearly with sequence length on hardware designed for parallelism.

Vanishing gradients meant information 50 tokens back was already compressed through 50 non-linearities. Gated recurrent units (LSTM, GRU) softened the crush but never eliminated it. Long-range dependencies — “the book I read last summer on a plane to Kyoto was...” — routinely failed.

Fixed-width hidden states meant the encoder squeezed the entire source sequence into a single vector before the decoder saw anything. Doesn’t matter if the source is 5 tokens or 500; the bottleneck is the same shape.

The 2017 paper “Attention Is All You Need” proposed something radical: drop recurrence entirely. Let every position attend to every other position in parallel. Train in one big matrix multiplication instead of 1,024 sequential ones.

The result dominates every modality by 2026. Language (GPT-5, Claude 4, Llama 4), vision (ViT, DINOv2, SAM 3), audio (Whisper), biology (AlphaFold 3), robotics (RT-2). Same block, different inputs.

The Concept

Recurrence as a bottleneck. An RNN computes $h_t = f(h_{t-1}, x_t)$. Each step depends on the previous. You cannot compute h_5 before h_4 . On modern GPUs with 10,000+ parallel cores, this wastes 99% of the silicon on a long sequence.

Attention as a broadcast. Self-attention computes $output_i = \sum_j (a_{ij} * v_j)$ for every pair (i, j) simultaneously. The whole $N \times N$ attention matrix fills in one batched matmul. No step depends on another. GPUs love it.

The speedup is not a constant. It is the difference between $O(N)$ serial depth and $O(1)$ serial depth. In practice, transformers train 5-10× faster per epoch on matched hardware

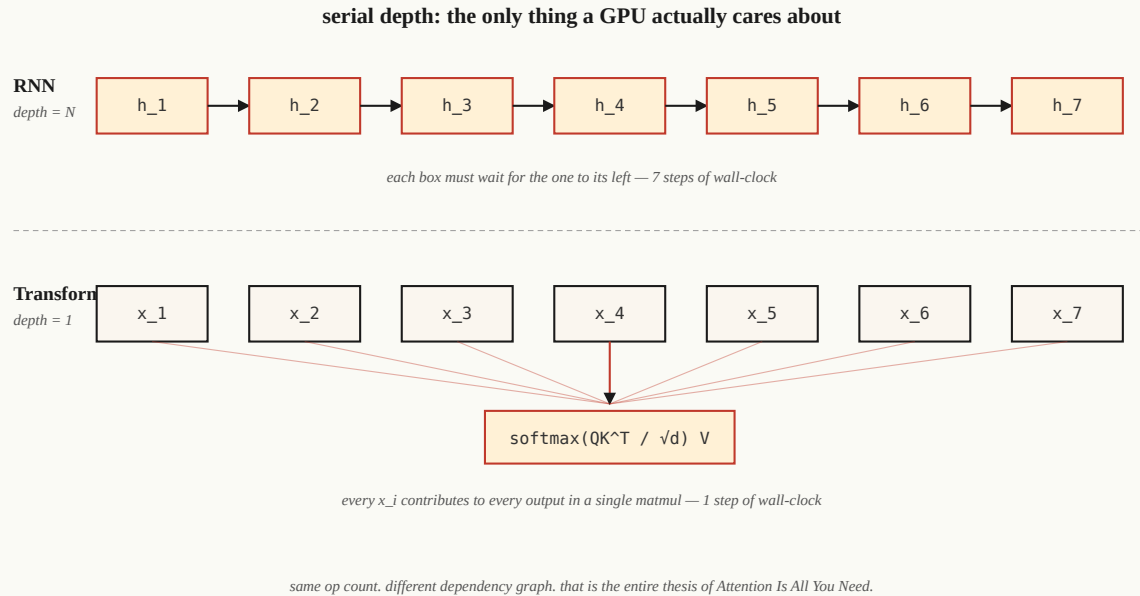


Figure 21: RNN sequential compute vs Transformer parallel attention

at $N=512$, and the gap widens with sequence length until you hit the $O(N^2)$ memory wall of attention (which Flash Attention later fixed — see Lesson 12).

What transformers cost. Attention memory scales as $O(N^2)$. For 2K context, fine. For 128K context, you need sliding windows, RoPE extrapolation, Flash Attention tiling, or linear attention variants. Recurrence was $O(N)$ in both time and memory; transformers trade time for memory and then win the time back through parallelism.

The inductive bias shift. RNNs assume locality and recency. Transformers assume nothing — every pair is a candidate for attention. That is why transformers need more data to train well but scale further once they have it. Chinchilla (2022) formalized this: given enough tokens, a transformer always beats an RNN of equal parameter count.

Interactive figure: rnn-vs-parallel. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/01-why-transformers>

Build It

No neural network here — we simulate the core bottleneck numerically so you feel the gap on your laptop.

Step 1: measure serial depth

See code/main.py. We build two functions. One encodes a sequence as a chain of additions (serial, like an RNN). One encodes it as a parallel reduction (broadcast, like attention). Same math, different dependency graph.

```
def rnn_style(xs):
    h = 0.0
    for x in xs:
```

```

    h = 0.9 * h + x    # can't parallelize: h depends on previous h
    return h

def attention_style(xs):
    return sum(xs) / len(xs) # every x is independent

```

We time both on sequences up to 100,000 elements. The RNN version is $O(N)$ and a single CPU pipeline. Even in pure Python, the attention-style reduction beats it at length $\geq 1,000$ because Python's `sum()` is implemented in C and iterates without interpreter overhead per step.

Step 2: count theoretical operations

Both algorithms do N adds. The difference is *dependency depth*: how many operations must happen sequentially before the next can start. RNN depth = N . Attention depth = $\log(N)$ with a tree reduction, or 1 with a parallel scan. Depth, not op count, decides GPU time.

Step 3: empirical scaling on long sequences

We print a timing table that makes the $O(N)$ gap visible. On a 2026 Mac laptop, sequences under 1,000 elements are too fast to measure. Sequences of 100,000 show a clean linear scan. Scale that to a 16,384-token transformer with a 12-layer LSTM equivalent and you see why training wall-clock was a blocker in 2016.

Use It

When to still pick an RNN in 2026:

Situation	Pick
Streaming inference, one token at a time, constant memory	RNN or state-space model (Mamba, RWKV)
Very long sequences (>1M tokens) where attention memory explodes	Linear attention, Mamba 2, Hyena
Edge device with no matmul accelerator	Depthwise-separable RNN still wins on FLOPs/watt
Anything else (training, batched inference, context up to 128K)	Transformer

State-space models (SSMs) like Mamba are essentially RNNs with structured parameterization that gives them the best of both: $O(N)$ scan memory, parallel training via selective scan. They recover 90% of transformer quality with better long-context scaling. In 2026 most frontier labs train hybrid SSM+transformer models (e.g. Jamba, Samba) — recurrence is not dead, it is a component.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/01-why-transformers>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/01-why-transformers/code>

1. **Easy.** Take `rnn_style` from `code/main.py` and replace the scalar hidden state with a length-64 vector of hidden states. Re-measure. How much does the serial overhead grow with hidden-state dimension?
2. **Medium.** Implement a parallel prefix-sum (Hillis-Steele scan) in pure Python. Verify it produces the same numerical output as a serial scan on length 1024. Count the depth.
3. **Hard.** Port the attention-style reduction to PyTorch on GPU. Time both as you sweep sequence length from 64 to 65,536. Plot and explain the curve shape.

Key Terms

Term	What people say	What it actually means
Recurrence	“RNNs are sequential”	Computation where step t depends on step $t-1$, forcing serial execution along the time axis.
Serial depth	“How deep the graph is”	Longest chain of dependent ops; bounds wall-clock even on infinite hardware.
Attention	“Let tokens look at each other”	Weighted sum $\sum_j a_{ij} v_j$ where a_{ij} comes from a similarity score between positions i and j .
Context window	“How much the model sees”	Number of positions an attention layer can take as input; quadratic memory cost scales here.
Inductive bias	“Assumptions baked into the architecture”	Prior about what the data looks like; CNNs assume translation invariance, RNNs assume recency.
State-space model	“RNN with algebra behind it”	Recurrence parameterized for parallel training via structured state-space matrices.
Quadratic bottleneck	“Why context costs so much”	Attention memory = $O(N^2)$ in sequence length; Flash Attention hides the constants, not the scaling.

Further Reading

- Vaswani et al. (2017). Attention Is All You Need — the paper that killed recurrence in mainstream NLP.
- Bahdanau, Cho, Bengio (2014). Neural MT by Jointly Learning to Align and Translate — where attention was born, bolted onto an RNN.
- Hochreiter, Schmidhuber (1997). Long Short-Term Memory — the original LSTM paper, for the record.
- Gu, Dao (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces — modern recurrent answer to transformers.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/01-why-transformers>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/01-why-transformers/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Self-Attention from Scratch

Attention is a lookup table where every word asks “who matters to me?” - and learns the answer.

Type: Build **Languages:** Python **Prerequisites:** Phase 3 (Deep Learning Core), Phase 5 Lesson 10 (Sequence-to-Sequence) **Time:** ~90 minutes

Learning Objectives

- Implement scaled dot-product self-attention from scratch using only NumPy, including query/key/value projections and the softmax-weighted sum
- Build a multi-head attention layer that splits heads, computes parallel attention, and concatenates results
- Trace how the attention matrix captures token relationships and explain why scaling by $\sqrt{d_k}$ prevents softmax saturation
- Apply causal masking to convert bidirectional attention into autoregressive (decoder-style) attention

The Problem

RNNs process sequences one token at a time. By the time you reach token 50, the information from token 1 has been squeezed through 50 compression steps. Long-range dependencies get crushed into a fixed-size hidden state - a bottleneck that no amount of LSTM gating fully solves.

The 2014 Bahdanau attention paper showed the fix: let the decoder look back at every encoder position and decide which ones matter for the current step. But it was still bolted onto an RNN. The 2017 “Attention Is All You Need” paper asked a sharper question: what if attention is the *only* mechanism? No recurrence. No convolution. Just attention.

Self-attention lets every position in a sequence attend to every other position in a single parallel step. That is what makes transformers fast, scalable, and dominant.

The Concept

The Database Lookup Analogy

Think of attention as a soft database lookup:

Traditional database:

Query: "capital of France" --> exact match --> "Paris"

Attention:

Query: "capital of France" --> similarity to ALL keys --> weighted blend of
↪ ALL values

Every token generates three vectors: - **Query (Q)**: "What am I looking for?" - **Key (K)**: "What do I contain?" - **Value (V)**: "What information do I provide if selected?"

The dot product between a query and all keys produces attention scores. High score means "this key matches my query." Those scores weight the values. The output is a weighted sum of values.

Q, K, V Computation

Each token embedding gets projected through three learned weight matrices:

Input embeddings (sequence of n tokens, each d -dimensional):

$X = [x_1, x_2, x_3, \dots, x_n]$ shape: (n, d)

Three weight matrices:

W_q shape: (d, d_k)

W_k shape: (d, d_k)

W_v shape: (d, d_v)

Projections:

$Q = X @ W_q$ shape: (n, d_k) each token's query

$K = X @ W_k$ shape: (n, d_k) each token's key

$V = X @ W_v$ shape: (n, d_v) each token's value

Visually, for one token:

```

      Wq
x_i  -----[*]-----> q_i    "What am I looking for?"
      |
      |      Wk
      +-----[*]-----> k_i    "What do I contain?"
      |
      |      Wv
      +-----[*]-----> v_i    "What do I offer?"

```

The Attention Matrix

Once you have Q, K, V for all tokens, attention scores form a matrix:

Scores = $Q @ K^T$ shape: (n, n)

	k1	k2	k3	k4	k5	
q1	2.1	0.3	0.1	0.8	0.2	<- how much q1 attends to each key
q2	0.4	1.9	0.7	0.1	0.3	
q3	0.2	0.6	2.3	0.5	0.1	
q4	0.9	0.1	0.4	1.7	0.6	
q5	0.1	0.3	0.2	0.5	2.0	

Each row: one token's attention over the entire sequence

Watch one query at a time sweep the keys: each row scores every token, softmax turns the scores into weights, and the context vector is the weighted blend of values.

Interactive figure: attention-matrix. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/02-self-attention-from-scratch>

Why Scale?

The dot products grow with dimension d_k . If $d_k = 64$, dot products can be in the range of tens, pushing softmax into regions where gradients vanish. The fix: divide by $\sqrt{d_k}$.

Scaled scores = $(Q @ K^T) / \sqrt{d_k}$

This keeps values in a range where softmax produces useful gradients.

Softmax Turns Scores into Weights

Softmax converts raw scores into a probability distribution across each row:

Raw scores for q_1 : [2.1, 0.3, 0.1, 0.8, 0.2]

|
softmax
|

Attention weights: [0.52, 0.09, 0.07, 0.14, 0.08] (sums to ~1.0)

Now each token has a set of weights saying how much to attend to every other token.

Weighted Sum of Values

The final output for each token is a weighted sum of all value vectors:

$$\text{output}_i = \sum (\text{attention_weight}[i][j] * v_j \text{ for all } j)$$

For token 1:

$$\text{output}_1 = 0.52 * v_1 + 0.09 * v_2 + 0.07 * v_3 + 0.14 * v_4 + 0.08 * v_5$$

Full Pipeline

Diagram. Rendered live in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/02-self-attention-from-scratch>

Formula in one line:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q @ K^T / \sqrt{d_k}) @ V$$

Interactive figure: softmax-attention-scaling. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/02-self-attention-from-scratch>

Build It

Step 1: Softmax from scratch

Softmax converts raw logits into probabilities. Subtract the max for numerical stability.

```
import numpy as np

def softmax(x):
    shifted = x - np.max(x, axis=-1, keepdims=True)
    exp_x = np.exp(shifted)
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)

logits = np.array([2.0, 1.0, 0.1])
print(f"logits: {logits}")
print(f"softmax: {softmax(logits)}")
print(f"sum: {softmax(logits).sum():.4f}")
```

Step 2: Scaled dot-product attention

The core function. Takes Q, K, V matrices and returns the attention output plus the weight matrix.

```
def scaled_dot_product_attention(Q, K, V):
    dk = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(dk)
    weights = softmax(scores)
    output = weights @ V
    return output, weights
```

Step 3: Self-attention class with learned projections

A full self-attention module with Wq, Wk, Wv weight matrices initialized with Xavier-like scaling.

```
class SelfAttention:
    def __init__(self, d_model, dk, dv, seed=42):
        rng = np.random.default_rng(seed)
        scale = np.sqrt(2.0 / (d_model + dk))
        self.Wq = rng.normal(0, scale, (d_model, dk))
        self.Wk = rng.normal(0, scale, (d_model, dk))
        scale_v = np.sqrt(2.0 / (d_model + dv))
        self.Wv = rng.normal(0, scale_v, (d_model, dv))
        self.dk = dk

    def forward(self, X):
        Q = X @ self.Wq
        K = X @ self.Wk
        V = X @ self.Wv
        output, weights = scaled_dot_product_attention(Q, K, V)
        return output, weights
```

Step 4: Run it on a sentence

Create fake embeddings for a sentence and watch the attention weights.

```

sentence = ["The", "cat", "sat", "on", "the", "mat"]
n_tokens = len(sentence)
d_model = 8
dk = 4
dv = 4

rng = np.random.default_rng(42)
X = rng.normal(0, 1, (n_tokens, d_model))

attn = SelfAttention(d_model, dk, dv, seed=42)
output, weights = attn.forward(X)

print("Attention weights (each row: where that token looks):\n")
print(f"{'':>6}", end="")
for token in sentence:
    print(f"{token:>6}", end="")
print()

for i, token in enumerate(sentence):
    print(f"{token:>6}", end="")
    for j in range(n_tokens):
        w = weights[i][j]
        print(f"{w:6.3f}", end="")
    print()

```

Step 5: Visualize attention with ASCII heatmap

Map attention weights to characters for a quick visual.

```

def ascii_heatmap(weights, tokens, chars=" ██████"):
    n = len(tokens)
    print(f"\n{'':>6}", end="")
    for t in tokens:
        print(f"{t:>6}", end="")
    print()

    for i in range(n):
        print(f"{tokens[i]:>6}", end="")
        for j in range(n):
            level = int(weights[i][j] * (len(chars) - 1) / weights.max())
            level = min(level, len(chars) - 1)
            print(f"{' ' + chars[level] + ' '}>6", end="")
        print()

ascii_heatmap(weights, sentence)

```

Use It

PyTorch's `nn.MultiheadAttention` does exactly what we built, plus multi-head splitting and output projection:

```

import torch
import torch.nn as nn

```

```

d_model = 8
n_heads = 2
seq_len = 6

mha = nn.MultiheadAttention(embed_dim=d_model, num_heads=n_heads,
    ↪ batch_first=True)

X_torch = torch.randn(1, seq_len, d_model)

output, attn_weights = mha(X_torch, X_torch, X_torch)

print(f"Input shape:           {X_torch.shape}")
print(f"Output shape:          {output.shape}")
print(f"Attention weight shape: {attn_weights.shape}")
print(f"\nAttn weights (averaged over heads):")
print(attn_weights[0].detach().numpy().round(3))

```

The key difference: multi-head attention runs multiple attention functions in parallel, each with its own Q, K, V projections of size $dk = d_model / n_heads$, then concatenates results. This lets the model attend to different relationship types simultaneously.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/02-self-attention-from-scratch>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/02-self-attention-from-scratch/code>

1. Modify `scaled_dot_product_attention` to accept an optional mask matrix that sets certain positions to negative infinity before softmax (this is how causal/decoder masking works)
2. Implement multi-head attention from scratch: split Q, K, V into `n_heads` chunks, run attention on each, concatenate, and project through a final weight matrix `Wo`
3. Take two different sentences of the same length, feed them through the same SelfAttention instance, and compare their attention patterns. What changes? What stays the same?

Key Terms

Term	What people say	What it actually means
Query (Q)	"The question vector"	A learned projection of the input that represents what information this token is looking for
Key (K)	"The label vector"	A learned projection that represents what information this token contains, matched against queries

Term	What people say	What it actually means
Value (V)	“The content vector”	A learned projection carrying the actual information that gets aggregated based on attention scores
Scaled dot-product attention	“The attention formula”	$\text{softmax}(QK^T / \sqrt{d_k}) @ V$ - scaling prevents softmax saturation in high dimensions
Self-attention	“The token looks at itself and others”	Attention where Q, K, V all come from the same sequence, letting every position attend to every other position
Attention weights	“How much focus”	A probability distribution over positions, produced by softmax over scaled dot products
Multi-head attention	“Parallel attention”	Running multiple attention functions with different projections, then concatenating results for richer representations

Further Reading

- Attention Is All You Need (Vaswani et al., 2017) - the original transformer paper
- The Illustrated Transformer (Jay Alammar) - best visual walkthrough of the full architecture
- The Annotated Transformer (Harvard NLP) - line-by-line PyTorch implementation with explanations

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/02-self-attention-from-scratch>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/02-self-attention-from-scratch/code>
- The chapter quiz, graded in the browser: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/02-self-attention-from-scratch>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Multi-Head Attention

One attention head learns one relation at a time. Eight heads learn eight. Heads are free. Take more of them.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 02 (Self-Attention from Scratch)
Time: ~75 minutes

The Problem

A single self-attention head computes one attention matrix. That matrix captures one kind of relationship — usually the one that minimizes loss on whatever the training signal is. If your data has subject-verb agreement, co-reference, long-range discourse, and syntactic chunking all tangled together, a single head smears them into a single soft-max distribution and loses half the signal.

The fix from the 2017 Vaswani paper: run several attention functions in parallel, each with its own Q, K, V projections, and concatenate the outputs. Each head operates in a smaller subspace of dimension $d_{\text{model}} / n_{\text{heads}}$. Total parameters stay the same. Expressive power goes up.

Multi-head attention is the default every transformer in 2026 ships with. The only argument is about *how many* heads and whether keys and values share projections (Grouped-Query Attention, Multi-Query Attention, Multi-head Latent Attention).

The Concept

Split. Take X of shape (N, d_{model}) . Project to Q, K, V each of shape (N, d_{model}) . Reshape to $(N, n_{\text{heads}}, d_{\text{head}})$ where $d_{\text{head}} = d_{\text{model}} / n_{\text{heads}}$. Transpose to $(n_{\text{heads}}, N, d_{\text{head}})$.

Attend in parallel. Run scaled dot-product attention inside each head. Each head produces (N, d_{head}) . The heads operate on different subspaces of the embedding and never talk during the attention computation itself.

Concatenate and project. Stack heads back to (N, d_{model}) and multiply by a learned output matrix W_o of shape $(d_{\text{model}}, d_{\text{model}})$. W_o is where heads get to mix.

Why it works. Each head can specialize without competing with the others for representational budget. Probing studies from 2019–2024 show distinct head roles: positional heads, head that attends to the previous token, copy heads, named-entity heads, induction heads (which underlie in-context learning).

The 2026 lineage of variations:

Variant	Q heads	K/V heads	Used by
Multi-head (MHA)	N	N	GPT-2, BERT, T5
Multi-query (MQA)	N	1	PaLM, Falcon

Variant	Q heads	K/V heads	Used by
Grouped-query (GQA)	N	G (e.g. N/8)	Llama 2 70B, Llama 3+, Qwen 2+, Mistral
Multi-head latent (MLA)	N	compressed to low-rank	DeepSeek-V2, V3

GQA is the modern default because it cuts KV-cache memory by a factor of N/G while keeping nearly full quality. MLA goes further by compressing K/V into a latent space, then projecting back at compute time — costs FLOPs, saves a lot more memory.

Interactive figure: multihead-split. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/03-multi-head-attention>

Build It

Step 1: split heads from the single-head attention we already have

Take the SelfAttention from Lesson 02 and wrap it with a split/concat pair. See code/main.py for a numpy implementation; the logic is:

```
def split_heads(X, n_heads):
    n, d = X.shape
    d_head = d // n_heads
    return X.reshape(n, n_heads, d_head).transpose(1, 0, 2)  # (heads, n, d_head)

def combine_heads(H):
    h, n, d_head = H.shape
    return H.transpose(1, 0, 2).reshape(n, h * d_head)
```

One reshape and one transpose. No loop. This is exactly what PyTorch does under `nn.MultiheadAttention`.

Step 2: run scaled-dot-product attention per head

Each head gets its own slice of Q, K, V. Attention becomes a batched matmul:

```
def mha_forward(X, W_q, W_k, W_v, W_o, n_heads):
    Q = X @ W_q
    K = X @ W_k
    V = X @ W_v
    Qh = split_heads(Q, n_heads)  # (heads, n, d_head)
    Kh = split_heads(K, n_heads)
    Vh = split_heads(V, n_heads)
    scores = Qh @ Kh.transpose(0, 2, 1) / np.sqrt(Qh.shape[-1])
    weights = softmax(scores, axis=-1)
    out = weights @ Vh  # (heads, n, d_head)
    concat = combine_heads(out)
    return concat @ W_o, weights
```

On real hardware `Qh @ Kh.transpose(...)` is one bmm. The GPU sees a single batched matmul of shape $(heads, N, d_head) \times (heads, d_head, N) \rightarrow (heads, N, N)$. Adding heads is free.

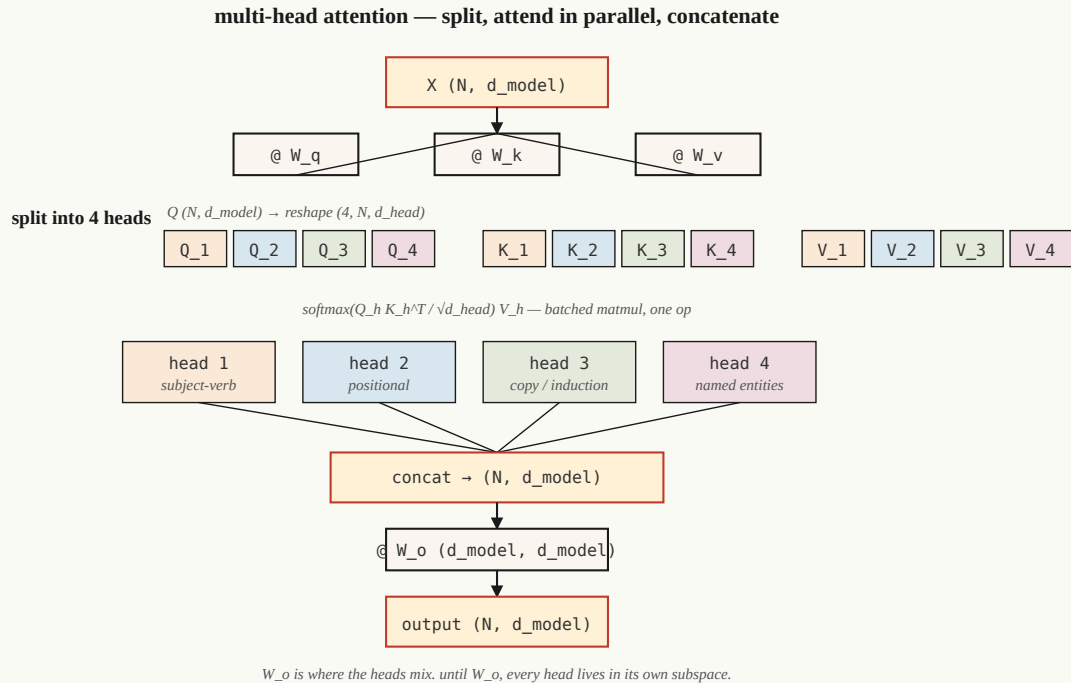


Figure 22: Multi-head attention splits, attends, concatenates

Step 3: Grouped-Query Attention variant

Only the key and value projections change. Q gets `n_heads` groups; K and V get `n_kv_heads` < `n_heads` groups and are repeated to match:

```
def gqa_project(X, W, n_kv_heads, n_heads):
    kv = split_heads(X @ W, n_kv_heads)           # (kv_heads, n, d_head)
    repeat = n_heads // n_kv_heads
    return np.repeat(kv, repeat, axis=0)          # (n_heads, n, d_head)
```

At inference this saves memory because only `n_kv_heads` copies live in the KV cache, not `n_heads`. Llama 3 70B uses 64 query heads with 8 KV heads — an 8× cache shrink.

Step 4: probe what each head learned

Run MHA on a short sentence with 4 heads. For each head, print the (N, N) attention matrix. You'll see different heads pick out different structure even with random initialization — that's partly signal, partly rotational symmetry in the subspaces.

Use It

In PyTorch, the one-line version:

```
import torch.nn as nn
```

```
mha = nn.MultiheadAttention(embed_dim=512, num_heads=8, batch_first=True)
```

GQA as of PyTorch 2.5+:

```

from torch.nn.functional import scaled_dot_product_attention

# scaled_dot_product_attention auto-dispatches Flash Attention on CUDA.
# For GQA, pass Q of shape (B, n_heads, N, d_head) and K,V of shape
# (B, n_kv_heads, N, d_head). PyTorch handles the repeat.
out = scaled_dot_product_attention(q, k, v, is_causal=True, enable_gqa=True)

```

How many heads? Rules of thumb from production models in 2026:

Model size	d_model	n_heads	d_head
Small (~125M)	768	12	64
Base (~350M)	1024	16	64
Large (~1B)	2048	16	128
Frontier (~70B)	8192	64	128

d_head almost always lands at 64 or 128. It is the unit of how much one head can “see.” Drop below 32 and heads start fighting the scaling factor $\sqrt{d_head}$; go above 256 and you lose the “many small specialists” benefit.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/03-multi-head-attention>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/03-multi-head-attention/code>

1. **Easy.** Take the MHA from code/main.py and change n_heads from 1 to 16 with d_model=64 fixed. Plot the loss of a tiny one-layer model on a synthetic copy task. Do more heads help, plateau, or hurt?
2. **Medium.** Implement MQA (one KV head shared across all query heads). Measure how much parameter count drops vs full MHA. Compute how much the KV-cache size shrinks at inference for N=2048.
3. **Hard.** Implement a tiny version of Multi-head Latent Attention: compress K,V to a rank-r latent, store the latent in the KV cache, decompress at attention time. At what r does cache memory cross below 1/8 of full MHA while quality stays within 1 bit of validation ppl?

Key Terms

Term	What people say	What it actually means
Head	“A single attention circuit”	One Q/K/V projection of dimension d_head = d_model / n_heads with its own attention matrix.
d_head	“Head dimension”	Per-head hidden width; almost always 64 or 128 in production.

Term	What people say	What it actually means
Split / combine	“Reshape tricks”	$(N, d_{\text{model}}) \leftrightarrow (n_{\text{heads}}, N, d_{\text{head}})$ reshape+transpose around attention.
W_o	“Output projection”	$(d_{\text{model}}, d_{\text{model}})$ matrix applied after concatenating heads; where heads mix.
MQA	“One KV head”	Multi-Query Attention: single shared K/V projection. Smallest KV cache, some quality loss.
GQA	“The default since Llama 2”	Grouped-Query Attention with $n_{\text{kv_heads}} < n_{\text{heads}}$; repeats to match Q.
MLA	“DeepSeek’s trick”	Multi-head Latent Attention: K,V compressed to low-rank latent, decompressed at attend time.
Induction head	“The circuit behind in-context learning”	A pair of heads that detect previous occurrences and copy what followed them.

Further Reading

- Vaswani et al. (2017). Attention Is All You Need §3.2.2 — the original multi-head spec.
- Shazeer (2019). Fast Transformer Decoding: One Write-Head is All You Need — the MQA paper.
- Ainslie et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints — how to convert MHA to GQA after training.
- DeepSeek-AI (2024). DeepSeek-V2 Technical Report — MLA and why it beats MHA/GQA on cache memory.
- Olsson et al. (2022). In-context Learning and Induction Heads — mechanistic look at what heads actually do.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/03-multi-head-attention>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/03-multi-head-attention/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Positional Encoding — Sinusoidal, RoPE, ALiBi

Attention is permutation-invariant. “The cat sat on the mat” and “mat the on sat cat the” produce the same output without positional signal. Three algorithms fix it — each with a different bet on what “position” means.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 02 (Self-Attention), Phase 7 · 03 (Multi-Head Attention) **Time:** ~45 minutes

The Problem

Scaled dot-product attention is order-blind. The attention matrix $\text{softmax}(QK^T / \sqrt{d})$ is computed from pairwise similarities. Shuffle the rows of X , get the rows of the output shuffled the same way. Nothing inside attention cares about position.

That is not a bug in a bag-of-words model. For language, code, audio, video — anything where order carries meaning — it is fatal.

The fix is to inject position into the embeddings somehow. Three eras of answers:

1. **Absolute sinusoidal** (Vaswani 2017). Add $\sin/\cos$ of position to the embedding. Simple, learnable-free, extrapolates poorly beyond trained lengths.
2. **RoPE — Rotary Position Embeddings** (Su 2021). Rotate Q and K vectors by an angle proportional to position. Encodes *relative* position directly in the dot product. Dominant in 2026.
3. **ALiBi — Attention with Linear Biases** (Press 2022). Skip embeddings entirely; add a per-head linear penalty to attention scores based on distance. Excellent length extrapolation.

As of 2026, essentially every frontier open model uses RoPE: Llama 2/3/4, Qwen 2/3, Mistral, Mixtral, DeepSeek-V3, Kimi. A handful of long-context models use ALiBi or its modern variants. Absolute sinusoidal is historical.

The Concept

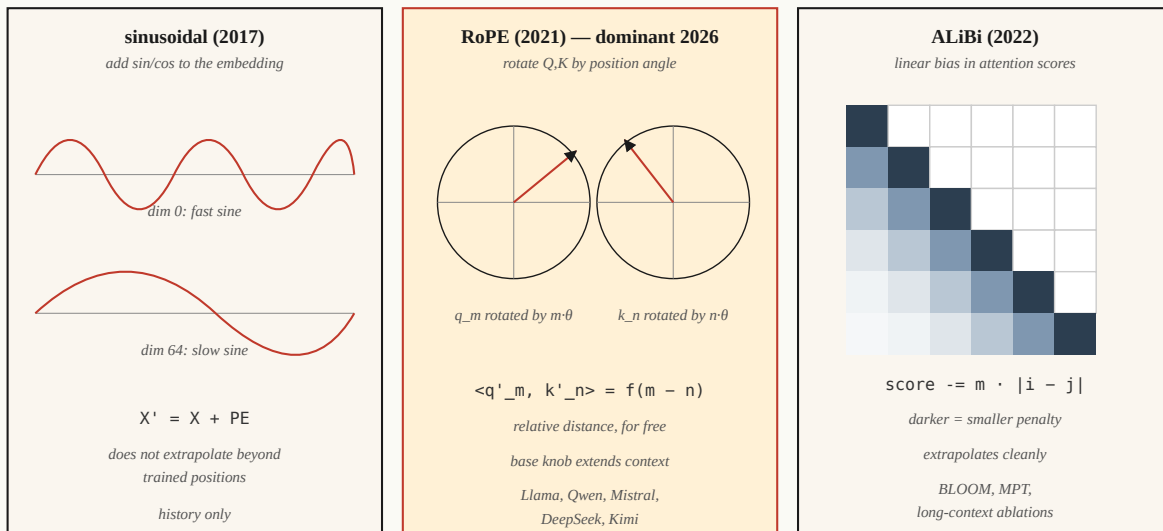
Absolute sinusoidal

Pre-compute a fixed matrix PE of shape $(\text{max_len}, d_{\text{model}})$:

```
PE[pos, 2i]   = sin(pos / 10000^(2i / d_model))
PE[pos, 2i+1] = cos(pos / 10000^(2i / d_model))
```

Then $X' = X + PE[:N]$ before attention. Each dimension is a sinusoid at a different frequency. The model learns to read position from the phase pattern. Fails beyond max_len : nothing told the model what happens at position 2048 when it only saw positions 0–2047.

three ways to tell attention about position



attention is permutation-invariant; pick the injection that matches your context length strategy

Figure 23: Sinusoidal absolute vs RoPE rotations vs ALiBi distance bias

RoPE

Rotate the Q and K vectors (not embeddings). For a pair of dimensions (2i, 2i+1):

$$\begin{bmatrix} q'_{2i} \\ q'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos(\text{pos} \cdot \theta_i) & -\sin(\text{pos} \cdot \theta_i) \\ \sin(\text{pos} \cdot \theta_i) & \cos(\text{pos} \cdot \theta_i) \end{bmatrix} \begin{bmatrix} q_{2i} \\ q_{2i+1} \end{bmatrix}$$

$$\theta_i = \text{base}^{-(2i / d_{\text{head}})}, \quad \text{base} = 10000 \text{ by default}$$

Apply the same rotation to keys with position pos_k. The dot product $q'_m \cdot k'_n$ becomes a function of (m - n) alone. That is: **the attention score depends only on the relative distance**, even though the rotation was keyed off absolute positions. Beautiful trick.

Extending RoPE: base can be scaled (NTK-aware, YaRN, LongRoPE) to extrapolate to longer contexts without retraining. Llama 3 extended from 8K to 128K context this way.

ALiBi

Skip the embedding trick. Bias the attention scores directly:

$$\text{attn_score}[i, j] = (q_i \cdot k_j) / \sqrt{d} - m_h \cdot |i - j|$$

Where m_h is a head-specific slope (e.g. $1 / 2^{(8 \cdot h / H)}$). Closer tokens get boosted; far tokens get penalized. No training-time cost. The paper shows length extrapolation beats sinusoidal and matches RoPE on its original trained length.

What to pick in 2026

Variant	Extrapolation	Training cost	Used by
Absolute sinusoidal	poor	free	original transformer, early BERT
Learned absolute	none	tiny	GPT-2, GPT-3
RoPE	good with scaling	free	Llama 2/3/4, Qwen 2/3, Mistral, DeepSeek-V3, Kimi
RoPE + YaRN	excellent	fine-tune stage	Qwen2-1M, Llama 3.1 128K
ALiBi	excellent	free	BLOOM, MPT, Baichuan

RoPE won because it slots into attention without changing the architecture, encodes relative position, and its base hyperparameter gives a clean knob for long-context fine-tuning.

Interactive figure: rope-explorer. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/04-positional-encoding>

Build It

Step 1: sinusoidal encoding

See code/main.py. A 4-line computation:

```
def sinusoidal(N, d):
    pe = [[0.0] * d for _ in range(N)]
    for pos in range(N):
        for i in range(d // 2):
            theta = pos / (10000 ** (2 * i / d))
            pe[pos][2 * i] = math.sin(theta)
            pe[pos][2 * i + 1] = math.cos(theta)
    return pe
```

Add this to the embedding matrix before the first attention layer.

Step 2: RoPE applied to Q, K

RoPE operates in-place on Q and K. For each pair of dims:

```
def apply_rope(x, pos, base=10000):
    d = len(x)
    out = list(x)
    for i in range(d // 2):
        theta = pos / (base ** (2 * i / d))
        c, s = math.cos(theta), math.sin(theta)
        a, b = x[2 * i], x[2 * i + 1]
        out[2 * i] = a * c - b * s
```

```

    out[2 * i + 1] = a * s + b * c
    return out

```

Crucial: apply the same function to Q at position m and K at position n. Their dot product picks up a $\cos((m-n) \cdot \theta_i)$ factor on every coordinate pair. Attention learns relative position for free.

Step 3: ALiBi slopes and bias

```

def alibi_bias(n_heads, seq_len):
    # slope_h = 2 ** (-8 * h / n_heads) for h = 1..n_heads
    slopes = [2 ** (-8 * (h + 1) / n_heads) for h in range(n_heads)]
    bias = []
    for m in range(seq_len):
        row = []
        for n in range(seq_len):
            row.append(slopes[m - n])
        bias.append(row)
    return bias # add to attention scores before softmax

```

Add bias[h] to the (seq_len, seq_len) attention score matrix of head h, then softmax.

Step 4: verify relative-distance property of RoPE

Pick two random vectors a, b. Rotate by (pos_a, pos_b). Then by (pos_a + k, pos_b + k). Both dot products must match within floating-point error. That property is the whole point of RoPE — it is invariant to the absolute offset, only the relative gap matters.

Use It

PyTorch 2.5+ ships RoPE utilities in torch.nn.functional. Most production code uses flash_attn or xformers where RoPE is applied inside the attention kernel.

```

from transformers import AutoModel
model = AutoModel.from_pretrained("meta-llama/Llama-3.2-3B")
# model.config.rope_scaling → {"type": "yarn", "factor": 32.0,
  ↳ "original_max_position_embeddings": 8192}

```

Long-context tricks in 2026:

- **NTK-aware interpolation.** Rescale base to $\text{base} * (\text{scale_factor})^{(d/(d-2))}$ when extending from 4K to 16K+.
- **YaRN.** Smarter interpolation that preserves attention entropy on long contexts. Llama 3.1 128K uses it.
- **LongRoPE.** Microsoft's 2024 method that uses evolutionary search to pick per-dimension scale factors. Phi-3-Long uses it.
- **Position interpolation + fine-tuning.** Just shrink positions by the extension factor and fine-tune for 1-5B tokens. Surprisingly effective.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/04-positional-encoding>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/04-positional-encoding/code>

1. **Easy.** Plot the sinusoidal PE matrix as a heatmap for `max_len=512`, `d=128`. Confirm the “stripes get wider as dimension index grows” pattern.
2. **Medium.** Implement NTK-aware RoPE scaling. Train a tiny LM on sequences of length 256, then test on length 1024 with and without scaling. Measure perplexity.
3. **Hard.** Implement ALiBi and RoPE in the same attention module. Train a 4-layer transformer on a copy task with sequences of length 512. Extrapolate to 2048 at test time. Compare degradation.

Key Terms

Term	What people say	What it actually means
Positional encoding	“Tells attention about order”	Any signal added to embeddings or attention that encodes position.
Sinusoidal	“The original one”	$\sin/\cos$ at geometric frequencies added to embeddings; doesn’t extrapolate.
RoPE	“Rotary embeddings”	Rotate Q, K by position-dependent angle; dot product encodes relative distance.
ALiBi	“Linear bias trick”	Add $-m \cdot \lfloor i-j \rfloor$ to attention scores; no embedding needed, great extrapolation.
base	“RoPE’s knob”	The frequency scaler in RoPE; increase to extend context at inference.
NTK-aware	“A RoPE scaling trick”	Rescale base so high-frequency dims aren’t squeezed when context expands.
YaRN	“The fancy one”	Per-dimension interpolation+extrapolation that preserves attention entropy.
Extrapolation	“Works beyond trained length”	Can the position scheme serve correct output past <code>max_len</code> seen in training?

Further Reading

- Vaswani et al. (2017). Attention Is All You Need §3.5 — original sinusoidal.
- Su et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding — RoPE paper.
- Press, Smith, Lewis (2021). Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation — ALiBi.
- Peng et al. (2023). YaRN: Efficient Context Window Extension of Large Language Models — state of the art RoPE scaling.
- Chen et al. (2023). Extending Context Window of Large Language Models via Positional Interpolation — Meta’s Llama 2 long-context paper.
- Ding et al. (2024). LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens — the Microsoft method used by Phi-3-Long and cited in the Use It section.
- HuggingFace Transformers — `modeling_rope_utils.py` — production-grade implementations of every RoPE scaling scheme (default, linear, dynamic, YaRN, LongRoPE, Llama-3).

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/04-positional-encoding>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/04-positional-encoding/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

The Full Transformer — Encoder + Decoder

Attention is the star. Everything else — residuals, normalization, feed-forward, cross-attention — is the scaffolding that lets you stack it deep.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 02 (Self-Attention), Phase 7 · 03 (Multi-Head Attention), Phase 7 · 04 (Positional Encoding) **Time:** ~75 minutes

The Problem

A single attention layer is a feature extractor, not a model. One matmul per layer is not enough capacity for language. You need depth — and depth breaks without the right plumbing.

The 2017 Vaswani paper packaged six design decisions that turned one attention layer into a stackable block. Every transformer since — encoder-only (BERT), decoder-only (GPT), encoder-decoder (T5) — inherits the same skeleton. In 2026 the blocks have been refined (RMSNorm, SwiGLU, pre-norm, RoPE) but the skeleton is identical.

This lesson is the skeleton. Next lessons specialize it — 06 for encoders, 07 for decoders, 08 for encoder-decoder.

The Concept

The six pieces

1. **Embedding + positional signal.** Tokens $\rightarrow$ vectors. Position injected via RoPE (modern) or sinusoidal (classic).
2. **Self-attention.** Every position attends to every other. Masked in decoders.
3. **Feed-forward network (FFN).** Position-wise two-layer MLP: $W_2 \cdot \text{activation}(W_1 \cdot x)$. Expansion ratio $4\times$ by default.
4. **Residual connection.** $x + \text{sublayer}(x)$. Without this, gradients vanish past ~ 6 layers.
5. **Layer normalization.** LayerNorm or RMSNorm (modern). Stabilizes the residual stream.
6. **Cross-attention (decoder only).** Queries come from the decoder, keys and values from the encoder output.

Watch a vector flow through one block: attention mixes across positions, the residual carries it forward, the FFN transforms it, and norm keeps the stream stable.

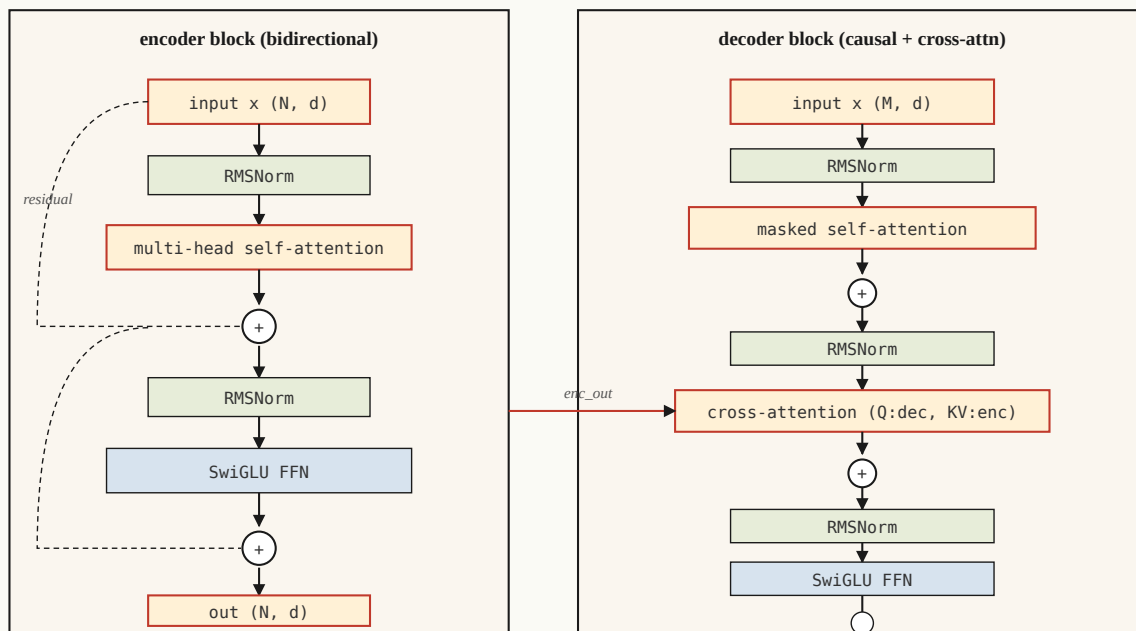
Interactive figure: transformer-block. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/05-full-transformer>

Encoder block (used by BERT, T5 encoder)

$x \rightarrow \text{LN} \rightarrow \text{MHA}(\text{self}) \rightarrow + \rightarrow \text{LN} \rightarrow \text{FFN} \rightarrow + \rightarrow \text{out}$

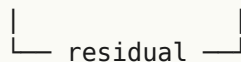
$\quad \quad \quad \wedge \quad \quad \quad \wedge$

encoder block vs decoder block — the 2026 skeleton



encoder: 2 sublayers per block. decoder: 3 sublayers per block. residuals are dashed, additions are circles.

Figure 24: Encoder and decoder block internals, wired



Encoder is bidirectional. No masking. All positions see all positions.

Decoder block (used by GPT, T5 decoder)

$x \rightarrow \text{LN} \rightarrow \text{MHA}(\text{masked self}) \rightarrow + \rightarrow \text{LN} \rightarrow \text{MHA}(\text{cross to encoder}) \rightarrow + \rightarrow \text{LN} \rightarrow \text{FFN} \rightarrow + \rightarrow \text{out}$

Decoder has three sublayers per block. The middle one — cross-attention — is the only place information flows from encoder to decoder. In a pure decoder-only architecture (GPT), cross-attention is omitted and you just have masked self-attention + FFN.

Pre-norm vs post-norm

Original paper: $x + \text{sublayer}(\text{LN}(x))$ vs $\text{LN}(x + \text{sublayer}(x))$. Post-norm lost favor around 2019 — it is harder to train deeply without careful warmup. Pre-norm (LN *before* sublayer) is the 2026 default: Llama, Qwen, GPT-3+, Mistral all use it.

The 2026 modernized block

Vaswani 2017 shipped LayerNorm + ReLU. Modern stacks replaced both. What production blocks actually look like:

Component	2017	2026
Normalization	LayerNorm	RMSNorm
FFN activation	ReLU	SwiGLU
FFN expansion	4×	2.6× (SwiGLU uses three matrices, total params match)
Position	Sinusoidal absolute	RoPE
Attention	Full MHA	GQA (or MLA)
Bias terms	Yes	No

RMSNorm drops the mean-centering of LayerNorm (one fewer subtraction), which saves compute and is empirically at least as stable. SwiGLU ($\text{Swish}(W_1 x) \odot W_3 x$) consistently outperforms ReLU/GELU FFN by ~ 0.5 point ppl in the Llama, PaLM and Qwen papers.

Parameter count

For one block with $d_{\text{model}} = d$ and FFN expansion r :

- MHA: $4 \cdot d^2$ (Q, K, V, O projections)
- FFN (SwiGLU): $3 \cdot d \cdot (r \cdot d) \approx 3rd^2$
- Norms: negligible

At $d = 4096$, $r = 2.6$, $\text{layers} = 32$ (roughly Llama 3 8B), total: $32 \cdot (4 \cdot 4096^2 + 3 \cdot 2.6 \cdot 4096^2) \approx 32 \cdot (16 + 32) \text{ M} = \sim 1.5\text{B}$ parameters per layer $\times 32 \approx 7\text{B}$ (plus embeddings and head). Matches published counts.

Build It

Step 1: the building blocks

Using the tiny Matrix class from Lesson 03 (copied to this file for independence):

- `layer_norm(x, eps=1e-5)` — subtract mean, divide by std.
- `rms_norm(x, eps=1e-6)` — divide by RMS. No mean subtraction.
- `gelu(x)` and `silu(x) * W3 x` (SwiGLU).
- `ffn_swiglu(x, W1, W2, W3)`.
- `encoder_block(x, params)` and `decoder_block(x, enc_out, params)`.

See `code/main.py` for the full wiring.

Step 2: wire a 2-layer encoder and a 2-layer decoder

Stack them. Pass the encoder output into every decoder cross-attention. Add a final LN before the output projection.

```
def encode(tokens, params):
    x = embed(tokens, params.emb) + sinusoidal(len(tokens), params.d)
    for block in params.encoder_blocks:
        x = encoder_block(x, block)
    return x

def decode(target_tokens, encoder_out, params):
    x = embed(target_tokens, params.emb) + sinusoidal(len(target_tokens), params.d)
```

```

for block in params.decoder_blocks:
    x = decoder_block(x, encoder_out, block)
return x

```

Step 3: run forward on a toy example

Feed a 6-token source and a 5-token target through. Verify the output shape is (5, vocab). No training — this lesson is about the architecture, not the loss.

Step 4: swap in RMSNorm + SwiGLU

Replace LayerNorm and ReLU-FFN with RMSNorm and SwiGLU. Confirm shapes still match. This is the 2026 modernization with one function substitution.

Use It

The PyTorch/TF reference implementations: `nn.TransformerEncoderLayer`, `nn.TransformerDecoderLayer`. But most 2026 production code rolls its own block because:

- Flash Attention is called inside attention, not via `nn.MultiheadAttention`.
- GQA / MLA are not in the stdlib reference.
- RoPE, RMSNorm, SwiGLU are not the PyTorch defaults.

HF transformers has clean reference blocks you should read: `modeling_llama.py` is the canonical 2026 decoder-only block. It's ~500 lines and worth walking through once.

Encoder vs decoder vs encoder-decoder — when to pick:

Need	Pick	Example
Classification, embeddings, QA over text	Encoder-only	BERT, DeBERTa, ModernBERT
Text generation, chat, code, reasoning	Decoder-only	GPT, Llama, Claude, Qwen
Structured input → structured output (translation, summarization)	Encoder-decoder	T5, BART, Whisper

Decoder-only won language because it scales cleanest and handles both comprehension and generation. Encoder-decoder is still best when the input has a clear “source sequence” identity (translation, speech recognition, structured tasks).

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/05-full-transformer>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/05-full-transformer/code>

1. **Easy.** Count the parameters in your encoder block at `d_model=512`, `n_heads=8`, `ffn_expansion=4`, `swiglu=True`. Validate by implementing the block and using `sum(p.numel() for p in block.parameters())`.
2. **Medium.** Switch from post-norm to pre-norm. Initialize both and measure the activation norm after 12 stacked layers on random input. Post-norm’s activations should explode; pre-norm’s should stay bounded.
3. **Hard.** Implement a 4-layer encoder-decoder on a toy copy task (copy x reversed). Train 100 steps. Report loss. Swap in RMSNorm + SwiGLU + RoPE — does loss drop?

Key Terms

Term	What people say	What it actually means
Block	“One transformer layer”	Stack of norm + attention + norm + FFN, wrapped in residual connections.
Residual	“Skip connection”	$x + f(x)$ output; enables gradient flow through deep stacks.
Pre-norm	“Normalize before, not after”	Modern: $x + \text{sublayer}(\text{LN}(x))$. Trains deeper without warmup gymnastics.
RMSNorm	“LayerNorm without the mean”	Divide by RMS; one less op, same empirical stability.
SwiGLU	“The FFN everyone switched to”	$\text{Swish}(W1 \cdot x) \odot W3 \cdot x \rightarrow W2$. Beats ReLU/GELU on LM ppl.
Cross-attention	“How the decoder sees the encoder”	MHA with Q from decoder, K/V from encoder outputs.
FFN expansion	“How wide the middle MLP is”	Ratio of hidden-size to <code>d_model</code> , usually 4 (LayerNorm) or 2.6 (SwiGLU).
Bias-free	“Drop the +b terms”	Modern stacks omit biases in linear layers; slight ppl improvement, smaller model.

Further Reading

- Vaswani et al. (2017). Attention Is All You Need — original block spec.
- Xiong et al. (2020). On Layer Normalization in the Transformer Architecture — why pre-norm beats post-norm deeply.
- Zhang, Sennrich (2019). Root Mean Square Layer Normalization — RMSNorm.
- Shazeer (2020). GLU Variants Improve Transformer — the SwiGLU paper.
- HuggingFace `modeling_llama.py` — canonical 2026 decoder-only block.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/05-full-transformer>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/05-full-transformer/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

BERT — Masked Language Modeling

GPT predicts the next word. BERT predicts a missing word. One sentence of difference — and half a decade of everything embedding-shaped.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 05 (Full Transformer), Phase 5 · 02 (Text Representation) **Time:** ~45 minutes

The Problem

In 2018 every NLP task — sentiment, NER, QA, entailment — trained its own model from scratch on its own labeled data. There was no pre-trained “understand English” checkpoint you could fine-tune. ELMo (2018) showed you could pre-train contextual embeddings with a bidirectional LSTM; it helped but did not generalize.

BERT (Devlin et al. 2018) asked: what if we took a transformer encoder, trained it on every sentence on the internet, and forced it to predict missing words from context on both sides? Then you fine-tune one head on your downstream task. Parameter efficiency was a revelation.

The result: within 18 months BERT and its variants (RoBERTa, ALBERT, ELECTRA) dominated every NLP leaderboard that existed. By 2020 every search engine, content moderation pipeline, and semantic-search system on earth had a BERT inside.

In 2026 encoder-only models are still the right tool for classification, retrieval, and structured extraction — they run 5-10× faster per token than decoders and their embeddings are the backbone of every modern retrieval stack. ModernBERT (Dec 2024) pushed the architecture to 8K context with Flash Attention + RoPE + GeGLU.

The Concept

The training signal

Take a sentence: the quick brown fox jumps over the lazy dog.

Mask 15% of tokens randomly:

input: the [MASK] brown fox jumps [MASK] the lazy dog
target: the quick brown fox jumps over the lazy dog

Train the model to predict the original tokens at masked positions. Because the encoder is bidirectional, predicting [MASK] at position 1 can use brown fox jumps at positions 2+. That is the thing GPT cannot do.

The BERT mask rules

Of the 15% of tokens selected for prediction:

- 80% are replaced with [MASK].
- 10% are replaced with a random token.
- 10% are left unchanged.

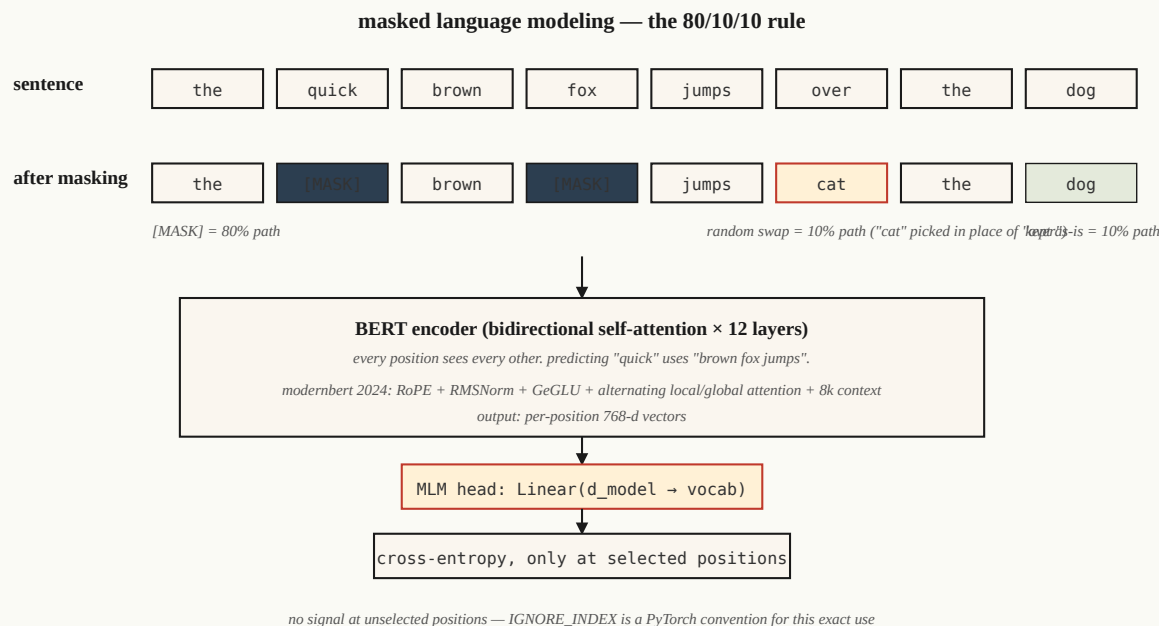


Figure 25: Masked language modeling: pick tokens, mask them, predict originals

Why not always [MASK]? Because [MASK] never appears at inference time. Training the model to expect [MASK] at 100% of masked positions would create a distribution shift between pre-training and fine-tuning. The 10% random + 10% unchanged keeps the model honest.

Next Sentence Prediction (NSP) — and why it was dropped

Original BERT also trained on NSP: given two sentences A and B, predict if B follows A. RoBERTa (2019) ablated it and showed NSP hurt, not helped. Modern encoders skip it.

What changed in 2026: ModernBERT

The 2024 ModernBERT paper rebuilt the block with 2026 primitives:

Component	Original BERT (2018)	ModernBERT (2024)
Positional	Learned absolute	RoPE
Activation	GELU	GeGLU
Normalization	LayerNorm	Pre-norm RMSNorm
Attention	Full dense	Alternating local (128) + global
Context length	512	8192
Tokenizer	WordPiece	BPE

And unlike the 2018 stack, it is Flash-Attention-native. Inference is 2–3× faster at sequence length 8K than DeBERTa-v3 with better GLUE scores.

Use cases that still pick an encoder in 2026

Task	Why encoder beats decoder
Retrieval / semantic search embeddings	Bidirectional context = better embedding quality per token
Classification (sentiment, intent, toxicity)	One forward pass; no generation overhead
NER / token labeling	Per-position output, natively bidirectional
Zero-shot entailment (NLI)	Classifier head on top of encoder
Reranker for RAG	Cross-encoder scoring, 10x faster than LLM rerankers

Interactive figure: transformer-residual. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/06-bert-masked-language-modeling>

Build It

Step 1: masking logic

See code/main.py. The function `create_mlm_batch` takes a list of token IDs, a vocab size, and a mask probability. Returns input IDs (with masks applied) and labels (only at masked positions, -100 elsewhere — PyTorch’s ignore index convention).

```
def create_mlm_batch(tokens, vocab_size, mask_prob=0.15, rng=None):
    input_ids = list(tokens)
    labels = [-100] * len(tokens)
    for i, t in enumerate(tokens):
        if rng.random() < mask_prob:
            labels[i] = t
            r = rng.random()
            if r < 0.8:
                input_ids[i] = MASK_ID
            elif r < 0.9:
                input_ids[i] = rng.randrange(vocab_size)
            # else: keep original
    return input_ids, labels
```

Step 2: run MLM prediction on a tiny corpus

Train a 2-layer encoder + MLM head on a vocabulary of 20 words, 200 sentences. No gradient — we do forward-pass sanity checks. Full training needs PyTorch.

Step 3: compare mask types

Show how the three-way rule keeps the model usable without [MASK]. Predict on an unmasked sentence and on a masked sentence. Both should produce reasonable token distributions because the model saw both patterns in training.

Step 4: fine-tune head

Replace the MLM head with a classification head on a toy sentiment dataset. Only the head trains; the encoder is frozen. This is the pattern every BERT application follows.

Use It

```
from transformers import AutoModel, AutoTokenizer

tok = AutoTokenizer.from_pretrained("answerdotai/ModernBERT-base")
model = AutoModel.from_pretrained("answerdotai/ModernBERT-base")

text = "Attention is all you need."
inputs = tok(text, return_tensors="pt")
out = model(**inputs).last_hidden_state  # (1, N, 768)
```

Embedding models are fine-tuned BERT. sentence-transformers models like all-MiniLM-L6-v2 are BERTs trained with contrastive loss. The encoder is the same. The loss changed.

Cross-encoder rerankers are also fine-tuned BERT. Pair-classification on [CLS] query [SEP] doc [SEP]. The bidirectional attention between query and doc is exactly what gives cross-encoders their quality edge over biencoders.

When not to pick BERT in 2026. Anything generative. The encoder has no sensible way to autoregressively produce tokens. Also: anything under 1B params where a small decoder can match quality with more flexibility (Phi-3-Mini, Qwen2-1.5B).

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/06-bert-masked-language-modeling>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/06-bert-masked-language-modeling/code>

1. **Easy.** Run code/main.py and print the mask distribution across 10,000 tokens. Confirm ~15% are selected, and of those ~80% become [MASK].
2. **Medium.** Implement whole-word masking: if a word is tokenized into subwords, mask all subwords together or none. Measure whether this improves MLM accuracy on a 500-sentence corpus.
3. **Hard.** Train a tiny (2-layer, d=64) BERT on 10,000 sentences from a public dataset. Fine-tune the [CLS] token for SST-2 sentiment. Compare against a decoder-only baseline at matched params — which wins?

Key Terms

Term	What people say	What it actually means
MLM	“Masked language modeling”	Training signal: randomly replace 15% of tokens with [MASK], predict the originals.

Term	What people say	What it actually means
Bidirectional	“Looks both ways”	Encoder attention has no causal mask — every position sees every other position.
[CLS]	“The pooler token”	A special token prepended to every sequence; its final embedding is used as the sentence-level representation.
[SEP]	“Segment separator”	Separates paired sequences (e.g. query/doc, sentence A/B).
NSP	“Next sentence prediction”	BERT’s second pretraining task; shown to be useless in RoBERTa, dropped after 2019.
Fine-tuning	“Adapt to a task”	Keep the encoder mostly frozen; train a small head on top for the downstream task.
Cross-encoder	“A reranker”	A BERT that takes both query and doc as input, outputs a relevance score.
ModernBERT	“2024 refresh”	Encoder rebuilt with RoPE, RMSNorm, GeGLU, alternating local/global attention, 8K context.

Further Reading

- Devlin et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding — original paper.
- Liu et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach — how to train BERT right; kills NSP.
- Clark et al. (2020). ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators — replaced-token detection beats MLM at matched compute.
- Warner et al. (2024). Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder — ModernBERT paper.
- HuggingFace `modeling_bert.py` — canonical encoder reference.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/06-bert-masked-language-modeling>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/06-bert-masked-language-modeling/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

GPT — Causal Language Modeling

BERT sees both sides. GPT sees only the past. The triangle mask is the most consequential single line of code in modern AI.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 02 (Self-Attention), Phase 7 · 05 (Full Transformer), Phase 7 · 06 (BERT) **Time:** ~75 minutes

The Problem

A language model answers one question: given the first $t-1$ tokens, what is the probability distribution over token t ? Train on that signal — next-token prediction — and you get a model that can generate arbitrary text one token at a time.

To train it end-to-end on a whole sequence in parallel, you need each position's prediction to depend only on earlier positions. Otherwise the model trivially cheats by looking at the answer.

The causal mask does this. It is a single upper-triangular matrix of $-\infty$ values added to attention scores before softmax. After softmax, those positions become 0. Each position can attend only to itself and earlier positions. And because you apply it once to the whole sequence, you get N parallel next-token predictions in one forward pass.

GPT-1 (2018), GPT-2 (2019), GPT-3 (2020), GPT-4 (2023), GPT-5 (2025), Claude, Llama, Qwen, Mistral, DeepSeek, Kimi — they are all decoder-only causal transformers with the same core loop. What separates them is data quality, scale and architectural refinements, and post-training (SFT, RLHF, DPO, and their successors).

The Concept

The mask

Given a sequence of length N , build an $N \times N$ matrix:

$$\begin{aligned} M[i, j] &= 0 && \text{if } j \leq i \\ M[i, j] &= -\infty && \text{if } j > i \end{aligned}$$

Add M to the raw attention scores before softmax. $\exp(-\infty) = 0$, so masked positions contribute zero weight. Each row of the attention matrix is a probability distribution over previous positions only.

Implementation cost: one `torch.tril()` call. Time to compute: nanoseconds. Impact on the field: everything.

Where the triangle comes from

The mask is usually presented as a patch bolted onto attention. Run the derivation in the other direction and it stops being mysterious: attention is the third refinement of a prefix average, and the triangle is the loop bounds of that average, written as a matrix.

causal mask — one triangle, half a decade of progress

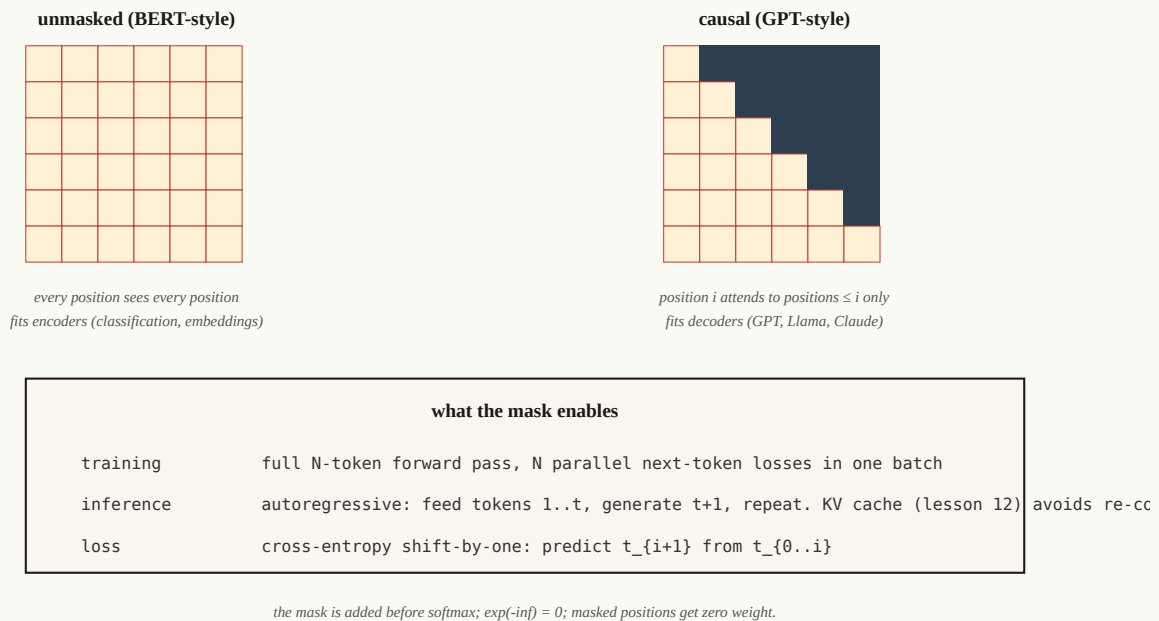


Figure 26: Causal mask creates a triangular attention matrix

Stage 1 — prefix average. The dumbest causal summary of a sequence: position i becomes the mean of positions $0 \dots i$. As a loop, that is $\text{out}[i] = X[:i+1].\text{mean}()$. The same computation is one matrix multiply. Take a lower-triangular matrix of ones, divide each row by its count, multiply:

```
import numpy as np
```

```
A = np.tril(np.ones((n, n)))
A = A / A.sum(axis=1, keepdims=True)
out = A @ X
```

Row i of A is $[1/(i+1), \dots, 1/(i+1), 0, \dots, 0]$. The zeros above the diagonal are the causality. Nothing about the future was masked out; the future was never in the sum.

Stage 2 — learned weights. A uniform average treats every past token as equally relevant. Replace the ones with a learned score matrix S . Now rows no longer sum to one by construction, so normalize each row with softmax instead of dividing by the count. Softmax never outputs an exact zero, which breaks causality — unless the future scores go in as $-\text{inf}$, because $\exp(-\text{inf}) = 0$:

```
def softmax(x, axis):
    e = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e / e.sum(axis=axis, keepdims=True)
```

```
S = S + np.triu(np.full((n, n), -np.inf), k=1)
A = softmax(S, axis=1)
out = A @ X
```

Same triangle, same row-stochastic matrix, same one matmul. The $-\text{inf}$ mask is not new machinery. It is stage 1's zero entries, translated into softmax's input domain.

Stage 3 — content-dependent weights. In stage 2, S is fixed after training: position 7 always weighs position 3 the same, whatever the tokens say. Let the scores depend on the tokens themselves: $S = Q @ K.T / \sqrt{d_k}$. Nothing else changes. Mask, softmax, matmul — identical.

Three stages, one invariant: a lower-triangular row-stochastic matrix times the sequence. Uniform average, learned static weights, content-dependent weights. The mask was never added to attention. It survived from the average.

Interactive figure: mask-derivation. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/07-gpt-causal-language-modeling>

Parallel training, serial inference

Training: forward-pass the whole (N , d_{model}) sequence once, compute N cross-entropy losses (one per position), sum, backprop. Parallel along the sequence. This is why GPT training scales — you process 1M tokens in a batch in one GPU pass.

Inference: you generate token by token. Feed $[t_1, t_2, t_3]$, get t_4 . Feed $[t_1, t_2, t_3, t_4]$, get t_5 . Feed $[t_1, t_2, t_3, t_4, t_5]$, get t_6 . The KV cache (Lesson 12) saves the hidden states of $t_1..t_n$ so you don't recompute them each step. But serial depth at inference = output length. That is the autoregressive tax and why decoding is the latency bottleneck of every LLM.

The loss — shift-by-one

Given tokens $[t_1, t_2, t_3, t_4]$:

- Input: $[t_1, t_2, t_3]$
- Targets: $[t_2, t_3, t_4]$

For every position i , compute $-\log P(\text{target}_i \mid \text{inputs}[:i+1])$. Sum. This is the cross-entropy for the whole sequence.

Every transformer LM you've heard of trains on this loss. Pre-training, fine-tuning, SFT — same loss, different data.

Decoding strategies

After training, sampling choices matter more than people think.

Method	What it does	When to use
Greedy	Argmax every step	Deterministic tasks, code completion
Temperature	Divide logits by T , sample	Creative tasks, higher T = more diversity
Top-k	Sample from top-k tokens only	Kills low-probability tails
Top-p (nucleus)	Sample from smallest set with cumulative prob $\geq p$	2020+ default; adapts to distribution shape
Min-p	Keep tokens with $p > \min_p * \max_p$	2024+; better at rejecting long tails than top-p
Speculative decoding	Draft model proposes N tokens, big model verifies	2-3× latency reduction at same quality

In 2026, min-p + temperature 0.7 is a reasonable default for open-weights models. Speculative decoding is table stakes for any production inference stack.

What made the “GPT recipe” work

1. **Decoder-only.** No encoder overhead. One pass of attention + FFN per layer.
2. **Scaling.** 124M → 1.5B → 175B → trillions. Chinchilla scaling laws (Lesson 13) tell you how to spend compute.
3. **In-context learning.** Emerged around 6B–13B. The model can follow few-shot examples without fine-tuning.
4. **RLHF.** Post-training on human preferences converted raw pretrained text into chat assistants.
5. **Pre-norm + RoPE + SwiGLU.** Stable training at scale.

The core architecture hasn’t changed much since GPT-2. Everything interesting has happened in data, scale, and post-training.

Interactive figure: causal-mask. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/07-gpt-causal-language-modeling>

Build It

Step 1: the causal mask

See code/main.py. A one-liner:

```
def causal_mask(n):
    return [[0.0 if j <= i else float("-inf") for j in range(n)] for i in range(n)]
```

Add it to attention scores before softmax. That’s the entire mechanism.

Step 2: a 2-layer GPT-ish model

Stack two decoder blocks (masked self-attention + FFN, no cross-attention). Add a token embedding, a positional encoding, and an unembedding (tied to the token embedding matrix — a standard trick since GPT-2).

Step 3: next-token prediction, end-to-end

On a 20-token toy vocab, produce logits at every position. Compute cross-entropy loss against the shift-by-one target. No gradient — this is a forward-pass sanity check.

Step 4: sampling

Implement greedy, temperature, top-k, top-p, min-p. Run each on a fixed prompt and compare outputs. A sampling function is 10 lines.

Use It

PyTorch, 2026 idiom:

```

from transformers import AutoModelForCausalLM, AutoTokenizer
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3.2-3B-Instruct")
tok = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-3B-Instruct")

prompt = "Attention is all you need because"
inputs = tok(prompt, return_tensors="pt")
out = model.generate(
    **inputs,
    max_new_tokens=64,
    temperature=0.7,
    top_p=0.9,
    do_sample=True,
)
print(tok.decode(out[0]))

```

Under the hood, `generate()` runs the forward pass, pulls the final-position logits, samples the next token, appends it, and repeats. Every production LLM inference stack (vLLM, TensorRT-LLM, llama.cpp, Ollama, MLX) implements the same loop with heavy optimization — batched prefill, continuous batching, KV cache paging, speculative decoding.

GPT vs BERT, one line each: GPT predicts $P(x_t \mid x_{<t})$. BERT predicts $P(x_{\text{masked}} \mid x_{\text{unmasked}})$. The loss determines whether the model can generate.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/07-gpt-causal-language-modeling>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/07-gpt-causal-language-modeling/code>

1. **Easy.** Run `code/main.py` and verify the causal attention matrix is lower-triangular after softmax. Spot-check: row 3 should have weights only in columns 0-3.
2. **Medium.** Implement beam search for width 4. Compare perplexity of beam-4 vs greedy on 10 short prompts. Does beam always win? (Hint: usually for translation, not for open-ended chat.)
3. **Hard.** Implement speculative decoding: use a tiny 2-layer model as the draft and a 6-layer model as the verifier. Measure wall-clock speedup on 100 completions of length 64. Confirm outputs match greedy of the verifier.

Key Terms

Term	What people say	What it actually means
Causal mask	“The triangle”	Upper-triangular $-\infty$ matrix added to attention scores so position i only sees positions $\leq i$.
Next-token prediction	“The loss”	Cross-entropy of the model’s distribution against the true next token at every position.

Term	What people say	What it actually means
Autoregressive	“Generate one at a time”	Feed output back as input; parallelism only during training, not during generation.
Logits	“Pre-softmax scores”	Raw output of the LM head before softmax; sampling happens on these.
Temperature	“Creativity knob”	Divide logits by T ; $T \rightarrow 0$ = greedy, $T \rightarrow \infty$ = uniform.
Top-p	“Nucleus sampling”	Truncate distribution to smallest set summing to $\geq p$; sample from what remains.
Min-p	“Better than top-p”	Keep tokens where $p \geq \min_p \times \max_p$; adapts cutoff to sharpness of distribution.
Speculative decoding	“Draft + verify”	Cheap model proposes N tokens; big model verifies in parallel.
Teacher forcing	“Training trick”	During training, feed the true previous token, not the model’s prediction. Standard for every seq2seq LM.

Further Reading

- Radford et al. (2018). Improving Language Understanding by Generative Pre-Training — GPT-1.
- Radford et al. (2019). Language Models are Unsupervised Multitask Learners — GPT-2.
- Brown et al. (2020). Language Models are Few-Shot Learners — GPT-3 and in-context learning.
- Leviathan, Kalman, Matias (2023). Fast Inference from Transformers via Speculative Decoding — spec decoding paper.
- HuggingFace `modeling_llama.py` — canonical causal-LM reference code.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/07-gpt-causal-language-modeling>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/07-gpt-causal-language-modeling/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

T5, BART — Encoder-Decoder Models

Encoders understand. Decoders generate. Put them back together and you get a model built for input → output tasks: translate, summarize, rewrite, transcribe.

Type: Learn **Languages:** Python **Prerequisites:** Phase 7 · 05 (Full Transformer), Phase 7 · 06 (BERT), Phase 7 · 07 (GPT) **Time:** ~45 minutes

The Problem

Decoder-only GPT and encoder-only BERT each strip down the 2017 architecture for a different goal. But many tasks are naturally input-output:

- Translation: English → French.
- Summarization: 5,000-token article → 200-token summary.
- Speech recognition: audio tokens → text tokens.
- Structured extraction: prose → JSON.

For these, encoder-decoder makes the cleanest fit. The encoder produces a dense representation of the source. The decoder generates the output, cross-attending to that representation at every step. Training is shift-by-one on the output side. Same loss as GPT, just conditioned on the encoder output.

Two papers defined the modern playbook:

1. **T5** (Raffel et al. 2019). “Text-to-Text Transfer Transformer.” Every NLP task reframed as text-in, text-out. Single architecture, single vocabulary, single loss. Pretrained on masked span prediction (corrupt spans in the input, decode them in the output).
2. **BART** (Lewis et al. 2019). “Bidirectional and Auto-Regressive Transformer.” Denoising autoencoder: corrupt input in multiple ways (shuffle, mask, delete, rotate), ask the decoder to reconstruct the original.

In 2026 the encoder-decoder format lives on where input structure matters:

- Whisper (speech → text).
- Google’s translation stack.
- Some code-completion / repair models that have distinct context-and-edit structures.
- Flan-T5 and variants for structured reasoning tasks.

Decoder-only won the spotlight, but encoder-decoder never went away.

The Concept

The forward loop

```
source tokens → encoder → (N_src, d_model) ┌
                                           │
target tokens → decoder block              │
                    └─ masked self-attention ─┘
```

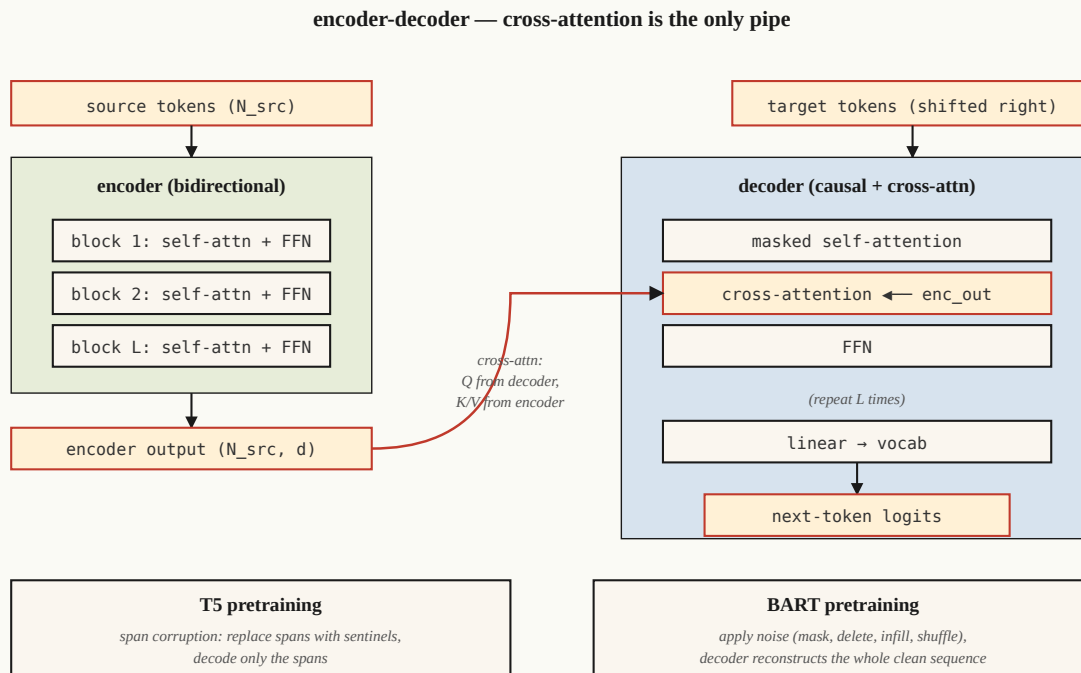
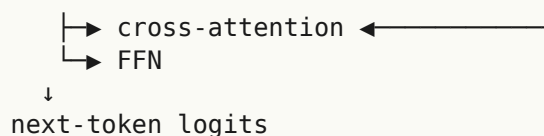


Figure 27: Encoder-decoder with cross-attention



Crucially, the encoder runs once per input. The decoder runs autoregressively but cross-attends to the *same* encoder output at every step. Caching the encoder output is a free speedup for long inputs.

T5 pretraining — span corruption

Pick random spans of the input (average length 3 tokens, 15% total). Replace each span with a unique sentinel: `<extra_id_0>`, `<extra_id_1>`, etc. The decoder outputs only the corrupted spans with their sentinel prefix:

source: The quick `<extra_id_0>` fox jumps `<extra_id_1>` dog

target: `<extra_id_0>` brown `<extra_id_1>` over the lazy

Cheaper signal than predicting the whole sequence. Competitive with MLM (BERT) and prefix-LM (UniLM) in the T5 paper's ablation.

BART pretraining — multi-noise denoising

BART tries five noising functions:

1. Token masking.
2. Token deletion.
3. Text infilling (mask a span, decoder inserts the right length).
4. Sentence permutation.

5. Document rotation.

Combining text infilling + sentence permutation produced the best downstream numbers. The decoder always reconstructs the original. BART's output is the full sequence, not just the corrupted spans — so pretraining compute is higher than T5.

Inference

Same autoregressive generation as GPT. Greedy / beam / top-p sampling apply. Beam search (width 4–5) is standard for translation and summarization because the output distribution is narrower than chat.

When to pick each variant in 2026

Task	Encoder-decoder?	Why
Translation	Yes, usually	Clear source sequence; fixed output distribution; beam search works
Speech-to-text	Yes (Whisper)	Input modality differs from output; encoder shapes audio features
Chat / reasoning	No, decoder-only	No persistent “input” — the conversation is the sequence
Code completion	Usually no	Decoder-only with long context wins; code models like Qwen 2.5
Summarization	Either works	Coder are decoder-only BART, PEGASUS beat earlier decoder-only baselines; modern decoder-only LLMs match them
Structured extraction	Either	T5 is clean because “text → text” absorbs any output format

The trend since ~2022: decoder-only takes over tasks that encoder-decoder used to own because (a) instruction-tuned decoder-only LLMs generalize to anything via prompting, (b) one architecture scales easier than two, (c) RLHF assumes a decoder. Encoder-decoder holds on where input modality differs (speech, images) or where beam search quality matters.

Interactive figure: encoder-decoder. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/08-t5-bart-encoder-decoder>

Build It

See `code/main.py`. We implement T5-style span corruption for a toy corpus — the most useful single piece of this lesson because it shows up in every encoder-decoder pretraining recipe since.

Step 1: span corruption

```
def corrupt_spans(tokens, mask_rate=0.15, mean_span=3.0, rng=None):
    """Pick spans summing to ~mask_rate of tokens. Return (corrupted_input,
    ↪ target)."""
    n = len(tokens)
    n_mask = max(1, int(n * mask_rate))
    n_spans = max(1, int(round(n_mask / mean_span)))
    ...
```

The target format is the T5 convention: `<sent0> span0 <sent1> span1 ....` The corrupted input interleaves unchanged tokens with the sentinel tokens at span locations.

Step 2: verify round-trip

Given the corrupted input and target, reconstruct the original sentence. If your corruption is reversible, the forward pass is well-defined. This is a sanity check — real training never does this, but the test is cheap and catches off-by-one bugs in your span bookkeeping.

Step 3: BART noising

Five functions: `token_mask`, `token_delete`, `text_infill`, `sentence_permute`, `document_rotate`. Compose two of them and show the result.

Use It

HuggingFace reference:

```
from transformers import T5ForConditionalGeneration, T5Tokenizer
tok = T5Tokenizer.from_pretrained("google/flan-t5-base")
model = T5ForConditionalGeneration.from_pretrained("google/flan-t5-base")

inputs = tok("translate English to French: Attention is all you need.",
    ↪ return_tensors="pt")
out = model.generate(**inputs, max_new_tokens=32)
print(tok.decode(out[0], skip_special_tokens=True))
```

The T5 trick: the task name goes into the input text. Same model handles dozens of tasks because each task is text-in, text-out. In 2026 this pattern has been generalized by instruction-tuned decoder-only models, but T5 codified it first.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/08-t5-bart-encoder-decoder>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/08-t5-bart-encoder-decoder/code>

1. **Easy.** Run `code/main.py`, apply span corruption to a 30-token sentence, verify that concatenating the non-sentinel source tokens with the decoded target spans reproduces the original.
2. **Medium.** Implement BART’s `text_infill` noise: replace random spans with a single `<mask>` token, and the decoder must infer the correct span length plus contents. Show one example.
3. **Hard.** Fine-tune `flan-t5-small` on a tiny English → pig-Latin corpus (200 pairs). Measure BLEU on a held-out 50-pair set. Compare against fine-tuning Llama-3.2-1B on the same data with the same compute.

Key Terms

Term	What people say	What it actually means
Encoder-decoder	“Seq2seq transformer”	Two stacks: bidirectional encoder for input, causal decoder with cross-attention for output.
Cross-attention	“Where source talks to target”	Decoder’s Q × encoder’s K/V. The only place encoder information enters the decoder.
Span corruption	“T5’s pretraining trick”	Replace random spans with sentinel tokens; decoder outputs the spans.
Denoising objective	“BART’s game”	Apply a noise function to the input, train the decoder to reconstruct the clean sequence.
Sentinel token	“The <code><extra_id_N></code> placeholder”	Special tokens that tag corrupted spans in the source and re-tag them in the target.
Flan	“Instruction-tuned T5”	T5 fine-tuned on >1,800 tasks; made encoder-decoder competitive at instruction-following.
Beam search	“Decoding strategy”	Keep top-k partial sequences at each step; standard for translation/summarization.
Teacher forcing	“Training-time input”	During training, feed the true previous output token to the decoder, not the sampled one.

Further Reading

- Raffel et al. (2019). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer — T5.
- Lewis et al. (2019). BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension — BART.
- Chung et al. (2022). Scaling Instruction-Finetuned Language Models — Flan-T5.
- Radford et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision — Whisper, the canonical 2026 encoder-decoder.
- HuggingFace `modeling_t5.py` — reference implementation.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/08-t5-bart-encoder-decoder>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/08-t5-bart-encoder-decoder/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Vision Transformers (ViT)

An image is a grid of patches. A sentence is a grid of tokens. The same transformer eats both.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 05 (Full Transformer), Phase 4 · 03 (CNNs), Phase 4 · 14 (Vision Transformers intro) **Time:** ~45 minutes

The Problem

Before 2020, computer vision meant convolutions. Every SOTA on ImageNet, COCO, and detection benchmarks used a CNN backbone. Transformers were for language.

Dosovitskiy et al. (2020) — “An Image is Worth 16x16 Words” — showed you can drop the convolutions entirely. Slice an image into fixed-size patches, linearly project each patch into an embedding, feed the sequence to a vanilla transformer encoder. At sufficient scale (ImageNet-21k pretraining or bigger), ViT matches or beats ResNet-based models.

ViT was the start of a broader pattern in 2026: one architecture, many modalities. Whisper tokenizes audio. ViT tokenizes images. Action tokens for robotics. Pixel tokens for video. The transformer doesn’t care — feed it a sequence and it learns.

By 2026, ViT and its descendants (DeiT, Swin, DINOv2, ViT-22B, SAM 3) own most of vision. CNNs still win on edge devices and latency-sensitive tasks. Everything else has a ViT somewhere in the stack.

The Concept

Step 1 — patchify

Split a $H \times W \times C$ image into an $N \times (P \cdot P \cdot C)$ sequence of flat patches. Typical setup: 224×224 image, 16×16 patches → 196 patches of 768 values each.

image (224, 224, 3) → 14×14 grid of $16 \times 16 \times 3$ patches → 196 vectors of length 768

Patch size is the lever. Smaller patches = more tokens, better resolution, quadratic attention cost. Larger patches = coarser, cheaper.

Step 2 — linear embedding

A single learned matrix projects each flat patch to `d_model`. Equivalent to a convolution of kernel size `P` and stride `P`. In PyTorch this is literally `nn.Conv2d(C, d_model, kernel_size=P, stride=P)` — a 2-line implementation.

Step 3 — prepend [CLS] token, add positional embeddings

- Prepend a learnable [CLS] token. Its final hidden state is the image representation used for classification.

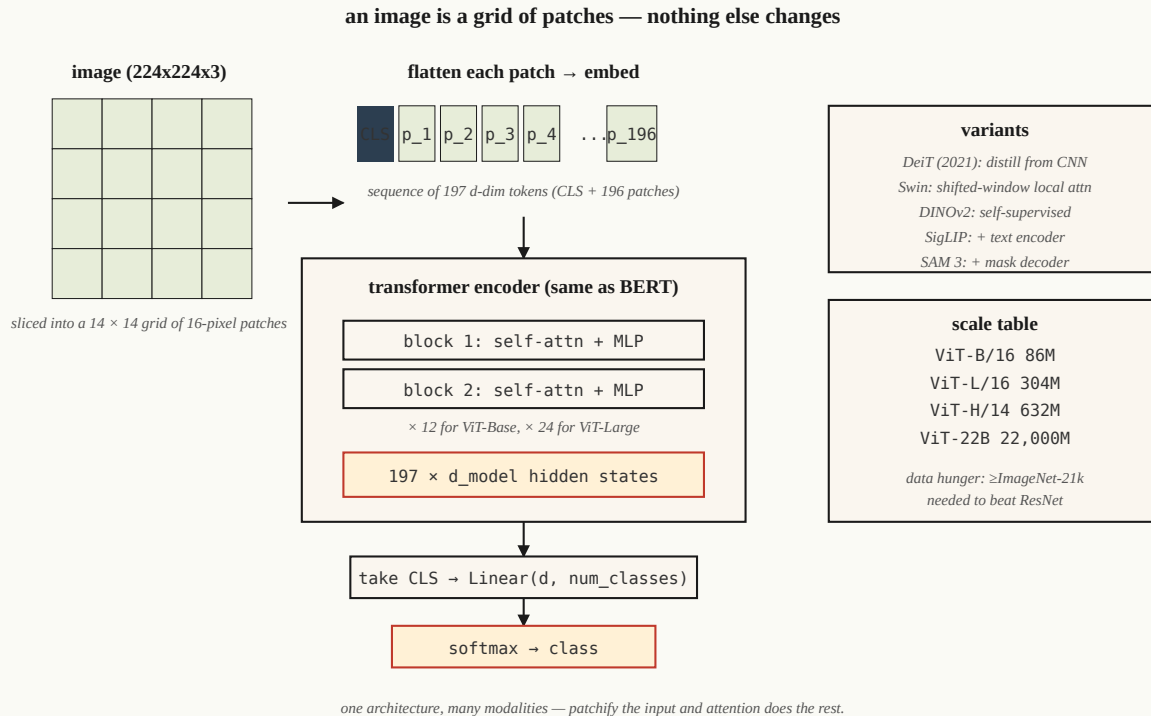


Figure 28: Image → patches → tokens → transformer

- Add learnable positional embeddings (ViT-original) or sinusoidal 2D (later variants).
- In 2024+ RoPE extended to 2D for position, sometimes without explicit embeddings.

Step 4 — standard transformer encoder

Stack L blocks of $\text{LayerNorm} \rightarrow \text{Self-Attention} \rightarrow + \rightarrow \text{LayerNorm} \rightarrow \text{MLP} \rightarrow +$. Identical to BERT. No vision-specific layers. This is the pedagogical punchline of the paper.

Step 5 — head

For classification: take [CLS] hidden state → linear → softmax. For DINOv2 or SAM, discard [CLS], use the patch embeddings directly.

Variants that mattered

Model	Year	Change
ViT	2020	The original. Fixed patch size, full global attention.
DeiT	2021	Distillation; trainable on ImageNet-1k only.
Swin	2021	Hierarchical with shifted windows. Fixed sub-quadratic cost.
DINOv2	2023	Self-supervised (no labels). Best general vision features.

Model	Year	Change
ViT-22B	2023	22B params; scaling laws apply.
SigLIP	2023	ViT + language pair, sigmoid contrastive loss.
SAM 3	2025	Segment anything; ViT-Large + promptable mask decoder.

Why it took a while

ViT needs *a lot* of data to match CNNs because it has none of the CNN inductive biases (translation invariance, locality). Without >100M labeled images or strong self-supervised pretraining, CNNs still win at matched compute. DeiT fixed this in 2021 with distillation tricks; DINOv2 fixed it permanently in 2023 with self-supervision.

Interactive figure: n5-patch-stream. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/09-vision-transformers>

Build It

See `code/main.py`. Pure-stdlib patchify + linear embedding + sanity checks. No training — ViT at any realistic scale needs PyTorch and hours of GPU time.

Step 1: fake image

A 24×24 RGB image as a list of rows of (R, G, B) tuples. We use 6×6 patches $\rightarrow$ 16 patches, 108-d embedding vector each.

Step 2: patchify

```
def patchify(image, P):
    H = len(image)
    W = len(image[0])
    patches = []
    for i in range(0, H, P):
        for j in range(0, W, P):
            patch = []
            for di in range(P):
                for dj in range(P):
                    patch.extend(image[i + di][j + dj])
            patches.append(patch)
    return patches
```

Raster order: row-major across the grid. Every ViT uses this ordering.

Step 3: linear embed

Multiply each flat patch by a random (`patch_flat_size`, `d_model`) matrix. Verify output shape is (`N_patches + 1`, `d_model`) after prepending [CLS].

Step 4: count parameters for a realistic ViT

Print the param count for ViT-Base: 12 layers, 12 heads, $d=768$, $\text{patch}=16$. Compare to ResNet-50 ($\sim 25\text{M}$). ViT-Base lands at $\sim 86\text{M}$. ViT-Large $\sim 307\text{M}$. ViT-Huge $\sim 632\text{M}$.

Use It

```
from transformers import ViTImageProcessor, ViTModel
import torch
from PIL import Image

processor = ViTImageProcessor.from_pretrained("google/vit-base-patch16-224-in21k")
model = ViTModel.from_pretrained("google/vit-base-patch16-224-in21k")

img = Image.open("cat.jpg")
inputs = processor(img, return_tensors="pt")
out = model(**inputs).last_hidden_state  # (1, 197, 768): [CLS] + 196 patches
cls_emb = out[:, 0]                     # image representation
```

DINOv2 embeddings are the 2026 default for image features. Freeze the backbone, train a tiny head. Works for classification, retrieval, detection, captioning. Meta's DINOv2 checkpoints outperform CLIP on every non-text vision task.

Patch-size picking. Small models use 16×16 (ViT-B/16). Dense prediction (segmentation) uses 8×8 or 14×14 (SAM, DINOv2). Very large models use 14×14 .

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/09-vision-transformers>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/09-vision-transformers/code>

1. **Easy.** Run `code/main.py`. Verify the number of patches equals $(H/P) * (W/P)$ and the flat patch dimension equals $P*P*C$.
2. **Medium.** Implement 2D sinusoidal positional embeddings — two independent sinusoidal codes for row and col of each patch, concatenated. Feed them into a tiny PyTorch ViT and compare accuracy vs learnable positional embeddings on CIFAR-10.
3. **Hard.** Build a 3-layer ViT (PyTorch), train on 1,000 MNIST images with 4×4 patches. Measure test accuracy. Now add DINOv2 pretraining on the same 1,000 images (simplified: just train the encoder to predict patch embeddings from masked patches). Does accuracy improve?

Key Terms

Term	What people say	What it actually means
Patch	“The vision-transformer token”	Flat vector of pixel values for a $P \times P \times C$ region of the image.
Patchify	“Chop + flatten”	Slice image into non-overlapping patches, flatten each to a vector.
[CLS] token	“The image summary”	Prepended learnable token; its final embedding is the image representation.
Inductive bias	“What the model assumes”	ViT has fewer priors than CNNs; needs more data to make up the gap.
DINOv2	“Self-supervised ViT”	Trained without labels using image augmentation + momentum teacher. Best general image features in 2026.
SigLIP	“CLIP’s successor”	ViT + text encoder trained with sigmoid contrastive loss; better than CLIP on matched compute.
Swin	“Windowed ViT”	Hierarchical ViT with local attention + shifted windows; sub-quadratic.
Register tokens	“2023 trick”	A few extra learnable tokens that soak up attention sinks; improves DINOv2 features.

Further Reading

- Dosovitskiy et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale — the ViT paper.
- Touvron et al. (2021). Training data-efficient image transformers & distillation through attention — DeiT.
- Liu et al. (2021). Swin Transformer: Hierarchical Vision Transformer using Shifted Windows — Swin.
- Oquab et al. (2023). DINOv2: Learning Robust Visual Features without Supervision — DINOv2.
- Darcet et al. (2023). Vision Transformers Need Registers — the register-token fix for DINOv2.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/09-vision-transformers>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/09-vision-transformers/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Audio Transformers — Whisper Architecture

Audio is an image of frequency over time. Whisper is a ViT that eats mel spectrograms and speaks back.

Type: Learn **Languages:** Python **Prerequisites:** Phase 7 · 05 (Full Transformer), Phase 7 · 08 (Encoder-Decoder), Phase 7 · 09 (ViT) **Time:** ~45 minutes

The Problem

Before Whisper (OpenAI, Radford et al. 2022), state-of-the-art automatic speech recognition (ASR) meant wav2vec 2.0 and HuBERT — self-supervised feature extractors plus a fine-tuned head. High quality, expensive data pipelines, domain-brittle. Multilingual speech recognition needed separate models per language family.

Whisper made three bets:

1. **Train on everything.** 680,000 hours of weakly-labeled audio scraped from the internet across 97 languages. No clean academic corpus. No phoneme labels.
2. **Multi-task single model.** One decoder trained jointly on transcription, translation, voice activity detection, language ID, and timestamping via task tokens.
3. **Standard encoder-decoder transformer.** Encoder consumes log-mel spectrograms. Decoder produces text tokens autoregressively. No vocoder, no CTC, no HMM.

The result: Whisper large-v3 is robust across accents, noise, and languages that have zero clean labeled data. It is the default speech front-end for every open-source voice assistant and most commercial ones in 2026.

The Concept

Step 1 — resample + window

Audio at 16 kHz. Clip/pad to 30 seconds. Compute log-mel spectrogram: 80 mel bins, 10 ms stride → ~3,000 frames × 80 features. This is the “input image” that Whisper sees.

Step 2 — convolutional stem

Two Conv1D layers with kernel 3 and stride 2 reduce the 3,000 frames to 1,500. Halves sequence length without adding a lot of parameters.

Step 3 — encoder

A 24-layer (for large) transformer encoder over 1,500 timesteps. Sinusoidal positional encoding, self-attention, GELU FFN. Produces $1,500 \times 1,280$ hidden states.

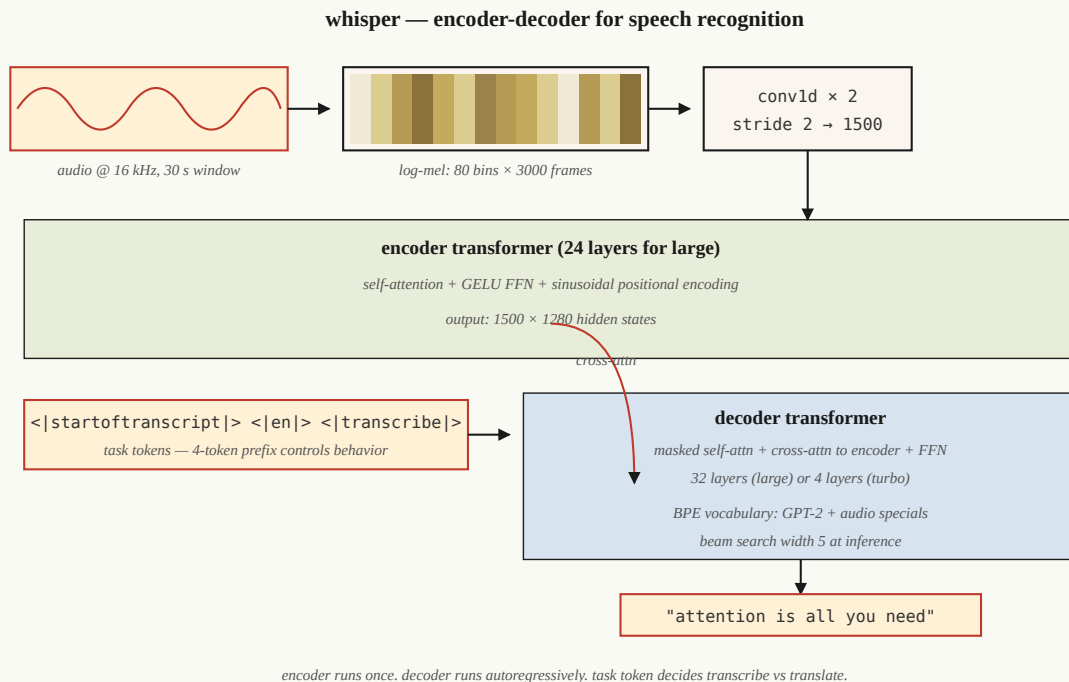


Figure 29: Whisper pipeline: audio → mel → encoder → decoder → text

Step 4 — decoder

A 24-layer transformer decoder. It autoregressively produces tokens from a BPE vocabulary that is a superset of GPT-2's with a few audio-specific special tokens.

Step 5 — task tokens

The decoder prompt starts with control tokens that tell the model what to do:

```
<|startoftranscript|> <|en|> <|transcribe|> <|0.00|>
```

or

```
<|startoftranscript|> <|fr|> <|translate|> <|0.00|>
```

The model was trained on this convention. You control task by prefix. The 2026 equivalent of instruction-tuning, but applied to speech.

Step 6 — output

Beam search (width 5) with a log-prob threshold. Timestamps are predicted every 0.02 seconds of audio when the <|notimestamps|> token is absent.

Whisper sizes

Model	Params	Layers	d_model	Heads	VRAM (fp16)
Tiny	39M	4	384	6	~1 GB
Base	74M	6	512	8	~1 GB

Model	Params	Layers	d_model	Heads	VRAM (fp16)
Small	244M	12	768	12	~2 GB
Medium	769M	24	1024	16	~5 GB
Large	1550M	32	1280	20	~10 GB
Large-v3	1550M	32	1280	20	~10 GB
Large-v3-turbo	809M	32	1280	20	~6 GB (4-layer decoder)

Large-v3-turbo (2024) cut the decoder from 32 layers to 4. 8× faster decoding with <1 WER point regression. That decode speed unlock is why Whisper-turbo is the default for real-time voice agents in 2026.

What Whisper does not do

- No diarization (who is speaking). Pair with pyannote for that.
- No real-time streaming natively — the 30-second window is fixed. Modern wrappers (faster-whisper, WhisperX) bolt on streaming via VAD + overlap.
- No long-form context beyond 30 s without external chunking. Works well in practice because human speech rarely needs long-range context for transcription.

2026 landscape

Task	Model	Notes
English ASR	Whisper-turbo, Moonshine	Moonshine is 4× faster on edge
Multilingual ASR	Whisper-large-v3	97 languages
Streaming ASR	faster-whisper + VAD	150 ms latency targets achievable
TTS	Piper, XTTS-v2, Kokoro	Encoder-decoder pattern, but Whisper-shaped
Audio + language	AudioLM, SeamlessM4T	Text tokens + audio tokens in one transformer

Interactive figure: n5-mel-decode. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/10-audio-transformers-whisper>

Build It

See code/main.py. We don't train Whisper — we build the log-mel spectrogram pipeline + task-token prompt formatter. Those are the parts you actually touch in production.

Step 1: synthesize audio

Generate a 1-second sine wave at 440 Hz sampled at 16 kHz. 16,000 samples.

Step 2: log-mel spectrogram (simplified)

Full mel spectrogram needs FFT. We do a simplified framing + per-frame energy version that shows the pipeline without requiring librosa:

```
def frame_signal(x, frame_size=400, hop=160):
    frames = []
    for start in range(0, len(x) - frame_size + 1, hop):
        frames.append(x[start:start + frame_size])
    return frames
```

Frame = 25 ms, hop = 10 ms. Matches Whisper's windowing. Per-frame energy stands in for mel bins for pedagogy.

Step 3: pad to 30 s

Whisper always processes 30-second chunks. Pad (or clip) the spectrogram to 3,000 frames.

Step 4: build the prompt tokens

```
def whisper_prompt(lang="en", task="transcribe", timestamps=True):
    tokens = ["<|startoftranscript|>", f"<|{lang}|>", f"<|{task}|>"]
    if not timestamps:
        tokens.append("<|notimestamps|>")
    return tokens
```

That is the whole task-control surface. A 4-token prefix.

Use It

```
import whisper
model = whisper.load_model("large-v3-turbo")
result = model.transcribe("meeting.wav", language="en", task="transcribe")
print(result["text"])
print(result["segments"][0]["start"], result["segments"][0]["end"])
```

Faster, OpenAI-compatible:

```
from faster_whisper import WhisperModel
model = WhisperModel("large-v3-turbo", compute_type="int8_float16")
segments, info = model.transcribe("meeting.wav", vad_filter=True)
for s in segments:
    print(f"{s.start:.2f} - {s.end:.2f}: {s.text}")
```

When to pick Whisper in 2026:

- Multilingual ASR with one model.
- Robust transcription of noisy, diverse audio.
- Research / prototype ASR — fastest starting point.

When to pick something else:

- Ultra-low latency streaming on edge — Moonshine beats Whisper at matched quality.
- Real-time conversational AI needing <200 ms — dedicated streaming ASR.
- Speaker diarization — Whisper does not do this; bolt on pyannote.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/10-audio-transformers-whisper>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/10-audio-transformers-whisper/code>

1. **Easy.** Run `code/main.py`. Confirm the frame count for a 1-second signal at 16 kHz with 10 ms hop is ~ 100 frames. For 30 seconds: $\sim 3,000$ frames.
2. **Medium.** Build the full log-mel spectrogram using `numpy.fft`. Verify 80 mel bins match `librosa.feature.melspectrogram(n_mels=80)` within numerical error.
3. **Hard.** Implement streaming inference: chunk audio into 10 s windows with 2 s overlap, run Whisper on each chunk, merge transcripts. Measure word-error rate vs single-pass on a 5-minute podcast sample.

Key Terms

Term	What people say	What it actually means
Mel spectrogram	“Audio image”	2D representation: frequency bins on one axis, time frames on the other; log-scaled energy per cell.
Log-mel	“What Whisper sees”	Mel spectrogram passed through log; approximates human perception of loudness.
Frame	“One time slice”	A 25 ms window of samples; overlapping at 10 ms stride.
Task token	“Prompt prefix for speech”	Special tokens like <code>< transcribe ></code> / <code>< translate ></code> in the decoder prompt.
Voice activity detection (VAD)	“Find the speech”	Gate that removes silence before ASR; cuts cost massively.
CTC	“Connectionist Temporal Classification”	Classic ASR loss for alignment-free training; Whisper does NOT use it.
Whisper-turbo	“Small decoder, full encoder”	large-v3 encoder + 4-layer decoder; 8× faster decoding.
Faster-whisper	“The production wrapper”	CTranslate2 reimplementation; int8 quantization; 4× faster than OpenAI’s reference.

Further Reading

- Radford et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision — Whisper paper.

- OpenAI Whisper repo — reference code + model weights. Read `whisper/model.py` to see the Conv1D stem + encoder + decoder top-to-bottom in ~400 lines.
- OpenAI Whisper — `whisper/decoding.py` — the beam-search + task-token logic described in Steps 5-6 is here; 500 lines, fully readable.
- Baevski et al. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations — precursor; still SOTA features in some settings.
- SYSTRAN/faster-whisper — production wrapper, 4× faster than reference.
- Jia et al. (2024). Moonshine: Speech Recognition for Live Transcription and Voice Commands — 2024 edge-friendly ASR, Whisper-shaped but smaller.
- HuggingFace blog — “Fine-Tune Whisper For Multilingual ASR with `⬜` Transformers” — canonical fine-tuning recipe including mel spectrogram preprocessor and token-timestamp handling.
- HuggingFace `modeling_whisper.py` — full implementation (encoder, decoder, cross-attention, generation) that mirrors the lesson’s architecture diagram.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/10-audio-transformers-whisper>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/10-audio-transformers-whisper/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Mixture of Experts (MoE)

A dense 70B transformer activates every parameter for every token. A 671B MoE activates only 37B per token and beats it on every benchmark. Sparsity is the most important scaling idea of the decade.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 05 (Full Transformer), Phase 7 · 07 (GPT) **Time:** ~45 minutes

The Problem

A dense transformer's FLOPs at inference equal its parameter count (times 2 for forward pass). Scale up a dense model and every token pays the full bill. By 2024 the frontier was hitting a compute wall: to be meaningfully smarter, you needed exponentially more FLOPs per token.

Mixture of Experts breaks this link. Replace each FFN with E independent experts + a router that picks k experts per token. Total parameters = $E \times \text{FFN_size}$. Active parameters per token = $k \times \text{FFN_size}$. Typical 2026 configuration: $E=256$, $k=8$. Storage scales with E, compute scales with k.

The 2026 frontier is almost entirely MoE: DeepSeek-V3 (671B total / 37B active), Mixtral 8×22B, Qwen2.5-MoE, Llama 4, Kimi K2, gpt-oss. On Artificial Analysis's independent leaderboard, the top 10 open-source models are all MoE.

The Concept

The FFN swap

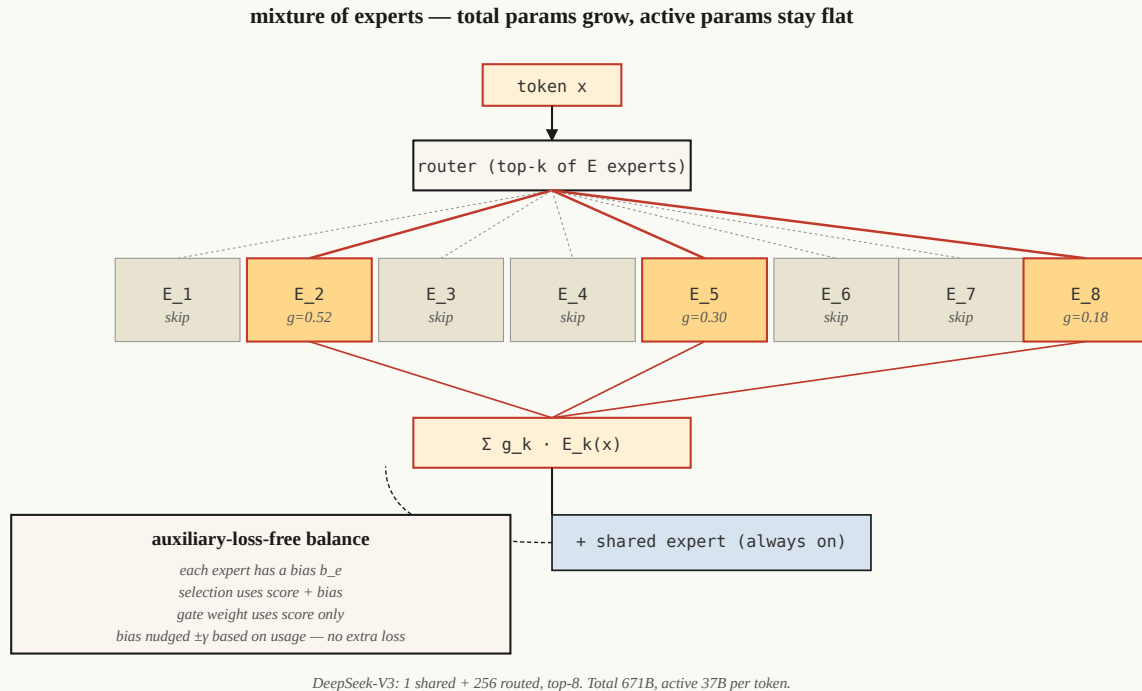
Dense transformer block:

```
h = x + attn(norm(x))
h = h + FFN(norm(h))
```

MoE block:

```
h = x + attn(norm(x))
scores = router(norm(h))                # (N_tokens, E)
top_k = argmax_k(scores)                # pick k of E per token
h = h + sum_{e in top_k}(
    gate(scores[e]) * Expert_e(norm(h))
)
```

Every expert is an independent FFN (typically SwiGLU). The router is a single linear layer. Each token picks its own k experts and gets a gated mixture of their outputs.

Figure 30: MoE layer: router selects k of E experts per token

The load-balancing problem

If the router puts 90% of tokens through expert 3, the other experts starve. Three fixes have been tried:

1. **Auxiliary load-balancing loss** (Switch Transformer, Mixtral). Add a penalty proportional to the variance in expert usage. Works, but adds a hyperparameter and a second gradient signal.
2. **Expert capacity + token dropping** (early Switch). Each expert processes at most $C \times N/E$ tokens; overflow tokens skip the layer. Hurts quality.
3. **Auxiliary-loss-free balancing** (DeepSeek-V3). Add a learned per-expert bias that shifts the router's top-k selection. Bias is updated outside the training loss. No penalty on the main objective. 2024's big unlock.

DeepSeek-V3's approach: after each training step, for every expert, check if its usage is above or below the target. Nudge the bias by $\pm \gamma$. Selection uses scores + bias. Expert probabilities used for gating are the raw scores unchanged. Decouples routing from expression.

Shared experts

DeepSeek-V2/V3 also split experts into *shared* and *routed*. Every token passes through all shared experts. Routed experts are picked via top-k. Shared experts capture common knowledge; routed experts specialize. V3 runs 1 shared expert plus top-8 of 256 routed.

Fine-grained experts

Classic MoE (GShard, Switch): each expert is as wide as a full FFN. E is small (8–64), k is small (1–2).

Modern fine-grained MoE (DeepSeek-V3, Qwen-MoE): each expert is narrower (1/8 FFN size). E is large (256+), k is larger (8+). Same total parameters, but combinations scale much faster. $C(256, 8) = 400$ trillion possible “experts” per token. Quality goes up, latency stays flat.

The cost profile

Per token, per layer:

Config	Active params / token	Total params
Mixtral 8×22B	~39B	141B
Llama 3 70B (dense)	70B	70B
DeepSeek-V3	37B	671B
Kimi K2 (MoE)	~32B	1T

DeepSeek-V3 beats Llama 3 70B (dense) on almost every benchmark while doing **fewer active FLOPs per token**. More parameters = more knowledge. More active FLOPs = more compute per token. MoE decouples them.

The catch: memory

All experts live on GPU regardless of which ones fire. A 671B model needs ~1.3 TB of VRAM for fp16 weights. Frontier MoE deployment requires expert parallelism — shard experts across GPUs, route tokens across the network. Latency is dominated by the all-to-all communication, not the matmul.

Interactive figure: expert-routing. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/11-mixture-of-experts>

Build It

See code/main.py. A compact MoE layer in pure stdlib with:

- n_experts=8 SwiGLU-ish experts (one linear each, for illustration)
- top-k=2 routing
- softmax-normalized gating weights
- auxiliary-loss-free balancing via per-expert bias

Step 1: the router

```
def route(hidden, W_router, top_k, bias):
    scores = [sum(h * w for h, w in zip(hidden, W_router[e])) for e in
    ↪ range(len(W_router))]
    biased = [s + b for s, b in zip(scores, bias)]
    top_idx = sorted(range(len(biased)), key=lambda i: -biased[i])[:top_k]
    # softmax over ORIGINAL scores of the chosen experts
    chosen = [scores[i] for i in top_idx]
    m = max(chosen)
    exps = [math.exp(c - m) for c in chosen]
    s = sum(exps)
```

```
gates = [e / s for e in exps]
return top_idx, gates
```

Bias affects selection, not gate weight. That is the DeepSeek-V3 trick — bias corrects load imbalance without steering the model’s predictions.

Step 2: run 100 tokens through the router

Track which experts fire how often. Without the bias, usage is skewed. With a bias update loop ($-\gamma$ for over-used experts, $+\gamma$ for under-used), usage converges to a uniform distribution over a few iterations.

Step 3: param count comparison

Print the “dense equivalent” of an MoE config. DeepSeek-V3-shaped: 256 routed + 1 shared, 8 active, `d_model=7168`. The total parameter count is eye-watering. The active count is a seventh of a dense Llama 3 70B.

Use It

HuggingFace loading:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
model = AutoModelForCausalLM.from_pretrained("mistralai/Mixtral-8x22B-v0.1")
```

2026 production inference: vLLM supports MoE routing natively. SGLang has the fastest expert-parallel path. Both automatically handle top-k selection and expert parallelism.

When to pick MoE: - You want frontier quality at lower inference cost per token. - You have the VRAM / expert-parallel infrastructure. - Your workload is token-heavy (chat, code) not context-heavy (long docs).

When NOT to pick MoE: - Edge deployment — you pay full storage for any active FLOP. - Latency-critical single-user serving — expert routing adds overhead. - Small models ($<7B$) — MoE’s quality advantage only appears above a compute threshold ($\sim 6B$ active params).

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/11-mixture-of-experts>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/11-mixture-of-experts/code>

1. **Easy.** Run `code/main.py`. Watch how the auxiliary-loss-free bias update evens out expert usage over 50 iterations.
2. **Medium.** Replace the learned router with a hash-based router (deterministic, no learning). Compare quality and balance. Why is the learned router better?
3. **Hard.** Implement GRPO-style “rollout-matched routing” (DeepSeek-V3.2 trick): log which experts fire during inference, force the same routing during gradient computation. Measure the effect on a toy policy-gradient setup.

Key Terms

Term	What people say	What it actually means
Expert	“One FFN among many”	An independent feed-forward network; parameters dedicated to a sparse slice of the FFN computation.
Router	“The gate”	A tiny linear layer that scores each token against each expert; top-k selection.
Top-k routing	“k active experts per token”	Each token’s FFN computation goes through exactly k experts, weighted by gate.
Auxiliary loss	“Load-balance penalty”	Extra loss term that penalizes skewed expert usage.
Auxiliary-loss-free	“DeepSeek-V3’s trick”	Balance via per-expert bias on the router’s selection only; no extra gradient.
Shared expert	“Always on”	Extra expert through which every token passes; captures common knowledge.
Expert parallelism	“Shard by expert”	Distribute different experts to different GPUs; route tokens across the network.
Sparsity	“Active params < total params”	The ratio $k \times \text{expert_size} / (E \times \text{expert_size})$; $37/671 \approx 5.5\%$ for DeepSeek-V3.

Further Reading

- Shazeer et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer — the idea.
- Fedus, Zoph, Shazeer (2022). Switch Transformer: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity — Switch, the classic MoE.
- Jiang et al. (2024). Mixtral of Experts — Mixtral 8×7B.
- DeepSeek-AI (2024). DeepSeek-V3 Technical Report — MLA + auxiliary-loss-free MoE + MTP.
- Wang et al. (2024). Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts — the bias-based balancing paper.
- Dai et al. (2024). DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models — the fine-grained + shared-expert split this lesson’s router uses.
- Kim et al. (2022). DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training — original shared-expert paper.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/11-mixture-of-experts>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/11-mixture-of-experts/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

KV Cache, Flash Attention & Inference Optimization

Training is parallel and FLOP-bound. Inference is serial and memory-bound. Different bottleneck, different tricks.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 02 (Self-Attention), Phase 7 · 05 (Full Transformer), Phase 7 · 07 (GPT) **Time:** ~75 minutes

The Problem

A naive autoregressive decoder does $O(N^2)$ work to generate N tokens: at each step it recomputes attention over the full prefix. For a 4K-token response that is 16M attention operations, most of them redundant. Every hidden state of a prefix token is deterministic once computed — you only need to run the new token’s query against the cached keys and values of everything before.

On top of that, attention itself moves a lot of data. Standard attention materializes an $N \times N$ score matrix, $N \times d$ softmax output, $N \times d$ final output — too many reads and writes to HBM. For $N \geq 2K$, attention becomes memory-bound before it becomes FLOP-bound. Classic attention kernels underuse modern GPUs by 4–10×.

Two optimizations, both from Dao et al., pushed frontier inference from “slow” to “fast”:

1. **KV cache.** Store the K and V vectors of every prefix token. Each new token’s attention is one query against the cached keys. Inference reduces from $O(N^2)$ to $O(N)$ per generation step.
2. **Flash Attention.** Tile the attention computation so the full $N \times N$ matrix never hits HBM. All of softmax + matmul happens in SRAM. 2–4× wall-clock speedup on A100; 5–10× on H100 with FP8.

By 2026 both are universal. Every production inference stack (vLLM, TensorRT-LLM, SGLang, llama.cpp) assumes them. Every frontier model ships with Flash Attention enabled.

The Concept

KV cache math

Per decoder layer, per token, per head:

$$\text{bytes_per_token_per_layer} = 2 * \underset{\substack{\uparrow \\ K \text{ and } V}}{d_head} * dtype_size$$

For a 7B model with 32 layers, 32 heads, $d_head=128$, fp16:

per token per layer = $2 * 128 * 2 = 512$ bytes
per token (32 layers) = 16 KB
per 32K context = 512 MB

KV cache grows. Flash Attention keeps the hot path in SRAM.

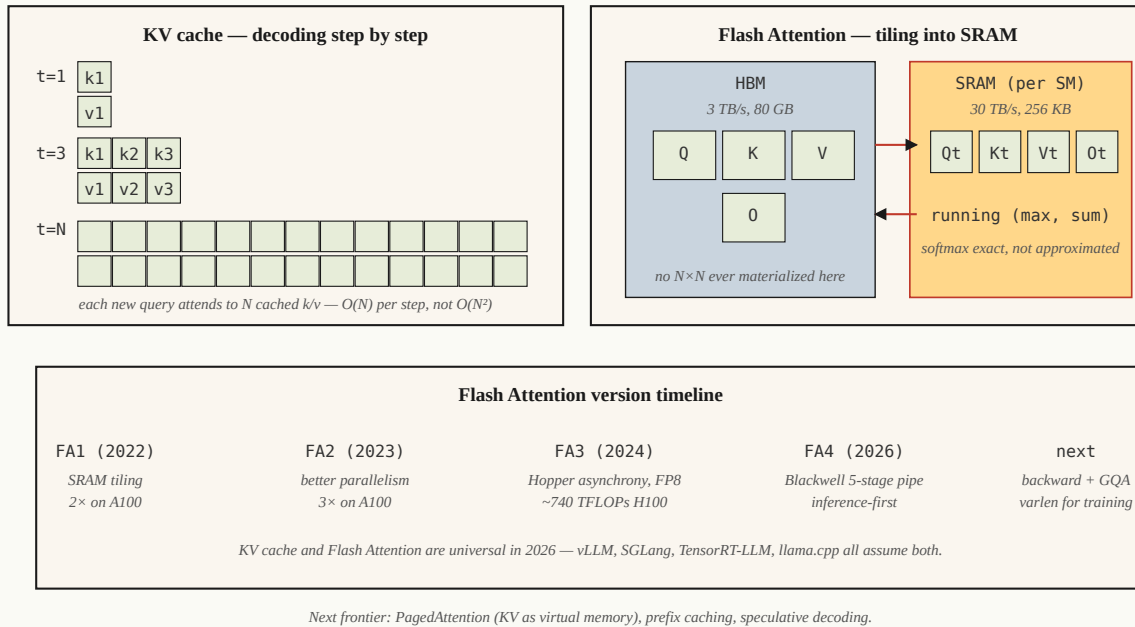


Figure 31: KV cache growth and Flash Attention tiling

For Llama 3 70B (80 layers, $d_{\text{head}}=128$, GQA with 8 KV heads):

per token per layer = $2 * 8 * 128 * 2 = 4096$ bytes (4 KB)
 per 32K context = 10.4 GB

That 10 GB is why Llama 3 70B at 128K context needs most of a 40 GB A100 just for KV cache at batch size 1.

GQA is the KV-cache win. MHA with 64 heads would be 32 GB. MLA compresses even further.

Drag the dimensions and watch the cache size move. Push the sequence length or batch up and see how fast it blows past a single GPU:

Interactive figure: kv-cache-sizer. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/12-kv-cache-flash-attention>

Flash Attention — the tiling trick

Standard attention:

$S = Q @ K^T$ (HBM read, $N \times N$, HBM write)
 $P = \text{softmax}(S)$ (HBM read, HBM write)
 $O = P @ V$ (HBM read, HBM write)

Three HBM round trips. On H100, HBM bandwidth is 3 TB/s; SRAM is 30 TB/s. Every HBM trip is a factor-of-10 slowdown vs keeping everything on-chip.

Flash Attention:

```

for each block of Q (tile size ~128 × 128):
    load Q_tile into SRAM
    for each block of K, V:
        load K_tile, V_tile into SRAM
        compute S_tile = Q_tile @ K_tile^T      (SRAM)
        running softmax aggregation            (SRAM)
        accumulate into O_tile                  (SRAM)
    write O_tile to HBM

```

One HBM trip per tile. Total memory footprint drops from $O(N^2)$ to $O(N)$. Backward pass recomputes some values from the forward pass instead of storing them — another memory win.

Numerical trick. Running softmax maintains (max, sum) across tiles so the final normalization is exact. Not an approximation — Flash Attention computes bit-identical output to standard attention (modulo fp16 non-associativity).

Version evolution:

Version	Year	Key change	Speedup on reference hardware
Flash 1	2022	Tiled SRAM kernel	2× on A100
Flash 2	2023	Better parallelism, causal-first ordering	3× on A100
Flash 3	2024	Hopper asynchrony, FP8	1.5–2× on H100 (~740 TFLOPs FP16)
Flash 4	2026	Blackwell 5-stage pipeline, software exp2	Inference-first (forward only initially)

Flash 4 is forward-pass only at launch. Training still uses Flash 3. GQA and varlen support for Flash 4 is pending (mid-2026).

Speculative decoding — the other latency win

Cheap model proposes N tokens. Big model verifies all N in parallel. If verification accepts k tokens, you paid 1 big-model forward pass for k generations. Typical $k=3-5$ on code and prose.

2026 defaults: - **EAGLE 2 / Medusa**. Integrated draft heads that share the verifier’s hidden states. 2–3× speedup with no quality loss. - **Speculative decoding with draft model**. 2–4× speedup on consumer hardware. - **Lookahead decoding**. Jacobi iteration; no draft model needed. Niche but free.

Continuous batching

Classic batched inference: wait for the slowest sequence to finish, then start a new batch. Wastes GPU when short responses finish early.

Continuous batching (first shipped in Orca, now in vLLM, TensorRT-LLM, SGLang): swap new requests into the batch as soon as old ones finish. 5–10× throughput gain for typical chat workloads.

PagedAttention — KV cache as virtual memory

vLLM's headline feature. KV cache is allocated in 16-token blocks; a page table maps logical positions to physical blocks. Lets you share KV across parallel samples (beam search, parallel sampling), hot-swap prefixes for prompt caching, and defragment memory. 4× throughput improvement over naive contiguous allocation.

Interactive figure: flash-attention-memory. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/12-kv-cache-flash-attention>

Build It

See code/main.py. We implement:

1. A naive $O(N^2)$ incremental decoder.
2. A $O(N)$ KV-cached decoder.
3. A tiled softmax that simulates Flash Attention's running-max algorithm.

Step 1: KV cache

```
class KVCache:
    def __init__(self, n_layers, n_heads, d_head):
        self.K = [[[ for _ in range(n_heads)] for _ in range(n_layers)]]
        self.V = [[[ for _ in range(n_heads)] for _ in range(n_layers)]]

    def append(self, layer, head, k, v):
        self.K[layer][head].append(k)
        self.V[layer][head].append(v)

    def read(self, layer, head):
        return self.K[layer][head], self.V[layer][head]
```

Simple: keep growing per-token K, V vectors in per-layer, per-head lists.

Step 2: tiled softmax

```
def tiled_softmax_dot(q, K, V, tile=4):
    """Flash-attention-style softmax( $qK^T$ )V with running max/sum."""
    m = float("-inf")
    s = 0.0
    out = [0.0] * len(V[0])
    for start in range(0, len(K), tile):
        k_block = K[start:start + tile]
        v_block = V[start:start + tile]
        scores = [sum(qi * ki for qi, ki in zip(q, k)) for k in k_block]
        new_m = max(m, *scores)
        exp_old = math.exp(m - new_m) if m != float("-inf") else 0.0
        exp_new = [math.exp(sc - new_m) for sc in scores]
        s = s * exp_old + sum(exp_new)
        for j in range(len(out)):
            out[j] = out[j] * exp_old + sum(e * v[j] for e, v in zip(exp_new,
↪ v_block))
```

```

        m = new_m
    return [o / s for o in out]

```

Bit-identical output to $\text{softmax}(qK^T V)$ in one shot, but at any time the working set is a $\text{tile} \times d_{\text{head}}$ block, not the full $N \times d_{\text{head}}$.

Step 3: compare naive vs cached decoding on 100-token generation

Count attention operations. Naive: $O(N^2) = 5050$. Cached: $O(N) = 100$. The code prints both.

Use It

```

# HuggingFace transformers auto-enables KV cache on decoder-only generate().
from transformers import AutoModelForCausalLM
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-3B",
    attn_implementation="flash_attention_2", # use FA3 if Hopper
    torch_dtype="bfloat16",
)
# generate() uses KV cache automatically

```

vLLM production:

```

pip install vllm
vllm serve meta-llama/Llama-3.1-70B-Instruct \
    --tensor-parallel-size 4 \
    --max-model-len 32768 \
    --enable-prefix-caching \
    --kv-cache-dtype fp8

```

Prefix caching across requests is a big 2026 win — the same system prompt, few-shot examples, or long context document reuses KV across calls. For agent workloads with repeated tool prompts, prefix caching is routinely 5× throughput gain.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/12-kv-cache-flash-attention>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/12-kv-cache-flash-attention/code>

1. **Easy.** Run `code/main.py`. Confirm the naive and cached decoders produce the same output; note the op-count difference.
2. **Medium.** Implement prefix caching: given a prompt P and several completions, run one forward pass over P to fill the KV cache, then branch per-completion. Measure speedup vs re-encoding P for each.
3. **Hard.** Implement a toy PagedAttention: KV cache in fixed 16-token blocks with a free-list. When a sequence finishes, return its blocks to the pool. Simulate 1,000 chat completions with varying lengths. Compare memory fragmentation vs contiguous allocation.

Key Terms

Term	What people say	What it actually means
KV cache	“The trick that makes decoding fast”	Stored K and V from every prefix token; new queries attend to them instead of recomputing.
HBM	“GPU main memory”	High Bandwidth Memory; 80 GB on H100, 192 GB on B200. ~3 TB/s bandwidth.
SRAM	“On-chip memory”	Per-SM fast memory, ~256 KB per SM on H100. ~30 TB/s bandwidth.
Flash Attention	“Tiled attention kernel”	Computes attention without materializing $N \times N$ in HBM.
Continuous batching	“No-wait batching”	Swap finished sequences out, new ones in, without draining the batch.
PagedAttention	“vLLM’s headline”	KV cache allocated in fixed blocks with a page table; eliminates fragmentation.
Prefix caching	“Reuse long prompts”	Cache KV for a shared prefix across requests; major cost cut for agents.
Speculative decoding	“Draft + verify”	Cheap draft model proposes tokens; big model verifies k in one pass.

Further Reading

- Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness — Flash 1.
- Dao (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning — Flash 2.
- Shah et al. (2024). FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision — Flash 3.
- FlashAttention-4 release notes (Dao-AILab, 2026) — Blackwell 5-stage pipeline and the software-exp2 trick; read the repo README for the forward-only launch caveats this lesson mentions.
- Kwon et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention — vLLM paper.
- Leviathan et al. (2023). Fast Inference from Transformers via Speculative Decoding — spec decoding.
- Li et al. (2024). EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty — EAGLE-1/2 paper for the integrated-draft approach the lesson cites.
- Cai et al. (2024). Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads — the Medusa approach referenced alongside EAGLE.
- vLLM docs — PagedAttention — the canonical deep dive on the 16-token block and page-table design.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/12-kv-cache-flash-attention>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/12-kv-cache-flash-attention/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Scaling Laws

The 2020 Kaplan paper said: bigger model, lower loss. The 2022 Hoffmann paper said: you were under-training. Compute goes into two buckets — parameters and tokens — and the split is not obvious.

Type: Learn **Languages:** Python **Prerequisites:** Phase 7 · 05 (Full Transformer), Phase 7 · 07 (GPT) **Time:** ~45 minutes

The Problem

When you have C FLOPs of training compute and want the best model, you face two knobs:

1. **How many parameters (N)?** Bigger model, higher capacity.
2. **How many training tokens (D)?** More data, better use of capacity.

FLOPs scale approximately as $6 \times N \times D$. You can push N up and D down, or D up and N down. Which is better?

Before 2022, the answer was “push N hard.” GPT-3 (2020) was 175B parameters trained on ~300B tokens. A ratio of about 1.7 tokens per parameter. The Kaplan scaling laws backed this up.

Hoffmann et al. (2022), training a small family of models called Chinchilla, found something different: optimal ratio is closer to **20 tokens per parameter**. GPT-3 was 10× undertrained. Chinchilla (70B params, 1.4T tokens) beat GPT-3 (175B, 300B tokens) on every benchmark at 2.5× less inference cost.

2026 is Chinchilla’s world — with one important twist. Llama 3 8B was trained on 15 trillion tokens, a ratio of 1,875 tokens per parameter. Ninety-four times past Chinchilla-optimal. Inference cost matters more than training cost for models that will be used at scale, so over-training (past Chinchilla) for a smaller deployable footprint is the 2026 default.

The Concept

The Hoffmann law

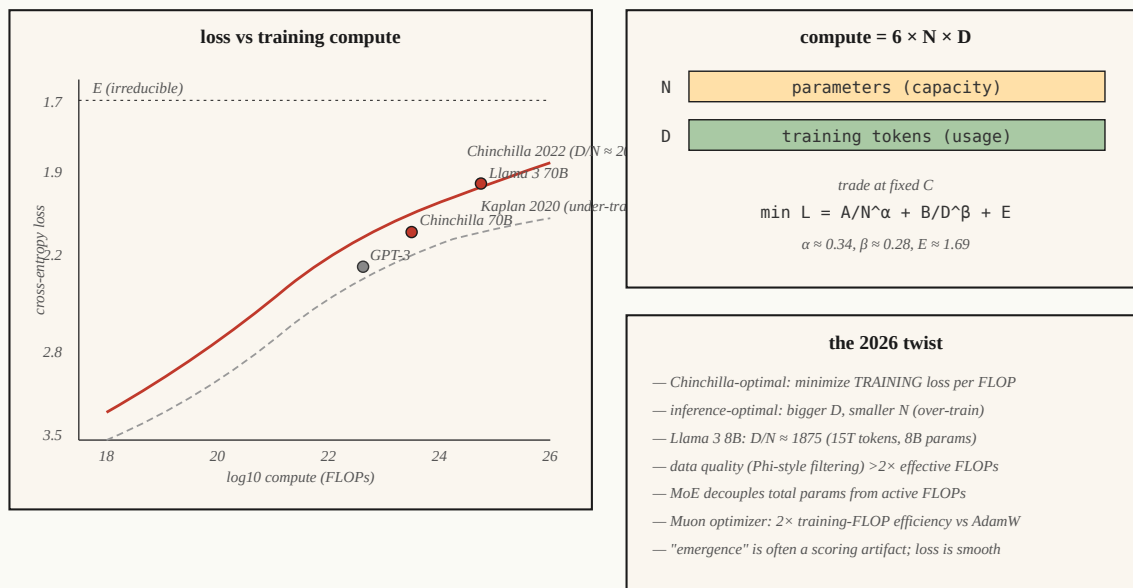
From the Chinchilla paper, loss follows:

$$L(N, D) = A / N^\alpha + B / D^\beta + E$$

- N = parameters (non-embedding).
- D = training tokens.
- $\alpha \approx 0.34$, $\beta \approx 0.28$ (roughly symmetric).
- $E \approx 1.69$, the irreducible loss ceiling.
- $A \approx 406$, $B \approx 411$.

Two terms trade against each other as you scale. Take the derivative w.r.t. N at fixed compute ($C = 6ND$) and solve:

scaling laws — parameters, tokens, compute, and what changed in 2026



same power-law shape, different absolute constants. still the map frontier labs plan against.

Figure 32: Chinchilla curves: loss vs compute at various N/D ratios

$$N_{\text{opt}} \approx 0.6 \times (C/6)^{0.5}$$

$$D_{\text{opt}} \approx 0.6 \times (C/6)^{0.5}$$

$$D_{\text{opt}} / N_{\text{opt}} \approx 20$$

Compute-optimal: 20 tokens per parameter.

Why over-training anyway

Chinchilla-optimal minimizes training loss per training FLOP. But you pay training cost once; inference cost forever.

For a chatbot that serves a trillion tokens per month, inference dominates total cost. Llama's approach: train smaller, longer. 8B at 15T tokens is deeply inference-optimized:

- Fits on consumer GPUs.
- Latency is a fraction of 70B Chinchilla-optimal.
- Quality is close enough for most tasks.

DeepMind's 2024 paper ("Over-training is the new optimal") formalized this. For inference-dominated workloads, the right ratio is closer to 100–500 tokens per parameter depending on serving volume.

Emergence vs smoothness

Claim: certain abilities (arithmetic, multi-step reasoning, chain-of-thought following) "emerge" suddenly at some scale.

Schaeffer et al. (2023) argued this is a measurement artifact: emergent metrics use discontinuous scoring (exact match, accuracy at threshold) that hide smooth improvement in the underlying logits. Continuous metrics (cross-entropy) show smooth curves.

In 2026 the consensus is: predictions via continuous loss are reliable. Benchmark jumps are often scorer artifacts. Plan budgets against continuous metrics.

The 2026 picture

Scaling laws still work, but:

Factor	Changed how
Data quality	Curating “good” tokens (Phi-style) shifts curves by $>2\times$ effective compute
MoE	Total params decouple from active FLOPs; scaling laws per-active-FLOP
Post-training	Some capabilities (instruction following, code) shift with SFT+RLHF more than pretraining
Multimodality	Image + text tokens scale together; separate curves per modality
Synthetic data	Models generate training data; effective compute can compound

The Muon optimizer (Kimi Moonlight, 2024) showed a $\sim 2\times$ effective-compute gain over AdamW at matched data. Some 2026 training runs use Muon by default. Changes the absolute constant in the scaling law, not its shape.

Interactive figure: scaling-laws. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/13-scaling-laws>

Build It

See code/main.py. We implement the Chinchilla loss equation and solve for compute-optimal (N , D) at each of several compute budgets.

Step 1: Chinchilla loss

```
def chinchilla_loss(N, D, A=406.4, B=410.7, alpha=0.34, beta=0.28, E=1.69):
    return A / N ** alpha + B / D ** beta + E
```

Plot L as a contour over (N, D) at fixed $C = 6ND$. Find the minimum.

Step 2: compute-optimal frontier

For compute budgets from $1e17$ to $1e25$ FLOPs, find (N, D) that minimize loss subject to $6ND = C$. Verify the ratio $D/N \approx 20$.

Step 3: over-training cost

Compute the extra loss you pay to train a $10\times$ smaller model ($1/10$ of optimal N , $10\times$ the optimal D). Reports the inference FLOP savings (proportional to N) in exchange.

Step 4: compare to real models

Drop in known (N, D) pairs for GPT-3, Chinchilla, Llama 3 8B, DeepSeek-V3 (active params), and compare predicted vs reported loss.

Use It

You're unlikely to train a frontier model yourself. But scaling laws tell you:

1. **Whether your fine-tune has enough data.** If your task-specific data is below 20 tokens per param of the base model, expect saturation at some loss floor.
2. **Whether to pick a bigger base model.** If you're spending all your budget on inference, prefer a smaller, longer-trained model.
3. **Where the returns diminish.** Beyond $1000\times$ Chinchilla-optimal, log-loss changes become noise.

The research trajectory in 2026:

- **Data-constrained regime.** The web has a finite number of high-quality tokens ($\sim 5\text{--}10$ trillion English after filtering). Frontier pretraining is approaching this ceiling. Synthetic data, multilingual, multimodal, and RLHF-scaled fine-tuning are the next levers.
- **Compute-multiplier tricks.** Muon optimizer, MoE, better data curation — each shifts the absolute constants, not the asymptote.
- **Scaling laws for RL.** Open question. Early evidence suggests power-law in RL samples but with very different exponents than pretraining.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/13-scaling-laws>

Exercises

Starter code and the lesson's working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/13-scaling-laws/code>

1. **Easy.** Run `code/main.py`. Print Chinchilla-optimal (N, D) for compute budgets $1e20, 1e22, 1e24$. Compare to the real model table.
2. **Medium.** Implement the Hoffmann loss-as-function-of-compute curve. Plot loss vs $\log_2 C$ for the compute-optimal frontier. Identify when the law predicts we'd need $>10^{28}$ FLOPs for the next 0.1 reduction in cross-entropy.
3. **Hard.** Fit your own scaling law on 5 tiny models (100K to 10M params) trained on the same dataset. Estimate α and E . How well do your exponents match published ones?

Key Terms

Term	What people say	What it actually means
Parameters (N)	"Model size"	Non-embedding weight count; determines capacity.
Tokens (D)	"Training data"	Number of training tokens seen; determines how well the parameters get used.

Term	What people say	What it actually means
Compute (C)	“FLOPs spent”	Approximately $6 \times N \times D$ for a standard transformer.
Chinchilla-optimal	“ $D/N \approx 20$ ”	Ratio that minimizes loss per FLOP of pretraining.
Over-training	“Past Chinchilla”	Spend extra training FLOPs to save inference FLOPs; $D/N \gg 20$.
Irreducible loss	“The floor”	The E term in the scaling law; the entropy of the data itself.
Emergent capability	“Sudden jumps at scale”	Often a scorer artifact; continuous loss is smooth.
Effective compute	“Training-efficiency multiplier”	Better data / optimizer / architecture multiplies how far a FLOP goes.

Further Reading

- Kaplan et al. (2020). Scaling Laws for Neural Language Models — the first scaling law paper; undertrained.
- Hoffmann et al. (2022). Training Compute-Optimal Large Language Models — Chinchilla.
- Schaeffer et al. (2023). Are Emergent Abilities of Large Language Models a Mirage? — emergence as measurement artifact.
- Sardana, Frankle (2024). Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws — why Llama’s over-training is right for its workload.
- Jordan et al. (2024). Muon: An optimizer for hidden layers in neural networks — $2\times$ compute multiplier.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/13-scaling-laws>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/13-scaling-laws/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Build a Transformer from Scratch — The Capstone

Thirteen lessons. One model. No shortcuts.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 01 through 13. Don't skip. **Time:** ~120 minutes

The Problem

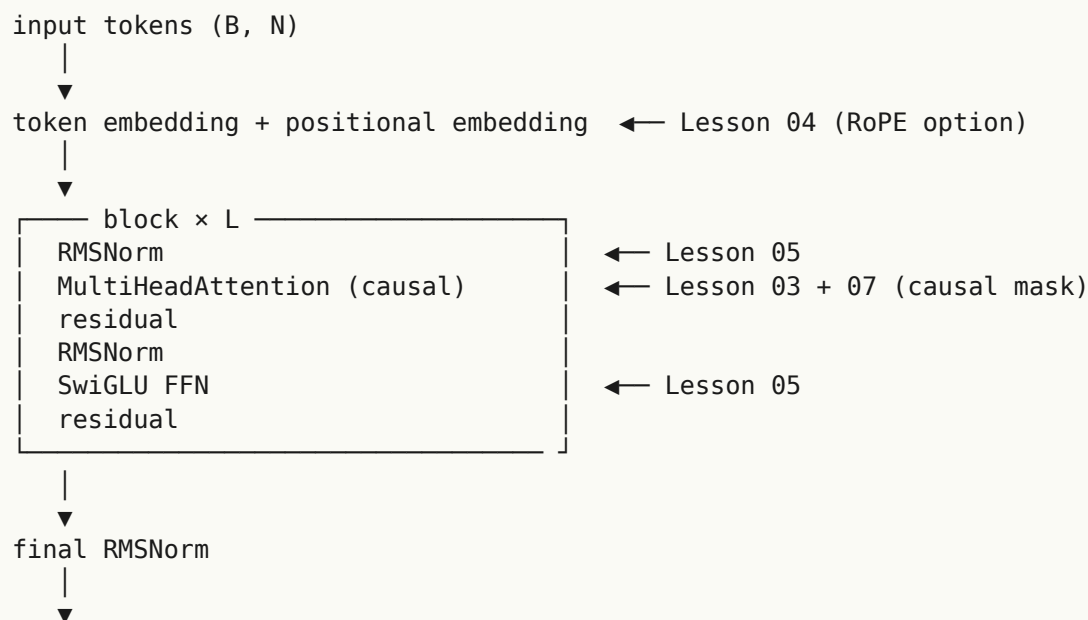
You've read every paper. You've implemented attention, multi-head splits, positional encodings, encoder and decoder blocks, BERT and GPT losses, MoE, KV cache. Now make them work together on a real task.

The capstone: train a small decoder-only transformer end-to-end on a character-level language modeling task. It reads Shakespeare. It generates new Shakespeare. It is small enough to train on a laptop in under 10 minutes. It is correct enough that swapping in a bigger dataset and longer training gets you a real LM.

This is the “nanoGPT” of the course. It is not original — Karpathy's 2023 nanoGPT tutorial is the reference implementation every student writes at least once. We lift the shape and retool it around what we've covered.

The Concept

The architecture, annotated:



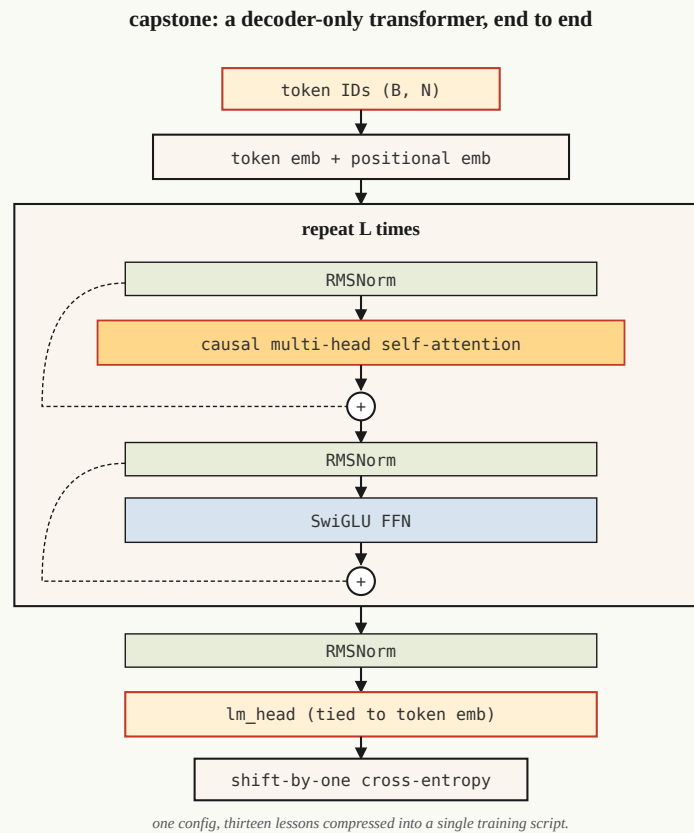
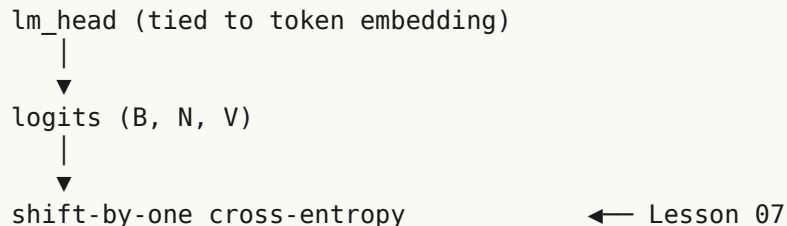


Figure 33: Transformer-from-scratch block diagram



What we ship

- `GPTConfig` — one place to configure all hyperparameters.
- `MultiHeadAttention` — causal, batched, with optional Flash-style pathway (PyTorch's `scaled_dot_product_attention`).
- `SwiGLUFFN` — modern FFN.
- `Block` — pre-norm, residual-wrapped attention + FFN.
- `GPT` — embeddings, stacked blocks, LM head, `generate()`.
- Training loop with AdamW, cosine LR, gradient clipping.
- Char-level tokenizer on Shakespeare text.

What we don't ship

- RoPE — implemented conceptually in Lesson 04. Here we use learned positional embeddings for simplicity. The exercises ask you to swap in RoPE.
- KV cache during generation — each generation step recomputes attention over the full prefix. Slower but simpler. The exercises ask you to add a KV cache.
- Flash Attention — PyTorch 2.0+ auto-dispatches if the inputs match; we use `F.scaled_dot_product_attention`.
- MoE — single FFN per block. You saw MoE in Lesson 11.

Target metrics

On a Mac M2 laptop, a 4-layer, 4-head, `d_model=128` GPT trained for 2,000 steps on `tinysakespeare.txt`:

- Training loss converges from ~ 4.2 (random) to ~ 1.5 in about 6 minutes.
- Sampled output looks Shakespeare-shaped: archaic words, line breaks, proper names like "ROMEO:" emerge.
- Val loss (held-out final 10% of text) tracks training loss closely; no overfitting at this size/budget.

Interactive figure: n5-block-stack. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/14-build-a-transformer-capstone>

Build It

This lesson uses PyTorch. Install torch (CPU build is fine). See `code/main.py`. The script handles:

- Downloading `tinysakespeare.txt` if missing (or reading a local copy).
- Byte-level char tokenizer.
- Train/val split at 90/10.
- Training loop with `bf16` autocast on supported hardware.
- Sampling after training completes.

Step 1: data

```
text = open("tinyshakespeare.txt").read()
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda xs: "".join(itos[x] for x in xs)
```

65 unique characters. Tiny vocabulary. Fits a 4-byte vocab_size. No BPE, no tokenizer drama.

Step 2: model

See code/main.py. The block is textbook from Lesson 05 — pre-norm, RMSNorm, SwiGLU, causal MHA. Parameter count for 4/4/128: ~800K.

Step 3: training loop

Get a random batch of length-256 token windows. Forward. Shift-by-one cross-entropy. Backward. AdamW step. Log. Repeat.

```
for step in range(max_steps):
    x, y = get_batch("train")
    logits = model(x)
    loss = F.cross_entropy(logits.view(-1, vocab_size), y.view(-1))
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()
    opt.zero_grad()
```

Step 4: sample

Given a prompt, repeatedly forward, sample from top-p logits, append, and continue. Stop after 500 tokens.

Step 5: read the output

After 2,000 steps:

ROMEO:

Away and mild will not thy friend, that thou shalt wit:

The chief that well shame and hath been his friends,

...

Not Shakespeare. But Shakespeare-shaped. A clear win for ~800K parameters and 6 minutes on a laptop.

Use It

This capstone is a reference architecture. Three extensions to ship it to something real:

1. **Swap the tokenizer.** Use BPE (e.g. tiktoken.get_encoding("cl100k_base")). Vocab size jumps from 65 to ~50,000. Model capacity needs to scale up to compensate.
2. **Train on a bigger corpus.** Use OpenWebText or fineweb-edu (HuggingFace). 10B tokens on a single A100 takes ~24 hours for a 125M-param GPT.

3. **Add RoPE + KV cache + Flash Attention.** The exercises below walk you through each.

This ends up as a 125M-parameter GPT that generates fluent English. Not a frontier model. But the same code path — just bigger — is what Karpathy, EleutherAI, and the Allen Institute use to train research checkpoints in 2026.

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/14-build-a-transformer-capstone>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/14-build-a-transformer-capstone/code>

1. **Easy.** Run `code/main.py`. Verify your trained model’s final-step validation loss is under 2.0. Change `max_steps` from 2,000 to 5,000 — does val loss keep improving?
2. **Medium.** Replace learned positional embeddings with RoPE. Apply the rotation to Q and K inside `MultiHeadAttention`. Train and verify val loss is at least as low.
3. **Medium.** Implement a KV cache in the sampling loop. Generate 500 tokens with and without cache. Wall-clock should improve by 5–20× on a laptop.
4. **Hard.** Add a second head to the model that predicts the next-plus-one token (MTP — Multi-Token Prediction from DeepSeek-V3). Train jointly. Does it help?
5. **Hard.** Replace the single FFN per block with a 4-expert MoE. Router + top-2 routing. See how val loss changes at matched active parameters.

Key Terms

Term	What people say	What it actually means
nanoGPT	“Karpathy’s tutorial repo”	Minimal decoder-only transformer training code, ~300 LOC; the canonical reference.
tinysakespeare	“The standard toy corpus”	~1.1 MB of text; every character-LM tutorial since 2015 uses it.
Tied embeddings	“Share input/output matrix”	LM head weight = transpose of token embedding matrix; saves parameters, improves quality.
bf16	“Training precision trick”	Run forward/back in bf16, keep optimizer state in fp32; standard since 2021.
autocast	“Stops spikes”	Cap global grad norm at 1.0; prevents training blowups.
Gradient clipping	“The 2020+ default”	LR ramps up linearly (warmup) then decays cosine-shaped to 10% of peak.
Cosine LR schedule	“Model FLOP Utilization”	Achieved FLOPs / theoretical peak; 40% dense, 30% MoE is strong in 2026.
MFU	“Held-out loss”	Cross-entropy on data the model never saw; overfit detector.

Further Reading

- The Annotated Transformer (Harvard NLP) — the classic annotated implementation.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/14-build-a-transformer-capstone>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/14-build-a-transformer-capstone/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Attention Variants — Sliding Window, Sparse, Differential

Full attention is a circle. Every token sees every token, and memory pays the price.
Four variants bend the shape of the circle and recover half the cost.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 02 (Self-Attention), Phase 7 · 03 (Multi-Head), Phase 7 · 12 (KV Cache / Flash Attention) **Time:** ~60 minutes

The Problem

Full attention costs $O(N^2)$ memory and $O(N^2)$ compute in sequence length. For a 128K-context Llama 3 70B that is 16 billion attention entries per layer, times 80 layers. Flash Attention (Lesson 12) hides the $O(N^2)$ activation memory but does not change the arithmetic cost — every token still attends to every other token.

Three classes of variants change the topology of the attention matrix itself:

1. **Sliding window attention (SWA).** Each token attends to a fixed window of neighbors, not the full prefix. Memory and compute drop to $O(N \cdot W)$ where W is the window. Gemma 2/3, Mistral 7B's first layers, Phi-3-Long.
2. **Sparse / block attention.** Only selected pairs (i, j) get scored; the rest are forced to zero weight. Longformer, BigBird, OpenAI sparse transformer.
3. **Differential attention.** Compute two attention maps with separate Q/K projections, subtract one from the other. Kills the “attention sink” that bleeds weight into the first few tokens. Microsoft's DIFF Transformer (2024).

These coexist. A 2026 frontier model often mixes them: most layers are SWA-1024, every fifth is global full attention, and a handful are differential heads that clean up retrieval. Gemma 3's 5:1 SWA-to-global ratio is the current textbook default.

The Concept

Sliding Window Attention (SWA)

Each query at position i attends only to positions in $[i - W, i]$ (causal SWA) or $[i - W/2, i + W/2]$ (bidirectional). Tokens outside the window get $-\infty$ in the score matrix.

full causal:	sliding window (W=4):
positions 0-7	positions 0-7, W=4
0 1 2 3 4 5 6 7	0 1 2 3 4 5 6 7
0 x	0 x
1 x x	1 x x
2 x x x	2 x x x
3 x x x x	3 x x x x
4 x x x x x	4 x x x x
5 x x x x x x	5 x x x x

```

6 | x x x x x x x   6 |           x x x x
7 | x x x x x x x x  7 |           x x x x

```

For $N = 8192$ and $W = 1024$, the score matrix has 1024×8192 non-zero rows in expectation — an $8\times$ reduction.

KV cache shrinks with SWA. Only the last W tokens of K and V need to be kept per layer. For a Gemma-3-ish config (1024 window, 128K context), KV cache drops $128\times$.

Quality cost. SWA-only transformers struggle with long-range retrieval. The fix: interleave SWA layers with full-attention layers. Gemma 3 uses 5:1 SWA:global. Mistral 7B used a causal-SWA stack where information “flows forward” through overlapping windows — each layer extends effective receptive field by W , and after L layers the model can attend $L \times W$ tokens back.

Sparse / Block Attention

Pick an $N \times N$ sparsity pattern ahead of time. Three canonical shapes:

- **Local + strided (OpenAI sparse transformer).** Attend to the last W tokens plus every stride-th token before that. Captures both local and long-range at $O(N \cdot \sqrt{N})$ compute.
- **Longformer / BigBird.** Local window + a small set of global tokens (e.g. [CLS]) that attend to everyone and are attended by everyone + random-sparse links. Empirical $2\times$ context at matched quality.
- **Native Sparse Attention (DeepSeek, 2025).** Learn which blocks of (Q, K) matter; skip the zero blocks at kernel level. FlashAttention-compatible.

Sparse attention is a kernel-engineering story. The math is simple (mask the score matrix); the win comes from never loading the zero entries into SRAM. FlashAttention-3 and the 2026 FlexAttention API make custom sparse patterns first-class in PyTorch.

Differential Attention (DIFF Transformer, 2024)

Regular attention has an “attention sink” problem: softmax forces every row to sum to 1, so tokens that don’t want to attend to anything in particular dump weight on the first token (or the first few). This steals capacity that should have gone to real content.

Differential attention fixes this by computing **two** attention maps and subtracting:

```

A1 = softmax(Q1 K1^T / √d)
A2 = softmax(Q2 K2^T / √d)
DiffAttn = (A1 - λ · A2) V

```

where λ is a learned scalar (typically 0.5–0.8). $A1$ captures real content weights; $A2$ captures the sink. Subtraction cancels the sink, reallocates weight to relevant tokens.

Reported results (Microsoft 2024): 5–10% lower perplexity, 1.5–2× longer effective context at same trained length, sharper needle-in-haystack retrieval.

Variant Comparison

Variant	Compute	KV cache	Quality vs full	Production use
Full attention	$O(N^2)$	$O(N)$ per layer	baseline	every model’s default layer

Variant	Compute	KV cache	Quality vs full	Production use
SWA (window 1024)	$O(N \cdot W)$	$O(W)$ per layer	-0.1 ppl, good with global layers	Gemma 2/3, Phi-3-Long
Local + strided sparse BigBird (local + global + random)	$O(N \cdot \sqrt{N})$ $O(N)$ approx	mixed	similar to SWA matches full at $2\times$ context	OpenAI sparse transformer, Longformer, early long-context BERT
Native Sparse (DeepSeek-V3.2)	$O(N \cdot \text{active fraction})$	$O(N)$	within 0.05 ppl	DeepSeek-V3.2, 2025
Differential	$O(2 \cdot N^2)$	$O(2N)$	-5 to -10% ppl	DIFF Transformer, early 2026 models

Interactive figure: gqa-kv-sharing. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/15-attention-variants>

Build It

See `code/main.py`. We implement a causal mask comparator that shows full, SWA, local+strided, and differential attention side by side on a toy sequence.

Step 1: full causal mask (baseline)

```
def causal_mask(n):
    return [[0.0 if j <= i else float("-inf") for j in range(n)] for i in range(n)]
```

Baseline from Lesson 07. Lower triangular; zero weight above the diagonal.

Step 2: sliding window causal mask

```
def swa_mask(n, window):
    M = [[float("-inf")] * n for _ in range(n)]
    for i in range(n):
        lo = max(0, i - window + 1)
        for j in range(lo, i + 1):
            M[i][j] = 0.0
    return M
```

One parameter — `window`. For `window >= n`, you recover full causal attention. For `window = 1`, each token attends only to itself.

Step 3: local + strided sparse mask

```
def strided_mask(n, window, stride):
    M = [[float("-inf")] * n for _ in range(n)]
```

```

for i in range(n):
    lo = max(0, i - window + 1)
    for j in range(lo, i + 1):
        M[i][j] = 0.0
    for j in range(0, i + 1, stride):
        M[i][j] = 0.0
return M

```

Dense local window plus every stride-th token back to the start of the sequence. Receptive field grows in log steps with additional layers.

Step 4: differential attention

```

def diff_attention(Q1, K1, Q2, K2, V, lam):
    A1 = softmax_causal(Q1 @ K1.T / sqrt_d)
    A2 = softmax_causal(Q2 @ K2.T / sqrt_d)
    return (A1 - lam * A2) @ V

```

Two attention passes, subtract with a learned mixing coefficient. In the code we compare the attention-sink heatmap of single vs differential and watch the sink collapse.

Step 5: KV cache sizes

Print the cache size per layer at $N = 131072$ for each variant. SWA and sparse variants drop by 10-100×. Differential doubles. Pay your memory bill consciously.

Use It

2026 production patterns:

```

from transformers import AutoModelForCausalLM
# Gemma 3 mixes SWA (window=1024) and global layers at 5:1.
model = AutoModelForCausalLM.from_pretrained("google/gemma-3-27b-it")
# print(model.config.sliding_window, model.config.layer_types)

```

FlexAttention in PyTorch 2.5+ accepts a mask function:

```

from torch.nn.attention.flex_attention import flex_attention, create_block_mask

def swa_pattern(b, h, q_idx, kv_idx):
    return (q_idx - kv_idx < 1024) & (q_idx >= kv_idx)

```

```

mask = create_block_mask(swa_pattern, B=batch, H=heads, Q_LEN=n, KV_LEN=n)
out = flex_attention(q, k, v, block_mask=mask)

```

This compiles to a custom Triton kernel. Within 10% of FlashAttention-3 speed for common patterns, and the mask function is a Python callable.

When to pick each:

- **Pure full attention** — every layer up to ~16K context, or when retrieval quality is paramount.
- **SWA + global mix** — long context (>32K), training and inference memory-bound. The 2026 default above 32K.
- **Sparse block attention** — custom kernel, custom pattern. Reserved for specialized workloads (retrieval, audio).

- **Differential attention** — any workload where attention-sink contamination hurts (long-context RAG, needle-in-haystack).

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/15-attention-variants>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/15-attention-variants/code>

1. **Easy.** Run `code/main.py`. Verify SWA at `window=4` zeroes everything outside the last 4 tokens per row. Verify `window=n` reproduces full causal attention bit-identically.
2. **Medium.** Implement causal SWA with `window=1024` on top of the Lesson 07 capstone. Train for 1,000 steps on `tinyshakespeare`. How much does val loss regress vs full attention? How much does peak memory drop?
3. **Hard.** Implement a Gemma-3-style 5:1 layer mix (5 SWA, 1 global) in the capstone model. Compare loss, memory, and generation quality against pure-SWA and pure-global baselines at matched parameters.
4. **Hard.** Implement differential attention with a learned λ per head. Train on a synthetic retrieval task (one needle, 2,000 distractors). Measure retrieval accuracy vs a single-attention baseline at matched parameters.

Key Terms

Term	What people say	What it actually means
Sliding window attention (SWA)	“Local attention”	Each query attends to its last W tokens; KV cache shrinks to $O(W)$.
Effective receptive field	“How far back the model sees”	In an L -layer SWA stack with window W , up to $L \times W$ tokens.
Longformer / BigBird	“Local + global + random”	Sparse patterns with a few always-attending global tokens; early long-context approach.
Native Sparse Attention	“DeepSeek’s kernel trick”	Learn block-level sparsity; skip zero blocks at the kernel level while keeping quality.
Differential attention	“Two maps, one subtracts”	DIFF Transformer: subtract a learned λ times a second attention map from the first to cancel attention sinks.
Attention sink	“Weight bleeds to token 0”	Softmax normalization forces rows to sum to 1; uninformative queries dump weight on position 0.
FlexAttention	“Mask-as-Python”	PyTorch 2.5+ API that compiles arbitrary mask functions into FlashAttention-shape kernels.

Term	What people say	What it actually means
Layer type mix	“5:1 SWA-to-global”	Interleave sparse and full attention layers in a stack to keep quality at lower memory.

Further Reading

- Beltagy, Peters, Cohan (2020). Longformer: The Long-Document Transformer — the canonical sliding-window + global-token paper.
- Zaheer et al. (2020). Big Bird: Transformers for Longer Sequences — local + global + random.
- Child et al. (2019). Generating Long Sequences with Sparse Transformers — OpenAI’s local+strided pattern.
- Gemma Team (2024). Gemma 2: Improving Open Language Models at a Practical Size — the 1:1 SWA:global mix.
- Gemma Team (2025). Gemma 3 technical report — the 5:1 mix with window=1024 that’s now the textbook default.
- Ye et al. (2024). Differential Transformer — DIFF Transformer paper.
- Yuan et al. (2025). Native Sparse Attention — DeepSeek-V3.2’s learned-sparsity attention.
- PyTorch — FlexAttention blog and docs — API reference for the mask-as-callable pattern in Use It.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/15-attention-variants>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/15-attention-variants/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.

Speculative Decoding — Draft, Verify, Repeat

Autoregressive decoding is serial. Each token waits for the previous one. Speculative decoding breaks the chain: a cheap model drafts N tokens, the expensive model verifies all N in one forward pass. When the draft is right you paid one big forward for N generations.

Type: Build **Languages:** Python **Prerequisites:** Phase 7 · 07 (GPT Causal LM), Phase 7 · 12 (KV Cache & Flash Attention) **Time:** ~60 minutes

The Problem

A 70B LLM sampling one token takes ~30 ms on an H100. A 3B draft model takes ~3 ms. If we let the 3B draft 5 tokens ahead, then run the 70B *once* to verify all 5, the total is $5 \times 3 + 30 = 45$ ms for up to 5 accepted tokens — versus $5 \times 30 = 150$ ms for straight-line generation. That is the full speculative-decoding pitch: trade a small amount of extra GPU memory (draft model) for 2–4× lower decode latency.

The trick has to preserve the distribution. Speculative sampling, introduced by Leviathan et al. (2023) and by Chen et al. concurrently, guarantees that the output sequence is **identically distributed** to what the big model would have produced on its own. No quality tradeoff. Just faster.

Four families of draft-verifier pairs dominate 2026 inference:

1. **Vanilla speculative (Leviathan 2023).** Separate draft model (e.g., Llama 3 1B) + verifier (e.g., Llama 3 70B).
2. **Medusa (Cai 2024).** Multiple decoding heads on the verifier predict positions $t+1 \dots t+k$ in parallel. No separate draft model.
3. **EAGLE family (Li 2024, 2025).** Lightweight draft that reuses the verifier’s hidden states; closer acceptance rate than vanilla; 3–4× typical.
4. **Lookahead decoding (Fu 2024).** Jacobi iteration; no draft model required at all. Self-speculation. Niche but dependency-free.

Every production inference stack in 2026 ships speculative decoding by default. vLLM, TensorRT-LLM, SGLang, and llama.cpp all support at least vanilla + EAGLE-2.

The Concept

The core algorithm

Given a verifier M_q and a cheaper draft M_p :

1. Let $x_1 \dots x_k$ be the prefix already decoded.
2. **Draft:** use M_p to autoregressively propose $d_{\{k+1\}}$, $d_{\{k+2\}}$, ..., $d_{\{k+N\}}$ with draft probabilities $p_1 \dots p_N$.

3. **Verify in parallel:** run M_q once on $x_1..x_k, d_{\{k+1\}}, \dots, d_{\{k+N\}}$, getting verifier probabilities $q_1..q_{\{N+1\}}$ for positions $k+1..k+N+1$.
4. **Accept/reject each draft token left to right:** for each i , accept with probability $\min(1, q_i(d_i) / p_i(d_i))$.
5. On first rejection at position j : sample t_j from the “residual” distribution $(q_j - p_j)_+$ normalized. All drafts after j are discarded.
6. On accepting all N : sample one extra token $t_{\{N+1\}}$ from $q_{\{N+1\}}$ (the free bonus token).

The residual distribution trick is the mathematical insight that keeps the output distributed exactly as if M_q had sampled from scratch.

What determines speedup

Let α = expected acceptance rate per draft token. Let c = draft-to-verifier cost ratio. Per step:

- Naive generation makes 1 big-model call per token.
- Speculative makes 1 big-model call per $(1 - \alpha^{\{N+1\}}) / (1 - \alpha) \approx 1/(1-\alpha)$ tokens when α is high.

Typical rule of thumb at $\alpha = 0.75$ and $N = 5$: 3× fewer big-model calls. Draft cost is 5× cheap. Total wall-clock drops ~2.5×.

α depends on:

- How well the draft approximates the verifier. Same family / same training data boosts α significantly.
- Decoding strategy. Greedy draft against greedy verifier: high α . Temperature sampling: harder to match; acceptance drops.
- Task type. Code and structured output accept more (predictable); free-form creative writing accepts less.

Medusa — drafts without a draft model

Medusa replaces the draft model with extra output heads on the verifier. At position t :

```
shared trunk → hidden  $h_t$ 
├─ head_0: predict token at  $t+1$  (standard LM head)
├─ head_1: predict token at  $t+2$ 
├─ head_2: predict token at  $t+3$ 
└─ head_3: predict token at  $t+4$ 
```

Each head outputs its own logits. At inference you sample from each head to get a candidate sequence, then verify with one forward pass using a tree-attention scheme that considers all candidate continuations at once.

Pros: no second model. Cons: adds trainable parameters; needs a supervised fine-tuning stage (~1B tokens); acceptance rate is a bit lower than vanilla speculative with a good draft.

EAGLE — better draft by reusing hidden states

EAGLE-1/2/3 (Li et al., 2024–2025) makes the draft model a tiny transformer (typically 1 layer) that ingests the verifier’s last-layer hidden states. Because the draft sees the verifier’s feature representation, its predictions correlate strongly with the verifier’s output distribution. Acceptance rates climb from ~0.6 (vanilla) to 0.85+.

EAGLE-3 (2025) added tree search over candidate continuations. vLLM and SGLang ship EAGLE-2/3 as the default spec pathway for Llama 3/4 and Qwen 3.

The KV cache dance

Verification feeds N draft tokens into the verifier in one forward pass. This extends the verifier’s KV cache by N entries. If some drafts are rejected, you must roll the cache back to the accepted prefix length.

Production implementations (vLLM’s `--speculative-model`, TensorRT-LLM’s LookaheadDecoder) handle this with scratch KV buffers. Write first, commit on acceptance. It’s not conceptually hard, but it is fiddly.

Interactive figure: draft-verify-tokens. This one moves. Watch it animate and drag its controls in the web edition: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/16-speculative-decoding>

Build It

See `code/main.py`. We implement the core speculative-sampling algorithm (rejection step + residual distribution) with:

- A “big model” that is a deterministic-softmax over a hand-coded distribution (so we can verify acceptance math analytically).
- A “draft model” that is a perturbation of the big model.
- An acceptance / rejection loop that produces the same marginal distribution as direct sampling.

Step 1: the rejection step

```
def accept_or_reject(q_prob, p_prob, draft_token, u):
    ratio = q_prob / p_prob if p_prob > 0 else float("inf")
    return u < min(1.0, ratio)
```

u is a uniform random number. q_prob is the verifier’s probability for the drafted token. p_prob is the draft model’s probability. The Leviathan theorem is that this Bernoulli decision, followed by sampling from the residual on rejection, preserves the verifier’s distribution exactly.

Step 2: residual distribution

```
def residual_dist(q, p):
    raw = [max(0.0, qi - pi) for qi, pi in zip(q, p)]
    s = sum(raw)
    return [r / s for r in raw]
```

Subtract p from q element-wise, clamp negative values to zero, renormalize. Sample from this on any rejection.

Step 3: one speculative step

```
def spec_step(prefix, q_model, p_model, N, rng):
    drafts = []
    p_probs = []
    ctx = list(prefix)
    for _ in range(N):
        p_dist = p_model(ctx)
        d = sample(p_dist, rng)
        drafts.append(d)
```

```

    p_probs.append(p_dist[d])
    ctx.append(d)

q_dists = [q_model(prefix + drafts[:i]) for i in range(N + 1)]

for i, d in enumerate(drafts):
    u = rng.random()
    q_prob = q_dists[i][d]
    p_prob = p_probs[i]
    if u < min(1.0, q_prob / p_prob if p_prob > 0 else float("inf")):
        prefix = prefix + [d]
    else:
        res = residual_dist(q_dists[i], p_model(prefix))
        prefix = prefix + [sample(res, rng)]
    return prefix
prefix = prefix + [sample(q_dists[N], rng)]
return prefix

```

Five accepted → one bonus → six tokens produced in one verifier pass.

Step 4: measure acceptance rate

Run 10,000 speculative steps at varying draft-quality levels. Plot acceptance rate vs. KL divergence between draft and verifier distributions. You should see a clean monotone relationship.

Step 5: verify distribution equivalence

Empirically: the histogram of tokens produced by the speculative loop should match the histogram produced by sampling directly from the verifier. This is the Leviathan theorem in practice. A chi-square test confirms within sampling error.

Use It

Production:

```

# vLLM with EAGLE
vllm serve meta-llama/Llama-3.1-70B-Instruct \
    --speculative-model /models/llama-3.1-eagle-70b \
    --speculative-draft-tensor-parallel-size 1 \
    --num-speculative-tokens 5

# vLLM with vanilla draft model
vllm serve meta-llama/Llama-3.1-70B-Instruct \
    --speculative-model meta-llama/Llama-3.2-1B-Instruct \
    --num-speculative-tokens 5

```

TensorRT-LLM has the fastest Medusa path as of mid-2026. faster-whisper wraps speculative decoding for Whisper-large with a small draft.

Picking a draft:

Strategy	When to pick	Speedup
Vanilla draft (1B/3B Llama family)	Fast prototype, no training	1.8–2.3×
Medusa heads	You can fine-tune the verifier	2–3×
EAGLE-2 / 3	Production, max speed	3–4×
Lookahead	No draft, no training, no extra params	1.3–1.6×

When NOT to spec-decode:

- Single-sequence generation of 1–5 tokens. Overhead dominates.
- Wildly creative / high-temperature sampling (α drops).
- Memory-constrained deployments (draft model adds VRAM).

This chapter ships an artifact. The course version of this lesson produces a reusable prompt or agent skill. It lives in the repository, ready to install: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/16-speculative-decoding>

Exercises

Starter code and the lesson’s working implementation: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/16-speculative-decoding/code>

1. **Easy.** Run code/main.py. Confirm the speculative token distribution matches the verifier’s direct-sample distribution on 50,000 tokens within chi-square $p > 0.05$.
2. **Medium.** Plot speedup (tokens per big-model forward) as a function of N for $\alpha = 0.5, 0.7, 0.85$. Identify the optimal N for each α . (Hint: expected tokens per verify call = $(1 - \alpha^{N+1}) / (1 - \alpha)$.)
3. **Hard.** Implement a tiny Medusa: take the capstone GPT from Lesson 14, add 3 extra LM heads that predict positions $t+2, t+3, t+4$. Train on tinyshakespeare with a joint multi-head loss. Compare acceptance rates vs a vanilla draft made by truncating the same model.
4. **Hard.** Implement rollback: start with a 10-token prefix KV cache, feed 5 draft tokens, simulate a rejection at position 3. Verify your cache reads correctly match “prefix + first 2 accepted drafts” at the next iteration.

Key Terms

Term	What people say	What it actually means
Draft model	“The cheap one”	A smaller model that proposes candidate tokens; usually 10–50× cheaper than the verifier.
Verifier	“The big one”	The target model whose distribution we preserve; runs once per speculative step.
Acceptance rate (α)	“How often the draft is right”	Per-token probability that the verifier accepts the draft. 0.7–0.9 typical.

Term	What people say	What it actually means
Residual distribution	“The rejection fallback”	$(q - p)_+ + \text{normalized}$; sampling from this on rejection preserves the verifier’s distribution.
Bonus token	“The free one”	When all N drafts accepted, sample one more from the verifier’s next-step distribution.
Medusa	“Draft-less speculative”	Multiple LM heads on the verifier predict positions $t+1..t+k$ in parallel.
EAGLE	“Hidden-state draft”	Tiny transformer draft conditioned on the verifier’s last-layer hidden states.
Lookahead decoding	“Jacobi iteration”	Self-speculation using a fixed-point iteration; no draft model.
Tree attention	“Verify many candidates at once”	Branching verification that considers several draft continuations simultaneously.
KV rollback	“Undo rejected drafts”	Scratch KV buffer; commit on acceptance, discard on reject.

Further Reading

- Leviathan, Kalman, Matias (2023). Fast Inference from Transformers via Speculative Decoding — the core algorithm and the equivalence theorem.
- Chen et al. (2023). Accelerating Large Language Model Decoding with Speculative Sampling — concurrent introduction; clean Bernoulli-rejection proof.
- Cai et al. (2024). Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads — Medusa paper; tree-attention verification.
- Li et al. (2024). EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty — EAGLE-1; hidden-state-conditioned draft.
- Li et al. (2024). EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees — EAGLE-2; dynamic tree depth.
- Li et al. (2025). EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test — EAGLE-3.
- Fu et al. (2024). Break the Sequential Dependency of LLM Inference Using Lookahead Decoding — lookahead, no-draft approach.
- vLLM docs — Speculative Decoding — canonical production reference with all four strategies wired up.
- SafeAILab / EAGLE reference implementation — the reference code for EAGLE-1/2/3.

Continue online. The living edition of this chapter has more than the page can hold:

- Animated, interactive figures and the web text: <https://aiengineeringfromscratch.com/lesson?path=phases/07-transformers-deep-dive/16-speculative-decoding>
- Runnable code for every step: <https://github.com/rohitg00/ai-engineering-from-scratch/tree/main/phases/07-transformers-deep-dive/16-speculative-decoding/code>

The repository moves faster than any printing. When the book and the repo disagree, trust the repo.